

NATIONAL POSTGRADUATE ENTRANCE EXAMINATION

408

计算机学科专业基础综合

系统复习讲义 · 章节训练 · 三套模拟卷

数据结构 · 计算机组成原理 · 操作系统 · 计算机网络

2026 REVIEW EDITION

luoyue

github.com/luoyueyuguang

目录

PART I 第一部分

数据结构



1 线性表

1.1 线性表的定义与 ADT

定义 1.1: 线性表

线性表 (Linear List) 是由 n ($n \geq 0$) 个数据元素 $a_1, a_2, \dots, a_n$ 构成的有限序列, 满足:

- 1. 除首元素 a_1 外, 每个元素有且仅有一个直接前驱;
- 2. 除末元素 a_n 外, 每个元素有且仅有一个直接后继;
- 3. 元素个数 n 称为表长, $n = 0$ 时为空表。

线性表是逻辑结构概念, 不规定存储方式。实现时可选择顺序存储或链式存储。

结论 1.1: 线性表的抽象数据类型 (ADT)

线性表 ADT 的核心操作集 (以 C 风格描述):

线性表 ADT 操作	
操作	语义
InitList(&L)	初始化空表
DestroyList(&L)	销毁表, 释放空间
ClearList(&L)	清空为初始状态
ListEmpty(L)	判空, 返回 bool
ListLength(L)	返回元素个数
GetElem(L, i, &e)	取第 i 个元素 ($1 \leq i \leq n$)
LocateElem(L, e)	定位第一个等于 e 的元素位置
PriorElem(L, cur, &pre)	取前驱
NextElem(L, cur, &next)	取后继
ListInsert(&L, i, e)	在第 i 位前插入 e
ListDelete(&L, i, &e)	删除第 i 位元素, 用 e 返回
ListTraverse(L)	遍历访问

408 考试中重点关注 GetElem、LocateElem、ListInsert、ListDelete 的算法实现与复杂度, 且要能区分类似 LocateElem 与 GetElem 的差异 (按值 vs 按位)。

1.2 顺序表 (Sequential List)

定义 1.2: 顺序表

用一组地址连续的存储单元依次存储线性表的数据元素。若每个元素占 s 个存储单元，首地址为 $LOC(a_1)$ ，则第 i 个元素的地址为：

$$LOC(a_i) = LOC(a_1) + (i - 1) \times s, \quad 1 \leq i \leq n.$$

这一性质称为随机存取 (Random Access)：任意位置元素的访问时间为 $O(1)$ 。

要点 1.1: 顺序表的存储结构 (C 实现)

```
1 #define MaxSize 100          // 最大容量
2 typedef int ElemType;        // 元素类型，可按需替换
3 typedef struct {
4     ElemType data[MaxSize];    // 存储空间
5     int length;               // 当前长度
6 } SqList;                    // 顺序表类型
```

关键点：data 是静态数组，预先分配 MaxSize；length 记录实际元素个数（不是最后一个元素的下标）。408 常考「length 与下标的关系」——第 i 个元素对应 data[i-1]。

顺序表的基本操作

结论 1.2: 顺序表的插入操作

在第 i ($1 \leq i \leq n + 1$) 位插入新元素 e ：

1. 检查 i 是否合法，以及表是否已满；
2. 将第 i 到第 n 个元素依次后移一位；
3. 在位置 $i - 1$ 处放入 e ，length++。

```
1 bool ListInsert(SqList *L, int i, ElemType e) {
2     if (i < 1 || i > L->length + 1) return false;
3     if (L->length >= MaxSize) return false;
4     for (int j = L->length; j >= i; j--)
5         L->data[j] = L->data[j-1];
6     L->data[i-1] = e;
7     L->length++;
8     return true;
9 }
```

时间复杂度：

$$T(n) = \frac{1}{n+1} \sum_{i=1}^{n+1} (n - i + 1) = \frac{n}{2}.$$

等概率下平均移动 $n/2$ 个元素，为 $O(n)$ 。最好情况（插在末尾） $O(1)$ ，最坏情况（插在表头） $O(n)$ 。

结论 1.3: 顺序表的删除操作

删除第 i ($1 \leq i \leq n$) 位元素并用 e 返回:

```

1 bool ListDelete(SqList *L, int i, ElemType *e) {
2     if (i < 1 || i > L->length) return false;
3     *e = L->data[i-1];
4     for (int j = i; j < L->length; j++)
5         L->data[j-1] = L->data[j];
6     L->length--;
7     return true;
8 }

```

等概率下平均移动 $(n-1)/2$ 个元素, 为 $O(n)$ 。

结论 1.4: 顺序表按值查找

```

1 int LocateElem(SqList L, ElemType e) {
2     for (int i = 0; i < L.length; i++)
3         if (L.data[i] == e) return i + 1; // 返回位序
4     return 0; // 未找到
5 }

```

平均比较次数 $(n+1)/2$, 时间复杂度 $O(n)$ 。

与按位查找的区别: `GetElem(L, i, &e)` 直接通过下标访问 `L.data[i-1]`, 时间复杂度 $O(1)$ ——顺序表的随机存取特性。

1.3 单链表 (Singly Linked List)**定义 1.3: 单链表**

用一组任意的存储单元存放线性表元素, 每个结点包含:

- 数据域 (data): 存放元素值;
- 指针域 (next): 指向直接后继结点的指针。

n 个结点通过指针链成一个链表。最后一个结点的指针域为 `NULL`。

单链表不再支持随机存取——访问第 i 个元素需从头沿链走 $i-1$ 步, 时间复杂度 $O(n)$ 。

要点 1.2: 单链表的结点定义

```

1 typedef struct LNode {
2     ElemType data;
3     struct LNode *next;
4 } LNode, *LinkList;

```

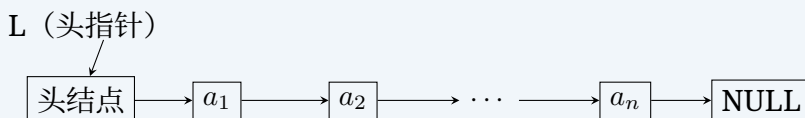
`LinkList` 是指向 `LNode` 的指针, 即头指针。后续代码中的 `L` 通常指头指针。

头结点 vs 无头结点

结论 1.5: 头结点的作用

408 考试中单链表通常带头结点 (head node)，即第一个结点 (头结点) 的 data 无实际意义，next 指向首元结点。好处：

1. 空表/非空表的操作代码统一——头指针 L 始终非空 (至少指向头结点)；
2. 链表头部的插入/删除不需要单独判定边界。



注意：题目既可能采用带头结点的链表，也可能采用不带头结点的链表，务必以题设为准。

单链表的基本操作

结论 1.6: 头插法建表

每次将新结点插入到头结点之后，形成逆序链表：

```

1  LinkedList CreateListHead(int n) {
2      LinkedList L = (LinkedList)malloc(sizeof(LNode));
3      L->next = NULL; // 头结点
4      for (int i = 0; i < n; i++) {
5          LNode *s = (LNode*)malloc(sizeof(LNode));
6          scanf("%d", &s->data);
7          s->next = L->next; // (1) s 后继 = 原首元
8          L->next = s; // (2) 头$1to$s
9      }
10     return L;
11 }
  
```

时间复杂度 $O(n)$ 。输入顺序与结果顺序相反。

结论 1.7: 尾插法建表

每次将新结点接到链表末尾，保持插入顺序：

```

1  LinkedList CreateListTail(int n) {
2      LinkedList L = (LinkedList)malloc(sizeof(LNode));
3      LNode *r = L; // r 始终指向当前尾结点
4      for (int i = 0; i < n; i++) {
5          LNode *s = (LNode*)malloc(sizeof(LNode));
6          scanf("%d", &s->data);
7          r->next = s; // 尾结点指向新结点
8          r = s; // 更新尾指针
9      }
10     r->next = NULL;
11     return L;
12 }
  
```

时间复杂度 $O(n)$ ，输入与结果顺序相同。

结论 1.8: 单链表按位查找

从第一个数据结点（不是头结点）开始计数：

```

1 LNode *GetElem(LinkList L, int i) {
2     if (i < 0) return NULL;
3     LNode *p = L;           // p 指向头结点 (第0个)
4     int j = 0;
5     while (p && j < i) {     // j=i 时 p 指向第i个结点
6         p = p->next;
7         j++;
8     }
9     return p;               // 若 i>表长 则返回 NULL
10 }
```

注意：GetElem(L, 0) 返回头结点；GetElem(L, 1) 返回首元结点。时间复杂度 $O(n)$ 。

结论 1.9: 单链表插入操作

在第 $i-1$ 个结点后插入新结点（即插入为第 i 个）：

```

1 bool ListInsert(LinkList L, int i, ElemType e) {
2     LNode *p = GetElem(L, i-1);   // 找第 i-1 个结点
3     if (!p) return false;
4     LNode *s = (LNode*)malloc(sizeof(LNode));
5     s->data = e;
6     s->next = p->next;             // (1) 新结点先接后继
7     p->next = s;                   // (2) 前驱再接新结点
8     return true;
9 }
```

关键：语句 (1) (2) 的顺序不能颠倒！若先执行 $p->next = s$ ，则原后继链丢失。时间复杂度：查找 $O(n)$ ，插入本身 $O(1)$ 。

结论 1.10: 单链表删除操作

删除第 i 个结点：

```

1 bool ListDelete(LinkList L, int i, ElemType *e) {
2     LNode *p = GetElem(L, i-1);   // 找第 i-1 个结点 (前驱)
3     if (!p || !p->next) return false; // p不存在或p无后继
4     LNode *q = p->next;           // q 指向第 i 个结点
5     *e = q->data;
6     p->next = q->next;             // 前驱跳过 q
7     free(q);
8     return true;
9 }
```


注意

408 常考陷阱——删除指定结点 p 本身：若只给指向 p 的指针（不知道前驱），可以将 p 的后继结点的 `data` 复制到 p ，然后删除后继结点。但此方法不适用于 p 是最后一个结点，也不适用于结点 `data` 量过大或包含不能简单复制的资源（如指针/文件句柄）。

1.4 双链表 (Doubly Linked List)

定义 1.4: 双链表

每个结点有两个指针域：`prior` 指向前驱，`next` 指向后继。

```
typedef struct DNode {
    ElemType data;
    struct DNode *prior, *next;
} DNode, *DLinkedList;
```

双链表可以从任意结点出发沿两个方向遍历，但存储密度低于单链表（多一个指针域）。

结论 1.11: 双链表的插入操作

在结点 p 之后插入新结点 s ：

```
s->next = p->next;           // (1) s接p的后继
s->prior = p;                 // (2) s接p
if (p->next) p->next->prior = s; // (3) p后继的前驱指向s
p->next = s;                  // (4) p接s
```

四条语句缺一不可，且顺序有讲究：先建立 s 的两个指针，再修改原链中指向 s 的两个指针。若 p 为尾结点则跳过 (3)。

结论 1.12: 双链表的删除操作

删除结点 p 的后继结点 q ：

```
DNode *q = p->next;
p->next = q->next;           // p跳过q
if (q->next) q->next->prior = p; // q后继的前驱指向p
free(q);
```

注意 q 不存在和 q 为尾结点的边界处理。

1.5 循环链表 (Circular Linked List)

定义 1.5: 循环链表

将单链表（或双链表）的最后一个结点的指针域指向头结点（而不是 `NULL`），形成一个环。

- 循环单链表：尾结点 `next` 指向头结点。判空条件：`L->next == L`。
- 循环双链表：头结点的 `prior` 指向尾结点；尾结点的 `next` 指向头结点。判空条件：`L->next == L && L->prior == L`。

循环链表适合需要循环遍历或从任意结点出发沿链反复扫描的场景。它避免了到达表尾后重新回到表头的特殊处理，但从一个任意结点查找到另一个任意结点通常仍需 $O(n)$ 时间，并不提供随机访问。

1.6 静态链表 (Static Linked List)

定义 1.6: 静态链表

用数组模拟链表：数组元素即结点，包含 data 和 next（记录后继结点的数组下标，-1 表示 NULL）。

```
#define MaxSize 100
typedef struct {
    ElemType data;
    int next;           // 后继下标
} SLinkList[MaxSize];
```

静态链表不需要动态内存分配 (malloc/free)，适用于不支持指针的语言或内存受限场景。408 中偶尔以选择题形式考察。

1.7 顺序表 vs 链表的比较

顺序表与链表的对比

比较维度	顺序表	链表
存储方式	连续地址	任意地址，指针链接
存取方式	随机存取 $O(1)$	顺序存取 $O(n)$
插入/删除	需移动元素 $O(n)$	修改指针 $O(1)$ （已定位时）
存储密度	≈ 1 （纯数据）	< 1 （含指针域）
缓存性能	好（局部性好）	差（结点分散）
动态扩容	静态数组需整体搬移	可按需分配单个结点

1.8 408 高频考点与真题风格

结论 1.13: 线性表 408 选择题常考点

- 1. 顺序表插入/删除的元素移动次数：平均 $\approx n/2$ ，注意区分插入与删除的差别。
- 2. 头结点与无头结点：空表判定、边界处理的代码差异。
- 3. 单链表插入语句的顺序：先连后继，再断前驱链。
- 4. 「删除 p 所指结点」陷阱：无前驱时可否用复制后继法、何时失效。
- 5. 双链表/循环链表判空条件：L->next == L 等。
- 6. 时间复杂度辨析：已定位结点的插入/删除是 $O(1)$ ，但定位本身需要时间——单链表按位查找 $O(n)$ ，顺序表按值查找 $O(n)$ 。
- 7. 静态链表：next 域存的是下标而非指针，增删元素不移动数据元素。

2 习题

2.1 基础过关题

- (填空题) 在一个长度为 n 的顺序表中删除第 i ($1 \leq i \leq n$) 个元素, 等概率下需要向前移动的元素个数的期望值为 _____; 若在第 i ($1 \leq i \leq n+1$) 个位置之前插入一个新元素, 等概率下需要向后移动的元素个数的期望值为 _____。
- (单项选择题) 在单链表中, 要将指针 s 所指的新结点插入到指针 p 所指结点之后, 正确的语句序列是
 (A) $s \rightarrow \text{next} = p \rightarrow \text{next}; p \rightarrow \text{next} = s;$
 (B) $p \rightarrow \text{next} = s; s \rightarrow \text{next} = p \rightarrow \text{next};$
 (C) $s \rightarrow \text{next} = p; p \rightarrow \text{next} = s;$
 (D) $p \rightarrow \text{next} = s \rightarrow \text{next}; s \rightarrow \text{next} = p;$
- (真题风格·综合题) 设单链表的结点结构为 $[\text{data} \mid \text{next}]$, 头指针为 L (带头结点)。编写算法, 将链表就地逆置, 即辅助空间复杂度为 $O(1)$ 。分析算法的时间复杂度与空间复杂度。
- (真题风格·综合题) 已知一个带头结点的单链表, 结点结构为 $[\text{data} \mid \text{next}]$ 。假设该链表只给出了头指针 L 。在不改变链表的前提下, 设计一个尽可能高效的算法, 查找链表中倒数第 k (k 为正整数) 个位置上的结点。若查找成功, 算法输出该结点的 data 值并返回 1; 否则只返回 0。
- (真题风格·综合题) 已知一个带头结点的单链表, 结点结构为 $[\text{data} \mid \text{next}]$ 。设计一个算法, 删除链表中所有数据值在区间 $[a, b]$ 内的结点 ($a \leq b$), 并分析算法的时间复杂度。
- (真题风格·综合题) 假定采用带头结点的单链表保存单词。当两个单词有相同的后缀时, 可共享相同的后缀存储空间。设 str1 和 str2 分别指向两个单词所在单链表的头结点, 链表结点结构为 $[\text{data} \mid \text{next}]$ 。设计一个尽可能高效的算法, 找出由 str1 和 str2 所指的两个链表共同后缀的起始位置。

2.2 真题风格综合训练

- (真题风格·综合题) 设将 n ($n > 1$) 个整数存放到一维数组 R 中。设计一个在时间和空间两方面都尽可能高效的算法, 将 R 中保存的序列循环左移 p ($0 < p < n$) 个位置, 即将 R 中的数据由 $(x_0, x_1, \dots, x_{n-1})$ 变换为 $(x_p, x_{p+1}, \dots, x_{n-1}, x_0, x_1, \dots, x_{p-1})$ 。
- (真题风格·综合题) 一个长度为 L ($L \geq 1$) 的升序序列 S , 处在第 $\lceil L/2 \rceil$ 个位置的数称为 S 的中位数。例如, 若序列 $S_1 = (11, 13, 15, 17, 19)$, 则 S_1 的中位数是 15。两个序列的中位数是含它们所有元素的升序序列的中位数。例如, 若 $S_2 = (2, 4, 6, 8, 20)$, 则 S_1 和 S_2 的中位数是 11。现有两个等长升序序列 A 和 B , 设计一个在时间和空间两方面都尽可能高效的算法, 找出两个序列 A 和 B 的中位数。
- (真题风格·综合题) 已知一个整数序列 $A = (a_0, a_1, \dots, a_{n-1})$, 其中 $0 \leq a_i < n$ ($0 \leq i < n$)。若存在 $a_{p_1} = a_{p_2} = \dots = a_{p_m} = x$ 且 $m > n/2$ ($0 \leq p_k < n$, $1 \leq k \leq m$), 则称 x 为 A 的主元素。例如 $A = (0, 5, 5, 3, 5, 7, 5, 5)$, 则 5 为主元素; 又如 $A = (0, 5, 5, 3, 5, 1, 5, 7)$, 则 A 中没有主元素。假设 A 中的 n 个元素保存在一个一维数组中, 设计一个尽可能高效的算法, 找出 A 的主元素。若存在主元素, 则输出该元素; 否则输出 -1。

- 10. (真题风格·综合题) 用单链表保存 m 个整数, 结点的结构为 $[data][next]$, 且 $|data| \leq n$ (n 为正整数)。设计一个时间复杂度尽可能高效的算法, 对于链表中 $data$ 的绝对值相等的结点, 仅保留第一次出现的结点而删除其余绝对值相等的结点。
- 11. (真题风格·综合题) 给定一个含 n ($n \geq 1$) 个整数的数组, 设计一个在时间上尽可能高效的算法, 找出数组中未出现的最小正整数。例如, 数组 $\{-5, 3, 2, 3\}$ 未出现的最小正整数是 1; 数组 $\{1, 2, 3\}$ 未出现的最小正整数是 4。
- 12. (真题风格·综合题) 已知一个带头结点的单链表, 结点结构为 $[data | next]$ 。设计一个算法, 将该链表按结点位置的奇偶性拆分为两个链表: 位置为奇数的结点保持原顺序组成链表 A , 位置为偶数的结点保持原顺序组成链表 B (结点编号从 1 开始计数, 即首元结点为第 1 个结点)。

拓展阅读:

- 邓俊辉《数据结构 (C++ 语言版)》第 2 章「向量」和第 3 章「列表」——从 ADT 分层视角理解线性表, 包括可扩充向量和有序列表。
- Sedgewick《算法》第 1.3 节「背包、队列和栈」——基于链表的实现技巧和迭代器设计。

3 栈与队列

3.1 栈的定义与 ADT

定义 3.1: 栈

栈 (Stack) 是一种操作受限的线性表: 只允许在表的一端进行插入和删除。这一端称为栈顶 (Top), 另一端称为栈底 (Bottom)。
栈的操作遵循后进先出 (LIFO, Last In First Out) 原则: 最后入栈的元素最先出栈。如同弹匣——后压入的子弹先弹出。
称插入操作为入栈 (Push), 称删除操作为出栈 (Pop)。

结论 3.1: 栈的抽象数据类型 (ADT)

栈 ADT 的核心操作集:

栈 ADT 操作	
操作	语义
InitStack(&S)	初始化空栈
DestroyStack(&S)	销毁栈, 释放空间
StackEmpty(S)	判空, 返回 bool
StackLength(S)	返回栈中元素个数
GetTop(S, &e)	取栈顶元素 (不出栈)
Push(&S, e)	入栈, 将 e 压入栈顶
Pop(&S, &e)	出栈, 弹出栈顶元素并用 e 返回

408 考试中, Push 和 Pop 的实现细节是高频考点: 指针的初始值、栈满/栈空的判定条件、操作前后 top 的变化。

3.2 顺序栈 (Sequential Stack)

定义 3.2: 顺序栈

用一组地址连续的存储单元存放栈元素。用指针 top 指示栈顶位置。
根据 top 的约定不同, 有两种常见实现:

1. **top** 指向栈顶元素 (408 最常用): 初始化 $\text{top} = -1$ (空栈), 入栈时先 $++\text{top}$ 再写入, 出栈时先读取再 $--\text{top}$ 。
2. **top** 指向栈顶下一空位: 初始化 $\text{top} = 0$, 入栈时先写入再 $++\text{top}$, 出栈时先 $--\text{top}$ 再读取。

要点 3.1: 顺序栈的存储结构与基本操作

约定: **top** 指向栈顶元素 ($\text{top} = -1$ 为空栈)。

```
1 #define MaxSize 100
2 typedef int ElemType;
3 typedef struct {
4     ElemType data[MaxSize]; // 存储空间
5     int top;                // 栈顶指针
6 } SqStack;
```

核心操作:

```
1 // 初始化
2 void InitStack(SqStack *S) {
3     S->top = -1;
4 }
5
6 // 判空
7 bool StackEmpty(SqStack S) {
8     return S.top == -1;
9 }
10
11 // 判满
12 bool StackFull(SqStack S) {
13     return S.top == MaxSize - 1;
14 }
15
16 // 入栈
17 bool Push(SqStack *S, ElemType e) {
18     if (S->top == MaxSize - 1) return false; // 栈满
19     S->data[++(S->top)] = e;                 // 先移指针再写入
20     return true;
21 }
22
23 // 出栈
```

```

24 bool Pop(SqStack *S, ElemType *e) {
25     if (S->top == -1) return false;           // 栈空
26     *e = S->data[(S->top)--];                 // 先读取再移指针
27     return true;
28 }
29
30 // 取栈顶 (不出栈)
31 bool GetTop(SqStack S, ElemType *e) {
32     if (S.top == -1) return false;
33     *e = S.data[S.top];
34     return true;
35 }

```

注意

408 易错点——top 初始值不同导致判空/判满条件变化：

不同 top 约定对比			
约定	初值	判空	判满
top = -1 (指向栈顶元素)	-1	top == -1	top == MaxSize-1
top = 0 (指向下一空位)	0	top == 0	top == MaxSize

务必根据题设中的 top 初始值来答题，408 选择题常在此设置陷阱。

结论 3.2: 共享栈

两个栈共享同一段数组空间：栈 1 从数组低地址端向高地址增长，栈 2 从高地址端向低地址增长。两个栈顶指针相遇即为栈满。

```

1 typedef struct {
2     ElemType data[MaxSize];
3     int top1;    // 栈1 栈顶, 初始 -1
4     int top2;    // 栈2 栈顶, 初始 MaxSize
5 } ShareStack;

```

判满条件：top1 + 1 == top2 (两个栈顶指针相邻)。

优点：两个栈互补，更充分地利用数组空间——栈 1 满载但栈 2 空时，总容量并未耗尽。408 中常以选择题形式考察共享栈的判满条件。

3.3 链栈 (Linked Stack)

定义 3.3: 链栈

用链表实现的栈，以链表的头部作为栈顶，便于 $O(1)$ 的入栈和出栈操作。
 由于操作均在头部进行，链栈不需要头结点（头指针即栈顶指针）。栈空时头指针为 NULL。

要点 3.2: 链栈的实现

```
1 typedef struct SNode {
2     ElemType data;
3     struct SNode *next;
4 } SNode, *LinkStack;
5
6 // 初始化
7 void InitStack(LinkStack *S) {
8     *S = NULL;
9 }
10
11 // 判空
12 bool StackEmpty(LinkStack S) {
13     return S == NULL;
14 }
15
16 // 入栈 (头插法)
17 bool Push(LinkStack *S, ElemType e) {
18     SNode *p = (SNode*)malloc(sizeof(SNode));
19     if (!p) return false;
20     p->data = e;
21     p->next = *S;          // 新结点指向原栈顶
22     *S = p;                // 更新栈顶
23     return true;
24 }
25
26 // 出栈
27 bool Pop(LinkStack *S, ElemType *e) {
28     if (*S == NULL) return false;
29     SNode *p = *S;
30     *e = p->data;
31     *S = p->next;          // 栈顶后移
32     free(p);
33     return true;
34 }
```

链栈不会栈满（除非内存耗尽），适合元素数量不可预测的场景。

3.4 队列的定义与 ADT

定义 3.4: 队列

队列（Queue）也是一种操作受限的线性表：只允许在表的一端插入，在另一端删除。

插入端称为队尾（Rear），删除端称为队头（Front）。

队列的操作遵循先进先出（FIFO, First In First Out）原则：最早入队的元素最早出队。如同排队——先进入队伍的先行离开。

称插入操作为入队（EnQueue），称删除操作为出队（DeQueue）。

结论 3.3: 队列的抽象数据类型 (ADT)

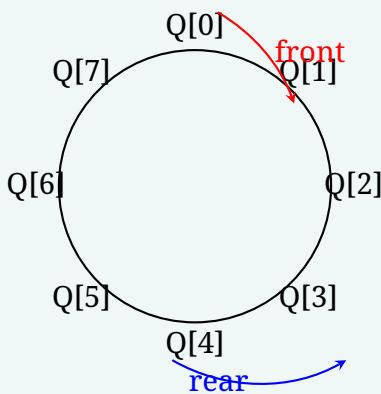
核心操作集:

队列 ADT 操作	
操作	语义
InitQueue(&Q)	初始化空队列
DestroyQueue(&Q)	销毁队列, 释放空间
QueueEmpty(Q)	判空, 返回 bool
QueueLength(Q)	返回队列中元素个数
GetHead(Q, &e)	取队头元素 (不出队)
EnQueue(&Q, e)	入队, 将 e 加入队尾
DeQueue(&Q, &e)	出队, 删除队头并用 e 返回

3.5 顺序队列与循环队列

定义 3.5: 循环队列

普通顺序队列: 入队在尾部追加, 出队从头部删除。问题在于出队后, 前面的空间无法复用 (假溢出)。
循环队列: 将数组 $Q[0..MaxSize - 1]$ 逻辑上视为环形——下标模 $MaxSize$ 循环。用 $front$ 指向队头, $rear$ 指向队尾 (或队尾下一空位), 入队和出队时指针循环前进。



要点 3.3: 循环队列的存储结构与基本操作

约定: $front$ 指向队头元素, $rear$ 指向队尾元素的下一个空位。

```
#define MaxSize 100
typedef struct {
    ElemType data[MaxSize];
    int front;    // 队头指针
    int rear;     // 队尾指针 (指向队尾下一空位)
} SqQueue;
```

核心操作:


```
1 // 初始化
2 void InitQueue(SqQueue *Q) {
3     Q->front = Q->rear = 0;
4 }
5
6 // 入队 (循环前进)
7 bool EnQueue(SqQueue *Q, ElemType e) {
8     if ((Q->rear + 1) % MaxSize == Q->front)
9         return false; // 队满
10    Q->data[Q->rear] = e; // 写入队尾
11    Q->rear = (Q->rear + 1) % MaxSize; // rear 循环+1
12    return true;
13 }
14
15 // 出队
16 bool DeQueue(SqQueue *Q, ElemType *e) {
17     if (Q->front == Q->rear)
18         return false; // 队空
19    *e = Q->data[Q->front];
20    Q->front = (Q->front + 1) % MaxSize; // front 循环+1
21    return true;
22 }
23
24 // 队列长度
25 int QueueLength(SqQueue Q) {
26     return (Q.rear - Q.front + MaxSize) % MaxSize;
27 }
```

结论 3.4: 循环队列判空/判满的三种方式

在上述 front 指向队头、rear 指向队尾下一空位的约定下，front == rear 既是空队条件，也是满队条件——需要额外手段区分：

判空/判满的三种策略		
策略	判空	判满
(1) 牺牲一个单元	front == rear	(rear+1)%MaxSize == front
(2) 增设长度计数	len == 0	len == MaxSize
(3) 增设 tag 标记	front==rear && tag==0	front==rear && tag==1

策略（1）是 408 最常用写法，代价是数组有效容量为 MaxSize - 1。策略（2）和（3）可以充分利用所有单元，但需要额外维护计数器或标记位。tag 标记在每次入队操作时置 1，出队操作时置 0。
注意：若定义 rear 指向队尾元素（而非下一空位），则需相应调整初始值、入队/出队的顺序和判空/判满条件——408 选择题务必仔细审题。

3.6 链队列 (Linked Queue)

定义 3.6: 链队列

用链表实现的队列。需要两个指针：`front` 指向队头结点，`rear` 指向队尾结点。入队操作在链尾进行，出队操作在链头进行。

通常带头结点：头结点便于统一空队和非空队的操作代码。空队时 `front == rear` (都指向头结点)。

要点 3.4: 链队列的实现

```
1  typedef struct QNode {
2     ElemType data;
3     struct QNode *next;
4 } QNode;
5  typedef struct {
6     QNode *front;    // 队头指针
7     QNode *rear;     // 队尾指针
8 } LinkQueue;
9
10 // 初始化 (带头结点)
11 void InitQueue(LinkQueue *Q) {
12     Q->front = Q->rear = (QNode*)malloc(sizeof(QNode));
13     Q->front->next = NULL;
14 }
15
16 // 判空
17 bool QueueEmpty(LinkQueue Q) {
18     return Q.front == Q.rear;
19 }
20
21 // 入队 (尾插法)
22 bool EnQueue(LinkQueue *Q, ElemType e) {
23     QNode *p = (QNode*)malloc(sizeof(QNode));
24     if (!p) return false;
25     p->data = e;
26     p->next = NULL;
27     Q->rear->next = p;    // 原队尾 $to$ 新结点
28     Q->rear = p;         // 更新队尾指针
29     return true;
30 }
31
32 // 出队
33 bool DeQueue(LinkQueue *Q, ElemType *e) {
34     if (Q->front == Q->rear) return false;    // 队空
35     QNode *p = Q->front->next;                // p 指向队头结点
36     *e = p->data;
37     Q->front->next = p->next;                  // 头结点跳过队头
38     if (Q->rear == p)                          // 若删除的是最后一个结点
39         Q->rear = Q->front;                  // 则 rear 也回到头结点
```

```

40     free(p);
41     return true;
42 }

```

注意出队时的特判：当队列只剩一个元素时，删除后队尾指针 `rear` 必须回指头结点，否则 `rear` 将指向已释放的内存（悬挂指针）。这是 408 常考的细节。

3.7 双端队列 (Deque)

定义 3.7: 双端队列

双端队列 (Double-Ended Queue, Deque) 是允许在两端都进行插入和删除操作的线性表。根据受限方式分为两类：

- 无限制双端队列：两端均可入队、出队。
- 输出受限的双端队列：一端可插入/删除，另一端只能插入。
- 输入受限的双端队列：一端可插入/删除，另一端只能删除。

双端队列是栈和队列的推广：若限定一端只入不出、另一端只出不入，则退化为队列；若限定所有操作在同一端，则退化为栈。

注意

408 对双端队列的考察通常为选择题：给定输入序列（如 1, 2, 3, 4），判断某种双端队列能产生/不能产生哪种输出序列。解题关键——在纸上模拟两端操作的可能性，标记出受限端的操作能力。

3.8 数组与特殊矩阵的压缩存储

结论 3.5: 数组的地址计算

设二维数组 $A[m][n]$ 的每个元素占 L 字节，首元素地址为 $\text{LOC}(A[0][0])$ 。

- 按行优先： $\text{LOC}(A[i][j]) = \text{LOC}(A[0][0]) + (i \cdot n + j)L$ ；
- 按列优先： $\text{LOC}(A[i][j]) = \text{LOC}(A[0][0]) + (j \cdot m + i)L$ 。

若题目下标从 1 开始，必须先把 i, j 分别减 1；若给出了各维下界，也要先减去对应下界。

结论 3.6: 特殊矩阵的压缩存储

- 对称矩阵：只存上三角或下三角，共 $n(n+1)/2$ 个元素。按行存下三角（含主对角线）、下标从 0 开始时， $i \geq j$ 的元素映射为 $k = i(i+1)/2 + j$ ；访问上三角元素 $A[i][j]$ 时改取 $A[j][i]$ 。
- 三角矩阵：只存一个三角区和一个公共常量，共 $n(n+1)/2 + 1$ 个存储单元。
- 三对角矩阵：只有主对角线及其上下相邻对角线可能非零，共 $3n - 2$ 个非零位置。
- 稀疏矩阵：非零元素很少时可用三元组 (i, j, v) 表或十字链表，存储量与非零元素个数 t 相关，而不是固定为 n^2 。

注意

矩阵压缩题必须先确认：下标从 0 还是 1 开始、存上三角还是下三角、按行还是按列、主对角线是否包含在内。公式不能跨约定直接套用。

3.9 栈的应用

括号匹配

结论 3.7: 括号匹配算法

检查表达式中的括号 (())、[]、{} 是否正确配对：

1. 扫描表达式，遇左括号则入栈；
2. 遇右括号：
 - 若栈空，则不匹配（右括号多了）；
 - 弹出栈顶左括号，检查是否与当前右括号同类——不匹配则报错；
3. 扫描完后，若栈非空则不匹配（左括号多了）；栈空则匹配成功。

```

1 bool BracketMatch(char *expr) {
2     SqStack S; InitStack(&S);
3     for (int i = 0; expr[i] != '\0'; i++) {
4         if (expr[i]=='(' || expr[i]=='[' || expr[i]=='{')
5             Push(&S, expr[i]);
6         else if (expr[i]==')' || expr[i]==']' || expr[i]=='}') {
7             if (StackEmpty(S)) return false;
8             char top; Pop(&S, &top);
9             if ((expr[i]==')' && top!='(') ||
10                (expr[i]==']' && top!='[') ||
11                (expr[i]=='}' && top!='{'))
12                 return false;
13         }
14     }
15     return StackEmpty(S); // 最后栈空才匹配
16 }

```

时间复杂度 $O(n)$ ，空间复杂度 $O(n)$ 。

表达式求值

定义 3.8: 中缀、后缀与前缀表达式

- 中缀表达式 (Infix)：运算符在两个操作数之间。如 $A + B$ 。符合人类习惯，但需要括号和优先级规则。
- 后缀表达式 (Postfix / 逆波兰表示法)：运算符在操作数之后。如 $AB+$ 。无需括号，天然适合栈计算。
- 前缀表达式 (Prefix / 波兰表示法)：运算符在操作数之前。如 $+AB$ 。

408 重点考察中缀转后缀的过程和后缀表达式求值。

结论 3.8: 中缀表达式转后缀表达式

算法（算符优先法）：

1. 初始化一个运算符栈（存运算符和左括号），结果队列存后缀表达式；
2. 从左到右扫描中缀表达式的每个 token：
 - 操作数：直接输出（进入后缀表达式）；
 - 左括号 (：入栈；
 - 右括号)：依次弹出栈顶运算符并输出，直到遇到左括号（左括号出栈但不输出）；
 - 运算符：若其优先级高于栈顶，则入栈；否则，反复弹出栈顶优先级不低于当前运算符的运算符并输出，然后将当前运算符入栈。
3. 扫描完毕后，将栈中剩余运算符依次弹出输出。

运算符优先级（408 标准，不含函数、位运算）：

运算符优先级表（优先级数字越大越高）

运算符	优先级	结合性
* / %	3	左结合
+ -	2	左结合
(	1（入栈后视为最低）	—

例题

示例： $A + B * (C - D) - E / F$ 转后缀：

当前 token	栈（栈底 → 栈顶）	输出
A	空	A
+	+	A
B	+	AB
*	+ *	AB
(	+ * (	AB
C	+ * (	ABC
-	+ * (-	ABC
D	+ * (-	ABCD
)	+ *	ABCD-
-	-	ABCD-*+
E	-	ABCD-*+E
/	- /	ABCD-*+E
F	- /	ABCD-*+EF
结束	空	ABCD-*+EF/

结果为 $ABCD - * + EF /$ 。

结论 3.9: 后缀表达式求值

算法:

1. 初始化一个操作数栈;
2. 从左到右扫描后缀表达式;
 - 操作数: 入栈;
 - 运算符: 弹出栈顶两个操作数 (先弹出的是右操作数, 后弹出的是左操作数), 计算后结果入栈。
3. 最终栈中唯一元素即表达式的值。

```

1 // 计算后缀表达式 (操作数为整数, 运算符 + - * /)
2 int EvalPostfix(char *postfix) {
3     SqStack S; InitStack(&S);
4     for (int i = 0; postfix[i]; i++) {
5         if (isdigit(postfix[i])) {
6             Push(&S, postfix[i] - '0'); // 操作数入栈
7         } else { // 运算符
8             int b, a; Pop(&S, &b); Pop(&S, &a); // b 先弹出 (右操作数)
9             switch (postfix[i]) {
10                case '+': Push(&S, a + b); break;
11                case '-': Push(&S, a - b); break;
12                case '*': Push(&S, a * b); break;
13                case '/': Push(&S, a / b); break;
14            }
15        }
16    }
17    int result; Pop(&S, &result);
18    return result;
19 }

```

注意

408 中缀转后缀易错点:

1. 左括号入栈后优先级最低——只有右括号才能让它出栈;
2. 相同优先级运算符, 左结合时, 栈内优先级高于栈外优先级 (即先算左边的), 所以当前运算符不高于栈顶时就要弹出栈顶;
3. 后缀表达式中操作数的顺序与中缀表达式完全相同——只有运算符位置改变。

递归与栈**结论 3.10: 递归的工作栈**

递归函数的调用过程天然符合栈的 LIFO 特性。每次函数调用时, 系统将返回地址、局部变量、参数等信息压入调用栈 (Call Stack), 返回时弹出。

以阶乘为例:

```

1 int Factorial(int n) {

```

```

2   if (n == 0 || n == 1) return 1;    // 递归基
3   return n * Factorial(n - 1);      // 递归调用
4 }
5 // Factorial(4):
6 // 调用: fact(4) $|to$ fact(3) $|to$ fact(2) $|to$ fact(1)
7 // 返回: 1 $|to$ 2 $|to$ 3 $|to$ 4 $|to$ 24

```

递归与栈的关系：

- 任何递归算法都可以用栈机械地转换为非递归算法；
- 递归函数本质是通过系统栈保存状态，自己显式维护栈可以达到同样的效果；
- 递归到非递归的转换有时能避免系统栈溢出（如用迭代 + 显式栈模拟深度优先遍历）。

408 常考点：给定一段递归代码，要求模拟调用栈的变化过程，或分析递归深度（即栈的最大深度）。

3.10 队列的应用

结论 3.11: 层次遍历 (BFS)

队列是实现广度优先遍历 (BFS, Breadth-First Search) 的核心数据结构。
以二叉树的层次遍历为例（详细见第 4 章）：

1. 根结点入队；
2. 当队列非空时：出队一个结点并访问之；将其左、右孩子（若存在）依次入队；
3. 重复步骤 2 直到队列为空。

```

1 // 二叉树层次遍历（假设已定义 BiTree 类型）
2 void LevelOrder(BiTree T) {
3     SqQueue Q; InitQueue(&Q);
4     if (T) EnQueue(&Q, T);
5     while (!QueueEmpty(Q)) {
6         BiTree p; DeQueue(&Q, &p);
7         visit(p);                // 访问结点
8         if (p->lchild) EnQueue(&Q, p->lchild);
9         if (p->rchild) EnQueue(&Q, p->rchild);
10    }
11 }

```

队列保证同一层的结点按从左到右的顺序被访问——这是 BFS 的关键性质。

结论 3.12: 缓冲区与生产者-消费者

在操作系统中，队列广泛用于缓冲区管理：

- 打印机缓冲队列：多个打印任务按提交顺序排队等待打印；
- 消息队列：生产者将消息入队，消费者从队头取出处理；
- CPU 就绪队列：就绪进程按优先级/时间片排队等待调度。

队列的 FIFO 特性天然保证了处理顺序的公平性。

3.11 栈与队列的 408 高频考点

结论 3.13: 408 选择/填空常考公式与结论

1. 循环队列的指针与元素个数:

$$\text{元素个数} = (\text{rear} - \text{front} + \text{MaxSize}) \bmod \text{MaxSize}$$

其中 front 指向队头, rear 指向队尾下一空位。

2. 循环队列判满 (牺牲一单元时):

$$(\text{rear} + 1) \bmod \text{MaxSize} = \text{front}$$

3. 共享栈判满:

$$\text{top1} + 1 = \text{top2}$$

4. n 个元素入栈, 可能的出栈序列数 (Catalan 数):

$$C_n = \frac{1}{n+1} \binom{2n}{n}$$

例如 $n = 3$ 时, 共有 $C_3 = 5$ 种可能的出栈序列 (全排列 $3! = 6$ 种中, 3, 1, 2 不可能合法出栈)。

5. 合法出栈序列的判断: 对任意三个元素 $i < j < k$, 若 k 先于 i 和 j 出栈, 则 i 必须后于 j 出栈 (大元素先出, 则其后的小元素出栈顺序必须与入栈顺序相反)。

6. 中缀转后缀的手工方法:

- 将所有运算用括号显式标注优先级 (全括号化);
- 将运算符移到对应右括号之后;
- 去掉所有括号。

例如 $A + B * C \rightarrow (A + (B * C)) \rightarrow A B C * +$ 。

7. 后缀表达式求值: 弹出时第一个是右操作数, 第二个是左操作数——减法和除法不可交换顺序。

注意

408 真题反复考察的循环队列细节:

1. 题目可能使用不同的指针定义 (front 指向队头前一位置、rear 指向队尾元素等), 判空/判满条件和入队/出队操作步骤必须根据题设推导, 切忌死记硬背公式。
2. 已知 front 和 rear 的值求元素个数时, 使用模运算公式, 注意 MaxSize 的具体值。
3. 「最多能存储多少个元素」——若采用牺牲一单元的方式, 答案为 $\text{MaxSize} - 1$; 否则为 MaxSize 。

4 习题

4.1 基础过关题

1. (真题风格·选择题) 设栈 S 的初始状态为空, 入栈序列为 $1, 2, 3, \dots, n$, 若出栈序列的第一个元素是 k ($2 \leq k \leq n$), 则以下说法正确的是

(A) 此时元素 $1, 2, \dots, k-1$ 必定全部在栈中, 且 1 在栈底
(B) 第 i 个出栈元素必定是 $n-i+1$
(C) 出栈序列的个数与 k 无关
(D) 出栈序列的可能个数为 C_{n-1}^{k-1}

2. (真题风格·选择题) 设循环队列的存储空间为 $Q[0..m-1]$, $front$ 指向队头元素, $rear$ 指向队尾元素的下一位置。则元素入队时 $rear$ 的变化为

(A) $rear = rear + 1$ (B) $rear = (rear + 1) \% (m - 1)$
(C) $rear = (rear + 1) \% m$ (D) $rear = (rear + 1) \% (m + 1)$

3. (真题风格·选择题) 中缀表达式 $A + B * (C - D) - E / F$ 对应的后缀表达式是

(A) $ABCD - * + EF / -$ (B) $AB + CD - * EF / -$
(C) $ABCD - * + EF /$ (D) $AB + CD - * E - F /$

4. (真题风格·选择题) 对后缀表达式 $ab + c * d - ef + /$ 进行求值 (设 a, b, c, d, e, f 均为操作数), 在从左到右扫描求值的过程中, 操作数栈中元素个数的最大值为

(A) 2 (B) 3
(C) 4 (D) 5

5. (真题风格·选择题) 设用数组 $A[0..n-1]$ 实现两个栈共享存储空间的共享栈。栈 1 的栈底在 $A[0]$ 处, 从低地址向高地址增长; 栈 2 的栈底在 $A[n-1]$ 处, 从高地址向低地址增长。 $top1$ 指向栈 1 的栈顶元素, $top2$ 指向栈 2 的栈顶元素。则栈满的条件是

(A) $top1 + top2 = n$ (B) $top1 + 1 = top2$
(C) $top1 = top2$ (D) $top1 + 1 < top2$

6. (真题风格·选择题) 设循环队列的存储空间为 $Q[0..m-1]$, $front$ 指向队头元素, $rear$ 指向队尾元素的下一位置。在采用「牺牲一个存储单元」判满策略时, 队列为空的判定条件是

(A) $front = rear$ (B) $(rear + 1) \bmod m = front$
(C) $front = 0$ 且 $rear = 0$ (D) $front = (rear + 1) \bmod m$

7. (基础题) 一个 6×8 的二维数组按行优先存储, 每个元素占 4 B, $A[0][0]$ 的地址为 1000, 则 $A[3][5]$ 的地址是多少?

8. (基础题) 一个 n 阶三对角矩阵按行压缩存储, 可能的非零元素最多有多少个?

4.2 综合训练

9. (真题风格·选择题) 以下算法中, 需要使用队列作为辅助数据结构的是

- (A) 二叉树的先序遍历 (B) 二叉树的层次遍历
(C) 图的深度优先遍历 (D) 中缀表达式转后缀表达式

10. (综合题) 给定入栈序列 $\{1, 2, 3, 4, 5\}$, 请:

- (1) 列出所有以 3 开头、以 5 结尾的合法出栈序列;
(2) 说明推导过程与判断合法性的关键原则。

11. (综合题) 设循环队列 $Q[0..M-1]$, **front** 指向队头元素, **rear** 指向队尾元素的下一个空位。回答以下问题:

- (1) 写出队列中元素个数的计算公式;
(2) 若 $M = 10$, **front** = 6, **rear** = 3, 求队列中元素个数;
(3) 若采用「牺牲一个单元」策略, 分别写出判空和判满条件。

拓展阅读:

- 邓俊辉《数据结构 (C++ 语言版)》第 4 章「栈与队列」——表达式求值的完整实现。
- Sedgewick《算法》第 1.3 节——栈与队列的 ADT 定义与迭代器设计。

5 串

5.1 串的定义与 ADT

定义 5.1: 串

串 (String) 是由零个或多个字符组成的有限序列, 记作:

$$S = 'a_1a_2 \cdots a_n' \quad (n \geq 0)$$

其中 S 为串名, n 为串长, $n = 0$ 时为空串, 记作 ε 。

相关术语:

- 子串: 串中任意连续字符组成的子序列。
- 主串: 包含子串的串。
- 串相等: 长度相等且对应位置字符均相同。
- 空格串: 由一个或多个空格组成的串 (不是空串)。

注意

408 辨析重点: 空串 **vs** 空格串。空串 ε 长度为 0; 空格串由空格字符组成, 长度 ≥ 1 。

5.2 朴素的模式匹配 (BF 算法)

定义 5.2: 模式匹配

给定主串 S (长 n) 和模式串 T (长 m , $n \geq m$), 在 S 中寻找首次等于 T 的子串的过程称为模式匹配。

结论 5.1: BF (Brute-Force) 算法

从主串每个位置开始, 逐个字符与模式串比较; 失配则主串回溯到下一位置, 模式串回到首字符。

```

1 int Index_BF(SString S, SString T) {
2     int i = 1, j = 1;
3     while (i <= S.length && j <= T.length) {
4         if (S.ch[i] == T.ch[j]) { i++; j++; }
5         else { i = i - j + 2; j = 1; } // i回溯, j复位
6     }
7     if (j > T.length) return i - T.length;
8     else return 0;
9 }

```

回溯公式 $i = i - j + 2$ 的理解: $i - j + 1$ 是本轮起始位置, $+1$ 即下一位置。

时间复杂度: 最好 $O(n)$, 最坏 $O(nm)$ (如 $S = \text{"aaaa...a"}$, $T = \text{"aaaab"}$)。只有在字符独立、失配位置具有特定概率分布等平均模型下, 期望代价才可接近线性; 脱离概率模型时不应无条件写成 “平均 $O(n + m)$ ”。

5.3 KMP 算法

定义 5.3: KMP 算法

KMP (Knuth-Morris-Pratt) 利用已匹配部分的信息, 使主串指针 i 不回溯, 模式串 j 根据 **next** 数组跳转。时间复杂度 $O(n + m)$ 。

结论 5.2: KMP 核心思想

当 $S[i] \neq T[j]$ 失配时, 已匹配的 $T[1..j-1] = S[i-j+1..i-1]$ 。若 $T[1..j-1]$ 的某个真前缀等于某个真后缀, 则 T 可「滑动」使该前缀对齐到后缀位置, i 不动, j 跳到该前缀长度 $+1$ 处继续比较。

next 数组的定义与手算

定义 5.4: next 数组 (下标从 1 开始)

$$\text{next}[j] = \begin{cases} 0, & j = 1 \\ \max\{k \mid 1 < k < j, T[1..k-1] = T[j-k+1..j-1]\}, & \text{集合非空} \\ 1, & \text{其他} \end{cases}$$

即 $\text{next}[j] =$ 前 $j-1$ 个字符的最长相等真前后缀长度 $+1$ 。

注意

408 默认下标从 1 开始, $\text{next}[1]=0$, $\text{next}[2]=1$ 恒成立。若下标从 0 开始, 则 $\text{next}[0]=-1$, $\text{next}[1]=0$ (每个值减 1)。

结论 5.3: 手算 next 数组示例

以 $T = \text{"abaabcac"}$ 为例:

j	1	2	3	4	5	6	7	8
$T[j]$	a	b	a	a	b	c	a	c
$\text{next}[j]$	0	1	1	2	2	3	1	2

计算过程 (以 $j = 6$ 为例): 前 5 个字符 "abaab" 的最长公共前后缀为 "ab" (长度 2), 故 $\text{next}[6]=3$ 。

KMP 匹配算法**结论 5.4: KMP 匹配代码**

```

1 int Index_KMP(SString S, SString T) {
2     int i = 1, j = 1;
3     int next[MAXSTRLEN];
4     GetNext(T, next);
5     while (i <= S.length && j <= T.length) {
6         if (j == 0 || S.ch[i] == T.ch[j])
7             { i++; j++; }
8         else
9             j = next[j];        // i不动! 仅j回溯
10    }
11    if (j > T.length) return i - T.length;
12    else return 0;
13 }
```

$j = 0$ 的含义: $\text{next}[1]=0$, 当首字符失配时 j 变 0, 下一轮 $i++$ 、 $j++$ 使 j 恢复为 1。

next 数组的递推求法**结论 5.5: GetNext 算法**

利用已知的 next 值递推计算:

```

1 void GetNext(SString T, int next[]) {
2     int j = 1, k = 0;
3     next[1] = 0;
4     while (j < T.length) {
5         if (k == 0 || T.ch[j] == T.ch[k]) {
6             j++; k++;
7             next[j] = k;
8         } else {
```

```

9         k = next[k];           // 失配时k回溯
10    }
11 }
12 }
```

时间复杂度 $O(m)$ 。核心： k 始终表示「当前已匹配的最长公共前后缀长度」。

nextval 数组

结论 5.6: nextval 改进

若 $T[j] = T[\text{next}[j]]$ ，则失配跳到 $\text{next}[j]$ 后字符相同仍会失配。 nextval 在此情况下继续向前跳：

```

1 void GetNextVal(SString T, int nextval[]) {
2     int j = 1, k = 0;
3     nextval[1] = 0;
4     while (j < T.length) {
5         if (k == 0 || T.ch[j] == T.ch[k]) {
6             j++; k++;
7             nextval[j] = (T.ch[j] != T.ch[k]) ? k : nextval[k];
8         } else {
9             k = nextval[k];
10        }
11    }
12 }
```

结论 5.7: 408 高频考点汇总

1. next 数组的手工计算——选择题高频，重点是先确认下标和定义约定。
2. $\text{next}[1]=0, \text{next}[2]=1$ 恒成立，可用于快速排除选项。
3. KMP 匹配过程中 i 永不回溯，比较次数 $\leq 2n$ 。
4. BF 的最坏时间复杂度 $O(nm)$ 及其触发条件。
5. nextval 的计算（偶尔出现）。

6 习题

6.1 基础过关题

- (单项选择题) 串的长度是指
 - 串中所含不同字母的个数
 - 串中所含字符的个数
 - 串中所含不同字符的个数
 - 串中所含非空格字符的个数
- (单项选择题) 设有两个串 p 和 q , 求 q 在 p 中首次出现的位置的运算称为
 - 连接
 - 模式匹配
 - 求子串
 - 求串长
- (填空题) 空串与空格串的区别是: 空串的长度为 ____, 空格串的长度 ____。
- (填空题) 设模式串 $T = \text{"ababaa"}$, 下标从 1 开始, 则 $\text{next}[4] = ______$, $\text{next}[5] = ______$ 。
- (简答题) 已知主串 $S = \text{"aaabaaaab"}$, 模式串 $T = \text{"aaaab"}$ 。用 BF 算法进行匹配, 写出每趟匹配的过程, 统计总的字符比较次数。
- (计算题) 求模式串 $T = \text{"abcabca"}$ 的 next 数组和 nextval 数组 (下标从 1 开始)。
- (简答题) KMP 算法相对于 BF 算法的优势是什么? 什么情况下 KMP 的优势最明显?

6.2 真题风格与综合训练

- (真题风格, 单项选择题) 已知模式串 $T = \text{"ababaabab"}$, 下标从 1 开始, 则 $\text{next}[6]$ 和 $\text{next}[7]$ 的值分别为
 - 3 和 2
 - 3 和 3
 - 4 和 2
 - 4 和 3
- (真题风格, 单项选择题) 设主串 S 的长度为 n , 模式串 T 的长度为 m 。若把构造 next 数组和随后执行匹配都计入, KMP 算法的总时间复杂度为
 - $O(n)$
 - $O(n + m)$
 - $O(nm)$
 - $O(m)$
- (真题风格, 单项选择题) 设有模式串 $T = \text{"aaab"}$, 下标从 1 开始, 以下关于 nextval 数组的说法中正确的是
 - $\text{nextval}[2] = 1$
 - $\text{nextval}[3] = 2$
 - $\text{nextval}[4] = 3$
 - $\text{nextval}[4] = 1$
- (真题风格, 综合题) 设主串 $S = \text{"ababcabcacbab"}$, 模式串 $T = \text{"abcac"}$ 。
 - 计算 T 的 next 数组 (下标从 1 开始);
 - 用 KMP 算法写出匹配过程 (画表格, 标出每趟的 i 、 j 起始值和匹配结果);
 - 统计总的字符比较次数。

12. (真题风格, 综合题) 给出求模式串 next 数组的算法 (GetNext), 分析其时间复杂度。若模式串为 $T = \text{"aaaaab"}$, 该算法的时间复杂度是否会退化为 $O(m^2)$? 说明理由。

拓展阅读:

- 邓俊辉《数据结构 (C++ 语言版)》第 9 章「串」——KMP 算法的完整推导, 包括 next 数组的改进 (nextval)。
- Sedgewick《算法》第 5.3 节「子字符串查找」——KMP 的 DFA 视角理解。

7 树与二叉树

7.1 树的定义与基本术语

定义 7.1: 树

树 (Tree) 是 n ($n \geq 0$) 个结点的有限集。当 $n = 0$ 时称为空树。任意一棵非空树满足:

1. 有且仅有一个特定的称为根 (Root) 的结点;
2. 当 $n > 1$ 时, 其余结点可分为 m ($m > 0$) 个互不相交的有限集 $T_1, T_2, \dots, T_m$, 其中每个集合本身又是一棵树, 称为根的子树 (Subtree)。

树的定义是递归的: 树由根和若干子树构成, 子树本身也是树。

结论 7.1: 树的基本术语

- 结点的度 (Degree): 结点拥有的子树个数。树的度为各结点度的最大值。
- 叶子结点 (Leaf): 度为 0 的结点, 也称终端结点。
- 分支结点: 度大于 0 的结点, 也称非终端结点。
- 孩子/双亲/兄弟: 结点的子树的根称为该结点的孩子 (Child); 该结点称为孩子的双亲 (Parent); 同一双亲的孩子互为兄弟 (Sibling)。
- 路径与路径长度: 从结点 v_0 到 v_k 的路径是结点序列 $v_0, v_1, \dots, v_k$, 其中 v_{i-1} 是 v_i 的双亲。路径上边的数目称为路径长度。
- 深度与高度: 结点的深度 (Depth) 是从根到该结点的路径长度 (根深为 0 或 1, 依约定); 树的高度 (Height) 是树中结点的最大深度。408 通常约定根深度为 1, 高度 = 最大层数。
- 祖先与子孙: 从根到某结点路径上的所有结点 (不含自身) 为该结点的祖先; 以某结点为根的子树中除该结点自身外的所有结点为其子孙。
- 有序树 vs 无序树: 子树有左右次序之分则为有序树, 否则为无序树。

注意

408 中树的深度通常从 1 开始计数 (根的深度/层数为 1), 树高 = $\max\{\text{结点深度}\}$ 。但部分教材 (如邓俊辉) 从 0 开始, 考试务必以题目约定为准。

要点 7.1: 树的基本性质

设树 T 有 n 个结点, 则:

1. T 中边数 $= n - 1$ 。
2. 按本章定义, 结点的度是其孩子(子树)数, 因此各结点度数之和等于边数: $\sum \deg_{\text{树}}(v) = n - 1$ 。将边定向为父 $\rightarrow$ 子时, 这也就是 $\sum \text{outdeg}(v) = n - 1$ 。
3. 若把树另行视为无向图, 则图论中的顶点度统计每条边的两个端点, 故 $\sum \deg_{\text{图}}(v) = 2(n - 1)$ 。这里的图论度数与上一项的“孩子数”不是同一概念, 不能混用。
4. 设按孩子数计、度为 k 的结点有 n_k 个, 则 $n = \sum n_k$, 且 $\sum k \cdot n_k = n - 1$ 。

7.2 二叉树的定义与性质**定义 7.2: 二叉树**

二叉树(Binary Tree)是 n ($n \geq 0$) 个结点的有限集, 每个结点至多有两棵子树(分别称为左子树和右子树), 且子树有左右之分, 次序不能任意颠倒。

注意

二叉树不是度为 2 的树——度为 2 的树要求至少有一个度为 2 的结点, 而二叉树可以全部结点度为 1 (如单链状)。二叉树是结点的有限集加上左右子树的递归结构, 它区分左右, 即使只有一棵子树也要指明是左子树还是右子树。

特殊二叉树**定义 7.3: 满二叉树**

一棵高度为 h 的二叉树, 若每一层的结点数都达到最大值(第 i 层有 2^{i-1} 个结点, $1 \leq i \leq h$), 且叶子结点只出现在第 h 层, 则称为满二叉树(Full Binary Tree)。结点总数 $n = 2^h - 1$ 。

定义 7.4: 完全二叉树

一棵高度为 h 、有 n 个结点的二叉树, 当且仅当其每个结点都与高度为 h 的满二叉树中编号 $1 \sim n$ 的结点一一对应时, 称为完全二叉树(Complete Binary Tree)。等价描述: 除最后一层外每层都满, 且最后一层的结点从左到右连续排列, 中间不留空位。

结论 7.2: 完全二叉树的性质

对于 n 个结点的完全二叉树(层序编号 $1 \sim n$):

1. 编号为 i 的结点, 其左孩子编号为 $2i$ (若 $2i \leq n$, 否则无左孩子);
2. 右孩子编号为 $2i + 1$ (若 $2i + 1 \leq n$, 否则无右孩子);
3. 双亲编号为 $\lfloor i/2 \rfloor$ ($i > 1$);
4. 叶子结点只可能在最后两层出现;
5. 若结点 i 为叶子, 则编号 $\geq i$ 的结点均为叶子; 若结点 i 只有左孩子, 则编号 $> i$ 的结点均为叶子。

二叉树的核心性质

要点 7.2: 二叉树核心性质

设二叉树中度为 0、1、2 的结点数分别为 n_0, n_1, n_2 , 总结点数 $n = n_0 + n_1 + n_2$ 。

1. (408 高频) $n_0 = n_2 + 1$ 。
证明: 由 $n = n_0 + n_1 + n_2$, 且边数 $= n - 1 = n_1 + 2n_2$ (每个度为 k 的结点贡献 k 条边指向孩子)。联立得 $n_0 = n_2 + 1$ 。
2. 第 i 层上最多有 2^{i-1} 个结点 ($i \geq 1$)。
3. 高度为 h 的二叉树最多有 $2^h - 1$ 个结点。
4. 具有 n 个结点的二叉树, 高度至少为 $\lceil \log_2(n+1) \rceil$ (或 $\lfloor \log_2 n \rfloor + 1$)。
5. (408 高频) 具有 n 个结点的完全二叉树的高度为 $\lceil \log_2(n+1) \rceil$ 或 $\lfloor \log_2 n \rfloor + 1$ 。

7.3 二叉树的存储结构

顺序存储结构

要点 7.3: 二叉树的顺序存储

将完全二叉树的结点按层序编号 $1 \sim n$, 用数组 $T[1..n]$ 存储: $T[i]$ 存编号为 i 的结点。

```
1 #define MaxSize 100
2 typedef int ElemType;
3 typedef ElemType SqBiTree[MaxSize]; // T[0] 可闲置或存结点数
```

对于非完全二叉树, 需要按完全二叉树的编号规则留出空位 (存放特殊标记如 0 或 '#'), 会造成空间浪费。最坏情况 (单链状, 深度为 k) 需要 $2^k - 1$ 个存储位置, 仅用 k 个。

二叉链表存储

要点 7.4: 二叉链表

每个结点含数据域和左右指针域:

```
1 typedef struct BiTNode {
2     ElemType data;
3     struct BiTNode *lchild, *rchild;
4 } BiTNode, *BiTree;
```

n 个结点的二叉链表共有 $2n$ 个指针域, 其中 $n - 1$ 个指向孩子 (每条边对应一个指针), 剩余 $n + 1$ 个为空指针域。线索二叉树正是利用这些空指针域。

三叉链表: 在上述结点中增加 `parent` 指针域, 便于查找双亲。

7.4 二叉树的遍历

遍历 (Traversal) 是按某种次序访问二叉树中的每个结点, 且每个结点仅访问一次。遍历是二叉树一切操作的基础。

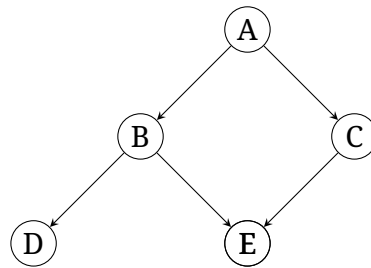


图 1: 二叉树的二叉链表表示: 每个结点通过 lchild/rchild 指针连接左右孩子

递归遍历

要点 7.5: 先序、中序、后序遍历的递归算法

三种深度优先遍历的核心区别在于访问根节点的时机:

```

1 // 先序遍历: 根 $lto$ 左 $lto$ 右
2 void PreOrder(BiTree T) {
3     if (T) {
4         visit(T);           // (1) 访问根
5         PreOrder(T->lchild); // (2) 遍历左子树
6         PreOrder(T->rchild); // (3) 遍历右子树
7     }
8 }
9
10 // 中序遍历: 左 $lto$ 根 $lto$ 右
11 void InOrder(BiTree T) {
12     if (T) {
13         InOrder(T->lchild); // (1) 遍历左子树
14         visit(T);           // (2) 访问根
15         InOrder(T->rchild); // (3) 遍历右子树
16     }
17 }
18
19 // 后序遍历: 左 $lto$ 右 $lto$ 根
20 void PostOrder(BiTree T) {
21     if (T) {
22         PostOrder(T->lchild); // (1) 遍历左子树
23         PostOrder(T->rchild); // (2) 遍历右子树
24         visit(T);           // (3) 访问根
25     }
26 }
  
```

结论 7.3: 递归遍历的时间/空间复杂度

每个结点访问一次且仅一次: 时间复杂度 $O(n)$ 。递归调用栈的最大深度 = 树的高度, 空间复杂度为 $O(h)$ 。退化为单链时 $h = n$, 故最坏为 $O(n)$; 仅在树高为 $O(\log n)$ (如平衡树) 时, 栈空间才是 $O(\log n)$ 。

非递归遍历

结论 7.4: 中序非递归遍历

用栈模拟递归调用栈，是 408 综合应用题的高频考点：

```

1 void InOrderNR(BiTree T) {
2     Stack S; InitStack(&S);
3     BiTree p = T;
4     while (p || !StackEmpty(S)) {
5         if (p) {
6             Push(&S, p);          // 沿左链走到底
7             p = p->lchild;
8         } else {
9             Pop(&S, &p);           // 退栈，访问
10            visit(p);
11            p = p->rchild;          // 转向右子树
12        }
13    }
14 }
```

核心思路：「沿左支深入，遇空则退栈访问，再转向右子树」。这也是 408 代码题最常考察的非递归模式。

结论 7.5: 先序非递归遍历

与中序类似，但入栈前即访问：

```

1 void PreOrderNR(BiTree T) {
2     Stack S; InitStack(&S);
3     BiTree p = T;
4     while (p || !StackEmpty(S)) {
5         if (p) {
6             visit(p);              // 先序：入栈前访问
7             Push(&S, p);
8             p = p->lchild;
9         } else {
10            Pop(&S, &p);
11            p = p->rchild;
12        }
13    }
14 }
```

结论 7.6: 后序非递归遍历

后序非递归较复杂：需要区分「从左子树返回」和「从右子树返回」两种退栈情形。

```

1 void PostOrderNR(BiTree T) {
2     Stack S; InitStack(&S);
3     BiTree p = T, lastVisited = NULL;
4     while (p || !StackEmpty(S)) {
5         if (p) {
```

```

6         Push(&S, p);
7         p = p->lchild;           // 沿左支深入
8     } else {
9         GetTop(S, &p);           // 查看栈顶, 不弹栈
10        if (p->rchild && p->rchild != lastVisited)
11            p = p->rchild;       // 右子树未访问, 转向
12        else {
13            Pop(&S, &p);         // 左右皆访问, 弹出
14            visit(p);
15            lastVisited = p;
16            p = NULL;           // 强制下次取栈顶
17        }
18    }
19 }
20 }

```

记忆技巧：入栈条件同先序；退栈时必须确保右子树已访问（用 `lastVisited` 标记），否则暂不弹出。

层次遍历

结论 7.7: 层次遍历

借助队列，逐层从左向右访问：

```

1 void LevelOrder(BiTree T) {
2     Queue Q; InitQueue(&Q);
3     if (T) EnQueue(&Q, T);
4     while (!QueueEmpty(Q)) {
5         DeQueue(&Q, &p);
6         visit(p);
7         if (p->lchild) EnQueue(&Q, p->lchild);
8         if (p->rchild) EnQueue(&Q, p->rchild);
9     }
10 }

```

时间复杂度 $O(n)$ ，空间复杂度 $O(w)$ ， w 为树的最大宽度（最坏 $O(n)$ ）。

7.5 遍历序列构造二叉树

结论 7.8: 由遍历序列构造二叉树

核心定理（408 高频）：在结点关键字互异且遍历序列合法的前提下，已知中序序列 + 先序序列或 中序序列 + 后序序列，可唯一确定一棵二叉树。已知先序 + 后序无法唯一确定二叉树（除非另有能够消除歧义的结构条件）。

构造方法（已知先序 + 中序）：

1. 先序的第一个结点为根；
2. 在中序中找到根的位置，根左侧为左子树中序，根右侧为右子树中序；
3. 根据左/右子树的结点数目，在先序中切分出左/右子树的先序；

4. 对左右子树递归执行上述步骤。

```

1 // pre[pl..pr] 为先序区间, in[il..ir] 为中序区间
2 BiTree Build(char pre[], char in[], int pl, int pr, int il, int ir) {
3     if (pl > pr) return NULL;
4     BiTree root = (BiTree)malloc(sizeof(BiTreeNode));
5     root->data = pre[pl]; // 根 = 先序首元
6     int k; // 根在中序中的位置
7     for (k = il; k <= ir; k++)
8         if (in[k] == pre[pl]) break;
9     if (k > ir) { // 序列不合法或关键字不一致
10        free(root);
11        return NULL;
12    }
13    int leftLen = k - il; // 左子树结点数
14    root->lchild = Build(pre, in, pl+1, pl+leftLen, il, k-1);
15    root->rchild = Build(pre, in, pl+leftLen+1, pr, k+1, ir);
16    return root;
17 }

```

后序 + 中序的构造完全对称：后序的最后结点为根，其余逻辑相同。

注意

408 选择题高频考察「给定两个遍历序列，判断能否唯一确定二叉树」以及「由序列还原二叉树的中间步骤」。务必掌握手动构造的流程：找根 → 切分中序 → 切分先序/后序 → 递归。

7.6 线索二叉树

定义 7.5: 线索二叉树

普通二叉链表中有 $n+1$ 个空指针域。线索二叉树将这些空指针域利用起来：

- 若结点无左孩子，则 lchild 指向前驱（某种遍历次序下的直接前驱）；
- 若结点无右孩子，则 rchild 指向后继（某种遍历次序下的直接后继）。

需要两个标志位 ltag、rtag 区分指针是指向孩子（0）还是线索（1）。

要点 7.6: 线索二叉树的结点结构

```

1 typedef struct ThreadNode {
2     ElemType data;
3     struct ThreadNode *lchild, *rchild;
4     int ltag, rtag; // 0: 孩子指针; 1: 线索指针
5 } ThreadNode, *ThreadTree;

```

中序线索化

结论 7.9: 中序线索化的算法

在递归中序遍历的过程中, 用一个全局变量 `pre` 记录刚刚访问过的结点, 以便建立线索:

```

1 ThreadNode *pre = NULL; // 全局变量, 记录前一个访问结点
2
3 void InThread(ThreadTree T) {
4     if (T) {
5         InThread(T->lchild);           // (1) 线索化左子树
6         if (!T->lchild) {               // (2) 处理当前结点
7             T->lchild = pre;
8             T->ltag = 1;
9         }
10        if (pre && !pre->rchild) {       // (3) 处理前驱的右线索
11            pre->rchild = T;
12            pre->rtag = 1;
13        }
14        pre = T;                       // (4) 更新 pre
15        InThread(T->rchild);           // (5) 线索化右子树
16    }
17 }
```

通过增设头结点, 可以将中序线索二叉树组织成循环链表以方便遍历。

线索二叉树的前驱与后继

结论 7.10: 中序线索二叉树: 找前驱与后继

这是 408 选择题/简答题的常考点。

找后继 `succ(p)`:

- 若 `p->rtag == 1`, 则 `p->rchild` 即为后继 (线索直接给出);
- 若 `p->rtag == 0` (有右孩子), 后继为右子树中第一个被访问的结点 (中序: 右子树的最左下结点)。

```

1 ThreadNode *InOrderSucc(ThreadNode *p) {
2     if (p->rtag == 1) return p->rchild; // 直接由线索给出
3     ThreadNode *q = p->rchild;        // 进入右子树
4     while (q->ltag == 0) q = q->lchild; // 找到最左下结点
5     return q;
6 }
```

找前驱 `pred(p)`:

- 若 `p->ltag == 1`, 则 `p->lchild` 即为前驱;
- 若 `p->ltag == 0` (有左孩子), 前驱为左子树中最后一个被访问的结点 (中序: 左子树的最右下结点)。

```

1 ThreadNode *InOrderPred(ThreadNode *p) {
2     if (p->ltag == 1) return p->lchild;
3     ThreadNode *q = p->lchild;
```

```

4   while (q->rtag == 0) q = q->rchild;
5   return q;
6   }

```

注意

408 常考陷阱：

1. 中序线索树中找前驱/后继的规律不适用于先序/后序线索树——分别有不同的查找规则。
2. 先序线索树中找后继：有左孩子则后继为左孩子，否则为右孩子（或线索）；找前驱需要知道双亲，一般需三叉链表。
3. 后序线索树中找前驱：有右孩子则前驱为右孩子，否则为左孩子（或线索）；找后继需要知道双亲。

7.7 树与森林的存储与遍历

树的存储结构

结论 7.11: 树的双亲表示法

用数组存储结点，每个结点附带双亲编号。根的双亲为 -1：

```

1  typedef struct {
2      ElemType data;
3      int parent;           // 双亲下标, -1 表示根
4  } PTNode;
5  typedef struct {
6      PTNode nodes[MaxSize];
7      int n;               // 结点数
8  } PTree;

```

优点：求双亲 $O(1)$ 。缺点：求孩子需遍历整个数组 $O(n)$ 。

结论 7.12: 孩子兄弟表示法（二叉树表示法）

408 最重要的树的存储方式：每个结点有两个指针：firstchild（指向第一个孩子）和 nextsibling（指向下一个兄弟）。

```

1  typedef struct CSNode {
2      ElemType data;
3      struct CSNode *firstchild, *nextsibling;
4  } CSNode, *CSTree;

```

这种表示法使得任何一棵树都可以唯一地转换为一棵二叉树——左孩子右兄弟。

树/森林与二叉树的转换

结论 7.13: 树 $\rightarrow$ 二叉树的转换规则

给定一棵树：

1. 每个结点只保留其第一个孩子的连线，删除与其他孩子的连线；
2. 同一层的兄弟结点之间用新的连线连接（从左向右）；
3. 以根为轴，将原本向下的孩子关系转为向左下方倾斜。

口诀：左孩子、右兄弟。转换后的二叉树右子树一定为空（因根无兄弟）。

结论 7.14: 森林 $\rightarrow$ 二叉树的转换规则

1. 先将森林中的每棵树各自转换为二叉树；
2. 将第一棵树的根作为总根，后续树的根依次作为前一棵树的根的右孩子。

森林转换成的二叉树，根有右子树（因为下一棵树的根成为其右孩子）。

树与森林的遍历

结论 7.15: 树的遍历

- 先根遍历：先访问根，再依次先根遍历各子树。结果与对应二叉树的先序遍历相同。
- 后根遍历：先依次后根遍历各子树，再访问根。结果与对应二叉树的中序遍历相同。
- 层次遍历：从上到下、从左到右依次访问。

注意：树没有中根遍历——因为树的中序无统一定义。

结论 7.16: 森林的遍历

设森林 $F = \{T_1, T_2, \dots, T_m\}$ ：

- 先序遍历：先访问 T_1 的根，再先序遍历 T_1 的子树森林，最后先序遍历 $\{T_2, \dots, T_m\}$ 。结果与对应二叉树的先序遍历相同。
- 中序遍历(后序遍历)：先中序遍历 T_1 的子树森林，访问 T_1 的根，再中序遍历 $\{T_2, \dots, T_m\}$ 。结果与对应二叉树的中序遍历相同。

结论：森林的先序 = 对应二叉树的先序；森林的中序 = 对应二叉树的中序。

7.8 Huffman 树与 Huffman 编码

定义 7.6: Huffman 树

给定 n 个权值 $\{w_1, w_2, \dots, w_n\}$ 作为 n 个叶子结点的权值，构造一棵二叉树。若该树的带权路径长度（WPL）达到最小，则称为最优二叉树，即 Huffman 树。

带权路径长度定义为：

$$\text{WPL} = \sum_{i=1}^n w_i \cdot l_i$$

其中 l_i 是第 i 个叶子结点到根的路径长度（边的条数，或深度 -1 ）。

结论 7.17: Huffman 树的构造算法

1. 将 n 个权值视为 n 棵仅含根结点的二叉树，构成森林 F ；
2. 从 F 中选取两棵根权值最小的树作为左右子树，构造一棵新树，新树根的权值为两子树根权值之和；
3. 将新树加入 F ，删除原来的两棵树；
4. 重复步骤 2—3，直到 F 中只剩一棵树——即为 Huffman 树。

可以使用最小堆（优先队列）高效实现：每次取出两个最小值，合并后压回。时间复杂度 $O(n \log n)$ 。

要点 7.7: Huffman 树的性质

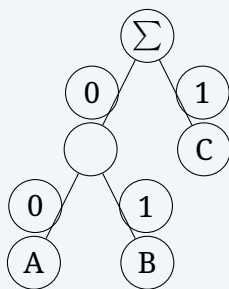
1. Huffman 树中没有度为 1 的结点 ($n_1 = 0$)。由 $n_0 = n_2 + 1$ 得总结点数 $= 2n_0 - 1$ 。
2. 权值越大的叶子离根越近，权值越小的叶子离根越远。这是 WPL 最小化的直观体现。
3. 给定权值集合，Huffman 树不唯一（左右子树可交换，等权结点次序可任意）。但 WPL 值唯一。
4. Huffman 树不一定是完全二叉树。

Huffman 编码**结论 7.18: Huffman 编码的构造**

以字符出现频率为权值构造 Huffman 树，然后对每条边标记：

- 从根到左孩子的边标记为 0；
- 从根到右孩子的边标记为 1。

从根到每个叶子结点的路径上的 0/1 序列即为该字符的 Huffman 编码。

**Huffman 编码的性质：**

1. 前缀编码 (Prefix Code)：任意字符的编码都不是另一字符编码的前缀。这保证了即时解码（无需预知码长）。
2. 由性质 1，Huffman 树中所有字符结点都是叶子结点，内部结点不对应任何字符。
3. 对于同一组频率，Huffman 编码不唯一（由树的不唯一性导致），但总编码长度 (= WPL) 唯一。

注意**408 高频计算题：**

1. 给定一组权值，要求构造 **Huffman** 树并画出结构图；
2. 计算 **WPL** 值（含中间步骤）；
3. 给出 **Huffman** 编码方案，计算压缩率或平均码长；
4. 判断某组编码是否为合法前缀编码。

快速计算 WPL 技巧：不需要等树画完再逐叶算——构造过程中每次合并两个最小权值，将它们的和累加，最终累加值即为 **WPL**。这是因为每个内部结点的权值等于其子树权值之和，所有内部结点权值之和 = **WPL**。

7.9 并查集

定义 7.7: 并查集

并查集（Disjoint Set / Union-Find）是一种管理不相交集合的数据结构，支持两种操作：

- **Find(x)**：查找元素 x 所属集合的代表元（根）；
- **Union(x, y)**：合并 x 和 y 所在的集合。

底层使用树（森林）的双亲表示法：每个集合对应一棵树，根结点为集合代表元。可用数组 `parent[i]` 存储双亲下标，根结点的 `parent` 为 `-1`。

要点 7.8: 并查集的基本实现

```

1  #define MaxSize 100
2  int parent[MaxSize];
3
4  void Init(int n) {                                // 初始每个元素自成一个集合
5      for (int i = 0; i < n; i++)
6          parent[i] = -1;
7  }
8
9  int Find(int x) {                                  // 查找根
10     while (parent[x] >= 0)
11         x = parent[x];
12     return x;
13 }
14
15 void Union(int x, int y) {                          // 合并
16     int rx = Find(x), ry = Find(y);
17     if (rx != ry) parent[rx] = ry;                 // 将 rx 的根指向 ry
18 }

```

结论 7.19: 并查集的优化

朴素实现中树可能退化为链，导致 Find 为 $O(n)$ 。两种经典优化：

1. 按秩合并 (**Union by Rank**)：总是将秩较小的树合并到秩较大的树上；秩通常是树高的上界，而不是集合元素个数。
2. 按大小合并 (**Union by Size**)：总是将元素较少的集合挂到元素较多的集合下。可用根结点 `parent` 的负值绝对值记录集合大小；下方代码采用的正是这种方式。
3. 路径压缩 (**Path Compression**)：在 Find 过程中，将查找路径上的所有结点直接指向根。与按秩或按大小合并结合使用时，摊还复杂度为 $O(\alpha(n))$ ，其中 α 为增长极慢的反 Ackermann 函数。

```

1 // 路径压缩的 Find
2 int Find(int x) {
3     if (parent[x] < 0) return x;
4     return parent[x] = Find(parent[x]); // 递归压缩
5 }
6
7 // 按大小合并：根结点 parent 的负值绝对值表示集合大小
8 void Union(int x, int y) {
9     int rx = Find(x), ry = Find(y);
10    if (rx == ry) return;
11    if (parent[rx] > parent[ry]) { // rx 的集合较小
12        parent[ry] += parent[rx]; // 更新 ry 集合的大小
13        parent[rx] = ry;
14    } else {
15        parent[rx] += parent[ry];
16        parent[ry] = rx;
17    }
18 }

```

注意

并查集在 408 中通常以选择题（考察优化效果和时间复杂度）或简答题（结合图论中的 Kruskal 算法）出现。需区分按秩合并与按大小合并的元数据含义，并理解它们和路径压缩的配合。

7.10 408 高频考点与真题风格**结论 7.20: 树与二叉树 408 选择题常考点**

1. 二叉树性质计算： $n_0 = n_2 + 1$ 是考察频率最高的公式，常结合 $n = n_0 + n_1 + n_2$ 出计算题——给定两个量求第三个。
2. 完全二叉树的性质：编号规则 ($2i$ 和 $2i + 1$)、叶子结点个数范围、高度计算。
3. 遍历序列分析：给定先序 + 中序（或后序 + 中序），还原二叉树。
4. 线索二叉树：中序线索化，找前驱/后继的步骤，区分 ltag/rtag 的含义。
5. 树/森林与二叉树的转换：孩子兄弟表示法，「左孩子右兄弟」规则。
6. **Huffman** 树：WPL 计算、Huffman 树的结点数 ($2n_0 - 1$)、给定编码判断是否为前缀码。

7. 二叉树的遍历序列与树形对应关系：如前序序列与中序序列相同的树是什么形态（只有右子树的单链树）。
8. 非递归遍历：中序非递归的栈变化过程，何时入栈、何时出栈。

8 习题

8.1 基础过关题

1. (真题风格·选择题) 在一棵二叉树中, 度为 2 的结点数为 5, 度为 1 的结点数为 6, 则叶子结点数为

(A) 5 (B) 6

(C) 7 (D) 11

2. (真题风格·选择题) 一棵完全二叉树共有 768 个结点, 则该二叉树中叶子结点的个数为

(A) 383 (B) 384

(C) 385 (D) 386

3. (真题风格·选择题) 已知一棵二叉树的先序遍历序列为 ABDECFG, 中序遍历序列为 DBEAFCG, 则该二叉树的后序遍历序列为

(A) DEBFGCA (B) EDBFGCA

(C) DEBFGAC (D) DEBGFCA

4. (真题风格·选择题) 一棵非空二叉树的先序序列与后序序列正好相反, 则该二叉树一定

(A) 所有结点无左孩子 (B) 所有结点无右孩子

(C) 只有一个叶结点 (D) 是满二叉树

5. (真题风格·选择题) 设给定的权值集合为 $\{2, 3, 4, 7, 8, 9\}$, 构造 Huffman 树, 则该树的带权路径长度 WPL 为

(A) 80 (B) 81

(C) 82 (D) 83

6. (真题风格·选择题) 一组字符 $\{A, B, C, D, E\}$ 出现的频率分别为 $\{5, 2, 8, 3, 13\}$, 对其进行 Huffman 编码, 则字符 E 的编码长度是

(A) 1 位 (B) 2 位

(C) 3 位 (D) 4 位

7. (真题风格·选择题) 已知一棵二叉树的先序遍历序列为 ABDFEC, 中序遍历序列为 BFDAEC, 则该二叉树的后序遍历序列为

(A) FDBECA (B) BFDECA

(C) FDEBCA (D) FDBEAC

8.2 真题风格与综合训练

8. (真题风格·选择题) 下列关于线索二叉树的叙述中, 正确的是

- (A) 先序线索二叉树中, 任意结点都可在不知道双亲的情况下直接找到其先序前驱
- (B) 中序线索二叉树中, 若某结点的 $rtag=1$, 则其右指针指向中序后继
- (C) 后序线索二叉树中, 任意结点都可在不知道双亲的情况下直接找到其后序后继
- (D) 只有叶结点的空指针域才可以改作线索

9. (真题风格·选择题) 在中序线索二叉树中, 指针 p 指向某结点且 $p \rightarrow rtag == 0$, 则 p 的直接中序后继是

- (A) $p \rightarrow rchild$
- (B) p 的右子树中最左下结点
- (C) p 的左子树中最右下结点
- (D) 无法直接确定

10. (真题风格·选择题) 设森林 F 对应的二叉树为 B , 它有 m 个结点, B 的根为 p , p 的右子树结点数为 n , 则森林 F 中第一棵树的结点个数是

- (A) $m - n$ (B) $m - n - 1$
- (C) $n + 1$ (D) 无法确定

11. (真题风格·选择题) 已知一棵二叉树的先序序列为 a, b, c, d , 中序序列为 b, a, d, c , 则下列说法正确的是

- (A) 该二叉树是满二叉树
- (B) 该二叉树的结点 b 和结点 d 在同一层
- (C) 该二叉树中结点 a 只有左孩子
- (D) 该二叉树的后序序列为 b, d, c, a

12. (综合题) 已知一棵二叉树的先序遍历序列为 ABDEGCF, 中序遍历序列为 DBGEACF。

- (1) 画出该二叉树的结构 (标明各结点的左右孩子);
- (2) 写出其后序遍历序列和层次遍历序列;
- (3) 将该二叉树转换为对应的森林, 画出森林结构图 (简述转换规则)。

13. (综合题, 真题风格) 设有字符集 $\{A, B, C, D, E, F\}$, 各字符在电文中出现的频率分别为 $\{0.05, 0.10, 0.15, 0.20, 0.25, 0.25\}$ 。

- (1) 构造 Huffman 树, 画出树的形态 (内部结点标出合并权值, 叶子结点标出字符);
- (2) 求各字符的 Huffman 编码 (约定左分支为 0, 右分支为 1);
- (3) 计算带权路径长度 WPL (写出计算过程);
- (4) 若将字符 F 的频率改为 0.40, 重新构造 Huffman 树并计算 WPL, 与 (3) 比较变化并解释原因。

拓展阅读:

- 邓俊辉《数据结构（C++ 语言版）》第 5 章「二叉树」——二叉树的遍历算法族（先/中/后序递归与非递归的统一框架）、Huffman 编码的贪心正确性证明。第 6 章「图」——并查集在 Kruskal 算法与图的连通性判定中的应用。
- Sedgewick《算法》（第 4 版）第 3.2–3.3 节「二叉查找树」与「平衡查找树」——从二叉树的遍历到 BST 上的查找与插入，为后续平衡树（AVL/红黑树）的学习建立基础。

9 图

9.1 图的基本概念

定义 9.1: 图的定义

图 $G = (V, E)$ 由顶点集 V 和边集 E 组成。 $|V| = n$ 为顶点数（阶）， $|E| = e$ 为边数。

- 有向图：边为有向边（弧） $\langle u, v \rangle$ ， u 为弧尾， v 为弧头。
- 无向图：边为无向边 (u, v) 。
- 简单图：无自环（顶点到自身的边），无重边。
- 完全图：无向完全图 K_n 有 $\frac{n(n-1)}{2}$ 条边；有向完全图有 $n(n-1)$ 条弧。
- 连通：无向图中若 u 到 v 有路径，称 u, v 连通。若任意两顶点连通，称图为连通图。
- 强连通：有向图中任意两顶点互达，称为强连通图。

结论 9.1: 图的基本术语速查

图论术语

术语	含义
度 $\deg(v)$	无向图中与 v 关联的边数； $\sum \deg(v) = 2e$
入度 $\text{id}(v)$ 、出度 $\text{od}(v)$	有向图中以 v 为弧头/弧尾的弧数； $\sum \text{id} = \sum \text{od} = e$
路径	顶点序列，相邻顶点间有边
简单路径	顶点不重复的路径
回路（环）	起点 = 终点的路径，长度 ≥ 1
连通分量	无向图的极大连通子图
强连通分量	有向图的极大强连通子图
生成树	连通图的极小连通子图，含全部 n 个顶点和 $n-1$ 条边
生成森林	非连通图中各连通分量的生成树的并

9.2 图的存储结构

要点 9.1: 邻接矩阵

用 $n \times n$ 矩阵 A 表示图:

$$A[i][j] = \begin{cases} 1 & (v_i, v_j) \text{ 或 } \langle v_i, v_j \rangle \in E \\ 0 & \text{否则} \end{cases}$$

带权图可将 1 替换为权值 w_{ij} , 无边的位置填 0 或 ∞ 。

```

1 #define MaxV 100
2 typedef char VType;           // 顶点类型
3 typedef int EType;            // 边上权值类型
4 typedef struct {
5     EType edges[MaxV][MaxV];   // 邻接矩阵
6     VType vexs[MaxV];          // 顶点信息
7     int n, e;                  // 顶点数、边数
8 } MGraph;

```

特点:

- 无向图的邻接矩阵对称; 有向图不一定。
- 空间复杂度 $O(n^2)$, 适合稠密图 ($e \approx n^2$)。
- 判断 (v_i, v_j) 是否有边仅需 $O(1)$ 。
- 求顶点的度: 无向图需扫描整行 $O(n)$; 有向图出度扫描行、入度扫描列。

要点 9.2: 邻接表

为每个顶点建立一条单链表, 链接其所有邻接点。

```

1 typedef struct ArcNode {
2     int adjvex;                // 邻接下标
3     EType weight;              // 边权值 (可选)
4     struct ArcNode *next;      // 下一条边
5 } ArcNode;
6
7 typedef struct {
8     VType data;                // 顶点信息
9     ArcNode *first;            // 第一条边
10 } VNode;
11
12 typedef struct {
13     VNode adjlist[MaxV];
14     int n, e;
15 } ALGraph;

```

特点:

- 空间复杂度 $O(n + e)$, 适合稀疏图。
- 无向图每条边存两次 (在两个顶点的链表中各出现一次)。
- 求有向图出度: 遍历该顶点的链表; 求入度: 需遍历全表或另建逆邻接表。

- 判断 (v_i, v_j) 是否有边：需遍历 v_i 的链表 $O(\deg(v_i))$ 。

结论 9.2: 邻接矩阵 vs 邻接表 (408 常考)

存储方式对比

操作	邻接矩阵	邻接表
空间	$O(n^2)$	$O(n + e)$
判断 (u, v) 是否有边	$O(1)$	$O(\deg(u))$
遍历某顶点的所有邻接边	$O(n)$	$O(\deg(u))$
适合场景	稠密图	稀疏图
唯一性	唯一	不唯一 (链表插入顺序可变)

9.3 图的遍历

结论 9.3: 深度优先搜索 (DFS)

从顶点 v 出发，沿一条路径走到尽头，回溯后再走其他分支。本质是递归/栈的过程。

```

1 bool visited[MaxV];
2 void DFS(ALGraph *G, int v) {
3     visited[v] = true;
4     // visit(G->adjlist[v]); // 访问顶点 v
5     for (ArcNode *p = G->adjlist[v].first; p; p = p->next)
6         if (!visited[p->adjvex])
7             DFS(G, p->adjvex);
8 }

```

邻接表表示时，DFS 时间复杂度 $O(n + e)$ ；邻接矩阵表示时 $O(n^2)$ 。

DFS 生成树/森林：递归过程中经过的边构成生成树（连通时）或生成森林（不连通时）。回边（back edge）是 DFS 树中指向祖先的边。

结论 9.4: 广度优先搜索 (BFS)

从顶点 v 出发，逐层访问。需要辅助队列。

```

1 void BFS(ALGraph *G, int v) {
2     int queue[MaxV], front = 0, rear = 0;
3     visited[v] = true;
4     queue[rear++] = v;
5     while (front < rear) {
6         int u = queue[front++];
7         // visit(G->adjlist[u]);
8         for (ArcNode *p = G->adjlist[u].first; p; p = p->next)
9             if (!visited[p->adjvex]) {
10                 visited[p->adjvex] = true;
11                 queue[rear++] = p->adjvex;
12             }

```

```

13     }
14 }

```

邻接表 $O(n + e)$ ，邻接矩阵 $O(n^2)$ 。

BFS 生成树：BFS 过程中首次发现某顶点所用的边构成生成树。BFS 生成树中从根到任意顶点的路径即最短路径（边数最少）。

注意

408 常考：DFS/BFS 遍历序列的唯一性。在起始顶点和按顶点编号递增扫描邻接点的规则均已确定时，邻接矩阵对应的访问次序确定；邻接表的访问次序还取决于各邻接链表中的结点顺序。若题目没有规定起点或邻接点选择顺序，则不能仅凭“邻接矩阵”断言遍历序列绝对唯一。

9.4 最小生成树 (MST)

定义 9.2: 最小生成树

连通带权无向图的生成树中，边权值和最小者称为最小生成树（Minimum Spanning Tree）。MST 可能不唯一，但边权和唯一。

结论 9.5: Prim 算法

从某一顶点开始，每次选一条连接「已在树中顶点」和「不在树中顶点」的最小权边，将对应顶点加入树。重复 $n - 1$ 次。

```

1 // Prim —  $O(n^2)$  实现 (邻接矩阵, 适合稠密图)
2 void Prim(MGraph G, int start) {
3     int lowcost[MaxV]; // lowcost[i]: i到当前MST的最小边权
4     int closest[MaxV]; // closest[i]: MST中与i最近的顶点
5     bool inTree[MaxV] = {false};
6     inTree[start] = true;
7     for (int i = 0; i < G.n; i++) {
8         lowcost[i] = G.edges[start][i];
9         closest[i] = start;
10    }
11    for (int k = 1; k < G.n; k++) {
12        int min = INF, u = -1;
13        for (int i = 0; i < G.n; i++)
14            if (!inTree[i] && lowcost[i] < min)
15                { min = lowcost[i]; u = i; }
16        inTree[u] = true;
17        // 输出边 (closest[u], u) 权 min
18        for (int i = 0; i < G.n; i++)
19            if (!inTree[i] && G.edges[u][i] < lowcost[i])
20                { lowcost[i] = G.edges[u][i]; closest[i] = u; }
21    }
22 }

```

时间复杂度 $O(n^2)$ ，与边数无关——适合稠密图。用堆优化可达 $O(e \log n)$ 。

结论 9.6: Kruskal 算法

按边权从小到大依次选边，若该边连接两个不同连通分量则加入 MST，否则丢弃。直到选出 $n - 1$ 条边。

时间复杂度 $O(e \log e)$ (排序主导)，适合稀疏图。需用并查集维护连通分量。

注意

408 选择题常考：给定一个图，手动执行 Prim (指定起点) 或 Kruskal 得到 MST。注意当有多条等权边时 MST 可能不唯一，但权值和唯一。

9.5 最短路径**结论 9.7: Dijkstra 算法**

求单源最短路径 (非负权值)。类似 Prim，贪心思想：

```

1 void Dijkstra(MGraph G, int s, int dist[], int path[]) {
2     bool final[MaxV] = {false};
3     for (int i = 0; i < G.n; i++) {
4         dist[i] = G.edges[s][i];
5         path[i] = (dist[i] < INF) ? s : -1;
6     }
7     final[s] = true; dist[s] = 0;
8     for (int k = 1; k < G.n; k++) {
9         int min = INF, u = -1;
10        for (int i = 0; i < G.n; i++)
11            if (!final[i] && dist[i] < min)
12                { min = dist[i]; u = i; }
13        final[u] = true;
14        for (int i = 0; i < G.n; i++)
15            if (!final[i] && dist[u] + G.edges[u][i] < dist[i])
16                { dist[i] = dist[u] + G.edges[u][i]; path[i] = u; }
17    }
18 }

```

时间复杂度 $O(n^2)$ (邻接矩阵)，堆优化可达 $O(e \log n)$ 。

与 **Prim** 的关键区别：Prim 的 `lowcost[i]` 是顶点 i 到当前 MST 集合的最小边权；Dijkstra 的 `dist[i]` 是源点到顶点 i 的累计路径长度。两者代码结构相似，更新条件不同。

结论 9.8: Floyd 算法

求所有顶点对间的最短路径。DP 思想：依次考虑以每个顶点为中间点时能否缩短路径。

```

1 void Floyd(MGraph G, int D[][MaxV], int path[][MaxV]) {
2     for (int i = 0; i < G.n; i++)
3         for (int j = 0; j < G.n; j++) {
4             D[i][j] = G.edges[i][j];
5             path[i][j] = (D[i][j] < INF && i != j) ? i : -1;
6         }
7     for (int k = 0; k < G.n; k++)

```

```

8         for (int i = 0; i < G.n; i++)
9             for (int j = 0; j < G.n; j++)
10                 if (D[i][k] + D[k][j] < D[i][j]) {
11                     D[i][j] = D[i][k] + D[k][j];
12                     path[i][j] = path[k][j];
13                 }
14     }

```

时间复杂度 $O(n^3)$ 。允许负权边，但不能有负权回路。

注意

408 高频考点：Dijkstra 算法中若存在负权边则失效（已确定最短的顶点可能在后续被更新）。Floyd 允许负权边但要求无负权回路。

9.6 有向无环图（DAG）与拓扑排序

定义 9.3: 拓扑排序

将有向无环图（DAG）的所有顶点排成一个线性序列，使得对每条弧 $\langle u, v \rangle$ ， u 在序列中出现于 v 之前。拓扑序列不唯一。

结论 9.9: 拓扑排序算法

基于入度：每次选一个入度为 0 的顶点输出并将其所有出边的终点入度减 1。重复直到所有顶点输出（成功）或不存在入度 0 的顶点（有回路）。

```

1 bool TopoSort(ALGraph *G, int topo[]) {
2     int indegree[MaxV] = {0}, cnt = 0;
3     int stack[MaxV], top = -1;
4     for (int i = 0; i < G->n; i++)          // 计算入度
5         for (ArcNode *p = G->adjlist[i].first; p; p = p->next)
6             indegree[p->adjvex]++;
7     for (int i = 0; i < G->n; i++)          // 入度为0入栈
8         if (indegree[i] == 0) stack[++top] = i;
9     while (top != -1) {
10         int u = stack[top--];
11         topo[cnt++] = u;
12         for (ArcNode *p = G->adjlist[u].first; p; p = p->next) {
13             int v = p->adjvex;
14             if (--indegree[v] == 0) stack[++top] = v;
15         }
16     }
17     return cnt == G->n;                    // cnt<n 则有回路
18 }

```

时间复杂度 $O(n + e)$ 。用栈或队列均可——用栈得到逆拓扑序更自然（DFS 的逆后序即拓扑序）。

9.7 AOE 网与关键路径

定义 9.4: AOV 网与 AOE 网

- **AOV 网** (Activity On Vertex): 顶点表示活动, 边表示活动间的优先关系。
- **AOE 网** (Activity On Edge): 边表示活动, 边权表示活动持续时间; 顶点表示事件 (某些活动完成的时刻)。

结论 9.10: 关键路径

AOE 网中:

- 事件 v_k 的最早发生时间 $ve(k)$: 从源点到 v_k 的最长路径长度。按拓扑序计算: $ve(k) = \max\{ve(j) + \text{weight}(\langle v_j, v_k \rangle)\}$ 。
- 事件 v_k 的最迟发生时间 $vl(k)$: 不推迟工期的前提下事件最迟发生时间。按逆拓扑序计算: $vl(k) = \min\{vl(j) - \text{weight}(\langle v_k, v_j \rangle)\}$, 汇点 $vl = ve$ 。
- 活动 $a_i = \langle v_j, v_k \rangle$ 的最早开始 $e(i) = ve(j)$, 最迟开始 $l(i) = vl(k) - \text{weight}$ 。
- 关键活动: $e(i) = l(i)$ 的活动。关键路径: 由关键活动构成的从源点到汇点的路径。

408 常考: 给定 AOE 网, 计算各事件的 ve 和 vl , 找出关键活动和关键路径。关键路径可能不唯一——缩短某条关键路径上的非公共关键活动不一定会缩短总工期。

9.8 408 高频考点速查

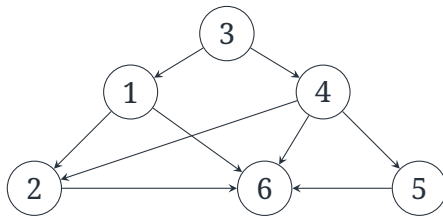
结论 9.11: 图—408 选择题常考点

1. 连通图边数范围: $n - 1 \leq e \leq C_n^2$ (最少 $n - 1$ 即生成树)。
2. **DFS/BFS** 遍历序列: 先确认起点与邻接点扫描顺序; 邻接表还受链表存储次序影响。
3. **Prim vs Kruskal**: 稠密图用 Prim $O(n^2)$, 稀疏图用 Kruskal $O(e \log e)$ 。
4. **Dijkstra** 适用条件: 非负权值; 与 Prim 代码对比。
5. 拓扑排序: DAG 才有拓扑序列; 不唯一的条件 (有多个人度同时为 0 的顶点)。
6. 关键路径: ve 和 vl 的计算; 关键活动 $e = l$; 缩短非关键活动不缩短工期。
7. 不同存储结构的遍历复杂度: 邻接表 $O(n + e)$ vs 邻接矩阵 $O(n^2)$ 。

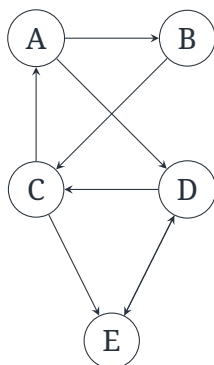
10 习题

10.1 基础过关题

1. (真题风格·选择题) 下列关于无向连通图特性的叙述中, 正确的是
- (A) 所有顶点的度之和为偶数
(B) 边数大于顶点个数减 1
(C) 至少有一个顶点的度为 1
(D) 以上均正确
2. (真题风格·选择题) 对 n 个顶点、 e 条边的有向图采用邻接表存储, 则拓扑排序算法的时间复杂度是
- (A) $O(n)$ (B) $O(n+e)$
(C) $O(n^2)$ (D) $O(ne)$
3. (真题风格·选择题) 下列关于图的叙述中, 正确的是
- (A) 回路是简单路径
(B) 存储稀疏图, 用邻接矩阵比邻接表更省空间
(C) 若有向图中存在拓扑序列, 则该图不存在回路
(D) 有向图的邻接矩阵一定是对称矩阵
4. (真题风格·选择题) 对有 n 个顶点、 e 条边且使用邻接表存储的有向图进行广度优先遍历, 其算法时间复杂度是
- (A) $O(n)$ (B) $O(e)$
(C) $O(n+e)$ (D) $O(n \times e)$
5. (真题风格·选择题) 对如下有向图进行拓扑排序, 得到的拓扑序列可能是



- (A) 3,1,4,2,5,6 (B) 3,1,4,2,6,5
(C) 3,1,5,4,2,6 (D) 3,1,2,4,5,6
6. (真题风格·选择题) 已知含有 5 个顶点的有向图 G 如下图所示, 则下列结论正确的是



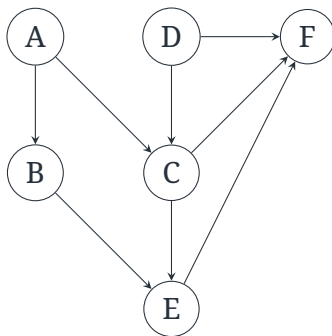
- (A) G 是强连通图
 (B) 用邻接表存储 G 所需空间一定为 $O(n^2)$
 (C) 顶点 C 的入度小于出度
 (D) G 的邻接矩阵一定是对称矩阵

7. (真题风格·选择题) 设无向图的邻接表按顶点编号递增排列, 从顶点 1 开始进行 BFS, 并始终按邻接表顺序扫描邻接点。若各顶点的邻接表为

1 : 2, 3; 2 : 1, 4, 5; 3 : 1, 6; 4 : 2; 5 : 2, 6; 6 : 3, 5,

则得到的 BFS 访问序列是

- (A) 1, 2, 3, 4, 5, 6 (B) 1, 2, 3, 6, 5, 4
 (C) 1, 3, 2, 6, 5, 4 (D) 1, 2, 4, 5, 3, 6
8. (真题风格·选择题) 已知如下有向图, 该图的拓扑排序序列的个数是

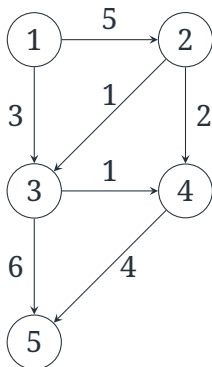


- (A) 1 (B) 2
 (C) 3 (D) 5

10.2 真题风格与综合训练

9. (真题风格·选择题) 下列关于生成树的说法中, 错误的是
- (A) 连通图的生成树包含图的所有顶点
 (B) 生成树中任意两顶点之间的路径是唯一的
 (C) 生成树是连通图的极小连通子图
 (D) 非连通图也可以有生成树

10. (真题风格·选择题) 用 Dijkstra 算法求下图中顶点 1 到其余各点的最短路径。算法依次确定最短路径的顶点次序是



- (A) 3,2,4,5 (B) 2,3,4,5
(C) 3,4,5,2 (D) 3,4,2,5
11. (真题风格·选择题) 已知一个无向图的邻接矩阵如下, 用 Prim 算法从顶点 A 开始构造最小生成树, 则依次加入生成树的边是

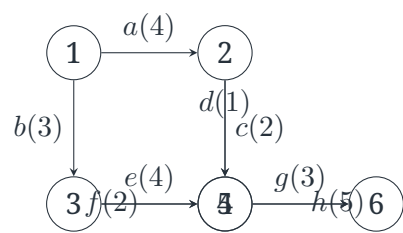
$$\begin{pmatrix} 0 & 6 & 1 & 5 & \infty & \infty \\ 6 & 0 & 5 & \infty & 3 & \infty \\ 1 & 5 & 0 & 5 & 6 & 4 \\ 5 & \infty & 5 & 0 & \infty & 2 \\ \infty & 3 & 6 & \infty & 0 & 6 \\ \infty & \infty & 4 & 2 & 6 & 0 \end{pmatrix}$$

(顶点顺序: A, B, C, D, E, F)

- (A) (A,C)(C,F)(F,D)(C,B)(B,E)
(B) (A,C)(C,F)(C,B)(F,D)(B,E)
(C) (A,C)(C,D)(F,D)(C,B)(B,E)
(D) (A,C)(C,D)(C,F)(F,B)(B,E)
12. (真题风格·选择题) 下列关于关键路径的叙述中, 正确的是
- (A) 缩短任意一个关键活动的持续时间, 必能缩短整个工程的工期
(B) 关键路径是 AOE 网中从源点到汇点的最短路径
(C) 一个 AOE 网中, 关键路径可能有多条
(D) 非关键活动提前完成可以缩短工期
13. (真题风格·选择题) 已知无向图 G 有 16 条边, 其中度为 4 的顶点有 3 个, 度为 3 的顶点有 4 个, 其余顶点的度均小于 3。则 G 至少有多少个顶点?

- (A) 10 (B) 11
(C) 12 (D) 13

14. (真题风格·选择题) 下列 AOE 网表示一项包含 8 个活动的工程，边上括号内为活动持续时间。下列选项中，两项活动均为关键活动的是



- (A) c 和 h (B) d 和 e
(C) f 和 d (D) f 和 h

15. (综合题) 已知带权有向图 $G = (V, E)$ 的邻接矩阵如下，顶点编号为 1~6。

$$A = \begin{pmatrix} 0 & \infty & 10 & \infty & 30 & 100 \\ \infty & 0 & 5 & \infty & \infty & \infty \\ \infty & \infty & 0 & 50 & \infty & \infty \\ \infty & \infty & \infty & 0 & \infty & 10 \\ \infty & \infty & \infty & 20 & 0 & 60 \\ \infty & \infty & \infty & \infty & \infty & 0 \end{pmatrix}$$

- (1) 画出该有向图；
(2) 写出 G 的全部拓扑序列；
(3) 用 Dijkstra 算法求顶点 1 到其余各顶点的最短路径，写出过程中 dist 数组的变化；
(4) 求顶点 1 到顶点 6 的最短路径及其长度。

拓展阅读：

- 邓俊辉《数据结构（C++ 语言版）》第 6 章「图」——完整的图 ADT 实现，包括 BFS/DFS、Prim/Kruskal/Dijkstra 的 C++ 代码和复杂度分析，以及强连通分量的 Tarjan 算法。
- Sedgewick《算法》（第 4 版）第 4 章「图」——无向图/有向图的 DFS/BFS 经典讨论，第 4.3 节最小生成树（含 Prim 即时版本与延时版本的对比），第 4.4 节最短路径（Dijkstra 与 Bellman-Ford）。
- CLRS《算法导论》第 22–25 章——图的算法严格理论分析，包括单源最短路径的最优子结构证明、所有结点对最短路径的矩阵乘法方法、最大流等。

11 查找

11.1 查找的基本概念

定义 11.1: 查找表与关键字

查找表 (Search Table) 是由同一类型的数据元素 (或记录) 构成的集合。对查找表常进行的操作有:

- 查询: 查找某个「特定的」数据元素是否在表中;
- 检索: 检索某个「特定的」数据元素的各种属性;
- 插入/删除: 在表中插入或删除一个数据元素。

若只做前两类操作, 称查找表为静态查找表 (Static Search Table); 若同时涉及插入/删除操作, 则称为动态查找表 (Dynamic Search Table)。

关键字 (Key): 数据元素中某个数据项的值, 用以标识该数据元素。

- 主关键字 (Primary Key): 能唯一标识一个数据元素的关键字。
- 次关键字 (Secondary Key): 不能唯一标识数据元素的关键字。

查找: 在查找表中确定一个其关键字等于给定值的数据元素 (或记录)。若找到, 称查找成功, 返回该元素的位置; 否则称查找失败 (不成功), 返回某种失败标志。

要点 11.1: 平均查找长度 ASL

查找算法的主要评价指标是平均查找长度 (Average Search Length), 定义为查找过程中对关键字执行比较次数的期望值:

$$ASL = \sum_{i=1}^n P_i \cdot C_i$$

其中 P_i 为查找第 i 个元素的概率 (通常假定等概率, 即 $P_i = 1/n$), C_i 为查找第 i 个元素所需的比较次数。

$ASL_{成功}$ 与 $ASL_{不成功}$ 是 408 考试中的两个核心计算量——前者对所有表中存在的元素的查找长度求期望; 后者对表中不存在的所有「可能的」查找值求期望, 通常依赖查找判定树的结构。

注意

408 考试中, $ASL_{成功}$ 计算通常假定等概率。本文对折半查找统一采用「关键字比较次数」: 失败结点代表一个查找不成功的区间, 其代价等于从根到该失败结点之前访问过的数据结点数, 不把空指针判断另算一次比较。若把根记为第 1 层, 则失败代价等于其父数据结点的层次; 题干另有约定时以题干为准。散列表则按实际探测到的槽位次数计数, 包括最终遇到的空槽。

11.2 顺序查找与折半查找

顺序查找 (Sequential Search)

定义 11.2: 顺序查找

从表的一端开始, 依次将每个元素的关键字与给定值进行比较。若找到, 返回位置; 若遍历完毕仍未找到, 则返回查找失败。又称为线性查找, 是最基本的查找方法。

要点 11.2: 带哨兵的顺序查找

哨兵技巧 (Sentinel): 将待查关键字存入表头 (或表尾) 作为「监视哨」, 使循环中只需判断关键字是否相等, 不必每轮另做越界判断。它减少了循环条件中的一次判断, 但关键字比较次数和 $O(n)$ 时间复杂度不变, 不能笼统地说运行时间减半。

以下给出在 $ST[1..n]$ 中查找 key 的带哨兵实现 ($ST[0]$ 为哨兵):

```

1 typedef struct {
2     ElemType *elem;    // 元素存储空间基址, 0号单元留空
3     int TableLen;      // 表长
4 } SSTable;
5
6 int Search_Seq(SSTable ST, KeyType key) {
7     ST.elem[0].key = key;    // 设置哨兵
8     int i;
9     for (i = ST.TableLen; ST.elem[i].key != key; i--);
10    return i;                // i=0 则查找失败
11 }

```

时间复杂度: $O(n)$ 。空间复杂度: $O(1)$ 。

结论 11.1: 顺序查找的 ASL 分析

等概率下:

$$ASL_{成功} = \sum_{i=1}^n \frac{1}{n} \cdot i = \frac{n+1}{2}$$

查找不成功时 (带哨兵版本哨兵在 0 号位置), 比较次数为 $n+1$ (关键值从 $ST[n]$ 比较到 $ST[0]$ 哨兵, 但哨兵比较也算一次)。故 $ASL_{不成功} = n+1$ (若不计哨兵则 $= n$, 以教材为准)。

优点: 对数据元素是否有序无要求; 顺序表/链表均可实现; 插入可在 $O(1)$ 完成 (表尾追加)。

缺点: n 较大时效率低。

折半查找 (Binary Search)**定义 11.3: 折半查找**

前提: 线性表必须采用顺序存储且按关键字有序 (递增或递减)。下列文字和代码以递增表为例; 若表按递减顺序排列, 需要将小于/大于的两个分支对调。

基本思想: 每次将待查区间分为两半, 将中间位置的关键字与给定值比较:

- 相等 $\rightarrow$ 查找成功;
- 给定值小于中间关键字 $\rightarrow$ 在前半区间继续查找;
- 给定值大于中间关键字 $\rightarrow$ 在后半区间继续查找。

要点 11.3: 折半查找的代码实现

```

1 int Binary_Search(SSTable ST, KeyType key) {
2     int low = 1, high = ST.TableLen;
3     while (low <= high) {
4         int mid = (low + high) / 2;

```

```

5      if (ST.elem[mid].key == key)
6          return mid;                // 查找成功
7      else if (ST.elem[mid].key > key)
8          high = mid - 1;            // 在前半区间
9      else
10         low = mid + 1;              // 在后半区间
11    }
12    return 0;                        // 查找失败
13 }

```

注意：mid = (low + high) / 2 在极端情况下可能溢出 (low+high 超出 int 范围)，工程中推荐 mid = low + (high - low) / 2。408 考试中默认不会出现溢出情况。

结论 11.2: 折半查找的判定树与 ASL

折半查找的过程可以用一棵判定树 (Decision Tree) 描述：

- 树中每个圆形结点对应表中一个元素 (成功查找)。
- 每个方形结点 (外结点/失败结点) 对应一个查找不成功的区间，总数为 $n + 1$ 。
- 判定树是一棵平衡二叉树——左右子树高度之差不超过 1。

对于 n 个元素，判定树的高度为 $\lceil \log_2(n + 1) \rceil$ 。

查找成功时的 **ASL**：等于判定树中所有圆形结点所在层次之和除以 n (根在第 1 层)，即

$$ASL_{\text{成功}} = \frac{1}{n} \sum_{i=1}^n l_i$$

其中 l_i 为第 i 个元素在判定树中的层次。

查找不成功时的 **ASL**：对 $n + 1$ 个失败区间分别统计查找过程中访问的数据结点数，再除以 $n + 1$ 。在本文「根为第 1 层且只计关键字比较」的约定下，该次数等于相应方形结点父结点的层次。若题目把空指针判断也计入，则每次失败查找需再加 1，必须按题干口径作答。

等概率下， n 较大时：

$$ASL_{\text{成功}} \approx \log_2(n + 1) - 1$$

时间复杂度： $O(\log n)$ 。

注意

408 高频考点——折半查找的判定树构造：

- 对于偶数个元素，mid = floor((low+high)/2) 和 mid = ceil((low+high)/2) 对应的判定树不同——前者右子树结点数 $\geq$ 左子树；后者左子树结点数 $\geq$ 右子树。题目若未直接给出取整规则，应根据给定的比较过程或判定树判断其约定。
- 判定树的后继与前驱：折半查找的判定树中，任一结点的中序前驱/后继即有序线性表中该元素的前一个/后一个元素。

分块查找（索引顺序查找）

定义 11.4: 分块查找

分块查找（Block Search / Indexed Sequential Search）在顺序查找和折半查找之间取得折中：

- 将查找表分为若干块（子表），块内元素可以无序，但块间必须有序——即前一块中所有元素的最大关键字小于后一块中所有元素的最小关键字。
- 建立索引表：每个索引项包含对应块的最大关键字和该块第一个元素的地址。

查找过程：

1. 先在索引表中确定关键字可能所在的块（索引表有序，可用折半查找或顺序查找）；
2. 再在对应块内顺序查找。

要点 11.4: 分块查找的 ASL

设表长为 n ，均匀分为 b 块，每块 s 个元素（ $n = bs$ ）。索引表用顺序查找，块内也用顺序查找，则等概率下：

- 索引表查找的 ASL: $(b + 1)/2$
- 块内查找的 ASL: $(s + 1)/2$
- 总 ASL $= \frac{b + 1}{2} + \frac{s + 1}{2} = \frac{s^2 + 2s + n}{2s}$ （代入 $b = n/s$ ）

当 $s = \sqrt{n}$ 时取得最小值 $ASL_{\min} = \sqrt{n} + 1$ 。

若索引表采用折半查找：

$$ASL \approx \log_2(b + 1) - 1 + \frac{s + 1}{2} \approx \log_2\left(\frac{n}{s} + 1\right) + \frac{s}{2}$$

11.3 二叉排序树（BST）

BST 的定义与查找

定义 11.5: 二叉排序树

二叉排序树（Binary Search Tree, BST）或者是一棵空树，或者是满足以下性质的二叉树：

1. 若左子树非空，则左子树上所有结点的关键字值均小于根结点的关键字值；
2. 若右子树非空，则右子树上所有结点的关键字值均大于根结点的关键字值；
3. 左、右子树本身也各是一棵二叉排序树。

注意：408 默认 BST 中不允许等值结点——即没有两个结点具有相同的关键字。若存在等值，按定义要么统一放在左子树，要么统一放在右子树，需看题设。

要点 11.5: BST 的查找算法

```
typedef struct BSTNode {
    KeyType key;
    struct BSTNode *lchild, *rchild;
```

```

4 } BSTNode, *BSTree;
5
6 BSTNode *BST_Search(BSTree T, KeyType key) {
7     while (T != NULL && T->key != key) {
8         if (key < T->key)
9             T = T->lchild;
10        else
11            T = T->rchild;
12    }
13    return T;    // 成功则返回结点指针, 失败返回 NULL
14 }

```

递归版同理, 本质是二分查找思想在树结构上的推广。时间复杂度: 最好 $O(1)$, 最坏 $O(h)$ (h 为树高)。平衡时 $h = O(\log n)$, 退化成单链时 $h = n$ 。

结论 11.3: BST 查找的 ASL 分析

BST 的 ASL 取决于树的形态:

- 最好情况 (平衡 BST, $h = \lceil \log_2(n+1) \rceil$): $ASL = O(\log n)$, 与折半查找的判定树相同。
- 最坏情况 (退化为单链表, $h = n$): $ASL = (n+1)/2$, 退化为顺序查找。
- 平均情况: 随机插入下期望树高 $O(\log n)$, $ASL \approx 1.38 \log_2 n$ 。

BST 的插入与删除

结论 11.4: BST 的插入

新结点一定作为叶子结点插入。过程: 若树空则直接插入为根; 否则沿树查找, 若关键字已存在则不插入 (或按题设处理), 若不存在则在查找失败的位置 (空指针处) 插入新结点。

```

1 bool BST_Insert(BSTree *T, KeyType key) {
2     if (*T == NULL) {
3         *T = (BSTNode*)malloc(sizeof(BSTNode));
4         (*T)->key = key;
5         (*T)->lchild = (*T)->rchild = NULL;
6         return true;
7     }
8     if (key == (*T)->key) return false;    // 已存在
9     if (key < (*T)->key)
10        return BST_Insert(&(*T)->lchild, key);
11    else
12        return BST_Insert(&(*T)->rchild, key);
13 }

```

时间复杂度: $O(h)$, 主要开销在查找插入位置。

结论 11.5: BST 的删除 (三种情况)

删除结点 z , 分三种情况处理:

情况 (1): z 为叶子结点——直接删除, 令其父结点对应指针置为 NULL。

情况 (2): z 只有一棵子树 (左或右)——用 z 的唯一子树的根替代 z 。

情况 (3): z 有两棵子树——两种等价做法:

- 方案 A (直接前驱替代): 找到 z 在中序序列中的直接前驱 (即左子树中的最右结点 p), 将 p 的关键字复制到 z , 然后删除 p 。由于 p 没有右子树 (它是最右结点), 删除 p 退化为情况 (1) 或 (2)。
- 方案 B (直接后继替代): 找到 z 在中序序列中的直接后继 (即右子树中的最左结点 s), 将 s 的关键字复制到 z , 然后删除 s 。由于 s 没有左子树, 退化为情况 (1) 或 (2)。

408 两种方案均可, 除非题目指定用哪一种。注意: 删除操作总是归结为删除一个至多只有一棵子树的结点。

注意

408 常考 BST 删除后的中序遍历序列——无论用前驱替代还是后继替代, 删除后其余结点的中序相对次序不变 (因为替代结点本身就是被删结点的中序前驱或后继)。

11.4 平衡二叉树 (AVL)

定义 11.6: AVL 树的定义

平衡二叉树 (Balanced Binary Tree), 又称 AVL 树 (由 Adelson-Velsky 和 Landis 提出):

1. 或者是一棵空树;
2. 或者是满足以下条件的二叉排序树: 任意结点的左、右子树高度之差 (平衡因子) 的绝对值不超过 1。

平衡因子 $bf = \text{左子树高度} - \text{右子树高度}$ 。AVL 中 $bf \in \{-1, 0, 1\}$ 。含有 n 个结点的 AVL 树, 最大高度为 $O(\log n)$, 最坏情况下 $h \approx 1.44 \log_2 n$ 。

要点 11.6: AVL 结点的定义

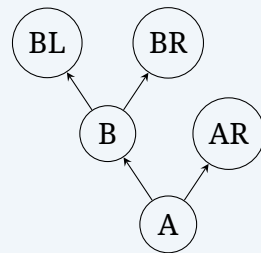
```
typedef struct AVLNode {
    KeyType key;
    int bf; // 平衡因子
    struct AVLNode *lchild, *rchild;
} AVLNode, *AVLTree;
```

AVL 的四种旋转

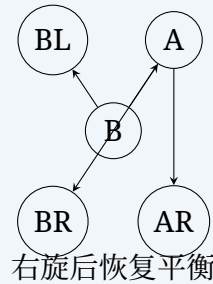
结论 11.6: LL 型——右单旋转

情况: 在结点 A 的左子树的左子树上插入新结点导致 A 的平衡因子变为 +2。

调整: 对 A 执行右旋 (Right Rotation) ——将 A 的左孩子 B 上提为根, A 成为 B 的右孩子, B 的原右子树挂为 A 的左子树。



插入前 (A 的 $bf = -1$)
在 BL 中插入后 A 的 $bf = +2$, LL 型



右旋后恢复平衡

旋转操作:

```

1 // 右旋: 以p为轴, 将p的左孩子上提
2 AVLNode *RotateRight(AVLNode *p) {
3     AVLNode *L = p->lchild;
4     p->lchild = L->rchild;
5     L->rchild = p;
6     return L; // 返回新根
7 }
  
```

结论 11.7: RR 型——左单旋转

情况: 在结点 A 的右子树的右子树上插入新结点导致 A 的平衡因子变为 -2 。

调整: 对 A 执行左旋 (Left Rotation) —— 将 A 的右孩子 B 上提为根, A 成为 B 的左孩子, B 的原左子树挂为 A 的右子树。LL 与 RR 互为镜像。

```

1 AVLNode *RotateLeft(AVLNode *p) {
2     AVLNode *R = p->rchild;
3     p->rchild = R->lchild;
4     R->lchild = p;
5     return R; // 返回新根
6 }
  
```

结论 11.8: LR 型——先左旋后右旋

情况: 在结点 A 的左子树的右子树 (即 B 的右子树 C) 上插入新结点导致 A 不平衡。

调整: 先对 B (A 的左孩子) 执行左旋, 使 B 的左子树变为 C 的左子树, C 代替 B 成为 A 的左孩子; 再对 A 执行右旋。即「先左后右」双旋转。

```

1 // LR: 先左旋 L, 再右旋 p
2 p->lchild = RotateLeft(p->lchild);
3 return RotateRight(p);
  
```

结论 11.9: RL 型——先右旋后左旋

情况: 在结点 A 的右子树的左子树上插入新结点导致 A 不平衡。

调整: 先对 B (A 的右孩子) 执行右旋, 再对 A 执行左旋。即「先右后左」双旋转。为 LR 的镜像。


```

1 // RL: 先右旋 R, 再左旋 p
2 p->rchild = RotateRight(p->rchild);
3 return RotateLeft(p);

```

注意

408 高频考点——AVL 旋转小结:

- 从插入位置向上回溯, 找到第一个 $|bf| = 2$ 的结点 (最小不平衡子树的根), 以它为轴进行调整。
- 判断旋转类型看「从被破坏平衡的结点出发, 沿插入方向走两步」: 两步都向左 $\rightarrow$ LL; 先左后右 $\rightarrow$ LR; 都向右 $\rightarrow$ RR; 先右后左 $\rightarrow$ RL。
- LR 和 RL 中, 新结点插入到 C 的左子树还是右子树会影响旋转后各结点平衡因子的具体值 ($0/\pm 1$), 408 选择题和综合题均可能考察。
- AVL 的删除也可能导致不平衡, 调整方法类似——从被删结点的父结点开始向上检查, 对每个失衡结点依据其较高子树的方向执行对应旋转 (可能需要多次调整, 不同于插入只需调整一次)。

11.5 B-树与 B+ 树

B-树

定义 11.7: m 阶 B-树

一棵 m 阶 (order) B-树, 或者为空树, 或者是满足以下性质的平衡多路查找树:

1. 每个结点至多有 m 棵子树 (即至多 $m - 1$ 个关键字)。
2. 除根结点外, 每个非终端结点至少有 $\lceil m/2 \rceil$ 棵子树 (即至少 $\lceil m/2 \rceil - 1$ 个关键字)。根结点若非叶结点, 则至少有 2 棵子树。
3. 每个结点中的关键字有序排列: $K_1 < K_2 < \dots < K_n$; 对每个关键字 K_i , 其左侧指针 P_{i-1} 所指子树中所有关键字均 $< K_i$, 右侧指针 P_i 所指子树中所有关键字均 $> K_i$ 。
4. 所有叶结点 (失败结点/外结点) 出现在同一层, 且不带信息 (可视为空指针)。

本文术语约定: 为避免「叶结点」的双重含义, 存放关键字的结点统称数据结点, 最底层存放关键字的结点称最底层数据结点; 其空子树指针指向的概念结点称外部失败结点。B-树高度 h 统一记为数据结点的层数 (根为第 1 层), 不计外部失败结点层。若题干采用边数或把外部结点计层, 需先换算口径。

要点 11.7: B-树的结点结构

```

1 #define MAXM 5          // B-树的阶
2 typedef struct BTreeNode {
3     int keynum;           // 结点中关键字个数
4     KeyType key[MAXM - 1]; // 关键字数组
5     struct BTreeNode *ptr[MAXM]; // 子树指针数组
6     // 可包含指向数据记录的指针
7 } BTreeNode, *BTree;

```

408 常考性质： n 个关键字的 m 阶 B-树，除根以外的非叶结点关键字数 k 满足：

$$\lceil \frac{m}{2} \rceil - 1 \leq k \leq m - 1$$

结论 11.10: B-树的查找

B-树的查找与 BST 类似，但每个结点内还需在有序关键字数组中查找（可用顺序或折半查找）：

1. 在根结点中查找，若找到则成功；
2. 否则，根据关键字大小关系确定应在哪个子树中继续查找；
3. 重复直到找到（成功）或到达外部失败结点（不成功）。

时间复杂度： $O(\log_m n)$ 级别（ $O(h)$ ， h 为 B-树高度）。

结论 11.11: B-树的插入与分裂

插入操作先将关键字插入到对应的最底层数据结点：

1. 找到该关键字应插入的叶结点位置；
2. 若该结点关键字个数 $< m - 1$ ，直接插入；
3. 若插入后关键字个数 $= m$ （超出上限），则进行分裂（Split）：
 - 将结点从中间位置 $\lceil m/2 \rceil$ 处分裂为两个结点，中间关键字提升到父结点；
 - 若父结点也因此超上限，则父结点继续分裂，分裂可能向上传播至根；
 - 若根也分裂，则产生新根，B-树高度增加 1。

408 典型操作题： 给定一个初始 B-树（通常 3 阶或 5 阶），依次插入若干关键字，画出每一步的结果。

结论 11.12: B-树的删除

删除分为两步：先找到待删关键字所在结点，然后执行删除。若待删关键字不在最底层数据结点中，则用其直接前驱（或直接后继）替代，再删除替代结点（归结为删除最底层数据结点中的关键字）。

删除后可能导致结点关键字个数低于下限（ $\lceil m/2 \rceil - 1$ ，根除外），需进行调整：

- **借（Borrow）：** 若左/右兄弟结点富裕（关键字数 $\geq \lceil m/2 \rceil$ ），从兄弟「借」一个关键字过来——通过父结点中转。
- **合并（Merge）：** 若左/右兄弟都不富裕，则与兄弟（及父结点中的分隔关键字）合并为一个结点。合并可能导致父结点关键字也低于下限，向上递归。

注意

408 高频考点——B-树性质：

- n 个关键字的非空 m 阶 B-树，按本文「数据结点层数」定义的最大高度满足：

$$h_{\max} \leq \log_{\lceil m/2 \rceil} \left(\frac{n+1}{2} \right) + 1$$

- 最小高度满足 $h_{\min} = \lceil \log_m (n+1) \rceil$ ；因为高度为 h 时至多有 $m^h - 1$ 个关键字。

- 结点分裂时，若 m 为奇数，中间关键字 $\lceil m/2 \rceil$ 上升；若 m 为偶数， $\lceil m/2 \rceil$ 或 $\lceil m/2 \rceil + 1$ 均可（取决于实现，但通常取 $\lceil m/2 \rceil$ ，即中间偏左）。
- B-树的所有外部失败结点在同一层，等价地，所有最底层数据结点在同一层，这是其「平衡性」的体现。

B+ 树

定义 11.8: m 阶 B+ 树

B+ 树是 B-树的一种变形，广泛应用于数据库索引。与 B-树的主要区别：

1. 每个非叶结点至多有 m 棵子树，至少有 $\lceil m/2 \rceil$ 棵（根除外）。
2. 按 408 教材的常用定义，非叶结点有 n 棵子树时含有 n 个索引关键字；非叶根至少有 2 棵子树，因而含 2 至 m 个关键字。
3. 非叶结点仅起索引作用，不存储指向数据记录的指针，只存储关键字和子树指针。
4. 所有关键字都出现在叶结点中，叶结点包含全部关键字及指向数据记录的指针。
5. 叶结点之间按关键字大小顺序链接（通常为单链表），支持高效的范围查询。
6. 查找时即使中间结点有关键字匹配，也必须继续走到叶结点才能确认——因为 B+ 树中非叶结点的关键字只是索引项，不一定对应实际存在的记录（被删除的记录可能仍在非叶结点中作为分界值）。

B-树与 B+ 树的对比（408 常考）

对比维度	B-树	B+ 树
关键字存储	每个结点都存关键字 + 记录指针	非叶仅存关键字（索引），叶存全部 + 记录指针
查找成功结束位置	任意层（找到即停）	必须到叶结点
叶结点结构	无链接	有指向下一叶结点的指针
范围查询效率	需中序遍历	$O(\log_m n + k)$ ，沿叶结点链直接扫描
根结点关键字数	$[1, m - 1]$ （非叶时至少 2 个子树）	非叶根 $[2, m]$ ；根即叶时 $[1, m]$
每个关键字出现次数	仅一次	非叶结点中可出现多次（作为索引分界值）

注意

408 常考对比题：

- B+ 树支持顺序访问和随机访问两种方式——顺序访问通过叶结点链表实现；B-树只能通过中序遍历顺序访问。
- B+ 树更适合文件系统和数据库索引的原因是：叶结点链支持高效范围查询（Range Query）；非叶结点不存数据，每个结点可容纳更多关键字，树更矮，减少磁盘 I/O。
- B-树更适合「单点查询」，因为可能在非叶结点就命中。

11.6 散列表 (Hash Table)

定义 11.9: 散列表与散列函数

散列表 (Hash Table): 根据关键字直接计算存储位置的数据结构。基本思想: 以关键字 key 为自变量, 通过散列函数 (Hash Function) $H(key)$ 计算出对应的存储地址。理想情况下, 查找无需比较即可直接定位——期望时间复杂度 $O(1)$ 。但不同的关键字可能映射到同一地址, 称为冲突 (Collision)。散列技术的核心问题: 设计好的散列函数和冲突处理方法。

要点 11.8: 散列函数的构造方法

常见的散列函数设计方法:

1. 除留余数法 (Division Method):

$$H(key) = key \bmod p$$

p 为不大于表长 m 的素数 (或不包含小于 20 的质因数的合数)。408 中最常见的方法。

2. 平方取中法 (Mid-Square Method): 取关键字平方后的中间几位作为散列地址。适用于关键字的每一位分布都不够均匀的场景。
3. 直接定址法: $H(key) = a \cdot key + b$ (a, b 为常数)。适用于关键字基本连续的情况, 不会产生冲突, 但空间利用率低。
4. 数字分析法: 选取关键字中分布较均匀的若干位作为散列地址。适用于关键字位数较多且某些位分布均匀的场景。
5. 折叠法: 将关键字分割为位数相等的几段, 叠加后取后几位。如移位叠加和边界叠加。

408 考试中, 除留余数法是最常考的散列函数构造方法。

冲突处理方法

要点 11.9: 开放定址法 (Open Addressing)

发生冲突时, 按某种探测序列在散列表中寻找下一个空闲位置:

$$H_i = (H(key) + d_i) \bmod m, \quad i = 0, 1, 2, \dots$$

三种常见的探测方式:

- (1) 线性探测法 (Linear Probing): $d_i = i$ (即 $0, 1, 2, 3, \dots$)。

- 优点: 实现简单。
- 缺点: 容易产生聚集 (Clustering) 现象——冲突的关键字会聚集在相邻的位置, 降低查找效率。

- (2) 二次探测法 (Quadratic Probing): $d_i = 1^2, -1^2, 2^2, -2^2, \dots$ (即 $0, 1, -1, 4, -4, \dots$), 即

$$H_i = (H(key) + i^2) \bmod m \quad \text{或} \quad H_i = (H(key) \pm i^2) \bmod m$$

可以缓解聚集现象。若采用 $\pm i^2$ 的对称探测序列, 并希望覆盖全部位置, 常取表长 $m = 4k + 3$ 的素数; 只采用单向 i^2 时通常只能覆盖部分位置, 不能把该条件笼统套用于所有二次探测实

现。

(3) 双重散列法 (**Double Hashing**): $d_i = i \times H_2(key)$, 其中 $H_2(key)$ 是另一个散列函数。 $H_2(key)$ 应与 m 互质。是最有效的开放定址法之一。

通用规则: 无论哪种探测方式, 开放定址法不能真正删除元素——删除只能做逻辑删除 (标记删除), 否则会切断探测序列导致后续查找失败。

要点 11.10: 链地址法 (Separate Chaining)

将所有散列地址相同的关键字链接在同一个单链表中。散列表的每个槽位是一个链表的头指针。

```
1 typedef struct HashNode {
2     KeyType key;
3     struct HashNode *next;
4 } HashNode;
5
6 typedef struct {
7     HashNode *table[MAXSIZE]; // 链表头指针数组
8     int size;
9 } HashTable;
```

优点:

- 不会出现「堆积」现象;
- 删除操作简单 (直接链表删除);
- 装填因子 α 可大于 1 (但太大影响效率);
- 适用于表长不确定的场景。

缺点: 需要额外的指针空间。

散列表的 ASL 与装填因子

定义 11.10: 装填因子

装填因子 (Load Factor) $\alpha = (\text{表中填入的记录数}) / (\text{散列表长度})$ 。 α 越大, 表明表越满, 冲突的可能性越大, 查找效率越低。

结论 11.13: 散列表的 ASL 计算

开放定址法——线性探测:

- $ASL_{\text{成功}} \approx \frac{1}{2} \left(1 + \frac{1}{1-\alpha} \right)$
- $ASL_{\text{不成功}} \approx \frac{1}{2} \left(1 + \frac{1}{(1-\alpha)^2} \right)$

链地址法:

- $ASL_{\text{成功}} \approx 1 + \frac{\alpha}{2}$
- $ASL_{\text{不成功}} \approx \alpha$ (只计与表中关键字的比较次数时, 等于桶内链表的平均长度)

上述是简单均匀散列下的期望值。若题目把访问桶头或判断空链表也记为一次操作, 则不成功

查找还要按题设口径计入该操作，不能把不同的计数口径混用。

408 实际计算要求：通常不直接套用上述近似公式，而是要求手工计算具体散列表的 ASL。计算步骤 ($ASL_{成功}$)：

1. 根据给定的关键字序列、散列函数和冲突处理方法，构造完整的散列表；
2. 对每个关键字，统计查找它所需的探测次数（含第一次散列）；
3. 求平均值： $ASL_{成功} = \frac{1}{n} \sum_{i=1}^n \text{探测次数}_i$ 。

计算步骤 ($ASL_{不成功}$)：

1. 对散列表的每个槽位（0 到 $m-1$ ），分别计算：若查找一个散列到该槽位但不存在的关键词，需要多少次探测才能判定「不存在」；
2. 对所有槽位的探测次数求和再除以 m 。

注意：若散列函数把不成功关键字等概率映射到表的全部 m 个初始地址，则 $ASL_{不成功}$ 的分母是 m （不是已有记录数 n ）。若散列函数的可能初始地址只有其中 q 个，则应对这 q 个初始地址求平均，不能机械地除以表长。

注意

408 高频考点——散列表 ASL 的关键细节：

- 查找成功的探测次数 = 从第一次散列算起到找到目标的探测次数（含第一次）。
- 查找不成功的探测次数 = 从某槽位第一次散列算起，沿探测序列直到遇到空位置（表明该关键字一定不存在）的探测次数。线性探测中若全表满则需探测 m 次。
- 链地址法中， $ASL_{不成功}$ 等于在每个槽位的链表中查找不存在的关键词所需的比较次数——即该链表长度（因为需要遍历完整个链表才能确定不存在）。
- 二次探测法和双重散列法中， $ASL_{不成功}$ 的判断条件是「探测到空槽」或「探测完整个表仍未找到」。

11.7 408 高频考点速查

结论 11.14: 查找——408 选择题常考点

1. 折半查找的判定树： n 个元素对应判定树中 n 个圆形结点和 $n+1$ 个方形结点；树形是平衡 BST；不同取整方式对应不同判定树形态。
2. ASL 计算：折半查找 $ASL_{成功}$ 和 $ASL_{不成功}$ ——前者分母 n ，后者分母 $n+1$ （方形结点数）。
3. AVL 旋转判断：从插入点向上找第一个 $|bf| = 2$ 的结点，根据「自上而下两步」判断 LL/LR/RL/RR。
4. B-树性质：除根外每个非叶结点至少 $\lceil m/2 \rceil$ 棵子树；所有叶结点同层；插入可能导致连锁分裂。
5. B+ 树 vs B-树：B+ 树非叶仅索引、叶链、查找必须到叶；B-树任意层可命中。
6. 散列表 ASL：手工计算成功/不成功 ASL，区分分母（ n vs m ），区分探测次数起算点。
7. 装填因子 α ： $\alpha = n/m$ ；决定散列表效率的核心参数。
8. BST 删除：度 2 结点转换为度 0/1 结点的删除；中序前驱/后继替代。

12 习题

12.1 基础过关题

1. (真题风格·选择题) 已知一个长度为 16 的顺序表 L , 其元素按关键字有序排列。若采用折半查找法查找一个不存在的元素, 则比较次数最多为

(A) 4 (B) 5

(C) 6 (D) 7

2. (真题风格·选择题) 在一棵 m 阶 B-树中, 除根结点外的所有非终端结点至少有多少棵子树?

(A) $\lfloor m/2 \rfloor$ (B) $\lceil m/2 \rceil$

(C) $\lfloor m/2 \rfloor + 1$ (D) $m - 1$

3. (真题风格·选择题) 一棵 3 阶 B-树的根结点含关键字 $[40, 70]$, 三个孩子从左到右分别含关键字 $[10, 20]$ 、 $[50, 60]$ 、 $[80, 90]$ 。依次删除关键字 10 和 20; 结点下溢时优先向右兄弟借关键字。调整完成后, 根结点中的关键字是

(A) $[40, 70]$ (B) $[50, 70]$

(C) $[50, 80]$ (D) $[60, 70]$

4. (真题风格·选择题) 设散列表长 $m = 14$, 散列函数 $H(k) = k \bmod 11$ 。表中已有 4 个记录: 15, 38, 61, 84, 用线性探测法处理冲突。则关键字 49 的地址是

(A) 8 (B) 3

(C) 5 (D) 9

5. (真题风格·选择题) 下列选项中, 属于 B+ 树特点的是

(A) 非叶结点包含关键字, 也包含指向记录的指针

(B) 叶结点仅包含关键字, 不包含指向记录的指针

(C) 所有叶结点通过指针链接成有序链表

(D) 结点中的关键字个数可以任意

6. (真题风格·选择题) B-树中所有外部失败结点均在同一层上, 这是由 B-树的什么特点决定的?

(A) 所有结点至少 $\lceil m/2 \rceil$ 棵子树

(B) 所有关键字按递增顺序排列

(C) 从根到叶的所有路径上关键字个数相同

(D) 插入和删除操作始终保持树的平衡

7. (真题风格·选择题) 下列关于 B-树与 B+ 树的比较中, 错误的是

- (A) B-树中的关键字可与记录关联, B+ 树的非叶结点只保存索引项
- (B) B+ 树的所有关键字都出现在叶结点中
- (C) B+ 树的叶结点通过指针链接在一起
- (D) B-树和 B+ 树的成功查找都必须到达最底层数据结点

12.2 真题风格与综合训练

8. (真题风格·综合题) 依次输入关键字序列 (30, 15, 25, 40, 10, 20, 35), 构造一棵二叉排序树 (BST)。

- (1) 画出该 BST;
- (2) 计算等概率下查找成功的平均查找长度 ASL;
- (3) 删除关键字 25 后画出 BST (用前驱替代)。

9. (真题风格·综合题) 设散列表长 $m = 13$, 散列函数 $H(k) = k \bmod 13$ 。用线性探测法处理冲突。依次插入关键字序列 (19, 14, 23, 1, 68, 20, 84, 27, 55, 11, 10, 79)。

- (1) 画出最终的散列表;
- (2) 计算等概率下查找成功的平均查找长度 ASL;
- (3) 计算查找不成功的平均查找长度 ASL。

10. (真题风格·综合题) 设有关键字序列 (40, 30, 20, 60, 50, 80, 70, 10), 依次插入一棵初始为空的 AVL 树。

- (1) 画出每次插入后 AVL 树的变化过程, 标注旋转类型 (LL/RR/LR/RL);
- (2) 写出该 AVL 树的中序遍历序列。

拓展阅读:

- 邓俊辉《数据结构 (C++ 语言版)》第 7 章「二叉搜索树」、第 8 章「高级搜索树」(含 AVL 和 B-树)。
- Sedgewick《算法》第 3.2 节 (二叉查找树)、第 3.3 节 (平衡查找树——红黑树和 B-树)。

13 排序

13.1 排序的基本概念

定义 13.1: 排序

排序 (Sorting) 是将一个数据元素的任意序列重新排列成一个按关键字有序的序列。设含 n 个记录的序列为

$$\{R_1, R_2, \dots, R_n\},$$

其对应的关键字序列为 $\{k_1, k_2, \dots, k_n\}$ 。排序即确定一种排列 $p(1), p(2), \dots, p(n)$ 使得

$$k_{p(1)} \leq k_{p(2)} \leq \dots \leq k_{p(n)}.$$

定义 13.2: 排序的稳定性

若待排序记录中存在两个或两个以上关键字相等的记录 R_i 与 R_j ($i < j$, 即 R_i 在序列中先于 R_j), 排序后二者的相对次序保持不变 (即 R_i 仍在 R_j 之前), 则称该排序算法是稳定的; 否则称不稳定的。

注意

稳定性是 408 高频考点。408 选择题常给出一组记录, 要求判断某排序算法执行后相同关键字记录的相对次序。稳定的算法: 直接插入排序、冒泡排序、归并排序、基数排序。不稳定的: 希尔排序、简单选择排序、快速排序、堆排序。

要点 13.1: 内部排序与外部排序

- 内部排序: 排序过程中全部记录存放在内存。本章第 2-7 节均属内部排序。
- 外部排序: 待排序记录数量太大, 无法一次全部装入内存, 排序过程中需在内存、外存之间多次交换数据。典型场景: 大文件排序。

排序算法的评价指标:

- 时间复杂度: 比较次数 + 移动次数。
- 空间复杂度: 除存储待排序记录之外所需的辅助空间。
- 稳定性: 见上文。

408 考试中, 内部排序是绝对重点, 外部排序仅考查基本概念与简单计算。

13.2 插入排序

直接插入排序 (Straight Insertion Sort)

结论 13.1: 直接插入排序——算法原理与代码

核心思想: 将待排序记录逐个插入到已排好序的有序序列中。类似打扑克牌时理牌的过程。

```

1 // 直接插入排序 (升序)
2 void InsertSort(int A[], int n) {
3     int i, j, temp;
4     for (i = 1; i < n; i++) {           // A[0..i-1] 有序, 插入 A[i]
5         temp = A[i];                   // 暂存待插入记录
6         for (j = i - 1; j >= 0; j--) {
7             if (A[j] > temp) {
8                 A[j + 1] = A[j];
9             }
10        }
11        A[j + 1] = temp;
12    }
13 }
```

```

6     for (j = i - 1; j >= 0 && A[j] > temp; j--)
7         A[j + 1] = A[j];           // 大于 temp 的记录后移
8     A[j + 1] = temp;               // 插入到正确位置
9 }
10 }

```

算法过程（以 [5, 2, 4, 6, 1, 3] 为例）：

i	哨兵	当前序列
初始	-	[5 2, 4, 6, 1, 3]
1	2	[2, 5 4, 6, 1, 3]
2	4	[2, 4, 5 6, 1, 3]
3	6	[2, 4, 5, 6 1, 3]
4	1	[1, 2, 4, 5, 6 3]
5	3	[1, 2, 3, 4, 5, 6]

复杂度分析：

- 最好情况（初始有序）：每趟只比较 1 次，不移动，共 $n - 1$ 次比较 $\Rightarrow O(n)$ 。
- 最坏情况（初始逆序）：第 i 趟比较 i 次，共 $\sum_{i=1}^{n-1} i = n(n-1)/2$ 次比较；移动次数同量级 $\Rightarrow O(n^2)$ 。
- 平均情况： $O(n^2)$ 。

空间复杂度： $O(1)$ （仅一个辅助变量 `temp`）。稳定性：稳定（判断条件 `A[j] > temp` 保证相等时不移位）。

折半插入排序（Binary Insertion Sort）

结论 13.2: 折半插入排序

在直接插入排序中，查找插入位置时使用二分查找代替顺序查找，以减少比较次数。但移动次数不变。

```

1 void BinInsertSort(int A[], int n) {
2     int i, j, low, high, mid, temp;
3     for (i = 1; i < n; i++) {
4         temp = A[i];
5         low = 0; high = i - 1;
6         while (low <= high) {           // 二分查找插入位置
7             mid = (low + high) / 2;
8             if (A[mid] > temp) high = mid - 1;
9             else low = mid + 1;         // 相等时插在后面 $!to$ 稳定
10        }
11        for (j = i - 1; j >= low; j--)    // 统一后移
12            A[j + 1] = A[j];
13        A[low] = temp;
14    }
15 }

```

复杂度分析：

- 每次插入的折半查找比较次数为 $O(\log i)$ ；总比较次数最坏为 $O(n \log n)$ （具体次数取决

于查找区间和实现细节，不能用上述求和式作为所有输入的精确值)。

- 移动次数：仍为 $O(n^2)$ (最坏和平均)。
- 总时间复杂度：仍为 $O(n^2)$ ，因为移动仍是瓶颈。

稳定性：稳定 (当 $A[mid] == temp$ 时，继续在右半区查找，保证相对次序不变)。

希尔排序 (Shell Sort)

结论 13.3: 希尔排序——缩小增量排序

希尔排序是对直接插入排序的改进：先将整个序列按某个增量 d 分成若干子序列，分别进行直接插入排序；然后缩小增量，再进行排序；直至增量 $d = 1$ 时，整体做一次直接插入排序。

```

1 void ShellSort(int A[], int n) {
2     int d, i, j, temp;
3     for (d = n / 2; d >= 1; d /= 2) { // 增量序列，每次减半
4         for (i = d; i < n; i++) { // 对各子表插入排序
5             temp = A[i];
6             for (j = i - d; j >= 0 && A[j] > temp; j -= d)
7                 A[j + d] = A[j];
8             A[j + d] = temp;
9         }
10    }
11 }
```

关键理解：

- 增量序列的选择影响效率。常用的有 $d_1 = n/2$, $d_{k+1} = \lfloor d_k/2 \rfloor$ (希尔原始增量)；更好的有 Hibbard 增量 $1, 3, 7, \dots, 2^k - 1$ 。
- 当 $d = 1$ 时就是一次直接插入排序，但由于前面的预处理使序列「基本有序」，最后一次排序工作量大减。

复杂度分析：

- 时间复杂度：高度依赖增量序列，不能脱离增量和输入分布给出统一的“平均复杂度”。希尔原始增量的最坏复杂度为 $O(n^2)$ ，Hibbard 增量可得到更好的上界；408 题目若要求精确复杂度，应以题设指定的增量序列和教材结论为准。
- 空间复杂度： $O(1)$ 。
- 稳定性：不稳定。相同关键字可能被分到不同子序列中，相对次序可能改变。

注意

408 常考——希尔排序的手工模拟：给出初始序列和增量序列，要求写出每趟排序后的结果。注意不同增量下子序列的划分方式：第 i 个子序列的下标为 $i, i + d, i + 2d, \dots$ 。同组内元素进行直接插入排序，不同组之间交替进行。

例题

对序列

{49, 38, 65, 97, 76, 13, 27, 49', 55, 4}

进行希尔排序，增量依次取 5, 3, 1。第一趟 $d = 5$ 时，子序列为

$$\{49, 13\}, \{38, 27\}, \{65, 49'\}, \\ \{97, 55\}, \{76, 4\}.$$

各子序列分别完成直接插入排序后，整体序列为

$$\{13, 27, 49', 55, 4, 49, 38, 65, 97, 76\}.$$

此时 49 和 49' 的相对次序已经改变，因此希尔排序不稳定。

13.3 交换排序

冒泡排序 (Bubble Sort)

结论 13.4: 冒泡排序——算法原理与代码

每趟从前往后两两比较相邻元素，若逆序则交换。每趟可将当前未排序部分的最大元素「冒」到最后。若某一趟没有发生交换，说明已有序，可提前结束。

```

1 void BubbleSort(int A[], int n) {
2     int i, j, temp;
3     bool swapped;
4     for (i = 0; i < n - 1; i++) {
5         swapped = false;
6         for (j = 0; j < n - 1 - i; j++) { // 每趟缩小比较范围
7             if (A[j] > A[j + 1]) { // 升序，注意不是 >=
8                 temp = A[j];
9                 A[j] = A[j + 1];
10                A[j + 1] = temp;
11                swapped = true;
12            }
13        }
14        if (!swapped) break; // 本趟无交换，已有序
15    }
16 }
```

复杂度分析：

- 最好情况（初始有序）：比较 $n - 1$ 次，移动 0 次 $\Rightarrow O(n)$ 。
- 最坏情况（初始逆序）：比较 $\frac{n(n-1)}{2}$ 次，移动 $\frac{3n(n-1)}{2}$ 次（每次交换 3 步） $\Rightarrow O(n^2)$ 。

空间复杂度： $O(1)$ 。稳定性：稳定（严格大于才交换， $A[j] > A[j+1]$ 而非 $\geq$ ）。

快速排序 (Quick Sort)

结论 13.5: 快速排序——算法原理

快速排序是实际应用中最快的通用内排序算法之一。核心思想是分治：

- 1. 划分 (**Partition**)：从序列中选一个枢轴 (**pivot**)，将序列分为左右两部分——左边全部 $\leq$ **pivot**，右边全部 $\geq$ **pivot**。**pivot** 落位到最终位置。
- 2. 递归地对左右子序列进行快速排序。

要点 13.2: 一趟划分 (Partition) 过程——Lomuto 版

以第一个元素为枢轴。交替从右找小、从左找大，直到两指针相遇。

```
1 int Partition(int A[], int low, int high) {
2     int pivot = A[low];           // 选第一个元素为枢轴
3     while (low < high) {
4         while (low < high && A[high] >= pivot) // 从右找小于 pivot 的
5             high--;
6         A[low] = A[high];          // 将小数移到左边
7         while (low < high && A[low] <= pivot) // 从左找大于 pivot 的
8             low++;
9         A[high] = A[low];          // 将大数移到右边
10    }
11    A[low] = pivot;                 // pivot 落位
12    return low;                    // 返回枢轴最终位置
13 }
```

划分过程演示 (**pivot = A[0] = 49**)：

初始: 49, 38, 65, 97, 76, 13, 27, 49'								
右找小	49	38	65	97	76	13	27	49'
low↑							←high	
交换	27	38	65	97	76	13	27	49'
↑low							↑high	
左找大	27	38	65	97	76	13		49'
			→low				↑high	
交换	27	38	65	97	76	13	65	49'
			↑low			↑high		
右找小	27	38	65	97	76	13		49'
			↑low			←high		
交换	27	38	13	97	76	13	65	49'
			→low			↑high		
左找大	27	38	13	97	76	13	65	49'
			↑low			↑high		
					←high			
相遇	27	38	13	49	76	97	65	49'

一趟划分结果：{27, 38, 13} 49 {76, 97, 65, 49'}，枢轴 49 落位。注意 49 和 49' 的相对次序可能改变——快排不稳定。

结论 13.6: 快速排序——完整代码

```

1 void QuickSort(int A[], int low, int high) {
2     if (low < high) {
3         int pivotpos = Partition(A, low, high);
4         QuickSort(A, low, pivotpos - 1); // 左半部分递归
5         QuickSort(A, pivotpos + 1, high); // 右半部分递归
6     }
7 }

```

要点 13.3: 快速排序——复杂度分析

设 $T(n)$ 为排序 n 个记录所需时间，一趟 Partition 需要 $O(n)$ 时间。

- 最好情况（每次划分均匀，pivot 恰在中位）：

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n) \implies T(n) = O(n \log n).$$

- 最坏情况（每次划分极不均衡，如初始有序/逆序且 pivot 固定选首元素）：

$$T(n) = T(n-1) + O(n) \implies T(n) = O(n^2).$$

- 平均情况：假设 pivot 等概率落在任一位置，可以证明 $T(n) = O(n \log n)$ 。

空间复杂度：递归工作栈深度。最好 $\lceil \log_2(n+1) \rceil = O(\log n)$ ；最坏 $O(n)$ （需 $n-1$ 次递归调用）；平均 $O(\log n)$ 。

稳定性：不稳定。Partition 过程中，跳跃式交换会打乱相同关键字的相对次序。

注意**408 核心考点——快排的 Partition：**

- 给定序列，写出每趟排序后的结果。若题目把“一趟”定义为一次 Partition，则一趟结束时 pivot 落位、左右子序列分开；若按递归树的一层计趟数，则应明确这是递归深度，二者不能混用。
- 最坏情况的触发条件：初始序列基本有序或基本逆序，且 pivot 选法固定（如始终选第一个/最后一个）时退化为 $O(n^2)$ 。优化方法：随机选 pivot、三数取中。
- 递归次数 $\neq$ 比较次数：递归深度 = 栈深度，每趟比较次数 $\approx$ 子序列长度。
- 快排的划分次数与递归深度要分开：总 Partition 次数在平衡和随机情形通常为 $\Theta(n)$ ，最坏为 $n-1$ ；递归深度最好/平均约为 $O(\log n)$ ，最坏为 $O(n)$ 。

13.4 选择排序

简单选择排序 (Simple Selection Sort)

结论 13.7: 简单选择排序

每趟从待排序部分选出最小元素，与待排序部分的第一个元素交换。

```

1 void SelectSort(int A[], int n) {
2     int i, j, min, temp;
3     for (i = 0; i < n - 1; i++) {
4         min = i;
5         for (j = i + 1; j < n; j++)
6             if (A[j] < A[min]) min = j;
7         if (min != i) {
8             temp = A[i]; A[i] = A[min]; A[min] = temp;
9         }
10    }
11 }

```

复杂度分析：

- 比较次数： $\sum_{i=0}^{n-2} (n-1-i) = \frac{n(n-1)}{2}$ ，与初始序列无关——始终 $O(n^2)$ 。
- 移动次数：最好 0，最坏 $3(n-1)$ 次。

空间复杂度： $O(1)$ 。稳定性：不稳定。例如 $[5, 5', 2] \rightarrow [2, 5', 5]$ ，第一个 5 被换到了后面。

堆排序 (Heap Sort)

定义 13.3: 堆

堆是一棵完全二叉树，且满足任一非叶结点的关键字均不大于（或不小于）其左右孩子结点的关键字。

- 小根堆 (**Min-Heap**)： $A[i] \leq A[2i+1]$ 且 $A[i] \leq A[2i+2]$ (根最小)。
- 大根堆 (**Max-Heap**)： $A[i] \geq A[2i+1]$ 且 $A[i] \geq A[2i+2]$ (根最大)。

堆排序通常用大根堆实现升序排序。

要点 13.4: 堆的存储

完全二叉树适合顺序存储（数组）。对下标从 0 开始的数组 $A[0..n-1]$ ：

- $A[i]$ 的左孩子： $A[2i+1]$ ；右孩子： $A[2i+2]$ 。
- $A[i]$ 的双亲： $A[(i-1)/2]$ 。
- 第一个非叶结点下标： $\lfloor n/2 \rfloor - 1$ 。
- 第一个叶子结点下标： $\lfloor n/2 \rfloor$ 。

408 注意：408 教材可能用下标 1 开始 ($A[1..n]$)，此时左孩子 $2i$ ，右孩子 $2i+1$ ，双亲 $\lfloor i/2 \rfloor$ ，考场上务必看清下标约定。

结论 13.8: 向下调整算法 (HeapAdjust / SiftDown)

假设以 $A[k]$ 为根的子树，除 $A[k]$ 本身外，左右子树均已满足堆性质。调整 $A[k]$ 下沉到合适位置，使整棵子树满足堆性质。

```

1 // 大根堆向下调整：将 A[k] 向下调整到合适位置
2 void HeapAdjust(int A[], int k, int n) {
3     int temp = A[k];           // 暂存根结点
4     for (int i = 2 * k + 1; i < n; i = 2 * i + 1) {
5         // i 指向左孩子；选较大的孩子
6         if (i + 1 < n && A[i] < A[i + 1]) // 右孩子存在且更大
7             i++;
8         if (temp >= A[i]) break;        // 已满足堆性质
9         A[k] = A[i];                   // 大孩子上移
10        k = i;                          // 继续向下筛选
11    }
12    A[k] = temp;
13 }

```

时间复杂度：每次调整沿树高下降，比较次数 $\leq 2\lfloor \log_2 n \rfloor$ ，即 $O(\log n)$ 。

结论 13.9: 建堆过程

从最后一个非叶结点 ($\lfloor n/2 \rfloor - 1$ ，下标 0 开始) 开始，向前遍历到根，对每个结点执行一次向下调整。

```

1 void BuildMaxHeap(int A[], int n) {
2     for (int i = n / 2 - 1; i >= 0; i--)
3         HeapAdjust(A, i, n);
4 }

```

建堆时间复杂度： $O(n)$ 。

例 序列 {4, 10, 3, 5, 1, 2} ($n = 6$) 建大根堆过程：

第一个非叶： $i = 2$ (值为 3)，子树 {3, 2} 已满足大根堆 ✓
 $i = 1$ (值为 10)，子树 {10, 5, 1} 满足 ✓
 $i = 0$ (值为 4)，调整：4 与 10 交换 $\rightarrow$ {10, 4, 3, 5, 1, 2};
 4 再与 5 交换 $\rightarrow$ {10, 5, 3, 4, 1, 2} ——建堆完成。

结论 13.10: 堆排序——完整算法

堆排序分为两步：(1) 建初堆 ($O(n)$)；(2) 反复输出堆顶，将堆尾元素移至堆顶后向下调整 ($n - 1$ 次，每次 $O(\log n)$)。

```

1 void HeapSort(int A[], int n) {
2     BuildMaxHeap(A, n);           // (1) 建大根堆
3     for (int i = n - 1; i > 0; i--) {
4         // 交换堆顶和堆尾 (当前未排序部分最后一个)
5         int temp = A[0]; A[0] = A[i]; A[i] = temp;
6         HeapAdjust(A, 0, i);      // (2) 调整剩余 i 个元素为新堆
7     }
8 }

```

例 续上例建堆后 {10, 5, 3, 4, 1, 2}：

输出 10，堆尾 2 移至堆顶 {2, 5, 3, 4, 1} $\rightarrow$ 调整 {5, 4, 3, 2, 1}；

输出 5，堆尾 1 移至堆顶 {1, 4, 3, 2} $\rightarrow$ 调整 {4, 2, 3, 1}；

依此类推，最终得 {1, 2, 3, 4, 5, 10}。

复杂度分析：

- 时间复杂度：建堆 $O(n) + n - 1$ 次调整 $O(n \log n) = O(n \log n)$ 。最好/最坏/平均均为 $O(n \log n)$ 。
- 空间复杂度： $O(1)$ （仅用一个辅助变量）。
- 稳定性：不稳定。堆排序中元素沿着树中路径跳跃移动，相同关键字相对次序可能改变。

注意

408 核心考点——堆排序：

- 建堆过程的手工模拟：给定初始序列，逐步画出完全二叉树并自底向上调整。
- 堆排序各趟结果：交换堆顶与堆尾 → 剩余元素重新调整为堆。每趟确定一个最大元素在末尾。
- 堆的插入与删除（堆本身的操作，不一定是堆排序）：
 - 插入：新元素放末尾，然后向上调整（上浮）；
 - 删除堆顶：堆尾元素移置堆顶，然后向下调整。
- 建堆 $O(n)$ 的证明思路：第 h 层最多 2^{h-1} 个结点，每结点最多下移 $(H - h)$ 层 ($H = \lfloor \log_2 n \rfloor + 1$)，总调整次数 $\sum_{h=1}^H 2^{h-1}(H - h) = O(n)$ 。

13.5 归并排序 (Merge Sort)

结论 13.11: 二路归并排序——原理与代码

归并排序是分治法的又一典型。基本思想：将两个（或以上）有序表合并为一个新的有序表。
二路归并（Merge）——合并两个相邻有序段：

```

1 // 将 A[low..mid] 和 A[mid+1..high] 两个有序段合并为一段
2 void Merge(int A[], int low, int mid, int high) {
3     int *B = (int*)malloc((high - low + 1) * sizeof(int));
4     int i = low, j = mid + 1, k = 0;
5     while (i <= mid && j <= high) { // 两段均未取完
6         if (A[i] <= A[j]) B[k++] = A[i++]; // 注意 <= 保证稳定性
7         else B[k++] = A[j++];
8     }
9     while (i <= mid) B[k++] = A[i++]; // 剩余左段
10    while (j <= high) B[k++] = A[j++]; // 剩余右段
11    for (i = low, k = 0; i <= high; i++, k++)
12        A[i] = B[k];
13    free(B);
14 }

```

递归归并排序：

```

1 void MergeSort(int A[], int low, int high) {
2     if (low < high) {
3         int mid = (low + high) / 2;
4         MergeSort(A, low, mid); // 左半排序
5         MergeSort(A, mid + 1, high); // 右半排序
6         Merge(A, low, mid, high); // 合并
7     }
8 }

```

} }

复杂度分析：

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n) \implies T(n) = O(n \log n).$$

- 时间复杂度： $O(n \log n)$ ，与初始序列无关——每趟归并均需 $O(n)$ ，共 $\lceil \log_2 n \rceil$ 趟。
- 空间复杂度： $O(n)$ （归并过程需要辅助数组）。
- 稳定性：稳定。Merge 中 $A[i] \leq A[j]$ 保证左段元素优先，相等时不改变相对次序。

注意

408 常考——归并排序各趟结果：给定初始序列，写出每趟归并后的结果。例如 {49, 38, 65, 97, 76, 13, 27}：

趟 1（段长 1→2）：{38, 49, 65, 97, 13, 76, 27}（最后一段只有 27 单独成段）

趟 2（段长 2→4）：{38, 49, 65, 97, 13, 27, 76}

趟 3（段长 4→8）：{13, 27, 38, 49, 65, 76, 97}

另外，归并排序也可用迭代（非递归）实现（自底向上），程序结构适合用 for 循环控制段长逐步翻倍。

13.6 基数排序（Radix Sort）

定义 13.4: 基数排序

基数排序是一种不基于比较的排序算法。它借助多关键字排序思想，将单关键字按各位的值进行排序。

结论 13.12: 基数排序原理

以十进制整数为例。设每个关键字由 d 位十进制数字组成，每位的取值范围为 $[0, 9]$ （即基数 $r = 10$ ）。

两种策略：

- 最高位优先（**MSD, Most Significant Digit**）：先按最高位排，再递归对各组排次高位。
- 最低位优先（**LSD, Least Significant Digit**）：先按最低位排，再排次低位……最后排最高位。LSD 需要每趟排序是稳定的。

LSD 基数排序过程（以 {278, 109, 63, 930, 589, 184, 505, 269, 8, 83} 为例）：

1. 第一趟（个位）：按个位数分配到 10 个队列（0–9），收集。
结果：{930, 63, 83, 184, 505, 278, 8, 109, 589, 269}。
2. 第二趟（十位）：按十位数分配、收集。
结果：{505, 8, 109, 930, 63, 269, 278, 83, 184, 589}。
3. 第三趟（百位）：按百位数分配、收集。
结果：{8, 63, 83, 109, 184, 269, 278, 505, 589, 930}（有序）。

关键：每趟的分配和收集必须是稳定的（通常用队列实现，先进先出），否则前一趟的排序成果会被破坏。

结论 13.13: 基数排序复杂度分析

设待排序记录数为 n ，关键字有 d 位，基数为 r （每位有 r 种取值）。

- 时间复杂度： $O(d(n+r))$ 。每趟分配 $O(n)$ （扫描全部记录），收集 $O(r)$ （扫描每个队列）。共 d 趟。
- 空间复杂度： $O(r)$ （ r 个队列的头尾指针）+ $O(n)$ （队列结点）= $O(n+r)$ 。
- 稳定性：稳定（若每趟采用的分配/收集方法保持稳定）。

适用场景：

- 关键字可分解为多个独立「位」，且每位取值范围不大。
- d 较小、 r 较小、 n 较大时效率优于基于比较的 $O(n \log n)$ 。
- 典型应用：字符串排序、多关键字排序（如先按班级排再按成绩排）。

注意

408 常考——基数排序不是基于比较的排序，其复杂度可以突破基于比较的 $\Omega(n \log n)$ 下界！排序趟数取决于关键字的位数（或「基数分解」方式），每趟排序不改变相同关键字记录的相对次序。

13.7 各种内部排序算法的比较**内部排序算法综合比较表**

算法	最好时间	平均时间	最坏时间	空间	稳定性
直接插入排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
折半插入排序	$O(n \log n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
希尔排序	依增量而定	依增量/输入而定	$O(n^2)$	$O(1)$	不稳定
冒泡排序	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
快速排序	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	平均 $O(\log n)$ 最坏 $O(n)$	不稳定
简单选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
堆排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	不稳定
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	稳定
基数排序	$O(d(n+r))$	$O(d(n+r))$	$O(d(n+r))$	$O(n+r)$	稳定

结论 13.14: 排序算法选择策略

1. n 较小时（如 $n \leq 50$ ）：直接插入排序或简单选择排序。移动次数：直接插入 > 直接选择，但直接选择比较次数固定。
2. n 较大时：选 $O(n \log n)$ 算法。
 - 快速排序：平均最快，是通用内排序的首选。但初始基本有序时退化，需用随机化或三数取中优化。
 - 堆排序：不会退化，辅助空间少（ $O(1)$ ），但实际常数因子较大，通常比快排慢。适合空间受限且需保证 $O(n \log n)$ 的场景。

- 归并排序：稳定，适合外部排序的基础。但需要 $O(n)$ 辅助空间。
3. 初始序列基本有序：直接插入排序或冒泡排序可达 $O(n)$ 。
 4. 记录规模大、关键字位数少：基数排序可能优于 $O(n \log n)$ 。
 5. 要求稳定：直接插入、冒泡、归并、基数排序。

注意

408 高频选择题考点——排序趟数：

- 冒泡排序趟数与初始序列有关：最好 1 趟（无交换即停），最坏 $n - 1$ 趟。
- 快速排序的递归树深度与初始序列有关（最好/平均约 $\log_2 n$ ，最坏为 n ）；若按一次 Partition 计“趟”，总划分次数不能写成 $O(\log n)$ 。
- 简单选择排序/堆排序每趟确定 1 个元素的最终位置：共 $n - 1$ 趟。
- 归并排序趟数 = $\lceil \log_2 n \rceil$ ，与初始序列无关。
- 基数排序趟数 = 关键字位数 d 。

13.8 外部排序 (External Sorting)

定义 13.5: 外部排序

外部排序指待排序文件太大，无法一次全部装入内存，需要多次在内存与外存之间交换数据。通常采用归并排序的思想。

结论 13.15: 外部排序基本过程

以二路平衡归并为例：

1. 将大文件按内存缓冲区大小分成若干段，每段读入内存进行内部排序，得到若干初始归并段 (**run**)，写回外存。
2. 对归并段进行多趟归并，每趟将归并段两两合并，段数减少，段长增大，直到只剩一个归并段 (有序文件)。

总时间 = 内部排序时间 + 外存读写时间 + 内部归并时间。其中外存 I/O 时间通常占主导。增大归并路数 k 可减少归并趟数 (趟数 = $\lceil \log_k m \rceil$, m 为初始段数)，但增大 k 会使每趟内部归并时选最小元的代价变大——需用败者树优化。

结论 13.16: 败者树 (Loser Tree)

败者树是一棵完全二叉树，用于在 k 路归并中高效选出 k 个归并段当前首记录中的最小值。

- 结构： k 个叶结点对应 k 个归并段的当前元素； $k - 1$ 个内部结点存放「失败者」（即比较中较大者）。树根上方额外放一个结点存放最终胜者（最小值）。
- 重建：输出胜者后，从对应叶结点到根路径上重赛，每次只需比较兄弟结点（或当前值与父结点值）。每选一个最小元需要沿路径进行 $O(\log k)$ 次比较。
- 与胜者树 (**Winner Tree**) 的区别：败者树中兄弟结点比较后「失败者」留存在父结点，重建时只需与路径上的父结点（保存的失败者）比较即可，比胜者树少访问兄弟结点，效率略高。

败者树的比较次数：初始建立败者树需要 $O(k)$ 次比较；之后每输出一个记录，沿路径重赛需要 $O(\log k)$ 次比较。总记录数为 N 时，总复杂度为 $O(N \log k)$ ，比较次数约为 $O(N \log k)$ （忽略末段不足 k 路等边界差异）。

结论 13.17: 置换-选择排序 (Replacement-Selection Sort)

产生初始归并段的算法。目的：在内存缓冲区大小受限的情况下，生成尽可能长的初始归并段（减少段数 m ，进而减少归并趟数）。

基本思想：在内存中维护一个大小为 W 的缓冲区（即堆）。

1. 读入 W 个记录建立小根堆。
2. 输出堆顶（当前段的最小记录）到当前初始段。
3. 读入下一记录，若其关键字 $\geq$ 刚输出的记录，则插入堆中继续参与当前段；否则标记为「下一段」暂存。
4. 堆空时当前段结束，用「下一段」暂存记录建新堆，开始新归并段。

效果：当输入接近有序时，可生成远大于缓冲区 W 的初始归并段，显著减少段数。

结论 13.18: 最佳归并树 (Optimal Merge Tree)

当各初始归并段的长度不相等时，以归并段长度为权，建立一棵带权路径长度 (WPL) 最小的 k 叉 Huffman 树，称为最佳归并树。按此树的归并顺序可使总 I/O 次数最少。

- 对于 k 路归并，若初始段数 m 不满足 $(m-1) \bmod (k-1) = 0$ （即不是严格 k 叉 Huffman 树），需添加长度为 0 的虚拟段补足。
- 总磁盘 I/O 次数 = $2 \times \text{WPL}$ （每归并一次，每个记录读写各一次。注：408 有时只统计归并趟数对应的读写次数）。

408 常考：给定若干归并段长度和归并路数 k ，计算最佳归并树的 WPL 或总 I/O 次数。

13.9 408 高频考点速查

结论 13.19: 排序——408 选择题常考点

1. 稳定性判断：会背表还不够——要能根据算法过程推理解释。希尔排序（跳跃分组）、快排（跳跃交换）、堆排序（沿树跳跃）、简单选择（远距离交换）不稳定。
2. 快排的 **Partition**：给定序列和 pivot 选法，写出每趟 Partition 结果（pivot 落位）。
3. 堆排序之建堆与调整：给定序列，画出完全二叉树，模拟自底向上建堆过程。写出每趟排序后序列。
4. 各排序算法的趟/轮次：冒泡、快排与初始序列相关；归并、基数与初始序列无关。
5. 时间复杂度对比：哪些是 $O(n \log n)$ ？哪些最好可 $O(n)$ ？哪三个 $O(n^2)$ ？
6. 基于比较的排序下界：基于比较的排序算法最坏情况时间下界为 $\Omega(n \log n)$ 。因此堆排序和归并排序在比较模型中是最优的。基数排序突破此下界因为它不基于比较。
7. 外部排序的败者树与最佳归并树：概念辨析与简单计算。

14 习题

14.1 基础过关题

- (单项选择题) 以下排序算法中, 不稳定的是
(A) 直接插入排序 (B) 冒泡排序
(C) 归并排序 (D) 快速排序
- (单项选择题) 对 n 个记录进行快速排序, 平均情况下的时间复杂度为
(A) $O(n)$ (B) $O(n \log n)$
(C) $O(n^2)$ (D) $O(\log n)$
- (单项选择题) 堆排序中, 对 n 个元素的序列建初堆, 时间复杂度为
(A) $O(1)$ (B) $O(n)$
(C) $O(n \log n)$ (D) $O(n^2)$
- (单项选择题) 初始序列基本有序时, 以下排序算法中效率最高的是
(A) 快速排序 (B) 直接插入排序
(C) 简单选择排序 (D) 堆排序
- (填空题) 基于比较的排序算法在最坏情况下的时间下界为 _____ (用大 O 或大 Ω 记法)。
- (填空题) 希尔排序的最后一趟增量必须为 _____。
- (填空题) 对序列 $\{6, 5, 3, 1, 8, 7, 2, 4\}$ 进行二路归并排序, 第一趟归并后的序列为 _____。
- (简答题) 简述堆排序中建堆过程为什么可以达到 $O(n)$ 的时间复杂度而非 $O(n \log n)$ 。给出证明思路即可。
- (算法设计) 编写算法, 用单链表实现直接插入排序。说明时间与空间复杂度。
- (算法设计) 编写算法, 将两个按升序排列的整型数组 $A[0..m-1]$ 与 $B[0..n-1]$ 归并到数组 $C[0..m+n-1]$ 中。要求 $O(m+n)$ 时间。

14.2 真题风格与综合训练

- (真题风格, 单项选择题) 对序列 $\{45, 38, 66, 90, 88, 10, 25, 45'\}$ 采用本章给出的“以首元素为枢轴、从右找小再从左找大”的交替填坑 Partition 算法进行一趟划分, 得到的序列是
(A) $\{10, 25, 38, 45, 88, 90, 66, 45'\}$ (B) $\{25, 38, 10, 45, 88, 90, 66, 45'\}$
(C) $\{10, 38, 25, 45', 45, 90, 66, 88\}$ (D) $\{25, 38, 10, 45', 45, 90, 88, 66\}$
- (真题风格, 单项选择题) 已知序列 $\{12, 10, 15, 8, 20, 6, 25, 18\}$, 将其调整为一个最小堆, 则调整后堆对应的序列是
(A) $\{6, 8, 12, 10, 20, 15, 25, 18\}$ (B) $\{6, 8, 15, 10, 20, 12, 25, 18\}$
(C) $\{6, 10, 12, 8, 20, 15, 25, 18\}$ (D) $\{6, 8, 12, 15, 20, 10, 25, 18\}$

13. (真题风格, 单项选择题) 以下关于堆排序的说法中, 正确的是

- (A) 堆排序是稳定的
- (B) 堆排序在最坏情况下的时间复杂度为 $O(n \log n)$
- (C) 建堆的时间复杂度为 $O(\log n)$
- (D) 堆排序每趟确定一个最小元素放在末尾

14. (真题风格, 综合题) 设待排序记录的关键字序列为 $\{49, 38, 65, 97, 76, 13, 27, 49'\}$ 。

- (1) 以第一个元素 49 为枢轴, 写出一趟快速排序 (Partition) 的结果;
- (2) 写出快速排序各趟排序完成后的完整序列;
- (3) 写出归并排序 (二路) 各趟归并后的完整序列;
- (4) 分析上述两种算法中 49 与 49' 的相对次序变化, 说明算法的稳定性。

15. (真题风格, 综合题) 设有关键字序列 $\{10, 18, 4, 3, 6, 12, 1, 9, 15, 8\}$ 。

- (1) 画出建初堆 (大根堆) 的过程 (每步调整后画出对应的完全二叉树);
- (2) 写出堆排序输出前 3 个最大元素后剩余元素构成的堆序列;
- (3) 写出希尔排序 (增量序列 5, 3, 1) 每趟排序后的序列。

16. (真题风格, 综合题) 有 8 个初始归并段, 各段记录数依次为 $\{9, 30, 12, 18, 3, 24, 6, 15\}$ 。

- (1) 采用 3 路平衡归并, 画出最佳归并树 (即三叉 Huffman 树);
- (2) 计算最佳归并树的 WPL, 并说明总 I/O 次数与 WPL 的关系;
- (3) 若归并过程中每趟需在 k 个归并段中选最小值, 简述败者树在 k 较大时的优势。

17. (真题风格, 综合题) 给定 n 个元素的序列。

- (1) 若要求排序算法是稳定的、时间复杂度 $O(n \log n)$ 且在最坏情况下不退化, 应选用哪种排序算法? 说明理由。
- (2) 若所有记录的关键字均不相同但已知序列基本有序 (仅有个别元素不在正确位置), 从时间效率角度应选哪种排序算法? 说明理由。
- (3) 若空间极其有限 (要求 $O(1)$ 额外空间), 且要求在最坏情况下保证 $O(n \log n)$ 时间, 应选哪种排序算法?

拓展阅读:

- 邓俊辉《数据结构 (C++ 语言版)》第 10 章「排序」——快速排序的三种轴点选取策略 (随机法、三者取中法), k -选取问题与选取中位数算法, 希尔排序的增量序列理论分析, 以及外部排序的完整讨论。
- Sedgewick《算法》第 2.1 节「初级排序算法」(选择/插入/希尔), 第 2.2 节「归并排序」, 第 2.3 节「快速排序」, 第 2.4 节「优先队列」(堆与堆排序)——对每种算法的性能和稳定性有深入的实验分析和数学推导, 第 2.5 节亦有排序算法综合对比。

15 算法设计专题

本章是数据结构课程的综合篇, 将前面各章 (线性表、树、图、查找、排序) 中的算法思想提炼为可复用的设计模式。数据结构综合应用题常以算法阅读、补全或设计为载体, 考查的不是死记

硬背，而是将基本数据结构和算法思想灵活组合的能力。

15.1 双指针与快慢指针

定义 15.1: 双指针技术

双指针 (Two Pointers) 指使用两个游标 (指针、下标) 协同遍历同一数据结构的设计模式。常见分类:

- 快慢指针: 两指针以不同速度移动 (如 $\times 1$ 和 $\times 2$), 用于判环、找中点;
- 前后指针: 两指针起点不同或移动节奏不同, 用于保持固定间距 (倒数第 k 个);
- 左右指针/碰撞指针: 两指针分别从序列两端向中间移动, 用于有序数组中的查找;
- 滑动窗口: 两指针界定可变长度的区间, 常用于子数组/子串问题。

快慢指针判环

结论 15.1: 快慢指针判环 (Floyd 判圈算法)

问题: 给定一个单链表, 判断其中是否存在环。若存在环, 找出环的入口结点。

算法思路:

1. 设置快指针 **fast** (每次走两步) 和慢指针 **slow** (每次走一步), 同时从头结点出发。
2. 若 **fast** 与 **slow** 相遇, 则链表有环; 若 **fast** 到达 **NULL**, 则无环。
3. 找环入口: 相遇后, 将一个指针重置到头结点, 两指针各走一步, 再次相遇处即为环入口。

```

1 // 判断链表是否有环, 若有则返回环入口结点
2 LNode *DetectCycle(LinkList L) {
3     LNode *slow = L->next, *fast = L->next; // 跳过不存储数据的头结点
4     while (fast && fast->next) {
5         slow = slow->next;
6         fast = fast->next->next;
7         if (slow == fast) { // 相遇则有环
8             LNode *p = L->next; // p 从头开始
9             while (p != slow) { // 各走一步, 再次相遇为入口
10                 p = p->next;
11                 slow = slow->next;
12             }
13             return p; // 环入口
14         }
15     }
16     return NULL; // 无环
17 }

```

正确性分析: 设头到环入口距离为 a , 环入口到相遇点距离为 b , 环长为 L 。相遇时慢指针走了 $a+b$ 步, 快指针走了 $2(a+b) = a+b+kL$ 步 (k 为快指针多绕圈数), 故 $a+b = kL$, 即 $a = kL - b$ 。从相遇点再走 a 步恰好到达环入口, 且从头走 a 步也到环入口——两者同时出发各走一步, 必相遇于入口。

复杂度: 时间 $O(n)$, 空间 $O(1)$ 。

注意

快慢指针是 408 风格算法题的常见背景。Floyd 判环通常令快指针每次走两步、慢指针每次走一步；若存在环，两者在模环长意义下的相对位置每轮前进 1，因而必会相遇。若任意改变步长，必须重新证明相对步长与环长的关系，不能仅凭「快一些」断言一定相遇。

查找倒数第 k 个结点**结论 15.2: 倒数第 k 个结点**

问题：在不计算链表长度的前提下，找到单链表中倒数第 k 个数据结点。

算法思路：「前后指针」——先让 **front** 指针向前走 k 步，然后 **back** 与 **front** 同步前进。当 **front** 到达末尾时，**back** 恰好指向倒数第 k 个结点。

```

1 LNode *FindKthFromEnd(LinkList L, int k) {
2     LNode *front = L->next, *back = L->next; // 从首元结点开始
3     for (int i = 0; i < k; i++) {
4         if (!front) return NULL; // k 超过表长
5         front = front->next;
6     }
7     while (front) { // front 走到末尾
8         front = front->next;
9         back = back->next;
10    }
11    return back; // back 即为倒数第 k 个
12 }

```

复杂度：时间 $O(n)$ ，空间 $O(1)$ 。

变体：

- 删除倒数第 k 个结点——需同时维护 **back** 的前驱，或让 **back** 从第 0 个结点（头结点）开始。
- 求链表中间结点——令 **front** 每次走两步，**back** 每次走一步（即快慢指针的另一应用）。

链表中间结点**结论 15.3: 求链表中间结点**

快慢指针的一次遍历实现：

```

1 LNode *FindMiddle(LinkList L) {
2     LNode *slow = L->next, *fast = L->next;
3     while (fast && fast->next) {
4         slow = slow->next;
5         fast = fast->next->next;
6     }
7     return slow; // 表长为奇数时指向正中间，偶数时指向后半段第一个
8 }

```

若偶数长度时希望指向前半段最后一个，则将 **fast** 初始条件设为 **fast->next && fast->next->next**。

复杂度：时间 $O(n)$ ，空间 $O(1)$ 。

15.2 链表逆置

链表逆置是 408 算法设计题的「元操作」——许多复杂操作（如部分逆置、回文判断、两数相加）都需要逆置作为子步骤。

就地逆置（全链表）

结论 15.4: 单链表就地逆置

算法思路：遍历原链表，逐个摘下结点并以「头插法」插入到新表头之后。也可用三指针法（pre, cur, next）逐个翻转指针方向。

方法一：头插法（带头结点）

```

1 void ReverseList(LinkList L) {
2     LNode *p = L->next;           // p 指向首元结点
3     L->next = NULL;               // 断开, L 成为新表头
4     while (p) {
5         LNode *q = p->next;       // 先保存后继
6         p->next = L->next;        // 头插
7         L->next = p;
8         p = q;                   // 继续下一结点
9     }
10 }
```

方法二：三指针反向链接（更直观）

```

1 LNode *ReverseListNoHead(LNode *head) {
2     LNode *prev = NULL, *cur = head, *next;
3     while (cur) {
4         next = cur->next;         // 保存后继
5         cur->next = prev;        // 反向
6         prev = cur;
7         cur = next;
8     }
9     return prev;                // 新表头
10 }
```

复杂度：时间 $O(n)$ ，空间 $O(1)$ 。

部分逆置

结论 15.5: 链表的区间逆置

问题：将链表中从第 m 到第 n 个结点之间的部分逆置 ($1 \leq m < n \leq$ 表长)。

算法思路：定位到第 $m-1$ 个结点，然后对区间内结点逐个「平移」到区间头部（即固定 pre，每次将当前结点的后继提到 pre 之后）。

```

1 void ReverseBetween(LinkList L, int m, int n) {
2     LNode *pre = L;
3     for (int i = 0; i < m - 1; i++) pre = pre->next; // 定位到 m-1
4     LNode *cur = pre->next;                          // cur 指向第 m 个
5     for (int i = 0; i < n - m; i++) {
```

```

6      LNode *t = cur->next;           // 要前移的结点
7      cur->next = t->next;           // cur 跳过 t
8      t->next = pre->next;           // t 插入到 pre 之后
9      pre->next = t;
10    }
11  }

```

关键技巧：该算法相当于在区间内部反复执行「将当前结点的后继结点移到区间最前面」。
复杂度：时间 $O(n - m) = O(n)$ ，空间 $O(1)$ 。

注意

408 逆置题常见变体：

- 不带头结点的链表逆置（需注意首结点变化）；
- 给定头尾指针的部分逆置（定位点不同）；
- 双链表逆置（需同时修改 `prior` 和 `next` 两个指针）；
- 每 k 个结点一组逆置（分段处理，余下不足 k 个时是否逆置由题设决定）。

15.3 二叉树递归算法设计

二叉树的递归算法是 408 综合题的另一大重点。掌握以下典型的递归设计，几乎可以覆盖所有二叉树类算法题。

二叉树的存储结构与遍历框架

要点 15.1: 二叉链表存储结构

```

1  typedef struct BiTNode {
2      ElemType data;
3      struct BiTNode *lchild, *rchild;
4  } BiTNode, *BiTree;

```

以下算法均基于此结构，采用递归遍历框架：

```

1  void Traverse(BiTree T) {
2      if (!T) return;           // 递归基：空树
3      // visit(T);              // 先序位置
4      Traverse(T->lchild);
5      // visit(T);              // 中序位置
6      Traverse(T->rchild);
7      // visit(T);              // 后序位置
8  }

```

在「先序/中序/后序位置」添加逻辑是实现所有二叉树递归算法的基本范式。

求二叉树深度

结论 15.6: 二叉树深度

```

1 int Depth(BiTree T) {
2     if (!T) return 0;           // 空树深度为 0
3     int ld = Depth(T->lchild);   // 左子树深度
4     int rd = Depth(T->rchild);   // 右子树深度
5     return (ld > rd ? ld : rd) + 1; // 取较大者加 1
6 }

```

扩展：求二叉树的最小深度——对左右孩子任一为空的情况特殊处理，取非空一侧的深度 +1。

求二叉树宽度

结论 15.7: 二叉树宽度

宽度定义为各层结点个数的最大值。需借助队列进行层序遍历。

```

1 int Width(BiTree T) {
2     if (!T) return 0;
3     BiTNode *queue[MaxSize];
4     int front = 0, rear = 1;
5     queue[0] = T;
6     int maxWidth = 0;
7     while (front < rear) {
8         int curLen = rear - front;           // 当前层的结点数
9         if (curLen > maxWidth) maxWidth = curLen;
10        for (int i = 0; i < curLen; i++) { // 处理当前层所有结点
11            BiTNode *p = queue[front++];
12            if (p->lchild) queue[rear++] = p->lchild;
13            if (p->rchild) queue[rear++] = p->rchild;
14        }
15    }
16    return maxWidth;
17 }

```

复杂度：时间、空间均为 $O(n)$ 。

统计结点数 / 叶子数

结论 15.8: 二叉树的结点统计

```

1 // 总结点数
2 int CountNodes(BiTree T) {
3     if (!T) return 0;
4     return CountNodes(T->lchild) + CountNodes(T->rchild) + 1;
5 }
6
7 // 叶子结点数（无左孩子也无右孩子）
8 int CountLeaves(BiTree T) {

```

```

9     if (!T) return 0;
10    if (!T->lchild && !T->rchild) return 1; // 叶子
11    return CountLeaves(T->lchild) + CountLeaves(T->rchild);
12 }
13
14 // 单分支结点数 (只有左孩子或只有右孩子)
15 int CountSingleChild(BiTree T) {
16     if (!T) return 0;
17     int isSingle = (!T->lchild ^ !T->rchild) ? 1 : 0; // XOR 异或
18     return CountSingleChild(T->lchild) + CountSingleChild(T->rchild) + isSingle
19     ↪ ;
20 }
21
22 // 第 k 层的结点数
23 int CountKthLevel(BiTree T, int k) {
24     if (!T) return 0;
25     if (k == 1) return 1;
26     return CountKthLevel(T->lchild, k - 1) + CountKthLevel(T->rchild, k - 1);
27 }

```

判断完全二叉树

结论 15.9: 完全二叉树的判定

算法思路：层序遍历。遇到第一个没有左孩子或没有右孩子的结点后，后续访问的所有结点都必须是叶子结点（无孩子）。

```

1 bool IsCompleteTree(BiTree T) {
2     if (!T) return true;
3     BiTNode *queue[MaxSize];
4     int front = 0, rear = 1;
5     queue[0] = T;
6     bool leafPhase = false; // 是否已进入"后面的结点必须是叶子"阶段
7     while (front < rear) {
8         BiTNode *p = queue[front++];
9         if (leafPhase && (p->lchild || p->rchild))
10            return false; // 必须为叶子但出现了孩子
11         if (p->lchild && p->rchild) {
12             queue[rear++] = p->lchild;
13             queue[rear++] = p->rchild;
14         } else if (!p->lchild && p->rchild) {
15             return false; // 有右无左 $!to$ 非完全二叉树
16         } else {
17             leafPhase = true; // 有左无右 或 无孩子
18             if (p->lchild) queue[rear++] = p->lchild;
19         }
20     }
21     return true;
22 }

```

时间复杂度： $O(n)$ 。关键判断条件：

- 若某结点有右孩子无左孩子 → 非完全二叉树；
- 若某结点只有左孩子（或无孩子），之后的所有结点都不能有孩子。

判断平衡二叉树

结论 15.10: 平衡二叉树的判定

判定条件：每个结点的左、右子树高度差绝对值不超过 1，且左右子树也都是平衡的。

朴素方法（先求高度再判平衡， $O(n^2)$ ）：

```

1 int Depth(BiTree T) {
2     if (!T) return 0;
3     int l = Depth(T->lchild), r = Depth(T->rchild);
4     return (l > r ? l : r) + 1;
5 }
6 bool IsBalanced(BiTree T) {
7     if (!T) return true;
8     int diff = Depth(T->lchild) - Depth(T->rchild);
9     if (diff > 1 || diff < -1) return false;
10    return IsBalanced(T->lchild) && IsBalanced(T->rchild);
11 }

```

优化方法（后序遍历，同时求高度和判断， $O(n)$ ）：

```

1 // 若平衡则返回树高，否则返回 -1
2 int CheckBalance(BiTree T) {
3     if (!T) return 0;
4     int lh = CheckBalance(T->lchild);
5     if (lh == -1) return -1; // 左子树已失衡
6     int rh = CheckBalance(T->rchild);
7     if (rh == -1) return -1; // 右子树已失衡
8     if (lh - rh > 1 || rh - lh > 1) return -1;
9     return (lh > rh ? lh : rh) + 1;
10 }
11 bool IsBalanced(BiTree T) {
12     return CheckBalance(T) != -1;
13 }

```

复杂度：优化方法时间为 $O(n)$ ，辅助空间为 $O(h)$ （递归栈深度， h 为树高）；平衡树时 $h = O(\log n)$ ，最坏退化树时为 $O(n)$ 。

设计要点：用返回值同时承载「是否平衡」和「高度」两个信息，避免各自重复计算。这是后序遍历的经典应用——先获得子树信息，再综合判断。

注意

408 二叉树递归题的通解：

1. 明确递归基：即空树返回什么？
2. 确定遍历顺序：先序（自顶向下传递）、后序（自底向上汇总）、还是中序（BST 特性）？
3. 定义返回值语义：不仅仅返回「结果」，可以返回包含多个信息的结构体或特殊值。
4. 常见的二叉树算法训练：求深度/宽度、判断完全二叉树、判断平衡二叉树、求双分支结

点数、交换左右子树。

15.4 二叉树非递归遍历的应用

当递归的栈深度可能导致栈溢出，或者需要中途暂停遍历时，需要使用非递归（显式栈/队列）方法。

非递归中序遍历

结论 15.11: 非递归中序遍历

「沿左链走到尽头再回溯」策略：

```
1 void InOrderNonRec(BiTree T) {
2     BiTNode *stack[MaxSize];
3     int top = -1;
4     BiTNode *p = T;
5     while (p || top != -1) {
6         if (p) {                                // 沿左链一直走，入栈
7             stack[++top] = p;
8             p = p->lchild;
9         } else {                                // 无左孩子，出栈访问，转向右子树
10            p = stack[top--];
11            visit(p);
12            p = p->rchild;
13        }
14    }
15 }
```

复杂度：时间 $O(n)$ ，辅助栈空间为 $O(h)$ ，其中 h 为树高；退化为链时最坏 $O(n)$ ，树高为 $O(\log n)$ 时为 $O(\log n)$ 。

非递归先序遍历

结论 15.12: 非递归先序遍历

```
1 void PreOrderNonRec(BiTree T) {
2     BiTNode *stack[MaxSize];
3     int top = -1;
4     if (T) stack[++top] = T;
5     while (top != -1) {
6         BiTNode *p = stack[top--];
7         visit(p);
8         if (p->rchild) stack[++top] = p->rchild; // 右孩子先进栈
9         if (p->lchild) stack[++top] = p->lchild; // 左孩子后进栈（先出栈）
10    }
11 }
```

非递归后序遍历

结论 15.13: 非递归后序遍历

双栈法（最简单的实现思路）：先用「根 → 右 → 左」的顺序入栈，再反转输出即得「左 → 右 → 根」的后序。或者用单栈 + 标记法：

```

1 void PostOrderNonRec(BiTree T) {
2     BiTNode *stack[MaxSize];
3     int tag[MaxSize];           // 0: 未访问右子树, 1: 已访问
4     int top = -1;
5     BiTNode *p = T;
6     do {
7         while (p) {             // 沿左链走到底
8             stack[++top] = p;
9             tag[top] = 0;
10            p = p->lchild;
11        }
12        while (top != -1 && tag[top] == 1) { // 右子树已访问, 访问根
13            visit(stack[top--]);
14        }
15        if (top != -1) {         // 转向右子树
16            tag[top] = 1;
17            p = stack[top]->rchild;
18        }
19    } while (top != -1 || p);
20 }

```

非递归遍历的经典应用

结论 15.14: 通过遍历序列构建表达式

非递归遍历（尤其是栈操作）在以下场景中非常有用：

- 寻找两结点的最近公共祖先（**LCA**）：分别记录从根到两个结点的路径（用非递归后序遍历实现），然后比较两条路径找到最后一个相同结点。时间复杂度 $O(n)$ 。
- 求从根到每个叶子结点的路径：先序遍历 + 栈维护当前路径，遇到叶子时输出栈中内容。
- 表达式树求值：后序遍历（递归/非递归均可）求值，叶子为操作数，内部结点为运算符。
- 二叉树线索化：中序遍历时记录前驱结点，修改其 `rchild` 为线索指向当前结点。

15.5 图的遍历应用

图的 DFS 和 BFS 不仅是遍历手段，更是解决图论问题的算法框架。

连通分量计数

结论 15.15: 无向图连通分量计数

算法思路：对每个未访问顶点启动一次 DFS（或 BFS），启动次数即连通分量个数。

```

1  int CountConnectedComponents(ALGraph *G) {
2      bool visited[MaxV] = {false};
3      int count = 0;
4      for (int v = 0; v < G->n; v++) {
5          if (!visited[v]) {
6              count++;           // 新连通分量
7              DFS(G, v, visited); // 遍历该连通分量内的所有顶点
8          }
9      }
10     return count;
11 }

```

复杂度：邻接表 $O(n + e)$ ，邻接矩阵 $O(n^2)$ 。

变体：

- 判断无向图是否连通——即连通分量计数是否为 1。
- 输出各连通分量的顶点集合——在 DFS 中将顶点加入当前分量的结果数组。
- 有向图的强连通分量——需要 Kosaraju 或 Tarjan 算法（408 仅要求了解概念）。

判断图中是否存在环

结论 15.16: 无向图判环

DFS 方法：遍历过程中，若遇到已访问顶点且该顶点不是当前结点的直接父结点，则存在环。

```

1  bool DFSCycle(ALGraph *G, int v, int parent, bool visited[]);
2
3  bool HasCycleUndirected(ALGraph *G) {
4      bool visited[MaxV] = {false};
5      for (int v = 0; v < G->n; v++)
6          if (!visited[v])
7              if (DFSCycle(G, v, -1, visited)) // -1 表示无父结点
8                  return true;
9      return false;
10 }
11 bool DFSCycle(ALGraph *G, int v, int parent, bool visited[]) {
12     visited[v] = true;
13     for (ArcNode *p = G->adjlist[v].first; p; p = p->next) {
14         int w = p->adjvex;
15         if (!visited[w]) {
16             if (DFSCycle(G, w, v, visited)) return true;
17         } else if (w != parent) { // 已访问且不是父结点 $|to$ 回边 $|to$ 有环
18             return true;
19         }
20     }
21     return false;

```

```
22 }
```

结论 15.17: 有向图判环

有向图判环不能简单地判断「已访问 + 不是父结点」, 因为存在指向已经被其他路径访问的顶点的情况 (非环)。需用三色标记法或用拓扑排序 (能否排序 n 个顶点)。

方法一: 三色 DFS

```
1  enum { WHITE, GRAY, BLACK };
2  int color[MaxV];
3  bool HasCycleDirected(ALGraph *G) {
4      for (int i = 0; i < G->n; i++) color[i] = WHITE;
5      for (int v = 0; v < G->n; v++)
6          if (color[v] == WHITE)
7              if (DFSCycleD(G, v)) return true;
8      return false;
9  }
10 bool DFSCycleD(ALGraph *G, int v) {
11     color[v] = GRAY;           // 正在访问
12     for (ArcNode *p = G->adjlist[v].first; p; p = p->next) {
13         int w = p->adjvex;
14         if (color[w] == GRAY) return true; // 回到当前路径上 $to$ 环
15         if (color[w] == WHITE && DFSCycleD(G, w))
16             return true;
17     }
18     color[v] = BLACK;         // 访问完毕
19     return false;
20 }
```

方法二: 拓扑排序法。运行拓扑排序, 若最终输出的顶点数少于 n , 则存在环。时间复杂度 $O(n + e)$ 。

复杂度: 均为 $O(n + e)$ (邻接表)。

15.6 分治思想

定义 15.2: 分治策略

分治 (Divide-and-Conquer) 将原问题递归地分解为若干个规模较小的同类子问题, 分别求解后合并得到原问题的解。标准步骤:

1. 分解 (**Divide**): 将原问题划分为 k 个规模更小的子问题 (通常 $k = 2$);
2. 求解 (**Conquer**): 递归求解各子问题; 当子问题足够小时直接求解;
3. 合并 (**Combine**): 将子问题的解合并为原问题的解。

分治算法的复杂度通常由递推式 $T(n) = a \cdot T(n/b) + f(n)$ 描述, 可用主定理求解。

数组主元素

结论 15.18: 寻找主元素 (Boyer-Moore 多数投票算法)

问题：长度为 n 的数组中，主元素指出现次数 $> \lfloor n/2 \rfloor$ 的元素。找出主元素或判定不存在。
 算法思路 ($O(n)$ 时间, $O(1)$ 空间):

1. 设置候选 `candidate` 和计数器 `count=0`。
2. 遍历数组：若 `count=0`，将当前元素设为 `candidate`；若当前元素 $=$ `candidate`，`count++`；否则 `count--`。
3. 再次遍历，验证 `candidate` 的出现次数是否 $> n/2$ 。若否，则不存在主元素。

```

1 // 通过返回值表示是否存在主元素, *result 输出主元素
2 bool MajorityElement(int A[], int n, int *result) {
3     int candidate = 0, count = 0;
4     for (int i = 0; i < n; i++) {
5         if (count == 0) candidate = A[i]; // 重置候选
6         count += (A[i] == candidate) ? 1 : -1;
7     }
8     count = 0;
9     for (int i = 0; i < n; i++) // 验证阶段
10        if (A[i] == candidate) count++;
11    if (count <= n / 2) return false;
12    *result = candidate;
13    return true;
14 }
  
```

正确性直觉：每次 `count` 归零，说明遍历过的前缀中候选元素和其他元素恰好两两抵消。由于主元素个数大于总数的一半，最终 `candidate` 必然是主元素。

复杂度：时间 $O(n)$ ，空间 $O(1)$ 。

注意

这是一类典型的 408 风格算法设计题：既要写出候选值的计数抵消过程，也要进行第二遍计数验证。算法不依赖复杂数据结构，但能体现「计数抵消」和正确性验证的思维。

逆序对计数

结论 15.19: 求数组逆序对个数

问题：数组中若 $i < j$ 且 $A[i] > A[j]$ ，称 $(A[i], A[j])$ 为一个逆序对。求数组中的逆序对总数。
 算法思路：在归并排序 (Merge Sort) 的归并过程中统计。当左侧子数组元素 $A[i]$ 大于右侧子数组元素 $B[j]$ 时，左侧剩余的所有元素 $(A[i \dots \text{mid}])$ 都与 $B[j]$ 构成逆序对。

```

1 long long MergeCount(int A[], int left, int mid, int right) {
2     int temp[right - left + 1];
3     int i = left, j = mid + 1, k = 0;
4     long long inv = 0;
5     while (i <= mid && j <= right) {
6         if (A[i] <= A[j]) {
7             temp[k++] = A[i++];
  
```

```

8         } else {
9             temp[k++] = A[j++];
10            inv += (mid - i + 1);           // A[i..mid] 均与 A[j] 构成逆序对
11        }
12    }
13    while (i <= mid) temp[k++] = A[i++];
14    while (j <= right) temp[k++] = A[j++];
15    for (i = 0; i < k; i++) A[left + i] = temp[i];
16    return inv;
17 }
18
19 long long CountInversions(int A[], int left, int right) {
20     if (left >= right) return 0;
21     int mid = left + (right - left) / 2;
22     long long leftInv = CountInversions(A, left, mid);
23     long long rightInv = CountInversions(A, mid + 1, right);
24     long long crossInv = MergeCount(A, left, mid, right);
25     return leftInv + rightInv + crossInv;
26 }

```

复杂度：时间 $O(n \log n)$ ，空间 $O(n)$ （归并辅助数组）。

变体：

- 求「顺序对」个数——将比较条件改为 $A[i] < A[j]$ 。
- 求「重要翻转对」—— $A[i] > 2 \times A[j]$ ，需在归并前独立扫描一次。

最大子段和

结论 15.20: 最大子段和

问题：给定含 n 个元素的数组，求连续子数组的最大和。

方法一：分治 ($O(n \log n)$)。将数组分为左右两半，最大子段要么全在左半、要么全在右半、要么跨越中点。跨越中点时，从中点向左右分别扫描求最大和。

方法二：DP/贪心 ($O(n)$ ，更常用)。

```

1 int MaxSubArray(int A[], int n) {
2     int curSum = 0, maxSum = A[0];           // 或 INT_MIN
3     for (int i = 0; i < n; i++) {
4         curSum = (curSum > 0 ? curSum : 0) + A[i]; // 若前缀和<0则舍弃
5         if (curSum > maxSum) maxSum = curSum;
6     }
7     return maxSum;
8 }

```

分治版本：

```

1 int MaxCrossing(int A[], int left, int mid, int right) {
2     int leftSum = A[mid], sum = 0;
3     for (int i = mid; i >= left; i--) {
4         sum += A[i];
5         if (sum > leftSum) leftSum = sum;
6     }

```

```

7   int rightSum = A[mid + 1]; sum = 0;
8   for (int i = mid + 1; i <= right; i++) {
9       sum += A[i];
10      if (sum > rightSum) rightSum = sum;
11  }
12  return leftSum + rightSum;
13 }
14 int MaxSubArrayDC(int A[], int left, int right) {
15     if (left == right) return A[left];
16     int mid = left + (right - left) / 2;
17     int lMax = MaxSubArrayDC(A, left, mid);
18     int rMax = MaxSubArrayDC(A, mid + 1, right);
19     int cMax = MaxCrossing(A, left, mid, right);
20     int max = lMax;
21     if (rMax > max) max = rMax;
22     if (cMax > max) max = cMax;
23     return max;
24 }

```

复杂度：分治 $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$ ；贪心/DP 为 $O(n)$ 。

15.7 算法设计题训练覆盖

注意

原按年份、题号和分值排列的索引尚未逐题对照权威原卷，本版不把它作为正式内容发布。下列内容只用于说明算法训练覆盖，不声明某一题对应具体考试年份。

结论 15.21: 训练规律总结

数据结构算法设计题应重点覆盖以下能力：

1. 线性表（数组/链表）：原地修改、双指针、循环移动、去重、拆分与归并；
2. 链表题集中在三种核心操作：逆置、双指针、归并/拆分；
3. 二叉树题集中在：递归遍历应用（WPL、深度判定、表达式树）、层序遍历；
4. 分治、贪心与空间换时间：中位数、主元素、未出现的最小正整数、逆序对等典型问题；
5. 图算法：基于 DFS/BFS 的可达性、路径约束、连通性与环检测；
6. 作答要求：必须给出核心伪代码，说明边界条件，并分别分析时间复杂度和辅助空间复杂度。

16 习题

16.1 基础过关题

- (单项选择题) 在单链表中, 用快慢指针法查找中间结点。慢、快指针均从首结点出发, 并执行 `while (fast && fast->next)`: 慢指针每次走一步, 快指针每次走两步。当链表长度为偶数时, 循环结束后慢指针指向的是
 - 前半段的最后一个结点
 - 后半段的第一个结点
 - 链表的最后一个结点
 - 仅由链表长度的奇偶性无法确定
- (单项选择题) 以下关于 Floyd 快慢指针判环算法的说法中, 错误的是
 - 快指针每次走两步可以保证在有环时一定与慢指针相遇
 - 快指针每次走三步也可以保证在有环时一定与慢指针相遇
 - 若检测阶段采用快指针每次走三步, 找到相遇点后仍可直接用经典“头指针 + 相遇点各走一步”定位环入口
 - 算法的时间复杂度为 $O(n)$, 空间复杂度为 $O(1)$
- (单项选择题) 判断有向图是否存在环的常用方法不包括
 - 三色 DFS 标记法
 - 拓扑排序
 - 并查集
 - 检查 DFS 过程中是否遇到 GRAY 结点
- (填空题) 在归并排序的归并过程中统计逆序对, 当取出右侧子数组元素时, 左侧子数组中剩余的元素个数即为当前元素产生的 _____ 数。该算法的时间复杂度为 _____。
- (填空题) 判断一棵二叉树是否为完全二叉树时, 若层序遍历中遇到了一个 _____ 的结点, 则其后访问的结点必须均为叶子结点。若某结点仅有右孩子而无左孩子, 可直接判定为 _____。
- (简答题) 简述用非递归中序遍历寻找二叉排序树中第 k 小元素的基本思路, 并说明时间复杂度。
- (简答题) 说明为什么 Boyer-Moore 多数投票算法需要「验证阶段」(即第二次遍历确认候选元素的出现次数)。如果不验证, 是否会出错? 请给出反例。
- (算法设计) 编写算法, 将带头结点的单链表 L 中数据域为奇数的结点移到数据域为偶数的结点之前, 且保持奇数和偶数各自内部的相对顺序不变。时间复杂度要求 $O(n)$, 空间复杂度 $O(1)$ 。

16.2 真题风格综合训练

- (真题风格, 单项选择题) 已知一棵完全二叉树共有 2024 个结点, 则其叶子结点数为
 - 1011
 - 1012
 - 1013
 - 1024
- (真题风格, 单项选择题) 对 n 个互异整数进行逆序对计数, 在最坏情况下逆序对的数量是

- (A) n (B) $n \log_2 n$
 (C) $\frac{n(n-1)}{2}$ (D) 2^n

11. (真题风格, 综合题) 设一棵关键字互异的二叉树以二叉链表存储, 结点结构为 (lchild, data, rchild)。按“左子树关键字严格小于根、右子树关键字严格大于根”的定义, 设计算法判断该树是否为二叉排序树 (BST)。要求:

- (1) 描述算法基本思想;
- (2) 用 C 语言实现算法;
- (3) 分析时间复杂度和空间复杂度。

12. (真题风格, 综合题) 一个长度为 n ($n > 1$) 且元素互异的严格递增整数数组经过非零次循环右移后得到数组 A 。等价地, 存在某个下标 p ($0 \leq p < n-1$), 使 $A[0 \dots p]$ 、 $A[p+1 \dots n-1]$ 各自严格递增, 且后一段的所有元素均小于前一段的所有元素。设计 $O(\log n)$ 时间复杂度的算法找出最小值。要求:

- (1) 描述算法思想;
- (2) 用 C 语言实现算法;
- (3) 说明为什么时间复杂度为 $O(\log n)$ 。

13. (真题风格, 综合题) 设非空有向图 $G = (V, E)$ 以邻接表存储。设计算法, 判断 G 是否为一棵以某个顶点为根、边由父结点指向孩子结点的有向根树 (out-arborescence)。也就是说, 应恰有一个入度为 0 的根, 其余顶点入度均为 1, 并且从根可到达所有顶点。要求:

- (1) 描述算法思想;
- (2) 用 C 语言实现算法;
- (3) 分析算法时空复杂度。

提示: 满足上述入度条件并且从根可达所有顶点时, 边数自动为 $n-1$, 且不可能存在有向环。

14. (真题风格, 综合题) 设一棵二叉树中各结点值互不相同, 其先序遍历序列和中序遍历序列分别存入一维数组 $pre[]$ 和 $in[]$ 中。设计算法, 用这两个序列重建该二叉树的二叉链表存储结构。要求:

- (1) 描述算法思想;
- (2) 用 C 语言实现算法;
- (3) 分析时间复杂度和空间复杂度。

15. (真题风格, 综合题) 给定长度为 n 的整数数组 A , 设计一个尽可能高效的算法, 找出数组中第 k 小的元素 ($1 \leq k \leq n$), 并使算法在平均情况下达到 $O(n)$ 时间复杂度。要求:

- (1) 描述算法的基本设计思想;
- (2) 用 C 语言实现算法 (允许调用自定义函数);
- (3) 说明算法在平均情况下时间复杂度为 $O(n)$ 的理由, 并分析最坏情况的时间复杂度。

16. (真题风格, 综合题) 假定采用带头结点的单链表保存单词, 每个结点存储一个字母。设计一个在时间上尽可能高效的算法, 判断该单词是否为回文 (正读与反读相同)。要求:

- (1) 给出算法的基本设计思想;
- (2) 用 C 语言实现算法;
- (3) 说明算法的时间复杂度和空间复杂度。

提示: 可综合利用本章中的「找中间结点」+「链表逆置」+「双指针比较」技术。

拓展阅读：

- 邓俊辉《数据结构（C++ 语言版）》第 2 章「向量」——有序向量查找、排序、归并各节的算法设计习题；第 3 章「列表」——列表的逆置、排序、查找算法设计习题；第 5 章「二叉树」——5.2 节「编码树」、5.3 节「遍历」的习题（涵盖非递归遍历、线索化、重构等）；第 6 章「图」——6.5 节「图遍历算法」的习题（欧拉路径、二部图判定、双连通分量等）。
- Sedgewick《算法》第 4 版第 1.4 节「算法分析」——介绍 TwoSum、ThreeSum 等问题的逐步优化，有助于理解算法设计的迭代过程；第 3.2 节「二叉查找树」——BST 的各种有序性操作；第 4.1 节「无向图」——连通分量、判环的具体代码与分析。
- Cormen, Leiserson, Rivest, Stein《算法导论》第 2.3 节「设计算法」——分治法的一般性讨论，包括最大子数组和归并排序中的逆序对。第 7 章「快速排序」——快速选择算法（期望线性时间选择第 k 小）。
- 《C 程序设计语言》(K&R)——若对 C 指针操作（链表、二叉树）不够熟练，建议通读第 5–6 章关于指针和结构体的内容。

PART II 第二部分

计算机组成原理

1 计算机系统概述

1.1 计算机发展历程

定义 1.1: 冯·诺依曼结构

冯·诺依曼 (John von Neumann) 于 1945 年提出「存储程序」(Stored Program) 概念, 奠定了现代通用计算机的基本架构。其核心思想可概括为:

1. 存储程序: 指令和数据以同等地位存放在同一个存储器中, 按地址访问。
2. 顺序执行: CPU 从存储器中逐条取出指令, 经译码后执行, 程序计数器 (PC) 自动指向下一条指令。
3. 二进制编码: 指令和数据均以二进制形式表示。
4. 五大部件: 计算机由运算器、控制器、存储器、输入设备和输出设备五大部分组成。

结论 1.1: 冯·诺依曼结构 vs 现代计算机结构

冯·诺依曼结构的特点: 以运算器为中心。

在冯·诺依曼原始设计中, 运算器 (ALU) 处于整个系统的中心位置: 输入/输出设备与存储器之间的数据传送都需要经过运算器, 运算器既是计算的核心, 也是数据流动的枢纽。这一设计导致了所谓的「冯·诺依曼瓶颈」——总线上的数据传送成为性能制约因素。

现代计算机结构的特点: 以存储器为中心。

现代计算机体系结构转向了「以存储器为中心」:

- CPU (运算器 + 控制器) 与存储器通过系统总线直接通信;
- I/O 设备通过 I/O 控制器/接口电路与总线相连, 数据可以在存储器和 I/O 设备之间直接传送 (DMA 方式), 无需经过 CPU 的运算器;
- 存储器成为数据交换的中心枢纽。

注意

408 常考点: 冯·诺依曼结构的核心特征是「存储程序」。多选题中常出现「冯·诺依曼计算机的特点」的判断, 注意:

- 「指令和数据以同等地位存放在同一存储器」——正确 (但现代机器中指令 Cache 和数据 Cache 分离是性能优化, 不改变「同一存储器」的逻辑模型);
- 「指令与数据均用二进制表示」——正确;
- 「指令顺序执行, PC 自增指向下一条」——正确 (是基本模型; 现代 CPU 的流水线和乱序执行是内部优化, 程序员看到的是顺序执行)。

结论 1.2: 计算机发展简史

计算机发展的四个时代

时代	时间	核心元件	特点
第一代	1946–1957	电子管	体积大、功耗高、机器语言编程
第二代	1958–1964	晶体管	出现高级语言 (FORTRAN)、批处理系统
第三代	1965–1971	中小规模 IC	操作系统成熟、出现微机雏形
第四代	1972–至今	大规模/超大规模 IC	微处理器诞生、PC 普及、多核并行

摩尔定律 (Moore's Law): 在经典表述中, 集成电路上可经济集成的晶体管数量大约每 18–24 个月增长一倍。它描述的是集成度趋势, 并不等价于处理器性能按同一比例自动增长; 功耗、存储墙和并行度都会影响实际性能。随着传统缩放放缓, 现代处理器更多依靠多核、向量化和专用加速器提升性能。

1.2 计算机硬件组成

定义 1.2: 计算机硬件五大部件

现代计算机的硬件系统由以下五大部件组成, 通过系统总线 (地址总线、数据总线、控制总线) 互联:

1. **运算器 (ALU, Arithmetic Logic Unit):** 执行算术运算和逻辑运算。核心部件包括 ALU、累加器 (ACC)、乘商寄存器 (MQ)、通用寄存器组等。
2. **控制器 (CU, Control Unit):** 指挥整个计算机系统协调工作。核心部件包括程序计数器 (PC)、指令寄存器 (IR)、指令译码器 (ID)、时序发生器和微操作信号发生器。运算器与控制器合称 CPU。
3. **存储器 (Memory):** 存放程序和数据。分为主存储器 (内存) 和辅助存储器 (外存)。主存由存储体、MAR (存储器地址寄存器)、MDR (存储器数据寄存器) 及读写电路组成。
4. **输入设备:** 将程序和数据从外界送入计算机, 如键盘、鼠标、扫描仪。
5. **输出设备:** 将处理结果从计算机送出到外界, 如显示器、打印机。

注意

408 常考辨析: CPU 包含哪两部分? 运算器 + 控制器 = CPU (中央处理器)。注意「主机」的概念在 408 中通常指 CPU + 主存储器 (不包括外存和 I/O 设备)。

结论 1.3: 运算器的核心寄存器

运算器常用寄存器	
寄存器	功能
ACC (Accumulator)	累加器, 存放 ALU 运算结果
MQ (Multiplier-Quotient)	乘商寄存器, 乘法时存乘数, 除法时存商
X (通用操作数寄存器)	存放一个 ALU 输入操作数
PSW/FLAGS	程序状态字/标志寄存器, 存放条件码 (ZF/OF/CF/SF)

不同体系结构中寄存器命名有所不同, 但功能等价。x86 中累加器为 EAX/RAX, ARM 中使用通用寄存器 r0-r15。

结论 1.4: 控制器的基本工作流程

控制器完成一条指令的基本流程 (取指-译码-执行):

- 1. 取指 (**Fetch**): 按 PC 所指地址从存储器取指令 →IR, PC 自增指向下一条指令。
- 2. 译码 (**Decode**): 指令译码器分析 IR 中的操作码, 产生相应的控制信号。
- 3. 执行 (**Execute**): 根据译码结果, 控制相关部件完成数据传送或运算。
- 4. 写回 (**Write Back**): 将结果写回寄存器或存储器 (如有需要)。

结论 1.5: 存储器的层次结构

存储器按速度和容量分为层次结构 (见后续章节详述):

CPU 寄存器 > Cache (L1/L2/L3) > 主存 (DRAM) > 辅存 (SSD/HDD)

从左到右: 速度递减、容量递增、单位成本递减。

1.3 计算机软件系统

定义 1.3: 系统软件 vs 应用软件

计算机软件分为两大类:

- 1. 系统软件 (**System Software**): 管理计算机系统资源、支撑其他软件运行的软件。包括:
 - 操作系统 (**OS**): 管理硬件资源、提供程序运行环境 (如 Linux、Windows、macOS)。
 - 语言处理程序: 编译器、汇编器、解释器, 将高级/汇编语言程序翻译为机器码。
 - 数据库管理系统 (**DBMS**): 如 MySQL、PostgreSQL。
 - 实用程序: 加载器、链接器、调试器。
- 2. 应用软件 (**Application Software**): 直接为用户完成特定任务的软件, 如办公软件、浏览器、游戏、科学计算程序等。

408 常考辨析: 操作系统属于系统软件; 数据库管理系统 (DBMS) 通常也归入系统软件范畴; 编译器、汇编器是系统软件。

结论 1.6: 三个级别的程序设计语言

三种语言级别对比			
特性	机器语言	汇编语言	高级语言
形式	二进制代码	助记符	类自然语言/数学表达式
可读性	极差	较差	良好
可移植性	无	差	好（需编译器适配）
执行效率	最高	高	取决于编译器优化
程序示例	B8 0001	MOV AX, 1	int a = 1;
翻译方式	直接执行	汇编器 → 机器码	编译器 → 机器码或解释执行

翻译过程：高级语言 $\xrightarrow{\text{编译器/解释器}}$ 汇编语言 $\xrightarrow{\text{汇编器}}$ 机器语言。注意：编译与解释是两种不同的翻译方式：

- 编译 (**Compilation**)：一次性将整个源程序翻译为目标代码，再执行目标代码（如 C/C++、Go、Rust）。
- 解释 (**Interpretation**)：逐条翻译并执行，不生成独立的目标文件（如 Python、JavaScript 的传统执行方式）。

1.4 计算机系统的层次结构

定义 1.4: 计算机系统的多层次模型

计算机系统可以抽象为层次模型：下层为上层提供接口（服务），上层通过调用下层接口实现更高级的功能。从底层到上层的典型分层如下：

结论 1.7: 从微程序到高级语言的六级层次

层次	层级	核心概念
L_0	微程序级/硬联逻辑	微指令、微操作，由硬件直接执行
L_1	传统机器语言级	二进制机器指令，由微程序解释执行
L_2	操作系统级	提供系统调用，管理资源与抽象
L_3	汇编语言级	助记符指令，由汇编器翻译为 L_1
L_4	高级语言级	C/C++/Java 等，由编译器翻译
L_5	应用层（用户层）	应用程序通过 API/库使用系统服务

关键概念：

- 翻译 (**Translation**)：将上层程序整体转换为下层等价程序，再在下层执行。 $L_3 \rightarrow L_1$ （汇编器）、 $L_4 \rightarrow L_1$ （编译器）均通过翻译。
- 解释 (**Interpretation**)：下层程序逐条处理上层程序的每条语句并直接执行，不生成完整的目标程序。 L_1 的机器指令通常由 L_0 的微程序解释执行。
- 虚拟机 (**Virtual Machine**)：通过「翻译 + 解释」的组合，使得上层语言不需要考虑底层的硬件细节，每一层对上层来说都是一台「虚拟机器」。典型例子：JVM 将 Java 字节

码解释/即时编译 (JIT) 在物理机上执行。

注意

408 常考点——翻译 vs 解释：

- 在层次结构模型中， L_3 (汇编) 到 L_1 (机器) 是翻译； L_4 (高级语言) 到 L_1 也是翻译。
- L_0 (微程序) 到 L_1 是解释——每一条机器指令由一段微程序解释执行。
- 注意：物理上不存在 L_2 – L_5 层的「硬件」，这些是虚拟机器（由下层软件 + 硬件模拟出来的抽象）。物理机器一般指 L_0 和 L_1 。

1.5 计算机性能指标

定义 1.5: 基本性能指标

衡量计算机性能的主要指标包括：

1. **机器字长 (Word Size)**：CPU 一次能处理的二进制数据的位数。通常与通用寄存器位数和 ALU 宽度一致。字长越长，计算精度越高，但硬件成本也越大。常见：32 位、64 位。
2. **主频 (Clock Frequency)**：CPU 内部时钟信号的频率，以 Hz 为单位。主频 f 决定了时钟周期 $T = 1/f$ 。例如主频 3.0 GHz $\rightarrow$ 时钟周期 $T \approx 0.33$ ns。
3. **CPI (Cycles Per Instruction)**：执行一条指令所需的平均时钟周期数。CPI 越小，CPU 效率越高。
4. **MIPS (Million Instructions Per Second)**：每秒执行百万条指令。
5. **MFLOPS (Million Floating-point Operations Per Second)**：每秒百万次浮点运算。类似还有 GFLOPS、TFLOPS。

要点 1.1: CPU 执行时间的核心公式

CPU 执行时间是衡量计算机性能的最终标尺。核心公式：

$$T_{\text{CPU}} = N \times \text{CPI} \times T_{\text{clk}} = \frac{N \times \text{CPI}}{f}$$

其中：

- N ：程序执行的总指令条数 (Instruction Count)。
- CPI：平均每条指令的时钟周期数 (Cycles Per Instruction)。
- T_{clk} ：时钟周期 (Clock Cycle Time)， $T_{\text{clk}} = 1/f$ 。
- f ：主频 (Clock Frequency)。

三个维度的优化方向：

- 减少 N ：编译器优化、更好的算法；
- 降低 CPI：微架构改进 (流水线、超标量、乱序执行)；
- 减少 T_{clk} (提高主频)：改进工艺、更好的电路设计。

结论 1.8: MIPS 与 MFLOPS 计算

$$\text{MIPS} = \frac{\text{指令条数}}{\text{执行时间} \times 10^6} = \frac{f}{\text{CPI} \times 10^6}$$

$$\text{MFLOPS} = \frac{\text{浮点运算次数}}{\text{执行时间} \times 10^6}$$

注意事项:

- MIPS 是与程序有关的指标——同一 CPU 运行不同程序时 MIPS 不同, 不能用 MIPS 直接比较不同指令系统的机器的性能。
- MIPS 忽略了指令功能的差异 (复杂指令 vs 简单指令执行的工作量不同), 因此有时会出现「MIPS 较高但实际性能却较差」的情况 (所谓「MIPS 陷阱」)。
- MFLOPS 更适用于科学计算领域的性能比较。

注意

408 高频计算考点——CPI 与 MIPS 的换算:

1. 已知主频和 MIPS, 求 CPI:

$$\text{CPI} = \frac{f}{\text{MIPS} \times 10^6}$$

2. 多类指令的加权 CPI: 若程序包含 k 类指令, 每类指令条数为 N_i , CPI 为 CPI_i , 则:

$$\text{CPI}_{\text{avg}} = \frac{\sum_{i=1}^k N_i \times \text{CPI}_i}{\sum_{i=1}^k N_i} = \sum_{i=1}^k \left(\frac{N_i}{\sum N_i} \right) \times \text{CPI}_i$$

3. 执行时间比较: 比较两台机器的性能时, 最终依据是 T_{CPU} (或与其成正比的 $N \times \text{CPI}/f$), 不能只比较 MIPS 或主频。

结论 1.9: 性能指标速查表

常见性能指标

指标	定义	备注
机器字长	CPU 一次处理的二进制位数	32/64 位
主频 f	时钟信号频率	GHz 级
T_{clk}	$1/f$, 时钟周期	例如 3 GHz $\rightarrow$ 0.33 ns
CPI	每条指令的平均时钟数	越小越好
IPC	$1/\text{CPI}$, 每周期指令数	越大越好
MIPS	每秒百万条指令	与程序相关
MFLOPS	每秒百万次浮点运算	科学计算常用
T_{CPU}	$N \times \text{CPI} \times T_{\text{clk}}$	终极性能指标

其中 $\text{IPC} = 1/\text{CPI}$ 。现代超标量处理器的 IPC 可以大于 1 (每个周期发射多条指令)。

结论 1.10: 性能计算实例

例 某计算机主频为 2.0 GHz, 运行某程序共执行 2×10^9 条指令, $\text{CPI} = 1.5$ 。求 CPU 执行时间和 MIPS。

解:

- $T_{\text{clk}} = 1/(2.0 \times 10^9) = 0.5 \times 10^{-9} \text{ s} = 0.5 \text{ ns}$
- $T_{\text{CPU}} = N \times \text{CPI} \times T_{\text{clk}} = 2 \times 10^9 \times 1.5 \times 0.5 \times 10^{-9} = 1.5 \text{ s}$
- $\text{MIPS} = f/(\text{CPI} \times 10^6) = 2 \times 10^9/(1.5 \times 10^6) \approx 1333.3$

例 某程序包含三类指令, A 类 10,000 条 ($\text{CPI} = 1$)、B 类 20,000 条 ($\text{CPI} = 2$)、C 类 10,000 条 ($\text{CPI} = 3$)。求该程序的平均 CPI。

解:

$$\text{CPI}_{\text{avg}} = \frac{10,000 \times 1 + 20,000 \times 2 + 10,000 \times 3}{10,000 + 20,000 + 10,000} = \frac{80,000}{40,000} = 2.0$$

1.6 408 高频考点与答题策略**结论 1.11: 本章 408 核心考点总结**

1. 冯·诺依曼结构的核心特征: 存储程序、五大部件、以运算器为中心 (注意与现代以存储器为中心的对比)。
2. 五大部件的功能与辨析: 运算器 (ALU + 寄存器)、控制器 (PC/IR/ID)、两者合称 CPU; 主机 = CPU + 主存。
3. 软件分类: 系统软件 (OS/编译器/汇编器/DBMS) vs 应用软件——这个知识点在选择题中常以「以下哪个属于系统软件」的形式出现。
4. 三个语言级别: 机器语言 (二进制)、汇编语言 (助记符 → 汇编器翻译)、高级语言 (编译器翻译或解释执行)。注意区分编译与解释。
5. 计算机层次结构: L_0 (微程序) 到 L_5 (应用层), 理解每一层的作用及层次间的翻译/解释关系。常考点: 哪几层是虚拟机? 哪几层是物理机?
6. CPU 执行时间公式: $T_{\text{CPU}} = N \times \text{CPI} \times T_{\text{clk}}$ 。这是串起所有性能指标的枢纽公式, 必须熟练运用。
7. MIPS 计算: $\text{MIPS} = f/(\text{CPI} \times 10^6)$, 注意单位的统一 (主频用 Hz)。
8. 多类指令的加权 CPI: 按频度加权求和, 408 计算题高频出现。

注意

常见易错点:

- 主频单位换算: $1 \text{ GHz} = 10^9 \text{ Hz}$; $1 \text{ ms} = 10^{-3} \text{ s}$; $1 \mu\text{s} = 10^{-6} \text{ s}$; $1 \text{ ns} = 10^{-9} \text{ s}$ 。
- MIPS 公式中分母的 10^6 不要遗漏。
- 冯·诺依曼计算机中「指令和数据放在同一存储器」——这里的「同一存储器」指的是逻辑上的统一编址主存, 不是指 Cache 的物理分离。
- 「五大部分」中不包括「总线」——总线和电源属于支撑结构。

2 习题

2.1 基础过关题

1. (单项选择题) 冯·诺依曼计算机中, 指令和数据在存储器中的存放方式是
(A) 分别存放在两个独立存储器中 (B) 以同等地位存放在同一个存储器中
(C) 指令存放在 ROM, 数据存放在 RAM (D) 指令存放在 Cache, 数据存放在主存
2. (单项选择题) 以下关于 CPU 组成的说法中, 正确的是
(A) CPU 仅包含运算器 (B) CPU 仅包含控制器
(C) CPU 包含运算器和控制器 (D) CPU 包含运算器、控制器和主存储器
3. (单项选择题) 某计算机主频为 1.2 GHz, 运行某程序的 CPI 为 2.0, 则该计算机的 MIPS 为
(A) 600 (B) 1200
(C) 2400 (D) 600×10^6
4. (单项选择题) 以下不属于系统软件的是
(A) 操作系统 (B) 编译器
(C) 数据库管理系统 (D) 浏览器
5. (填空题) 在计算机系统的层次结构中, 能直接被硬件识别和执行的层次是 _____ 级。汇编语言程序需要通过 _____ 翻译为机器语言。
6. (简答题) 简述冯·诺依曼结构的主要特点, 并说明现代计算机结构与其最主要的区别是什么。
7. (计算题) 某计算机主频为 2.5 GHz, 运行基准测试程序中共执行了 5×10^9 条指令, CPI 为 1.6。求: (1) CPU 执行时间; (2) 该计算机的 MIPS。

2.2 真题风格与综合训练

8. (真题风格, 单项选择题) 某程序在计算机 A 上运行需 10 s, A 的主频为 2 GHz。计算机 B 执行该程序的指令条数与 A 相同, 但平均 CPI 为 A 的 1.2 倍。若目标是在 B 上用 6 s 运行完该程序, 则 B 所需的主频为
(A) 2.56 GHz (B) 3.33 GHz
(C) 4.00 GHz (D) 4.44 GHz
9. (真题风格, 单项选择题) 某程序在某台计算机上运行, 该程序包含 A、B、C 三类指令, 其 CPI 分别为 1、2、3。已知程序中三类指令的条数之比为 2 : 3 : 5, 则该程序的平均 CPI 为
(A) 2.0 (B) 2.1
(C) 2.3 (D) 2.5
10. (真题风格, 综合题) 某计算机的主频为 3.0 GHz, 运行某程序时测得 MIPS 为 1200。回答以下问题:

- (1) 求该程序运行时的平均 CPI;
- (2) 若该程序共执行 3.6×10^9 条指令, 求 CPU 执行时间;
- (3) 若通过编译器优化, 将程序的指令条数减少 20%, 但 CPI 增加了 10%, 优化后的程序执行时间比优化前更快还是更慢? 请给出计算过程。

11. (真题风格, 综合题) 在计算机系统的层次结构中:

- (1) 描述从 L_4 (高级语言级) 到 L_1 (机器语言级) 的转换过程, 指出哪些步骤使用了翻译, 哪些步骤使用了 (或可能使用) 解释;
- (2) 说明 Java 语言程序的执行过程如何体现「虚拟机」的概念;
- (3) 为什么说操作系统层 (L_2) 是虚拟机而非物理机? 操作系统为上层提供了哪些抽象?

12. (真题风格, 综合题) 某处理器包含 4 类指令, 其 CPI 和在某个典型程序中的使用频度如下表:

指令分布			
	指令类型	CPI	频度
	整数 ALU	1	45%
	数据传送	2	30%
	浮点运算	4	15%
	分支跳转	3	10%

- (1) 计算该程序的平均 CPI;
- (2) 若该处理器的主频为 2.0 GHz, 求 MIPS;
- (3) 设计者计划将浮点运算的 CPI 从 4 降为 2, 同时主频提升到 2.5 GHz, 其他不变。改进后的整体性能提升为原来的多少倍? (用 T_{CPU} 之比表示)

拓展阅读:

- Patterson & Hennessy 《计算机组成与设计: 硬件/软件接口》第 1 章「计算机概念与技术」——1.1–1.6 节全面覆盖本章内容, 包括性能公式的导数和 Amdahl 定律。
- Bryant & O'Hallaron 《深入理解计算机系统》(CS:APP) 第 1 章「计算机系统漫游」——1.1–1.9 节从程序员视角理解计算机系统的抽象层次, 包含 Amdahl 定律 (第 1.9 节) 和并发/并行的初步讨论。

3 数据的表示与运算

3.1 进位计数制

定义 3.1: 进位计数制

对于任意 R 进制数 $N = (d_{n-1}d_{n-2} \dots d_1d_0.d_{-1}d_{-2} \dots d_{-m})_R$, 其十进制值为按权展开:

$$N_{10} = \sum_{i=-m}^{n-1} d_i \times R^i, \quad d_i \in \{0, 1, \dots, R-1\}.$$

常见进制：

- 二进制 (Binary, B): $R = 2$, 数码 $\{0, 1\}$, 后缀 B。
- 八进制 (Octal, O): $R = 8$, 数码 $\{0, 1, \dots, 7\}$, 后缀 O 或前缀 0。
- 十六进制 (Hex, H): $R = 16$, 数码 $\{0, 1, \dots, 9, A, B, C, D, E, F\}$, 后缀 H 或前缀 0x。
- 十进制 (Decimal, D): $R = 10$ 。

结论 3.1: 进制转换方法

任意进制 $\rightarrow$ 十进制：按权展开求和。

十进制 $\rightarrow R$ 进制：

- 整数部分：除 R 取余，逆序排列（先得低位）；
- 小数部分：乘 R 取整，顺序排列（先得高位）。小数部分可能无限循环，此时需指定精度。

二/八/十六进制互转：二进制是桥梁。

- 二进制 $\rightarrow$ 八进制：从小数点向两侧，每 3 位一组转换为八进制数码（不足补 0）。
- 二进制 $\rightarrow$ 十六进制：从小数点向两侧，每 4 位一组转换为十六进制数码。
- 八/十六进制 $\rightarrow$ 二进制：每位拆成 3/4 位二进制即可。

例 $10110111.1011\text{B} = (10\ 110\ 111.101\ 100)_2 = 267.54\text{O}$ （分组时小数部分右侧补两个 0）；同理 $= (1011\ 0111.1011)_2 = \text{B7.BH}$ 。

注意

408 常考点：小数部分除基取余/乘基取整的方向不能混淆——整数除基取余「逆序」，小数乘基取整「顺序」。此外八进制/十六进制与二进制的分组转换是快速解题的核心技巧。

3.2 数据编码、存放与校验**结论 3.2: 字符、BCD 与字节序**

- **ASCII:** 标准 ASCII 使用 7 位编码，共 128 个码点；计算机中通常占 1 字节。数字字符 '0'-'9' 连续编码，大小写字母也分别连续编码。
- **BCD:** 用 4 位二进制表示一位十进制数。8421 BCD 中 59_{10} 编码为 0101 1001，不是二进制真值 111011_2 。
- **大端方式：**多字节数据的最高有效字节放在低地址；**小端方式：**最低有效字节放在低地址。设 32 位数 $0x12345678$ 从地址 a 开始存放，大端的 a 单元为 12H，小端为 78H。
- **边界对齐：**一个 n 字节数据通常从 n 的整数倍地址开始存放。对齐可减少访存次数，但结构体中可能产生填充字节；题目若明确“紧凑存放”，则不得自行加入填充。

结论 3.3: 奇偶校验与海明码

- **奇偶校验：**增加 1 位校验位，使码字中 1 的总数为偶数（偶校验）或奇数（奇校验）。能检测任意奇数位错误，不能定位错误，也不能保证检测出偶数位错误。
- **海明码：**若数据位为 k 、校验位为 r ，能够纠正 1 位错误的基本条件是

$$2^r \geq k + r + 1.$$

校验位通常放在编号 1, 2, 4, 8, ... 的位置。各校验组重新计算得到的校验结果组成故障字 (syndrome); 故障字为 0 表示未检出错误, 非 0 时其数值给出出错位编号。

- 仅满足上式的普通海明码最小距离为 3, 可纠正 1 位错; 再增加 1 位全局奇偶校验位可构成 SEC-DED 码, 实现“纠正 1 位、检测 2 位”。

注意

数据的机器表示必须区分三件事: 数值编码 (补码/IEEE 754)、字节在内存中的排列 (大小端) 和数据对象的起始地址约束 (边界对齐)。大小端只改变字节次序, 不改变一个字节内各位的顺序。

3.3 机器数的表示

定义 3.2: 真值与机器数

真值是数学上的实际数值 (带正负号); 机器数是将符号数值化后在计算机中的编码表示。通常约定最高位为符号位: 0 表示正, 1 表示负。机器数位数固定 (如 8 位、16 位、32 位), 超出范围即溢出。

原码 (Sign-Magnitude)

要点 3.1: 原码的定义

设机器字长 $n+1$ 位 (含符号位), 真值 x :

- 定点整数:

$$[x]_{\text{原}} = \begin{cases} x, & 0 \leq x < 2^n \\ 2^n - x = 2^n + |x|, & -2^n < x \leq 0 \end{cases}$$

- 定点小数:

$$[x]_{\text{原}} = \begin{cases} x, & 0 \leq x < 1 \\ 1 - x, & -1 < x \leq 0 \end{cases}$$

结论 3.4: 原码的特点

- 表示范围 ($n+1$ 位, 含符号位): 整数 $[-(2^n - 1), 2^n - 1]$, 小数 $[-(1 - 2^{-n}), 1 - 2^{-n}]$ 。
- 零的表示不唯一: $[+0]_{\text{原}} = \underbrace{000\dots0}_{n\uparrow}$, $[-0]_{\text{原}} = \underbrace{100\dots0}_{n\uparrow}$ 。
- 优点: 与真值对应直观, 乘除时符号位单独处理。
- 缺点: 加减法需判断符号和绝对值大小, 电路复杂。

反码 (Ones' Complement)

要点 3.2: 反码的定义

设机器字长 $n + 1$ 位, 真值 x :

- 定点整数:

$$[x]_{\text{反}} = \begin{cases} x, & 0 \leq x < 2^n \\ (2^{n+1} - 1) + x, & -2^n < x \leq 0 \end{cases}$$

- 定点小数:

$$[x]_{\text{反}} = \begin{cases} x, & 0 \leq x < 1 \\ (2 - 2^{-n}) + x, & -1 < x \leq 0 \end{cases}$$

直观规则: 正数反码与原码相同; 负数反码 = 符号位保持 1, 其余各位按位取反。

注意

反码的零也有两种表示: $[+0]_{\text{反}} = 00 \dots 0$, $[-0]_{\text{反}} = 11 \dots 1$ 。反码在现代计算机中已不直接使用, 但其作为补码的中间过渡具有理论价值, 且 TCP/IP 校验和仍用反码加法。

补码 (Two's Complement)

定义 3.3: 补码的定义

设机器字长 $n + 1$ 位, 真值 x :

- 定点整数:

$$[x]_{\text{补}} = \begin{cases} x, & 0 \leq x < 2^n \\ 2^{n+1} + x, & -2^n \leq x < 0 \end{cases} \pmod{2^{n+1}}$$

- 定点小数:

$$[x]_{\text{补}} = \begin{cases} x, & 0 \leq x < 1 \\ 2 + x, & -1 \leq x < 0 \end{cases} \pmod{2}$$

直观规则: 正数补码 = 原码; 负数补码 = 反码末位 + 1 (即「按位取反 + 1」)。

结论 3.5: 补码的关键性质 (408 高频)

1. 零唯一: $[+0]_{\text{补}} = [-0]_{\text{补}} = 00 \dots 0$ 。
2. 表示范围不对称 ($n + 1$ 位): 整数 $[-2^n, 2^n - 1]$, 小数 $[-1, 1 - 2^{-n}]$ 。注意 -2^n (或 -1) 的补码就是其自身 $\underbrace{100 \dots 0}_{n \text{ 个}}$, 没有对应的原码。
3. 符号位参与运算: 补码加减法符号位与数值位统一处理, 结果是模 2^{n+1} (或模 2) 意义下的正确结果。
4. 由补码求真值: 符号位为 0 → 正数, 直接按权展开; 符号位为 1 → 负数, 再次求补 (取反 + 1) 后加负号。
5. 扩展: 正数高位补 0, 负数高位补 1 (符号扩展)。

注意

408 高频： $n + 1$ 位补码整数的表示范围为 $[-2^n, 2^n - 1]$ ，比原码/反码多一个最小负数。例如 8 位补码范围 $[-128, 127]$ 。这也是 C 中 `abs(INT_MIN)` 无法得到可表示正值的根本原因。

移码 (Biased Notation)

要点 3.3: 移码的定义

设机器字长 $n + 1$ 位，真值 x ：

$$[x]_{\text{移}} = 2^n + x, \quad -2^n \leq x < 2^n.$$

直观规则：将真值沿数轴平移 2^n （偏置量），使得所有编码变为非负数，数值大小关系与编码大小关系一致。

移码与补码的关系：符号位取反即为移码（ $[x]_{\text{移}} = [x]_{\text{补}}$ 的符号位翻转）。表示范围与补码相同，但零唯一： $[+0]_{\text{移}} = [-0]_{\text{移}} = 100 \dots 0$ 。

结论 3.6: 四种机器数速查表

设字长 $n + 1 = 5$ 位（1 位符号 + 4 位数值），整数表示：

四种机器数对比（5 位字长）					
真值（十进制）	真值（二进制）	原码	反码	补码	移码
+7	+0111	0,0111	0,0111	0,0111	1,0111
+0	+0000	0,0000	0,0000	0,0000	1,0000
-0	-0000	1,0000	1,1111	0,0000	1,0000
-7	-0111	1,0111	1,1000	1,1001	0,1001
-8	-1000	—	—	1,1000	0,1000

（逗号仅用于分隔符号位和数值位，实际存储中无此分隔。）

3.4 移位运算

结论 3.7: 逻辑移位与算术移位

- 逻辑移位：移出的位丢弃，空位一律补 0，通常用于无符号数。
- 算术左移：低位补 0；在不溢出的前提下相当于乘 2。若移位前后的有效符号特征被破坏，则发生溢出。
- 算术右移：高位补原符号位，近似相当于除 2；对负数舍入方向取决于机器和语言规定，不能简单等同于向 0 截断。
- 循环移位：移出的位回填到另一端；带进位循环移位还把进位标志位纳入循环链。

补码算术右移的核心：正数高位补 0，负数高位补 1；逻辑右移则无论符号都补 0。

3.5 定点数

定义 3.4: 定点表示法

小数点位置固定不变的表示法。分为：

- 定点整数：小数点固定在最低位之后（纯整数）。
- 定点小数：小数点固定在符号位之后、数值位之前（纯小数，绝对值 < 1 ）。

二者在硬件上无本质区别——同一套加法器、同一套规则，仅是编程者对小数的位置约定不同。

结论 3.8: $n + 1$ 位定点数的补码表示范围

定点数补码表示范围

类型	最小值	最大值
定点整数 ($n + 1$ 位补码)	-2^n	$2^n - 1$
定点小数 ($n + 1$ 位补码)	-1	$1 - 2^{-n}$

例：16 位补码定点整数范围 $[-32768, 32767]$ ；32 位补码定点小数精度为 2^{-31} 。

注意

现代通用计算机多采用浮点表示，定点主要用于嵌入式 DSP 和无 FPU 的微控制器。408 考试中定点数是浮点数理解的基础。

3.6 浮点数——IEEE 754 标准

浮点数的一般形式

定义 3.5: 浮点数的数学形式

任意实数 N 可表示为：

$$N = (-1)^S \times M \times R^E,$$

其中 S 为数符 (sign)， M 为尾数 (mantissa/significand)， R 为基数 (radix)， E 为阶码 (exponent)。

IEEE 754 标准规定 $R = 2$ ，并约定尾数为纯小数（规格化时 $1 \leq |M| < 2$ ）。

IEEE 754 格式

要点 3.4: IEEE 754 单精度与双精度格式

单精度 (float, 32 位) 和双精度 (double, 64 位) 的位分配：

S(1)	E(8)	M(23)	(单精度, 总 32 位)
S(1)	E(11)	M(52)	(双精度, 总 64 位)

- 数符 S : 0 正 1 负。
- 阶码 E : 用移码表示, 偏置值 (bias) $= 2^{k-1} - 1$ (k 为阶码位数)。单精度 $\text{bias} = 127$, 双精度 $\text{bias} = 1023$ 。真指数 $e = E - \text{bias}$ 。
- 尾数 M : 存储的是尾数的小数部分。规格化数有隐藏位: 整数部分隐含为 1 (即尾数真值 $= 1.M$), 故 23/52 位小数部分实际表示 24/53 位精度。

数值公式 (规格化数):

$$N = (-1)^S \times (1.M) \times 2^{E-\text{bias}}.$$

结论 3.9: IEEE 754 特殊值编码

阶码全 0 或全 1 用于表示特殊值:

IEEE 754 特殊值

类型	阶码 E	尾数 M	表示的数值
± 0	全 0	全 0	$(-1)^S \times 0$
非规格化数	全 0	$\neq 0$	$(-1)^S \times (0.M) \times 2^{1-\text{bias}}$
规格化数	$1 \sim 2^k - 2$	任意	$(-1)^S \times (1.M) \times 2^{E-\text{bias}}$
$\pm \infty$	全 1	全 0	$(-1)^S \times \infty$
NaN	全 1	$\neq 0$	Not a Number

说明:

- 非规格化数用于表示绝对值极小的数, 填补 0 与最小规格化数之间的间隙 (gradual underflow)。此时隐藏位变为 0, 指数固定为 $1 - \text{bias}$ 。
- 单精度规格化数的实际指数范围: $[-126, +127]$; 双精度: $[-1022, +1023]$ 。
- NaN 分为 quiet NaN (不触发异常) 和 signaling NaN (触发异常)。

注意

408 核心考点——IEEE 754 真值与编码互转:

1. 真值 $\rightarrow$ 编码: (1) 转为二进制科学计数法 $(-1)^S \times (1.M) \times 2^e$; (2) $E = e + \text{bias}$; (3) 依次填入 S 、 E 的二进制、 M 的小数部分 (高位补 0 至指定位数)。
2. 编码 $\rightarrow$ 真值: (1) 分离 S 、 E 、 M ; (2) 判断是否特殊值; (3) 规格化数用 $(-1)^S \times (1.M) \times 2^{E-\text{bias}}$ 计算。

例 将 -6.625 表示为 IEEE 754 单精度浮点数 (十六进制)。

- $6.625 = 110.101_2 = 1.10101 \times 2^2$, $S = 1$ 。
- $e = 2$, $E = 2 + 127 = 129 = 1000\ 0001_2$ 。
- M (23 位小数) $= 10101\ 00000\ 00000\ 00000\ 000$ 。
- 拼接: $1\ 10000001\ 101010000000000000000000 \rightarrow \text{C0D40000H}$ 。

3.7 定点运算

补码加减法

要点 3.5: 补码加减法基本公式

设 $[x]_{\text{补}}$ 和 $[y]_{\text{补}}$, 则:

$$\begin{aligned}[x + y]_{\text{补}} &= [x]_{\text{补}} + [y]_{\text{补}} \pmod{2^{n+1}} \\ [x - y]_{\text{补}} &= [x]_{\text{补}} + [-y]_{\text{补}} \pmod{2^{n+1}}\end{aligned}$$

其中 $[-y]_{\text{补}}$ 的求法: 将 $[y]_{\text{补}}$ 连同符号位在内按位取反, 末位 +1。

结论 3.10: 补码加减法溢出判断的三种方法

当两个同号数相加 (或异号数相减) 结果符号与操作数符号不同时, 发生溢出。408 考三种判断方法:

方法一: 单符号位 (进位判断法)

$$\text{溢出} = C_n \oplus C_{n-1}$$

即符号位进位 C_n 与最高数值位进位 C_{n-1} 异或。 $C_n \neq C_{n-1}$ 时溢出。

方法二: 双符号位 (变形补码) 用两位符号位, 正数符号 00, 负数符号 11。运算结果:

- 符号位为 00 或 11: 无溢出, 正常结果;
- 符号位为 01: 正溢出 (结果 > 最大正数);
- 符号位为 10: 负溢出 (结果 < 最小负数)。

即两符号位不同则溢出。

方法三: 操作数符号判断法 设 S_x, S_y, S_r 分别为 $[x]_{\text{补}}, [y]_{\text{补}}$, 结果的符号位:

$$\text{溢出} = (\overline{S_x} \cdot \overline{S_y} \cdot S_r) + (S_x \cdot S_y \cdot \overline{S_r})$$

含义: 两正相加得负 (正溢), 两负相加得正 (负溢)。

例 8 位补码, $(-100) + (-90)$: $[x]_{\text{补}} = 10011100$, $[y]_{\text{补}} = 10100110$, 相加得 01000010 (正)。
 $C_7 = 1, C_6 = 0, 1 \oplus 0 = 1$, 溢出。—— -190 超出 $[-128, 127]$ 。

注意

408 高频陷阱: $C_n \oplus C_{n-1}$ 方法中 C_n 是符号位的进位输出, C_{n-1} 是最高数值位向符号位的进位, 顺序不能搞反。另外, 无符号数加法用 C_n (即进位标志 CF) 判断溢出, 有符号数才用 $C_n \oplus C_{n-1}$ (即溢出标志 OF)。

原码乘法

结论 3.11: 原码一位乘法

设 $[x]_{\text{原}} = x_s \cdot x_1 x_2 \dots x_n$, $[y]_{\text{原}} = y_s \cdot y_1 y_2 \dots y_n$:

1. 符号位单独处理: $P_s = x_s \oplus y_s$ (异或)。
2. 数值部分做无符号乘法。每次看乘数最低位 y_n :

- $y_n = 1$: 部分积 $+|x|$, 然后右移一位;
- $y_n = 0$: 部分积 $+0$, 然后右移一位。

共 n 次加法、 n 次移位。

3. 最终乘积 $= P_s$ 拼接数值积 ($2n$ 位)。

```
// 原码一位乘——演示原理 (n位)
1 unsigned int TrueMul(unsigned int x, unsigned int y, int n) {
2     unsigned int acc = 0;           // 部分积 (高位)
3     for (int i = 0; i < n; i++) {
4         if (y & 1) acc += x;         // 乘数最低位=1, 加被乘数
5         y >>= 1;                     // 乘数右移
6         // 部分积右移, 最低位进入乘数寄存器的空位
7         y |= (acc & 1) << (n - 1);
8         acc >>= 1;
9     }
10    return (acc << n) | y;           // 拼接 2n 位积
11 }
12 }
```

补码乘法——Booth 算法

结论 3.12: Booth 算法 (补码一位乘)

设 $[x]_{\text{补}}$ (被乘数) 和 $[y]_{\text{补}}$ (乘数), Booth 算法一次处理乘数的一位, 根据当前位 y_i 和上一位 y_{i-1} (初始 $y_{-1} = 0$) 的差值决定操作:

Booth 算法操作表

y_i	y_{i-1}	操作
0	0	部分积 $+0$, 右移一位
0	1	部分积 $+ [x]_{\text{补}}$, 右移一位
1	0	部分积 $+ [-x]_{\text{补}}$, 右移一位
1	1	部分积 $+0$, 右移一位

算法共进行 n 个迭代周期; 每个周期根据 (Q_0, Q_{-1}) 决定加、减或不操作, 然后对 (A, Q, Q_{-1}) 做一次算术右移。加、减并非每轮都发生。

核心思想: 将乘数中连续的 1 串转化为一次减法和一次加法。011...10 的处理: 首位 1 做减法, 末位后做加法。 n 位补码乘法得到 $2n$ 位积 (含符号扩展)。

例 $[x]_{\text{补}} = 1101 (-3)$, $[y]_{\text{补}} = 0011 (+3)$, 4 位 Booth (结果 8 位):

- 初值: $A = 0000$, $Q = 0011$, $Q_{-1} = 0$ 。
- 第 1 步 ($Q_0 Q_{-1} = 10$): $A \leftarrow A + [-x]_{\text{补}} = 0000 + 0011 = 0011$, 算术右移 $\rightarrow A = 0001$, $Q = 1001$, $Q_{-1} = 1$ 。
- 第 2 步 ($Q_0 Q_{-1} = 11$): $A \leftarrow A + 0 = 0001$, 算术右移 $\rightarrow A = 0000$, $Q = 1100$, $Q_{-1} = 1$ 。
- 第 3 步 ($Q_0 Q_{-1} = 01$): $A \leftarrow A + [x]_{\text{补}} = 0000 + 1101 = 1101$, 算术右移 $\rightarrow A = 1110$, $Q = 1110$, $Q_{-1} = 0$ 。
- 第 4 步 ($Q_0 Q_{-1} = 00$): $A \leftarrow A + 0 = 1110$, 算术右移 $\rightarrow A = 1111$, $Q = 0111$, $Q_{-1} = 0$ 。

- 结果 $A:Q = 11110111$ ，即 -9 的 8 位补码。 $(-3) \times 3 = -9$ ，正确。

注意

Booth 算法的优势在于：当乘数中连续 1 较多时减少加法次数。但 408 主要考察手工模拟过程，理解「当前位与上一位比较」的查表逻辑即可。

原码除法——加减交替法

结论 3.13: 原码除法（不恢复余数法/加减交替法）

设 x, y 为被除数和除数的无符号数值部分，且 $y \neq 0$ ； n 为商的位数。符号位单独处理，以下只说明数值部分：

1. 初始余数 $R = 0$ ，按从被除数最高位到最低位的顺序逐位移入。
2. 第 i 次迭代（从高位到低位）：先执行 $R \leftarrow 2R + x_i$ ，其中 x_i 是当前移入的被除数位；若原余数非负则减去 y ，若原余数为负则加上 y 。
3. 根据本次运算后的余数符号确定商位：余数非负时商位置 1，否则置 0。
 - 原余数 $R \geq 0$ ： $R \leftarrow R - y$ ；
 - 原余数 $R < 0$ ： $R \leftarrow R + y$ 。
4. 最后若余数为负，做一次恢复： $R \leftarrow R + y$ 。

核心：上一轮余数非负，本轮移位后减除数；上一轮余数为负，本轮移位后加除数。再按本轮结果的符号确定商位，从而避免每轮都先恢复负余数。

```

1 // 原码加减交替除法（无符号数值部分， $n$  位商）
2 #include <stdbool.h>
3 #include <stdint.h>
4
5 // 成功返回 true；失败时不写输出参数。
6 bool TrueDiv(uint32_t x, uint32_t y, unsigned int n,
7             uint32_t *quot, uint32_t *rem) {
8     if (n == 0 || n > 32 || y == 0 || !quot || !rem) {
9         return false;
10    }
11    uint64_t limit = UINT64_C(1) << n;
12    if ((uint64_t)x >= limit || (uint64_t)y >= limit) {
13        return false; //  $x, y$  必须能用  $n$  位表示
14    }
15
16    int64_t R = 0; // 余数允许为负
17    uint32_t Q = 0;
18    for (int i = (int)n - 1; i >= 0; --i) {
19        bool was_nonnegative = (R >= 0);
20        uint32_t bit = (x >> (unsigned int)i) & UINT32_C(1);
21        R = 2 * R + (int64_t)bit; // 不左移负的有符号整数
22        if (was_nonnegative) {
23            R -= (int64_t)y;
24        } else {
25            R += (int64_t)y;

```

```

26     }
27
28     Q <= 1;
29     if (R >= 0) {
30         Q |= UINT32_C(1);
31     }
32 }
33 if (R < 0) {
34     R += (int64_t)y;           // 最后恢复
35 }
36 *quot = Q;
37 *rem = (uint32_t)R;
38 return true;
39 }

```

接口约定 $1 \leq n \leq 32$ 、 $y \neq 0$ ，且 x, y 都能用 n 位无符号数表示；仅在返回 `true` 时读取输出。迭代后有 $-y \leq R < y$ ，故上述 32 位输入模型中的 $2 * R + \text{bit}$ 可由 `int64_t` 安全表示。

注意

408 常考对比：原码恢复余数法 vs 不恢复余数法。前者每次减去除数，若余数为负则加回（恢复），再上商 0——恢复步骤多一次加法。后者将负余数按算法左移一位（数值上乘 2）后加除数，省去逐轮恢复操作，速度更快；这不表示 C 程序可以对负的有符号整数直接使用左移运算符。

3.8 浮点运算

结论 3.14: 浮点数加减法步骤

设两浮点数 $X = M_x \times 2^{E_x}$ ， $Y = M_y \times 2^{E_y}$ （尾数均含隐藏位，已规格化）。加减法五步：

步骤 1：对阶 (Exponent Alignment)「小阶向大阶看齐」：比较 E_x 与 E_y 。设 $\Delta E = |E_x - E_y|$ ，将阶码较小的数的尾数右移 ΔE 位（算术右移），阶码增加到与大阶一致。每一步右移会丢失低位，产生舍入误差。

步骤 2：尾数加减对阶后两数阶码相同，直接对尾数做有符号加减： $M \leftarrow M_x \pm M_y$ 。

步骤 3：结果规格化加减后尾数 M 可能不满足 $1 \leq |M| < 2$ ：

- 若 $|M| \geq 2$ （溢出）：右规——尾数右移一位，阶码 +1；
- 若 $|M| < 1$ （非规格化）：左规——尾数左移直至最高位为 1，阶码同步减小。可能需要多次左规。

步骤 4：舍入 (Rounding) 右规或对阶过程中，被移出的低位需处理。IEEE 754 规定四种舍入模式（见下表），默认就近舍入到偶数（Round to Nearest Even）。

步骤 5：溢出判断规格化后检查阶码：

- 阶码 ≥ 255 （单精度）/ ≥ 2047 （双精度）：上溢（overflow），置 $\pm\infty$ ；
- 阶码 ≤ 0 （不满足规格化范围）：下溢（underflow），结果可能为非规格化数或 0。

要点 3.6: IEEE 754 四种舍入模式

舍入模式

模式	规则
就近舍入 (Ties to Even)	舍入到最近的可表示值; 恰在中间时取最低有效位为 0 (偶数)
向 $+\infty$ 舍入	始终向上舍入 (ceil)
向 $-\infty$ 舍入	始终向下舍入 (floor)
向 0 舍入	截断 (truncate)

默认模式为就近舍入到偶数。设置保护位 (guard bit)、舍入位 (round bit) 和粘滞位 (sticky bit) 辅助舍入决策。

注意

408 高频考点: 浮点数加减五步骤。选择题常给两个 IEEE 754 编码, 要求写出加减结果编码。必须熟练掌握: (1) 分离阶码与尾数并补隐藏位, (2) 对阶时小阶对大阶, (3) 右规/左规的条件, (4) 溢出判断标准。阶码全 1 或全 0 归为特殊值处理。

3.9 ALU 基本结构

定义 3.6: 算术逻辑单元 (ALU)

ALU 是 CPU 中执行算术运算 (加、减、乘、除) 和逻辑运算 (与、或、非、异或) 的核心部件。ALU 的核心是加法器, 减法通过补码加法实现, 乘法/除法可分解为多次加法和移位。

结论 3.15: 加法器结构

一位全加器 (Full Adder, FA):

$$\text{Sum}_i = A_i \oplus B_i \oplus C_i, \quad C_{i+1} = A_i B_i + (A_i \oplus B_i) C_i.$$

其中 C_i 为进位输入, C_{i+1} 为进位输出。

串行进位加法器 (Ripple-Carry Adder): 将 n 个 FA 级联, 低位进位输出接高位进位输入。结构简单但延迟 $O(n)$ ——最坏情况下进位从最低位传到最高位。

先行进位加法器 (Carry Lookahead Adder, CLA): 定义进位生成 $G_i = A_i B_i$, 进位传播 $P_i = A_i \oplus B_i$ (或 $A_i + B_i$), 则:

$$C_{i+1} = G_i + P_i C_i.$$

展开得:

$$\begin{aligned} C_1 &= G_0 + P_0 C_0, \\ C_2 &= G_1 + P_1 G_0 + P_1 P_0 C_0, \\ C_3 &= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0, \quad \dots \end{aligned}$$

所有进位可并行计算, 延迟 $O(\log n)$ 。但门电路扇入随位数增长, 实际采用分组并行 (如 4 位一组 CLA, 组间串行或再加一级 CLA)。

ALU 的整体结构：

- 核心： n 位加法器（通常为 CLA 或其变体）。
- 前端：多路选择器根据控制信号选择操作数（及是否取反以做减法）。
- 输出：结果、进位标志（CF）、溢出标志（OF）、零标志（ZF）、符号标志（SF）。
- 逻辑运算：按位与/或/非/异或等通常通过旁路加法器实现。

注意

408 对 ALU 的考察通常以「SN74181（4 位 ALU 芯片）」「先行进位原理」「标志位的产生逻辑」为主。重点理解进位链中 G_i 和 P_i 的含义，以及 CF 和 OF 的区别。

3.10 基础过关题

1. (基础题) 32 位数 $0x12345678$ 从地址 $0x1000$ 开始按字节存放。若 $0x1000$ 单元的内容为 $78H$ ，则机器采用大端还是小端方式？
2. (基础题) 8 位补码 10010110 算术右移一位和逻辑右移一位的结果分别是什么？
3. (基础题) 采用偶校验时，数据位 1011001 的校验位应取何值？若传输后有两位同时翻转，偶校验是否一定能检出？
4. (基础题) 对 16 位数据构造可纠正一位错误的海明码，至少需要多少个校验位？
5. (真题风格·选择题) float 型数据通常用 IEEE 754 单精度浮点数格式表示。若编译器将 float 型变量 x 分配在一个 32 位浮点寄存器 FR1 中，且 $x = -8.25$ ，则 FR1 的内容是（ ）。
(A) C104 0000H
(B) C242 0000H
(C) C184 0000H
(D) C1C2 0000H
6. (真题风格·选择题) 32 位补码所能表示的整数范围是（ ）。
(A) $-2^{32} \sim 2^{31} - 1$
(B) $-2^{31} \sim 2^{31} - 1$
(C) $-2^{32} \sim 2^{32} - 1$
(D) $-2^{31} \sim 2^{32} - 1$
7. (真题风格·选择题) 若 short 型变量 $x = -8190$ ，则 x 的机器数为（ ）。
(A) E002H
(B) E001H
(C) 9FFFH
(D) 9FFE H
8. (真题风格·选择题) IEEE 754 单精度浮点格式表示的数中，最小的规格化正数是（ ）。

- (A) 1.0×2^{-126}
- (B) 1.0×2^{-127}
- (C) 1.0×2^{-128}
- (D) 1.0×2^{-149}

9. (真题风格·选择题) -0.4375 的 IEEE 754 单精度浮点数表示为 ()。

- (A) BEE0 0000H
- (B) BF60 0000H
- (C) BF70 0000H
- (D) C0E0 0000H

10. (真题风格·选择题) float 类型 (即 IEEE 754 单精度浮点数格式) 能表示的最大正整数是 ()。

- (A) $2^{127} - 2^{104}$
- (B) $2^{127} - 2^{103}$
- (C) $2^{128} - 2^{104}$
- (D) $2^{128} - 2^{103}$

11. (真题风格·选择题) 某 IEEE 754 单精度浮点数的机器码为 42E4 8000H, 则它的真值为 ()。

- (A) 114.25
- (B) 57.125
- (C) 228.5
- (D) 113.25

12. (真题风格·选择题) 浮点数加减运算过程中, 「对阶」操作的原则是 ()。

- (A) 将较小的阶码对齐到较大的阶码
- (B) 将较大的阶码对齐到较小的阶码
- (C) 将被加 (减) 数的阶码对齐到加 (减) 数的阶码
- (D) 无须对阶

13. (真题风格·选择题) 假定变量 i 、 f 和 d 的数据类型分别为 int、float 和 double (int 用补码表示, float 和 double 分别用 IEEE 754 单精度和双精度浮点数格式表示), 已知 $i = 785$, $f = 1.5678e3$, $d = 1.5e100$ 。若在 32 位机器中执行下列关系表达式, 则结果为「真」的是 ()。

- I. $i == (\text{int})(\text{float})i$
- II. $f == (\text{float})(\text{int})f$
- III. $f == (\text{float})(\text{double})f$
- IV. $(d + f) - d == f$

- (A) 仅 I 和 II
- (B) 仅 I 和 III
- (C) 仅 II 和 III
- (D) 仅 III 和 IV

14. (真题风格·选择题) 下列有关 IEEE 754 浮点数的叙述中, 正确的是 ()。

- (A) 规格化数的尾数部分隐含了整数位 1, 故 23 位尾数实际实现了 24 位精度
- (B) 非规格化数的阶码为全 0, 隐藏位为 1, 用于表示接近 0 的极小值
- (C) IEEE 754 单精度浮点数的阶码采用补码表示, 偏置值为 128
- (D) 当阶码全 1 且尾数全 0 时, 表示的数与符号位 S 无关

3.11 真题风格综合训练

1. (真题风格·选择题) 已知带符号整数用补码表示, float 型数据用 IEEE 754 标准表示, 假定变量 x 的类型只能是 int 或 float。当 x 的机器数为 C800 0000H 时, x 的值可能是 ()。
 - (A) -7×2^{27}
 - (B) -2^{126}
 - (C) -2^{27}
 - (D) -7×2^{25}
2. (真题风格·选择题) 已知 float 型变量用 IEEE 754 单精度浮点数格式表示。若 float 型变量 x 的机器数为 4730 0000H, 则 x 的值为 ()。
 - (A) 0.375×2^{14}
 - (B) 1.375×2^{14}
 - (C) 0.375×2^{15}
 - (D) 1.375×2^{15}
3. (真题风格·选择题) 假设 8 位字长的计算机中, 两个带符号整数 x 和 y 的补码表示分别为 $[x]_{\text{补}} = 6AH$, $[y]_{\text{补}} = 9FH$ 。则 $x + y$ 的结果和溢出情况是 ()。
 - (A) 结果为 09H, 无溢出
 - (B) 结果为 09H, 有溢出
 - (C) 结果为 F5H, 无溢出
 - (D) 结果为 F5H, 有溢出
4. (综合)IEEE 754 浮点数编解码。已知 IEEE 754 单精度浮点数 X 的十六进制表示为 C2280000H, Y 的十六进制表示为 42A40000H。
 - (a) 分别写出 X 和 Y 的十进制真值;
 - (b) 按照 IEEE 754 浮点加减法步骤, 计算 $X + Y$ (写出对阶、尾数加减、规格化、舍入过程), 并给出最终结果的十六进制编码。
5. (综合)补码运算与溢出判断。设字长 8 位 (含符号位), $x = -85$, $y = -64$ 。
 - (a) 写出 x 和 y 的补码;
 - (b) 用补码加法计算 $x + y$, 分别用进位异或法、双符号位法和操作数符号法判断是否溢出;
 - (c) 用补码加法计算 $x - y$, 判断是否溢出。
6. (综合)四种机器数对比。设某机器字长 6 位 (含 1 位符号位), 给出下列真值分别对应的原码、反码、补码和移码 (采用 6 位二进制表示)。

- (a) +23
- (b) -15
- (c) -0 (若存在)

并回答：(1) 原码中为何 +0 和 -0 编码不同？(2) 补码的最小负数为何比原码多一个？

3.12 拓展阅读

- **Patterson & Hennessy**《计算机组成与设计 (RISC-V 版)》第 3 章「计算机的算术运算」——3.2 加法和减法, 3.3 乘法 (Booth 算法), 3.4 除法 (恢复余数法和恢复余数法), 3.5 浮点数 (IEEE 754 标准的完整论述, 含舍入模式与精度分析), 可用于补充理解浮点运算。
- **Bryant & O'Hallaron**《深入理解计算机系统》(CS:APP) 第 2 章「信息的表示和处理」——2.1 信息存储, 2.2 整数表示 (补码编码的完整处理), 2.3 整数运算 (溢出判断的底层原理), 2.4 浮点数 (IEEE 754 细节与非规格化数)。CS:APP 的整数与浮点章节论述严密, 适合补充理解 P&H 中省略的编码细节。
- 唐朔飞《计算机组成原理》第 6 章「计算机的运算方法」——6.1 无符号数和有符号数, 6.2 定点运算, 6.3 浮点四则运算, 6.4 算术逻辑单元。不同版本的章节号和个别符号可能有差异, 复习时应与所用大纲和教材版本对照。
- 延伸思考: IEEE 754-2008 标准扩展了 16 位半精度(binary16)和 128 位四精度(binary128)格式, 本质与单/双精度一致。此外, 十进制浮点(decimal floating-point)格式用于金融计算, 避免二进制浮点的舍入偏差。感兴趣者可从 P&H 3.10 节(历史与扩展阅读)入手。

4 存储系统

4.1 存储器概述与分类

定义 4.1: 存储器的分类维度

从多个维度对存储器进行分类:

1. 按存储介质: 半导体存储器 (SRAM/DRAM/ROM/Flash)、磁表面存储器 (磁盘)、光存储器 (光盘)。
2. 按存取方式:
 - 随机存取 (**RAM**): 存取时间与物理位置无关, 访问任意单元时间相等。如 SRAM、DRAM。
 - 顺序存取 (**SAM**): 按物理顺序线性访问, 存取时间与位置相关。如磁带。
 - 直接存取 (**DAM**): 先大致定位到某个区域, 再在该区域内顺序查找——兼有随机和顺序的特点。如磁盘 (磁头先寻道, 再在磁道上顺序旋转)。
 - 相联存取 (**Associative**): 按内容而非地址访问, 给定数据的一部分匹配存储单元。如 Cache 中的 TAG 比较、TLB。
3. 按信息的可保存性:
 - 易失性 (Volatile): 断电后信息丢失, 如 SRAM、DRAM。

- 非易失性 (Non-volatile): 断电不丢失, 如 ROM、Flash、磁盘。
4. 按在计算机中的位置: 寄存器 (CPU 内部)、主存 (内存)、辅存 (外存)。

注意

- 408 常考区分:
- 「随机存取」≠「RAM」。ROM 也是随机存取的 (可按地址直接访问), 但它是非易失性的, 不属于 RAM。
 - 磁盘是「直接存取」, 不是随机存取——因为寻道和旋转延迟使不同位置的访问时间不同。
 - 相联存储器按内容检索, 不是按地址; Cache 的 TAG 比较即相联查找。

结论 4.1: 半导体存储器分类

半导体存储器分类			
类型	典型器件	读写特性	易失性
RAM (随机存取)	SRAM	可读可写	易失
	DRAM	可读可写	易失
ROM (只读存储器)	MROM	出厂固化的只读	非易失
	PROM	用户一次编程	非易失
	EPROM	紫外线擦除后可重写	非易失
	EEPROM	电擦除可重写	非易失
Flash	NAND/NOR Flash	块擦除、页编程	非易失

- Flash 的两种类型:
- **NOR Flash**: 支持按字节随机读, 读取速度快, 擦除慢, 适合代码存储 (如固件)。
 - **NAND Flash**: 按页读写, 按块擦除, 密度高, 适合大容量数据存储 (如 SSD、U 盘)。

4.2 存储系统的层次结构

定义 4.2: 存储层次 (Memory Hierarchy)

现代计算机采用多级存储结构, 从上到下容量递增、速度递减、每字节成本递减:

容量：最小

寄存器（CPU 内部）

访问速度：最快

较小

Cache（SRAM）

很快

较大

主存（DRAM）

较慢

最大

辅存（磁盘/SSD）

最慢

存储层次通过局部性在相邻层之间保存近期或常用数据：Cache 缓存主存中的块，虚拟存储系统可把主存看作外存中进程地址空间的缓存。这里的“缓存”是功能类比，并不意味着任意时刻都满足严格的物理包含关系；写回 Cache、非包含式多级 Cache 和寄存器状态都可能使简单的“上层必为下层子集”表述失效。

结论 4.2: 局部性原理 (Principle of Locality)

存储层次之所以有效，根本原因是程序的局部性原理（CS:APP §6.2）：

1. 时间局部性（Temporal Locality）：被引用过一次的存储位置在不久的将来很可能被再次引用。
 - 典型来源：循环中的指令和数据、临时变量、频繁调用的函数。
2. 空间局部性（Spatial Locality）：被引用过的存储位置的附近位置在不久的将来很可能被引用。
 - 典型来源：顺序执行的指令、数组的顺序访问、结构体字段。

「步长为 k 的引用模式」的局部性：若 $k = 1$ （连续访问）则空间局部性好； k 越大空间局部性越差；完全不规则的访问（如哈希表）局部性最差。

注意

408 高频考点：判断一段代码的局部性。关注：

- 数组访问是按行还是按列——C 语言按行存储，按行访问空间局部性好，按列访问则差。
- 循环体内的指令具有时间局部性（重复执行）。
- 函数递归调用：参数和返回地址有空间局部性（栈帧连续），递归调用本身时间局部性较弱（每次调用参数不同）。

结论 4.3: 命中率与平均访问时间

设 t_H 为上层命中访问时间。为避免“缺失总时间”和“缺失惩罚”两种口径混用，本文后续统一令 t_M 表示缺失惩罚（不含已经付出的上层命中检查时间）：

- 命中率 H ：访问上层命中的概率。
- 缺失率 $M = 1 - H$ 。
- 平均访问时间： $T_{\text{avg}} = t_H + (1 - H) \cdot t_M = t_H + M \cdot t_M$ 。

两级存储（Cache-主存）情况下， t_H 即 Cache 命中检查时间， t_M 为缺失后从主存取一整块到 Cache 并完成本次访问的额外代价。若题目把 t_M 定义为“缺失访问的总时间”（已经包含命中检查），则应改用 $T_{\text{avg}} = Ht_H + (1 - H)t_M$ ，不能与上式混用。

4.3 主存储器的基本结构

定义 4.3: 存储单元与寻址

主存由 M 个存储单元组成，每个单元有一个唯一的地址。常见的编址方式：

- 按字节编址：每个地址对应一个字节（8 位）。现代计算机普遍采用。
- 按字编址：每个地址对应一个字（如 32 位或 64 位）。

若地址线有 n 根，则可寻址 2^n 个存储单元。

存储芯片的关键参数：

- 地址线数 a ：确定片内寻址范围，共 2^a 个单元。
- 数据线数 d ：每个存储单元的位数（字长）。
- 容量： $2^a \times d$ 位。

存储芯片的扩展

结论 4.4: 位扩展

多片存储芯片的数据线并联，实现字长的扩展。

例：用 $16\text{K} \times 4$ 位芯片构成 $16\text{K} \times 8$ 位存储器。需 2 片，地址线和控制线并联，数据线分别接 D_0-D_3 和 D_4-D_7 。

位扩展后容量变大为原来的 k 倍，地址线数不变。

结论 4.5: 字扩展

地址线增加（高位地址通过译码器产生片选信号），实现寻址范围的扩展。

例：用 $16\text{K} \times 8$ 位芯片构成 $64\text{K} \times 8$ 位存储器。需 4 片。地址线增加 2 根（高位地址经 2-4 译码器产生 4 个片选信号 $\overline{CS}_0-\overline{CS}_3$ ），数据线并联即可。

字扩展后容量变大为原来的 k 倍，数据线数不变。

结论 4.6: 字位同时扩展

例：用 $16\text{K} \times 4$ 位芯片构成 $64\text{K} \times 8$ 位存储器。需 $4 \times 2 = 8$ 片。芯片每 2 片一组做位扩展（得到 $16\text{K} \times 8$ 位模块），4 组之间做字扩展。

设地址线共 $A_{15}-A_0$ （16 根， $2^{16} = 64\text{K}$ ），其中：

- $A_{13}-A_0$ （14 根）给每组芯片的片内地址（ $2^{14} = 16\text{K}$ ）；
- $A_{15}-A_{14}$ （2 根）经 2-4 译码器产生 4 个片选，分别选通 4 组。

注意

408 常考计算：

- 给定若干芯片和所需总容量，求所需的芯片数量和地址线分配方案。
- 注意区分「地址线数」和「数据线数」：位扩展改变数据线数，字扩展改变地址线数。
- 片选信号由高位地址经译码产生，低地址给片内寻址。区分「片内地址线」和「片选地址线」。

4.4 半导体存储器：SRAM 与 DRAM

定义 4.4: SRAM（静态随机存取存储器）

SRAM 的基本存储单元是六管 **CMOS** 双稳态触发器（6-T cell）：两个交叉耦合的反相器构成锁存器，4 个晶体管做读写控制。只要持续供电，数据就稳定保持。特点：

- 速度快：静态存储单元无需刷新，通常比 DRAM 延迟低；具体延迟取决于工艺、容量和层级，题目给出参数时应以题设为准。
- 功耗较高：每个 bit 需要 6 个晶体管持续耗电维持状态。
- 密度低：集成度远低于 DRAM，每 bit 成本高。
- 用途：CPU 内部的 Cache（L1/L2/L3），要求极低延迟。

定义 4.5: DRAM（动态随机存取存储器）

DRAM 的基本存储单元是单晶体管 + 一个微小电容（1T1C cell）：

- 数据以电荷形式存储在电容上——有电荷为「1」，无电荷为「0」。
- 电容会缓慢漏电，因此需要定期刷新（refresh）以保持数据。
- 刷新周期：器件规定的保持时间内必须完成对所有行的刷新；计算题通常直接给出总刷新周期 R 。

DRAM 的读操作是破坏性的——读出时会消耗电容上的电荷，读后必须立即重写（读后恢复）。刷新方式：

- 集中刷新：在一个刷新周期内留出一段专门时间，一次性对所有行刷新。这段期间内不能响应 CPU 访问（「死时间」），死时间 = 刷新行数 $\times$ 每行刷新时间。
- 分散刷新：把每个存储周期分成正常读写和刷新两个固定阶段，逐行刷新。没有集中的长“死时间”，但存储周期被拉长，刷新阶段不能进行正常访存。
- 异步刷新：将各行刷新均匀分散到题设给定的刷新周期 R 内，每隔 $R/(\text{行数})$ 左右刷新一行。它避免了集中刷新的长死时间，刷新开销也不像分散刷新那样占据每个存储周期。

结论 4.7: SRAM vs DRAM 对比 (408 重点)

SRAM 与 DRAM 对比		
特性	SRAM	DRAM
基本单元	六管触发器	单管 + 电容
是否需要刷新	否	是 (必须在规定保持时间内刷新)
读写速度	通常较快	通常较慢
集成度	低 (6 管/bit)	高 (1 管 +1 电容/bit)
每 bit 成本	高	低
功耗 (静态)	较高	较低 (但需刷新功耗)
破坏性读出	否	是
典型用途	Cache	主存

408 常见辨析:

- 「SRAM 速度快、成本高」→ 用于 Cache。
- 「DRAM 需要刷新」→ 这是 DRAM 的「动态」含义。
- 「DRAM 破坏性读出」→ 读后必须重写。
- 「SRAM 不需要刷新」→ 但不等于非易失——断电数据仍丢失。

注意

408 高频概念区分:

- 存取周期 (**Memory Cycle Time**): 两次独立存储器操作之间所需的最小间隔时间。DRAM 的存取周期 > 存取时间, 因为读后需要恢复。
- 存取时间 (**Access Time**): 从给出地址到数据出现在数据线上的时间。
- 带宽 (**Bandwidth**): 单位时间内传输的数据量 (字节/秒), $BW = \text{数据宽度} / \text{存取周期}$ 。

4.5 双口 RAM 与多模块存储器

定义 4.6: 双口 RAM (Dual-Port RAM)

双口 RAM 有两套独立的地址线、数据线和读写控制线, 允许两个设备同时访问不同的存储单元。若两个端口同时访问同一存储单元:

- 同时读: 允许。
- 一读一写或同时写: 冲突, 需仲裁逻辑处理 (写优先或读优先)。

Cache 中的多端口实现常基于双口 RAM。

多模块存储器

结论 4.8: 单体多字存储器

在单个存储周期内从同一模块读出多个字。优点是带宽高（一次取出多条指令/数据），缺点是：

- 需要更宽的数据通路（总线宽度增大）。
- 遇到转移指令时，多取出的指令可能被废弃，效率降低。
- 要求访问地址连续——若指令流发生跳转则效益丧失。

结论 4.9: 多体并行存储器——交叉编址

将主存分成多个独立的存储体（bank），各体有自己的地址寄存器和数据寄存器，可以重叠操作。关键是不同体之间的地址分配方式。

高位交叉编址（High-Order Interleaving）

用地址的高位选择存储体，低位作为体内地址。即连续的存储单元位于同一存储体中：

$$\text{体号} = \text{高位地址}, \quad \text{体内地址} = \text{低位地址}$$

特点：连续地址块通常集中在同一个存储体中，因此对单个连续地址流不能像低位交叉编址那样让相邻字在各体流水访问；它更适合让不同地址区域或不同访问流分布到不同存储体并行工作。

低位交叉编址（Low-Order Interleaving）

用地址的低位选择存储体，高位作为体内地址。即连续的存储单元分布在不同存储体上：

$$\text{体号} = \text{低位地址}, \quad \text{体内地址} = \text{高位地址}$$

特点：连续访存时可多个存储体流水线工作，大幅提高带宽，是主存储器主要的并行化技术。

带宽分析（408 高频计算）

设存储体数量为 m ，每个存储体的存取周期为 T （从给出地址到数据就绪的时间），数据宽度为 W 字节。

- 单体单字带宽： $BW = W/T$ 。
- m 体低位交叉流水：每间隔 T/m 启动一个体，数据连续输出。流水启动后，持续带宽为：

$$BW_{\text{interleave}} = \frac{m \cdot W}{T} \quad (\text{流水稳态})$$

即带宽提升 m 倍。

交叉编址对比

特征	高位交叉	低位交叉
体号由	高位地址	低位地址
连续单元分布	同一体内	不同体
流水线工作	否	是
提高单程序带宽	否	是
典型应用	多道程序	向量/流式访存

注意

408 高频计算——低位交叉编址存取的时机：设 $m = 4$ ，存取周期 T ，总线传输周期 τ （数据线上稳定数据所需的时间），则：

- 每隔 τ 可以启动一个存储体。
- 若 $\tau = T/4$ ，则在 T 时间内可以连续启动 4 个体，数据连续输出，带宽 $= 4W/T$ （理想 4 倍）。
- 若 $\tau > T/4$ ，则流水线会有气泡，带宽提升因子 $< m$ 。
- 408 真题常考：给定 T 、 τ 和 m ，求连续读取 n 个字所需时间。

计算公式：启动第一个体后，每隔 τ 启动下一个体。读取 n 个字的总时间为：

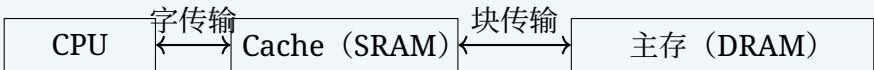
$$t = T + (n - 1) \times \max(\tau, T/m)$$

其中 T 为第一个体存取周期（流水线填充时间），后续每字以 $\max(\tau, T/m)$ 的间隔取出。

4.6 Cache 高速缓冲存储器

定义 4.7: Cache 的基本原理

Cache 位于 CPU 和主存之间，基于局部性原理：将当前最可能被访问的指令和数据放在 Cache 中，CPU 绝大部分访存请求由 Cache 满足。
Cache 按块（**Block / Cache Line**）为单位与主存交换数据。设 Cache 有 C 行，每行 B 字节（块大小），总容量 $C \times B$ 字节。



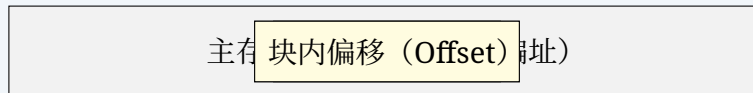
CPU 与 Cache 之间以字为单位传输（字长 = 机器字长），Cache 与主存之间以块为单位传输（块大小通常 32-128 字节）。

Cache 的地址映射

设主存地址位数为 n ，Cache 有 C 行，每行（块） B 字节。主存被分成 $2^n/B$ 个块，每个块与 Cache 中的一行对应。

结论 4.10: 直接映射 (Direct-Mapped Cache)

每个主存块只能映射到 Cache 中唯一的一行: $\text{Cache 行号} = (\text{主存块号}) \bmod C$ 。



- 块内偏移: $\log_2 B$ 位。确定数据在块内的具体字节位置。
- 索引 (**Index**): $\log_2 C$ 位。确定块映射到 Cache 中的哪一行。
- 标记 (**Tag**): 剩余高位。与 Cache 该行的 Tag 比较, 判断是否命中。

直接映射的优缺点:

- 硬件简单, 查找速度快 (只需比较一个 Tag)。
- 冲突缺失 (Conflict Miss) 率高——若两个频繁访问的块映射到同一行则互相颠簸 (thrashing)。

结论 4.11: 全相联映射 (Fully-Associative Cache)

每个主存块可以映射到 Cache 中的任意一行。

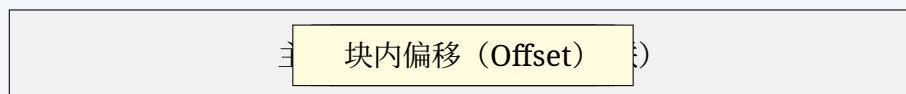
主存地址只分为标记 (**Tag**) 和块内偏移 (**Offset**) 两部分——没有 Index 字段。查找时需要将 Tag 与 Cache 中所有行并行比较 (相联查找)。

- 冲突缺失率最低 (只有容量缺失和强制缺失)。
- 硬件代价高——每行都要配备一个比较器, 大规模 Cache 中不可行。仅用于小容量 Cache (如 TLB)。

结论 4.12: 组相联映射 (Set-Associative Cache)

折中方案: 将 Cache 分成若干组 (Set), 每组内有 W 行 (W 路组相联)。每个主存块映射到唯一的一组, 但可以放入该组中的任意一行。

$$\text{组号} = (\text{主存块号}) \bmod S, \quad \text{其中 } S \text{ 为组数}$$



特殊情况:

- $W = 1$ 时: 就是直接映射 ($S = C$, 每组 1 行)。
- $W = C$ 时 ($S = 1$, 所有行属于同一组): 就是全相联映射。

地址位划分:

- 块内偏移: $\log_2 B$ 位。
- 组号: $\log_2 S = \log_2 (C/W)$ 位。
- 标记: $n - \log_2 S - \log_2 B$ 位。

注意

408 高频考点——Cache 地址位划分的计算：给定主存容量、Cache 容量、块大小和相联度，计算地址中 Tag / Index / Offset 各占多少位。步骤：

1. 确认主存总地址位数 N （按字节编址时 $N = \log_2(\text{主存容量})$ ）。
2. 块内偏移位数 $= \log_2(\text{块大小})$ 。
3. 计算组数 $S = \text{Cache 行数} / W$ ，组号位数 $= \log_2 S$ 。
4. Tag 位数 $= N - \log_2 S - \log_2(\text{块大小})$ 。

注意区分：「**Cache 总容量**」和「**Cache 数据区容量**」——前者还包含 **Tag** 和有效位等额外开销。

Cache 替换算法

当需要调入新块而组内（或全相联 Cache 中）无空闲行时，需要淘汰一个已有块。

结论 4.13: LRU (Least Recently Used) 替换算法

淘汰最近最少使用的块。基于时间局部性——最近用过的块更可能再次被使用。

硬件实现：

- 为每行维护一个计数器（或时间戳），每次命中或新装入时将计数器清零，其他行计数器加 1。计数器最大的行即为最近最少使用。
- W 路组相联时，每行需要 $\lceil \log_2 W \rceil$ 位的计数器。

LRU 能较好利用时间局部性，常作为教材中的基准替换算法。高相联度 Cache 中实现精确 LRU 的更新开销较大，实际硬件也常使用伪 LRU 或随机替换；题目给定哪种策略就按哪种策略模拟。

结论 4.14: FIFO (First-In First-Out) 替换算法

淘汰最早装入的块。不反映程序的局部性，命中率通常低于 LRU。

缺点：一个很早装入但频繁使用的块会被错误淘汰。

结论 4.15: 随机替换 (Random)

随机选择一个块淘汰。硬件最简单，不维护任何历史信息。在大容量高相联度 Cache 中，随机替换的效果与 LRU 相差不大。

注意

408 常考动手题：给定一个访问序列和 Cache 配置（容量/块大小/相联度），模拟 Cache 的访问过程，统计命中次数和命中率。需要：

1. 正确划分地址（确定每个地址对应的 Tag / Index）；
2. 按替换策略维护每行/每组的状态；
3. 每步判断命中/缺失。

常见陷阱：地址按字节编址时，需将地址除以块大小得到块号；注意区分「组号」和「Cache 行号」。

Cache 的写策略

结论 4.16: 写命中策略

当 CPU 要写入的数据在 Cache 中命中时：

- 写直达 (**Write-Through**)：同时写入 Cache 和主存。主存数据始终与 Cache 一致（简单、一致性好），但每次写都需要访问主存，速度慢。
- 写回 (**Write-Back**)：只写入 Cache，并标记该行为「脏」(dirty)。该行被替换时再写回主存。写操作快，但 Cache 和主存可能不一致。

408 重点：写回需要为每行维护一个「脏位」(dirty bit)；替换脏行时需要额外的写回开销；多核环境下写回策略带来缓存一致性问题。

结论 4.17: 写缺失策略

当 CPU 要写入的数据不在 Cache 中时：

- 写分配 (**Write-Allocate**)：先将对应块从主存调入 Cache，再写入 Cache（即「读缺失 + 写命中」）。依据空间局部性——写入的地址附近可能再次被访问。
- 非写分配 (**Write-No-Allocate / Write-Around**)：绕过 Cache，直接写入主存，不调入 Cache。

常见搭配：写回 + 写分配；写直达 + 非写分配。这是两种最典型的组合。

结论 4.18: 写缓冲 (Write Buffer)

写直达策略中，每次写都要等候主存的慢速写入周期，严重拖慢 CPU。解决方法是在 Cache 和主存之间加一个写缓冲（小型 FIFO 队列）：

- CPU 写 Cache 的同时将数据写入写缓冲，然后立即继续执行——主存的写入由写缓冲异步完成。
- 写缓冲满时 CPU 才需要 stall（停顿）。
- 后续读操作需先检查写缓冲中是否有待写回的对应地址数据。

Cache 性能分析

要点 4.1: Cache 平均访问时间

- t_c ：Cache 命中时间（包括 Tag 比较和选择）
- t_m ：Cache 缺失后访问主存并完成取块的额外惩罚（不含已经付出的 Cache 检查时间）
- h ：命中率 (Hit Rate)
- $m = 1 - h$ ：缺失率 (Miss Rate)

$$T_{\text{avg}} = t_c + m \cdot t_m$$

两级 Cache (L1 + L2)：

$$T_{\text{avg}} = t_{c1} + (1 - h_1)(t_{c2} + (1 - h_2)t_m)$$

其中 t_{c1} 为 L1 检查时间， t_{c2} 为 L1 缺失后访问 L2 并完成回填所需的额外时间， t_m 为 L2 也缺失后访问主存的额外惩罚； h_1, h_2 分别为 L1 命中率和“在 L1 缺失条件下”的 L2 命中率。

注意 (408 易错): 上述公式采用“各层额外时间”口径; 若题目把某层时间定义为从 CPU 发起访问到该层返回的总时间, 必须按题目给定口径重新列分支, 不能重复相加或漏加 L1 检查时间。

注意

408 综合题常考——Cache 容量计算: Cache 的总容量 = 数据区容量 + Tag 容量 + 有效位 + (若有写回) 脏位。

设 Cache 有 C 行, 每行 B 字节, Tag 宽度 T 位:

- 数据区: $C \times B$ 字节。
- Tag 存储: $C \times T$ 位 = $C \times T/8$ 字节。
- 有效位: $C \times 1$ 位 = $C/8$ 字节。
- 脏位 (写回时): $C \times 1$ 位。
- 替换状态 (组相联时): 位数取决于题设采用的实现。若明确采用每行年龄计数器, 可按每行 $\lceil \log_2 W \rceil$ 位计算; 精确 LRU、树形伪 LRU 等实现的状态位数并不相同。

所有项求和得 Cache 总容量。408 真题多次考察这种计算——算出 SRAM 总容量再推片数。

4.7 虚拟存储器 (Virtual Memory)

定义 4.8: 虚拟存储器的动机与基本原理

虚拟存储器将主存用作外存后备内容的「缓存」:

- 每个进程通常拥有独立的虚拟地址空间 (如 32 位虚拟地址空间为 2^{32} 字节 = 4GB), 其可寻址范围可以大于实际物理主存容量。
- 程序使用虚拟地址 (Virtual Address, VA), 由硬件 (MMU, Memory Management Unit) 在运行时转换为物理地址 (Physical Address, PA)。
- 虚拟地址空间中只有当前活跃部分驻留物理主存; 未驻留页的后备内容可能来自可执行文件、映射文件或交换区, 并非所有页面都必须预先写入 swap。

与 Cache 的类比:

Cache 与虚拟存储器的对应关系

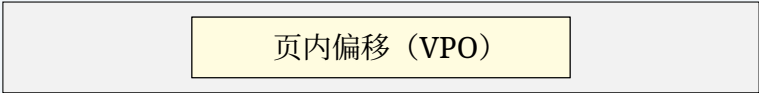
概念	Cache	虚拟存储器
上层（快的）	Cache	主存（DRAM）
下层（慢的）	主存（DRAM）	磁盘/SSD
传送粒度	较小的 Cache 块	通常大得多的页/页框
缺失	Cache Miss	Page Fault
缺失代价	取决于下一层，通常远低于缺页	涉及 OS；若需外存 I/O，代价高出多个数量级
映射与查找	主要由硬件完成	OS 建立页表，MMU/TLB 协助地址转换
替换策略	硬件实现的 LRU/伪 LRU/随机等	OS 页置换算法（如 Clock）
写策略	可采用写直达或写回	脏页淘汰时写回，干净页可直接丢弃

从“主存是外存内容的缓存”这一层次看，虚拟存储采用按需写回语义：对驻留页的普通写操作先修改主存并置脏，通常只在脏页被淘汰或显式同步时更新后备存储，而不会把每条存储指令都直写外存。虚拟存储主要由 OS 与 MMU 协同管理，Cache 则主要由硬件管理。

页式虚拟存储器

结论 4.19: 页式（Paging）虚拟存储器

将虚拟地址空间和物理地址空间都划分为固定大小的页（**Page**），虚拟页（Virtual Page, VP）映射到物理页框（Physical Page Frame, PP）。典型页大小：4KB（ 2^{12} 字节）。
虚拟地址构成：



页表（**Page Table**）：OS 为每个进程维护一张页表，记录每个虚拟页对应的物理页框号（PPN）和状态信息（有效位、脏位、访问权限等）。

$$\text{物理地址} = \text{PPN} \parallel \text{VPO}$$

页表存放于主存。每次访存理论上需要两次访问主存：先查页表（读 PTE），再访问数据。这是页式管理的根本开销。

TLB（Translation Lookaside Buffer）

定义 4.9: TLB

TLB 是页表的专用 Cache，缓存最近用过的页表项（PTE）。通常采用全相联或高组相联设计，命中时间极短（ < 1 个时钟周期）。

TLB 中的每项记录：

- Tag：虚拟页号（VPN）

- **Data**: 物理页框号 (PPN) + 权限位 + 脏位

TLB 命中时, 翻译 ($VA \rightarrow PA$) 在一个时钟周期内完成, 不再需要访问主存中的页表。TLB 缺失时, 硬件 (或 OS) 从主存中的页表找到对应 PTE 并装入 TLB。

结论 4.20: TLB + Cache 的综合访存流程

这是地址转换与存储层次的常见综合题。在同时有 TLB 和 Cache 的系统中, CPU 访存的完整流程:

1. CPU 发出虚拟地址 VA。
2. **TLB 查找**: 用 VPN 查找 TLB。
 - TLB 命中: 获得 PPN, 构成物理地址 PA。
 - TLB 缺失: 访问主存中的页表获取 PTE, 装入 TLB, 重新执行。
3. **Cache 查找**: 用物理地址 PA 的 Index 和 Offset 查找 Cache。
 - Cache 命中: 返回数据。
 - Cache 缺失: 从主存/下级 Cache 装入块。

四种典型情况的时间开销:

TLB + Cache 访存分类

情况	TLB	Cache
最佳	命中	命中
TLB 缺失后命中	缺失 (需访存页表一次)	命中 (用页表查出的 PA 再查 Cache)
TLB 命中后缺失	命中	缺失 (需从主存调入块)
最差	缺失	缺失

注意: 若 TLB 和 Cache 都命中, 则地址转换无需查主存页表, 数据可由 Cache 直接返回, 无需因本次取数再访问主存。

注意

常见辨析——物理寻址 **Cache** vs 虚拟寻址 **Cache**:

- 物理寻址 **Cache (PIPT, Physically Indexed, Physically Tagged)**: 用 PA 的 Index 选组、Tag 比较。需先完成地址翻译 (TLB 查完), TLB 在关键路径上。
- 虚拟寻址 **Cache (VIVT, Virtually Indexed, Virtually Tagged)**: 直接用 VA 的 Index 和 Tag, 省去 TLB。难点: 虚拟地址的同义/别名 (同一物理地址可能对应多个虚拟地址)。
- 虚拟索引物理标记 **Cache (VIPT, Virtually Indexed, Physically Tagged)**: 用 VA 的低位 (Index + Offset) 选组, PA 的 Tag 比较——Index 和 Offset 位在虚拟和物理地址中相同 (因为页大小对齐)。可以在 TLB 翻译的同时开始 Cache 的组选择, TLB 翻译完成后用 PPN 的高位做 Tag 比较。这是现代处理器 L1 Cache 的常见设计。

段式与段页式

结论 4.21: 段式 (Segmentation) 与段页式 (Segmented Paging)

段式虚拟存储器：按程序的逻辑结构（代码段、数据段、栈段等）划分变长段，段表 (Segment Table) 记录段的基址 (base) 和界限 (limit)。虚拟地址 = 段号 + 段内偏移。

优点：与程序结构一致，便于共享和保护。缺点：变长分配导致外部碎片。

段页式虚拟存储器：先按逻辑分段，每段内再分页。虚拟地址 = 段号 + 段内页号 + 页内偏移。结合了两者的优点——逻辑清晰、无外部碎片。

缺页中断 (Page Fault)

结论 4.22: 缺页处理流程

当页表中的有效位 (Valid bit) 为 0（即该虚拟页不在物理主存中）时，触发缺页异常 (Page Fault)：

1. CPU 保存当前指令的 PC 和必要现场。
2. 转入 OS 的缺页处理程序 (ISR / Fault Handler)。
3. OS 确认该访问属于合法虚拟页，并在相应后备存储（可执行文件、映射文件或 swap）中定位内容；按需清零页则无需从外存读取。随后在物理主存中分配一个空闲页框。
4. 若主存已满，OS 执行页面置换算法（如 LRU、Clock、改进的 Clock 算法）选出一个牺牲页写回磁盘（若脏）。
5. 从相应后备存储读入所需页，或按页面类型执行清零填充。
6. 更新页表 (VPN→PPN，标记 Valid)，刷新 TLB 中对应项。
7. 返回并重新执行引发缺页的故障指令。

若缺页需要外存 I/O，当前进程通常阻塞，调度器可让其他就绪进程使用 CPU；缺页处理延迟仍可能通过 I/O 竞争影响系统整体性能。

注意

408 常考辨析——Cache 缺失 vs 缺页：

- 在 408 通常讨论的硬件管理型 Cache 中，Cache 缺失由硬件自动从下层取块并回填，通常对 OS 透明；若题目明确给出软件管理型存储结构，则应服从题设。
- 缺页由 OS 软件处理（发出异常 → OS 缺页处理程序 → 磁盘 IO → 页表更新 → 返回），代价巨大。
- 二者在存储层次中处于不同级别：Cache 缺失是第 2-3 层之间，缺页是第 3-4 层之间。

4.8 408 高频考点速查

结论 4.23: 存储系统—408 选择题常考点

1. **SRAM vs DRAM**：SRAM 六管、快、贵、Cache；DRAM 单管 + 电容、需刷新、主存。DRAM 破坏性读出。
2. **ROM 系列**：MROM/PROM/EPROM/EEPROM/Flash 的可编程性和擦除方式。

3. **Cache** 地址划分: Tag / Index / Offset 各占多少位——给定参数必会计算。
4. 直接映射/全相联/组相联: 映射规则、冲突缺失率对比、地址划分。
5. **LRU** 替换: 手动模拟命中/缺失过程, 统计命中率。
6. 写策略组合: 写回 + 写分配, 写直达 + 非写分配。
7. 低位交叉编址: 连续访问的流水线带宽计算 (T 、 τ 、 m 的关系)。
8. **TLB** + **Cache** + 页表的综合访存流程: 虚拟地址 $\rightarrow$ TLB $\rightarrow$ PA $\rightarrow$ Cache $\rightarrow$ 数据。
9. 缺页处理: 由 OS 而非硬件处理, 属于异常 (fault) 类。
10. 局部性判断: 数组按行 vs 按列访问的空间局部性差异。

5 习题

5.1 基础过关题

1. (真题风格·选择题) 某计算机主存按字节编址, Cache 共有 16 个行, 块大小为 8 字节, 采用直接映射方式。主存地址为 1234E8F8H 的单元装入 Cache 后, 该单元所在 Cache 行的行号(从 0 开始)是 ____。
(a) 0
(b) 5
(c) 15
(d) 无法确定
2. (真题风格·选择题) 某计算机的 Cache 共有 16 块, 采用 2 路组相联映射(每组 2 块), 每块大小为 32 字节, 按字节编址。主存 129 号单元所在的主存块应装入的 Cache 组号是 ____。
(a) 0
(b) 1
(c) 4
(d) 6
3. (真题风格·选择题) 某计算机主存地址空间大小为 256MB, 按字节编址。数据 Cache 有 8 个 Cache 行, 行长为 64B, 采用直接映射方式。主存地址中 Tag 字段的位数是 ____。
(a) 11
(b) 17
(c) 19
(d) 28
4. (真题风格·选择题) 下列关于 Cache 映射方式的叙述中, 错误的是 ____。
(a) 直接映射中, 每个主存块只能映射到 Cache 的一个特定行
(b) 全相联映射中, 每个主存块可以映射到 Cache 的任意一行
(c) 组相联映射中, 每组包含多行, 主存块映射到特定组后, 可在组内任意一行存放
(d) 组相联映射的冲突缺失率高于直接映射
5. (真题风格·选择题) 下列关于 Cache 映射方式的叙述中, 错误的是 ____。
(a) 直接映射方式下, Cache 利用率最高
(b) 全相联映射方式下, Cache 命中时也需要进行 Tag 比较
(c) 组相联映射方式下, 每组只有一行时即为直接映射
(d) 组相联映射方式下, 每组包含所有行时即为全相联映射
6. (真题风格·选择题) 某计算机主存按字编址, 由 4 个存储体构成, 采用低位交叉编址。每个存储体的存取周期为 40 ns, 总线传输周期为 10 ns。忽略发地址等额外开销, CPU 从字地址 0 开始连续读取 4 个字, 需要的总时间是 ____。
(a) 40 ns
(b) 70 ns

(c) 80 ns

(d) 160 ns

7. (填空题) 某计算机主存按字节编址, 主存地址为 32 位。数据 Cache 容量为 32KB, 块大小为 64 字节, 采用 4 路组相联映射。则块内偏移占 ____ 位, 组号 (Set Index) 占 ____ 位, Tag 占 ____ 位。
8. (填空题) 一个由 8 个存储体构成的低位交叉编址存储器, 每个存储体的存取周期为 80 ns, 总线传输周期为 10 ns。流水线方式下连续访问时, 该存储器的最大带宽为 ____ MB/s (设每个存储体每次可读写 4 字节)。
9. (简答题) 简述时间局部性和空间局部性的含义, 并各举出一个程序中的典型例子。说明为什么存储层次结构依赖于局部性原理。
10. (简答题) 简述 Cache 的写直达 (Write-Through) 和写回 (Write-Back) 策略的区别, 并说明它们各自通常搭配的写缺失 (Write Miss) 策略是什么, 为什么这样搭配。

5.2 真题风格综合训练

1. (真题风格·综合题) **Cache** 映射与 **LRU** 替换。某计算机按字编址, Cache 共有 4 行, 块大小为 1 个字。CPU 要访问的字地址序列为:

0, 1, 2, 3, 1, 2, 3, 0, 1, 2, 3, 1, 2, 3, 0

设 Cache 采用 2 路组相联映射, LRU 替换策略, 初始 Cache 为空。

- (1) 写出每次访问时, 对应的 Cache 组号。
 - (2) 逐次模拟每次访问是否命中, 计算 Cache 命中次数和命中率。
 - (3) 若改为 4 路组相联 (全相联), 同样采用 LRU 替换, 命中率是多少? 比较两种映射方式的命中率差异, 并说明原因。
2. (真题风格·选择题) **Cache** 地址结构与命中分析。某计算机主存地址空间为 256MB, 按字节编址。数据 Cache 容量为 8KB, 块大小为 16 字节, 采用 4 路组相联映射。
- (1) 求主存地址中 Tag、组号 (Set Index) 和块内偏移各占多少位。
 - (2) 设 CPU 依次访问以下地址 (十进制): 0, 16, 32, 0, 64, 16, 128, 0。初始 Cache 为空。若采用 LRU 替换策略, 求 Cache 命中次数和命中率。
 - (3) 若替换策略改为 FIFO, 求命中次数。比较 LRU 和 FIFO 的结果并说明原因。
3. (真题风格·综合题) **TLB + Cache + 页表** 综合。某计算机系统按字节编址, 虚拟地址 32 位, 物理地址 28 位, 页大小 4KB。系统具有 TLB 和两级 Cache。
- (1) 虚拟地址中 VPN 和 VPO 各占多少位? 物理地址中 PPN 和 PPO 各占多少位?
 - (2) L1 数据 Cache 为 8 路组相联, 容量 32KB, 块大小 64 字节。写出物理地址中 Tag、组号和块内偏移各占多少位。
 - (3) 设 TLB 采用全相联映射, 共 16 项。当 CPU 执行访存指令访问虚拟地址 0x12345678 时, 若 TLB 命中且 L1 Cache 命中, 描述从虚拟地址到数据返回的完整过程 (依次列出各步骤和各部件的输入/输出)。
 - (4) 若 TLB 命中时 L1 Cache 缺失、L2 Cache 命中, 设 TLB 命中时间 1 个周期, L1 命中时间 2 个周期, L2 命中时间 (含从 L2 调入 L1) 15 个周期。假定 TLB、L1 和 L2 按上述顺序串行访问, 求这次访问的总延迟。
4. (真题风格·选择题) **LRU** 替换算法。某 Cache 采用 4 路组相联映射, 每组有 4 个 Cache 行。

对于某一特定组，初始为空。该组依次访问的主存块号为：5, 3, 7, 5, 2, 3, 4, 3, 5。

- (1) 采用 LRU 替换策略，逐次写出每次访问后该组中存放的主存块号，并统计命中次数。
 - (2) 采用精确 LRU 时需要记录各行怎样的信息？若只考虑能够区分 $k!$ 种新旧次序的理论最小状态编码，至少需要多少位？对本题 $k = 4$ 时是多少位？
 - (3) 为何在实际处理器中，4 路组相联通常使用近似 LRU (如二叉树伪 LRU) 而非真正的 LRU？
5. (真题风格·选择题) 低位交叉编址综合。某计算机主存按字编址，每字 4 字节，由 $m = 4$ 个存储体构成，采用低位交叉编址。每个存储体的存取周期 $T = 40 \text{ ns}$ ，总线传输周期 $\tau = 10 \text{ ns}$ ，每个存储体一次读写一个字。忽略发地址等额外开销，并假定改用高位交叉编址时这 16 个连续字均落在同一存储体。
- (1) 求该存储器在流水线方式下的最大带宽 (MB/s)。
 - (2) 若 CPU 连续从字地址 0 开始读取 16 个字 (共 64 字节)，需要多少时间？
 - (3) 若改用高位交叉编址，同样读取这 16 个连续字，需要多少时间？比较 (2) 和 (3) 的结果，说明低位交叉编址的优势。
 - (4) 若总线传输周期变为 20 ns ，而存取周期仍为 40 ns ，最大带宽有何变化？这个变化对 m 的取值有何启示？
6. (真题风格·选择题) 虚拟存储器与 Cache 综合。某计算机按字节编址，虚拟地址 32 位，物理地址 24 位，页大小 4KB。数据 Cache 采用 2 路组相联映射，容量 16KB，块大小 32 字节。
- (1) 虚拟地址中，VPN 占多少位？VPO 占多少位？
 - (2) 物理地址中，PPN 占多少位？PPO 占多少位？
 - (3) Cache 中，块内偏移占多少位？组号占多少位？Tag 占多少位？
 - (4) 设某进程的页表部分内容为：虚页号 $0x00123 \rightarrow$ 实页号 $0x0AB$ ，虚页号 $0x00124 \rightarrow$ 实页号 $0x0CD$ 。该进程访问虚拟地址 $0x00123456$ 。若 TLB 命中，且 Cache 也命中，写出完整的地址转换流程，并得出最终访问的 Cache 组号和 Tag。
7. (综合·局部性分析) 分析以下两段 C 代码的局部性表现，并估算其在 Cache 中的缺失率。设 Cache 块大小为 64 字节，int 占 4 字节， $N = 1024$ ，数组 a 按行优先存储。为使估算唯一，假定 Cache 容量小于 64 KiB，且代码 B 从一列转到下一列前，上一列涉及的各行 Cache 块均已被淘汰；忽略硬件预取和冲突细节。

```

1 // 代码A：按行访问
2 int sum = 0;
3 for (int i = 0; i < N; i++)
4     for (int j = 0; j < N; j++)
5         sum += a[i][j];
6
7 // 代码B：按列访问
8 int sum = 0;
9 for (int j = 0; j < N; j++)
10     for (int i = 0; i < N; i++)
11         sum += a[i][j];
12

```

- (1) 分别指出代码 A 和代码 B 的时间局部性和空间局部性表现，并解释原因。
- (2) 设 Cache 初始为空，容量远小于数组大小。估算代码 A 的缺失率 (忽略 sum 和循环变量的访问)。
- (3) 同等条件下，估算代码 B 的缺失率。

- (4) 若将代码 B 在虚拟存储系统（含 TLB、页表）中运行，除了 Cache 缺失率增加外，还可能带来什么性能问题？为什么？

5.3 拓展阅读

- **Patterson & Hennessy**《计算机组成与设计（RISC-V 版）》第 5 章「大而快：层次化存储系统」——完整论述存储层次、Cache 的直接映射、组相联和全相联三类映射方式及性能分析，可作为本章的扩展阅读。
- **Bryant & O'Hallaron**《深入理解计算机系统》（CS:APP）第 6 章「存储器层次结构」——§6.2 局部性原理的形式化讨论与量化实验；§6.3–§6.4 存储层次和 Cache 设计；§6.5 编写 Cache 友好的代码；§6.6 主存到 Cache 的地址映射详解。CS:APP 对局部性的量化分析和 Cache 模拟实验是理解 408 Cache 题的极佳补充。
- **CS:APP** 第 9 章「虚拟内存」——§9.3–§9.4 页表和 TLB 的详细工作机制；§9.5 缺页处理和内存映射；§9.6 地址翻译综合案例（Core i7 的完整地址翻译过程，含 TLB、多级页表、多级 Cache 的协同，是 408 TLB+Cache 综合题的理想参考）。
- 唐朔飞《计算机组成原理》第 4 章「存储器」——4.1 概述，4.2 主存储器（SRAM/DRAM 对比、刷新方式），4.3 高速缓冲存储器（Cache 地址映射、替换算法、写策略），4.5 辅助存储器。不同版本术语和章节号可能略有差异，应以所用教材版本为准。
- 延伸思考：现代处理器的 Cache 设计远不止 408 范围——多级 Cache 的包含策略（Inclusive、Exclusive、Non-Inclusive/Non-Exclusive）、硬件预取（Stride/Stream prefetcher）、非阻塞 Cache（MSHR）等。但 408 的核心仍是「给出参数 → 划分地址位 → 模拟访问序列 → 统计命中率」这一基本功。

6 指令系统

6.1 指令格式

定义 6.1: 指令的基本格式

一条机器指令由操作码（**Opcode**）和地址码（**Address**）两部分组成：

$$\underbrace{\text{操作码 OP}}_{\text{指定操作类型}} \quad \underbrace{\text{地址码 } A_1 A_2 \cdots A_n}_{\text{指定操作数地址}}$$

按地址码数量分类：

- 三地址指令：OP A1, A2, A3 —— (A1)OP(A2) → A3
- 二地址指令：OP A1, A2 —— (A1)OP(A2) → A1（或 A2）
- 一地址指令：OP A —— 如 ACC OP (A) → ACC
- 零地址指令：无地址码——如堆栈操作、空操作 NOP

结论 6.1: 定长操作码 vs 扩展操作码

- 定长操作码：所有指令操作码位数相同，译码简单、控制方便，但指令条数受限。
- 扩展操作码（哈夫曼编码思想）：操作码位数可变，短操作码留给高频指令，长操作码对应更多寻址方式或低频指令。这是 408 选择题中的高频考点——给定指令字长和地址码位数，计算最多能有多少条指令。

扩展操作码的设计约束：短操作码不能是长操作码的前缀（类似 Huffman 编码的前缀性质），否则译码歧义。

6.2 寻址方式

定义 6.2: 寻址方式

指令中获取操作数（或跳转地址）的方式。分为指令寻址和数据寻址两大类。

结论 6.2: 常见数据寻址方式

设指令字中的形式地址为 A ，有效地址为 EA 。

寻址方式汇总

寻址方式	EA 计算	访存次数（取操作数）	用途
立即寻址	操作数 = A （无需访存）	0	常量
直接寻址	$EA = A$	1	简单变量
间接寻址	$EA = (A)$	≥ 2	指针
寄存器寻址	操作数 = R_A	0	频繁使用变量
寄存器间接	$EA = (R_A)$	1	数组遍历
变址寻址	$EA = (R_X) + A$	1	循环/数组
基址寻址	$EA = (R_B) + A$	1	程序重定位
相对寻址	$EA = (PC) + A$	1	转移指令
堆栈寻址	栈顶隐含	0	表达式求值

变址寻址 vs 基址寻址：两者形式相同（ $EA = \text{寄存器} + A$ ），但语义不同：

- 变址： A 为基准， R_X 为偏移（可修改），用于数组。
- 基址： R_B 为基准， A 为偏移，用于程序重定位（OS 管理）。

注意

408 高频考点：给定一段 C 代码，分析其中某变量的访问使用了哪种寻址方式。例如 `for(i=0; i<n; i++) sum += a[i];` 中 `a[i]` 通常使用变址寻址（基址寄存器 = `a` 的首地址，变址寄存器 = `i`）。

6.3 CISC vs RISC

结论 6.3: CISC 与 RISC 对比

CISC vs RISC		
特征	CISC	RISC
指令长度	变长	定长（通常 32 位）
指令数量	通常较多	通常较少、强调规则性
寻址方式	多	少（仅 LOAD/STORE 访存）
指令执行周期	差异通常较大	设计上便于流水化，周期数较规则
控制实现	可采用微程序控制	常采用硬布线控制
寄存器数量	传统设计相对较少	通常较多
流水线实现	困难	容易
典型代表	x86	MIPS, ARM, RISC-V
设计哲学	用硬件复杂度换编译器简单性	用编译器复杂度换硬件简单性

注意

408 常考：x86 是 CISC 的代表，但现代 x86 处理器内部将 CISC 指令「翻译」为类 RISC 微操作（ μ ops）执行，融合了两者优先。

6.4 408 高频考点速查

结论 6.4: 指令系统—408 常考点

- 1. 扩展操作码设计：给定指令字长和地址码字段，求最大指令数。
- 2. 寻址方式：根据访存次数、EA 计算公式区分不同寻址方式。
- 3. **CISC vs RISC**：对比表每条都要记住。
- 4. 指令字长：定长 vs 变长，与操作码、地址码字段位数的关系。
- 5. 大端/小端 (**Endianness**)：给定一条指令或数据在内存中的字节存储顺序，判断是大端还是小端。

7 习题

7.1 基础过关题

- (单项选择题) 某计算机指令字长为 16 位, 采用扩展操作码设计, 二地址指令含两个 6 位地址字段。若二地址指令有 15 条, 一地址指令有 62 条, 则零地址指令最多有
(A) 8 条 (B) 16 条
(C) 64 条 (D) 128 条
- (单项选择题) 在指令寻址方式中, 直接寻址方式的有效地址为
(A) 程序计数器 PC 的值 (B) 指令中给出的形式地址
(C) 形式地址所指主存单元的内容 (D) 变址寄存器与形式地址之和
- (填空题) 变址寻址中, 操作数的有效地址 = ____ + ____。
- (填空题) 在 RISC 处理器中, 只有 ____ 和 ____ 指令可以访问存储器。
- (简答题) 解释变址寻址和基址寻址在功能上的区别。
- (计算题) 某计算机指令字长为 32 位, 采用二地址指令。操作码字段占 8 位, 两个地址字段各占 12 位。求: (1) 最多可有多少条二地址指令? (2) 若改为扩展操作码, 保留 100 条二地址指令, 一地址指令最多多少条?

7.2 真题风格与综合训练

- (真题风格, 单项选择题) 以下关于 CISC 和 RISC 的说法, 正确的是
(A) CISC 的指令长度固定
(B) RISC 的寻址方式种类更多
(C) 典型 load/store 型 RISC 中, 只有 LOAD/STORE 指令访问数据存储器
(D) CISC 比 RISC 更容易实现流水线
- (真题风格, 单项选择题) 一条指令的地址码字段给出的是操作数在寄存器中的编号, 则该寻址方式是
(A) 直接寻址 (B) 立即寻址
(C) 寄存器寻址 (D) 寄存器间接寻址
- (真题风格, 综合题) 某 16 位计算机采用单字长定长指令, 指令格式如下:

OP(4 位)	Md(2 位)	Rd(2 位)	Ms(2 位)	A/Rs(6 位)
---------	---------	---------	---------	-----------

其中 OP 为操作码, 设 ADD 的 OP=0010; Md、Ms 分别为目的和源寻址方式, 00 表示寄存器寻址, 01 表示寄存器间接寻址, 10 表示变址寻址 (变址寄存器为 R0), 11 表示立即寻址; Rd 为目的寄存器号。A/Rs 字段在寄存器或寄存器间接寻址时以低 2 位给出源寄存器号 (高 4 位置 0), 在变址/立即寻址时给出 6 位形式地址或立即数。寄存器编码为 R0=00、R1=01、R2=10、R3=11。

- (1) 该指令系统最多可有多少条指令?
- (2) 写出指令 `ADD R1, (R2)` (将 `R1` 与 `R2` 指向的内存单元内容相加, 结果存 `R1`) 的机器码 (二进制)。
- (3) 若某指令 `Ms=10`, `A/Rs=000101`, `R0` 的内容为十进制 1000, 求源操作数的有效地址。

拓展阅读:

- Patterson & Hennessy《计算机组成与设计》第 2 章「指令: 计算机的语言」——MIPS/RISC-V 指令集详解, 含寻址方式与指令格式设计原理。
- CS:APP 第 3 章「程序的机器级表示」——从 x86-64 汇编出发, 建立一个程序如何被翻译为机器指令的完整心智模型。

8 中央处理器 (CPU)

8.1 CPU 的功能与基本结构

定义 8.1: CPU 的组成

CPU 由运算器 (ALU) 和控制器 (CU) 两大部分组成:

- 运算器: 执行算术和逻辑运算。核心部件: ALU、累加器 ACC、通用寄存器组、状态寄存器 PSW (标志位: ZF/CF/OF/SF)。
- 控制器: 取指令、分析指令、执行指令, 控制各部件协调工作。核心部件: PC (程序计数器)、IR (指令寄存器)、ID (指令译码器)、时序发生器、微操作信号发生器。

CPU 与主存通过 MAR (存储器地址寄存器) 和 MDR (存储器数据寄存器) 交互。

结论 8.1: CPU 中的专用寄存器

CPU 主要寄存器

寄存器	缩写	功能
程序计数器	PC	存放下一条指令的地址, 自动 +1 (或 + 指令长度)
指令寄存器	IR	存放当前执行的指令
存储器地址寄存器	MAR	存放访存地址 (连接地址总线)
存储器数据寄存器	MDR	存放与主存交换的数据 (连接数据总线)
累加器	ACC	存放 ALU 运算结果
程序状态字	PSW	存放标志位 (ZF/CF/OF/SF/IF)
通用寄存器	R0-Rn	存放操作数和中间结果

408 常考: 哪些寄存器程序员可见 (通用寄存器、PSW、PC 部分可见), 哪些透明 (MAR、MDR、IR)。

8.2 指令执行过程

结论 8.2: 指令周期

一条指令的指令周期由取指和执行阶段构成，并可能包含间址阶段；若本条指令执行后检测到可响应的中断，还会进入中断周期：

取指 (Fetch) → 必要时进行间址 (Indirect) → 执行 (Execute)
→ 若响应中断，则进入中断周期 (Interrupt)

- 取指周期 (FE)：PC→MAR → 读主存 → MDR→IR，并把 PC 更新为下一条指令地址。
- 间址周期 (IND)：若指令含间接寻址，取操作数地址。
- 执行周期 (EX)：根据操作码执行具体操作。
- 中断周期 (INT)：保存断点并取得中断服务程序入口；具体由硬件自动完成哪些现场保存操作取决于体系结构。

注意

408 常考数据通路分析题：给定 CPU 数据通路图和控制信号，写出某条指令（如 ADD R1, (R2)）在每个时钟周期的数据流动路径和有效控制信号。

8.3 数据通路

定义 8.2: 数据通路

数据通路是 CPU 中执行单元（ALU、寄存器堆、多路选择器）和它们之间的互连总线。分为：

- 单总线结构：所有部件共享一条内部总线，一个时钟周期只能传送一个数据。结构简单但速度慢。
- 多总线结构：多条总线并行传送，速度快但控制复杂。
- 专用数据通路：各部件之间有专用连接，并行度高但硬件代价大。

8.4 控制器

结论 8.3: 硬布线控制器 vs 微程序控制器

两种控制器对比		
	硬布线控制器	微程序控制器
实现方式	组合逻辑电路	微程序（控制存储器 CM）
速度	快	较慢（需多次读 CM）
可修改性	困难（需重新设计电路）	容易（修改微程序）
设计复杂度	复杂（需化简逻辑表达式）	规整
典型应用	RISC	CISC

微程序控制器的核心概念：

- 微指令：控制存储器中的一个字，控制一个时钟周期内的所有微操作。
- 微程序：实现一条机器指令的微指令序列。
- **CM**（控制存储器）：存放全部微程序（通常在 CPU 内部 ROM）。
- μPC （微程序计数器）：顺序执行微程序时 +1；遇到转移微指令时加载新地址。

结论 8.4: 微指令的编码方式

- 直接编码：微指令的每一位直接控制一个微操作。简单快速，但微指令字长过长。
- 字段直接编码：互斥的微操作放在同一字段，通过译码器选择。折中方案，408 常考。
- 字段间接编码：一个字段的含义由另一个字段的值决定。进一步缩短字长但译码更复杂。

微指令格式：

- 水平型微指令：一次能控制多个并行微操作，字长较长。
- 垂直型微指令：类似机器指令格式，一次只控制一个微操作，需多步完成一条机器指令。

8.5 指令流水线

定义 8.3: 流水线

将指令执行过程分解为多个子过程，每个子过程由独立的硬件部件完成。多条指令的不同阶段在时间上重叠执行，提高 CPU 吞吐率（非单条指令执行速度）。

结论 8.5: 经典五级流水线

IF（取指）→ ID（译码/读寄存器）→ EX（执行/计算地址）
→ MEM（访存）→ WB（写回）

流水线性能指标：

- 吞吐率（**TP**）：单位时间完成的指令数。理想情况 $TP = \frac{1}{\max\{T_i\}}$ 。
- 加速比（**S**）： $S = \frac{\text{非流水线执行时间}}{\text{流水线执行时间}}$ 。 k 段流水线执行 n 条指令： $S = \frac{n \cdot k \cdot T}{k \cdot T + (n - 1) \cdot T} = \frac{nk}{k + n - 1}$ （各段等长时间 T ）。
- 效率（**E**）： $E = \frac{\text{时空图中有效面积}}{\text{总面积}}$ 。

8.6 流水线冒险 (Hazard)

结论 8.6: 三种流水线冒险

1. 结构冒险（**Structural Hazard**）：硬件资源冲突——多条指令同时访问同一功能部件（如单端口存储器）。解决：增加硬件资源（如指令 Cache 与数据 Cache 分离）、流水化功能部件。
2. 数据冒险（**Data Hazard**）：指令间存在数据依赖。类型：

- **RAW** (写后读): I1 写入后 I2 才读, 但 I2 在 I1 写入前就进入读阶段。
- **WAW** (写后写): 两条指令写同一位置, 顺序错。
- **WAR** (读后写): I2 在 I1 读之前就写入。

解决:

- **转发/旁路 (Forwarding/Bypassing)**: 将 ALU 结果直接从 EX/MEM 流水线寄存器「转发」回 EX 阶段输入, 无需等 WB 写回。可解决大部分 RAW。
- **阻塞/气泡 (Stall/Bubble)**: 硬件插入空操作周期。LW 指令后紧跟使用该数据的指令时, 即使转发也需阻塞 1 个周期 (Load-use Hazard)。
- **编译器调度**: 重排指令顺序以消除依赖。

3. **控制冒险 (Control Hazard)**: 分支/跳转指令导致 PC 值不确定。解决:

- **分支预测**: 静态预测 (总是预测不跳转)、动态预测 (基于历史的分支预测缓冲器 BHT/BTB)。
- **延迟分支**: 在分支指令后插入不受影响的指令 (延迟槽)。

注意

408 最常考的数据冒险场景:

1. 相邻 RAW: `add R1,R2,R3; sub R4,R1,R5` — 可通过转发解决 (无需阻塞)。
2. Load-use: `lw R1,0(R2); add R3,R1,R4` — 必须阻塞 1 个周期 (转发不够, 数据在 MEM 阶段才就绪)。

8.7 多核处理器基本概念

定义 8.4: 多核处理器

在一个芯片上集成多个处理器核心 (Core), 各核心共享最后一级 Cache 和主存, 通过片内互连通信。多核实现的是真正的并行 (不是并发)。

408 了解即可: SISD/SIMD/MIMD (Flynn 分类法)、共享 Cache 的一致性协议 (MESI 协议基本思想)、Amdahl 定律 (加速比上限由不可并行部分决定)。

8.8 408 高频考点速查

结论 8.7: CPU — 408 常考点

1. **数据通路分析**: 给定 CPU 结构图, 分析指令执行过程中各周期的数据流动。
2. **流水线性能计算**: 吞吐率、加速比、效率的公式和套用。
3. **流水线冒险**: 特别是数据冒险 (RAW/WAW/WAR) 的判别和转发/阻塞方案的时钟周期计算。
4. **微程序控制器**: 微指令的编码方式、CM 容量计算、 μ PC 的工作过程。
5. **CISC/RISC** 在流水线实现上的差异。

9 习题

9.1 基础过关题

1. (真题风格·选择题) 下列关于指令流水线设计方法的说法中, 错误的是
 - (A) 指令流水线可以提高 CPU 的吞吐率
 - (B) 指令流水线可以提高单条指令的执行速度
 - (C) 指令流水线不能提高单条指令的执行速度
 - (D) 指令流水线可能会增加指令的执行时间
2. (真题风格·选择题) 下列有关指令流水线的叙述中, 错误的是
 - (A) 流水段寄存器会增加单条指令的执行时间
 - (B) 数据冒险可以通过转发技术解决
 - (C) 控制冒险可以通过分支预测来缓解
 - (D) 指令流水线可以消除结构冒险
3. (真题风格·选择题) 在经典五级顺序流水线 (IF、ID、EX、MEM、WB) 中, 假定已配置常规的 EX/MEM、MEM/WB 转发通路, 以下哪种紧邻的数据相关仍需要通过阻塞 (插入气泡) 解决?
 - (A) 相邻两条 ALU 指令之间的 RAW 冒险
 - (B) Load 指令与其后紧跟的 ALU 指令之间的 RAW 冒险
 - (C) 两条 ALU 指令之间的 WAR 冒险
 - (D) 两条 ALU 指令之间的 WAW 冒险
4. (真题风格·选择题) 以下关于流水线冒险的说法, 正确的是
 - (A) 结构冒险是由于多条指令存在数据依赖引起的
 - (B) 数据冒险中, WAR 和 WAW 在五级流水线中不会发生
 - (C) 分支预测可以彻底消除控制冒险
 - (D) 所有数据冒险都可以通过转发技术消除, 无需阻塞
5. (微程序控制器计算题) 某微程序控制器的控制存储器 (CM) 容量为 256×32 位 (256 个微指令字, 每字 32 位)。微操作控制字段采用直接编码, 其中下地址字段占 8 位, 判别测试字段占 3 位, 其余位每位直接对应一个微操作控制信号。试回答:
 - (1) 该微程序控制器最多可控制多少个微操作信号?
 - (2) 假定 256 个字均可用于各机器指令的专属微程序, 每条机器指令固定占用 5 条微指令, 不另计共享的取指微程序, 则最多可支持多少条机器指令?
6. (流水线性能计算——填空题) 某经典五级流水线处理器, 各段执行时间分别为: IF = 10 ns, ID = 8 ns, EX = 10 ns, MEM = 10 ns, WB = 8 ns。

- (1) 该流水线的时钟周期应取 ____ ns;
 (2) 执行 1000 条指令（不发生阻塞）需要 ____ ns;
 (3) 相比非流水线（每条指令 $10 + 8 + 10 + 10 + 8 = 46$ ns），加速比为 ____（保留两位小数）。

9.2 真题风格综合训练

7. (真题风格·选择题) 某计算机的指令流水线由 4 个功能段组成，各段经过的时间分别为 Δt 、 $2\Delta t$ 、 Δt 、 $3\Delta t$ 。忽略流水寄存器开销，连续执行 n 条相同指令，则该流水线相对于顺序执行的加速比为

$$\begin{array}{ll} \text{(A)} \frac{7n}{3n+9} & \text{(B)} \frac{7n}{3n+6} \\ \text{(C)} \frac{7n}{3n+3} & \text{(D)} \frac{7n}{3n+12} \end{array}$$

8. (真题风格·选择题) 在经典五级流水线（IF/ID/EX/MEM/WB）中，考虑以下指令序列：

```
1 I1: lw  R1, 0(R2)
2 I2: add R3, R1, R4
3 I3: sub R5, R6, R7
4 I4: sw  R8, 0(R9)
```

假定寄存器堆允许同一周期先写回、后读出。若仅考虑数据冒险，且不使用转发技术，则 CPU 需要插入的总阻塞（气泡）周期数为

- (A) 0 (B) 1
(C) 2 (D) 3

9. (真题风格·综合题) 某 16 位计算机采用单总线 CPU 结构，数据通路示意图如下（文字描述）：

系统包含：PC、MAR、MDR、IR、通用寄存器 R0-R7、ALU、暂存器 Y 和 Z。ALU 有两个输入端 A 和 B（A 来自暂存器 Y，B 来自总线），一个输出端连接到暂存器 Z。所有部件通过单总线交换数据。PC 具有自增功能（ $PC \rightarrow PC+1$ ）。主存储器的读写由 MDR 和 MAR 配合完成。所有控制信号高电平有效。

回答以下问题：

- (1) 写出取指周期的微操作序列（数据流动路径及有效控制信号）。
 - (2) 写出指令 ADD R1, R2（功能： $R1 \leftarrow R1 + R2$ ）在执行周期的微操作序列。
 - (3) 写出指令 LOAD R3, (R4)（寄存器间接寻址，功能： $R3 \leftarrow \text{Mem}[R4]$ ）的完整指令周期微操作（取指 + 执行）。
10. (综合题·指令流水线分析) 某经典五级流水线处理器（IF/ID/EX/MEM/WB），执行以下指令序列：

```
1 I1: add  R1, R2, R3    % R1 <- R2 + R3
2 I2: lw   R4, 0(R1)     % R4 <- Mem[R1 + 0]
3 I3: sub  R5, R1, R6    % R5 <- R1 - R6
4 I4: add  R7, R4, R5    % R7 <- R4 + R5
5 I5: sw   R7, 0(R8)     % Mem[R8 + 0] <- R7
```

- (1) 指出上述指令序列中存在的所有数据冒险，说明冒险类型。
- (2) 若不存在转发机制，需要插入多少个阻塞周期？画出流水线时空图。

- (3) 若有完整的数据转发（包括向存储指令的数据输入转发），但紧邻的 Load-use 冒险仍需阻塞 1 拍，需要插入多少个阻塞周期？
- (4) 能否在保持程序语义的前提下交换 I3 与 I4？说明理由。

拓展阅读：

- Patterson & Hennessy 《计算机组成与设计》第 4 章「处理器」——MIPS 五级流水线的完整数据通路设计和冒险处理。408 的数据通路题几乎均源自此章的习题。
- CS:APP 第 4 章「处理器体系结构」——从顺序 Y86-64 到流水线实现，含转发逻辑的完整推导。

10 总线

10.1 总线的基本概念与分类

定义 10.1: 总线 (Bus)

总线是连接计算机各功能部件（CPU、存储器、I/O 设备）的一组公共信息传输线，是各部件共享的传输介质。总线采用分时共享的方式：同一时刻只允许一个部件向总线发送信息，但多个部件可以同时从总线接收信息。

总线传输的两种基本方式：

1. 串行传输：数据在一条数据线上逐位传输，适合远距离通信（如 USB、PCIe），抗干扰能力强，成本低。
2. 并行传输：数据在多条数据线上同时传输多位，近距离传输速率高，但需解决线间串扰和信号偏斜（skew）问题（如传统 PCI）。

结论 10.1: 总线的分类

一、按连接部件的不同（按所处位置）：

1. 片内总线（**Internal Bus / On-Chip Bus**）：CPU 芯片内部各部件（寄存器、ALU、控制器等）之间的连接线，由芯片制造商设计。属于芯片内部实现细节，对外不可见。
2. 系统总线（**System Bus**）：连接 CPU、主存和 I/O 接口的总线，是计算机系统主板的核心。按传输信息类型又可分为三类（见下）。
3. 通信总线（**Communication Bus**）：用于计算机系统之间或计算机与外部设备之间的数据传输。标准繁多，如 PCIe、USB、SATA、RS-232 等。

二、按传输信息类型——系统总线的三类信号线：

1. 地址总线（**Address Bus, AB**）：
 - 传输地址信息，由 CPU（或 DMA 控制器等总线主设备）向存储器或 I/O 端口发出。
 - 单向传输（从总线主设备到从设备）。
 - 地址总线的位数 n 决定了存储器最大可寻址空间 = 2^n 个单元（按字节编址时，最多可寻址 2^n 字节）。

2. 数据总线 (Data Bus, DB):

- 传输数据信息, 数据可以双向流动 (读操作时从存储器/IO 到 CPU; 写操作时从 CPU 到存储器/IO)。
- 双向传输 (三态门实现)。
- 数据总线的位数 (宽度) 通常与机器字长一致, 决定了一次可以传输的二进制位数。

3. 控制总线 (Control Bus, CB):

- 传输控制信号和时序信号, 如读/写信号 (MEMREAD、MEMWRITE、IOR、IOW)、中断请求 (INTR)、中断响应 (INTA)、总线请求 (BUSREQ)、总线允许 (BUSACK)、时钟 (CLK)、复位 (RESET) 等。
- 每条控制线的方向是固定单向的, 但不同控制线的方向可以不同——有的从 CPU 发出到外设, 有的从外设发送到 CPU。

注意

408 常考辨析:

- 地址总线是单向的 (CPU → 存储器/IO); 数据总线是双向的; 控制总线各线方向固定但不统一——不能说「控制总线是单向的」或「控制总线是双向的」。
- 地址总线的位数决定的是可寻址的最大范围 (地址空间大小), 而非实际的物理内存容量。
- 「总线宽度」一般指的是数据总线的位数。

结论 10.2: 总线主设备与从设备

总线主设备 (Bus Master): 可以主动发起总线操作的设备 (如 CPU、DMA 控制器)。

总线从设备 (Bus Slave): 只能在主设备控制下进行信息传输的设备 (如存储器、I/O 接口)。一次总线传输中, 主设备首先发出地址和读/写命令, 然后与从设备交换数据。仲裁机制决定了在多个主设备竞争总线时由谁获得控制权。

10.2 总线结构

总线的物理连接方式决定了系统的整体性能、可扩展性和成本。典型的总线结构有三种。

结论 10.3: 单总线结构 (Single Bus Architecture)

CPU、主存和所有 I/O 设备都连接在同一条系统总线上。

- 优点: 结构简单, 成本低, 易于扩展 (增加新设备只需连接到总线上)。
- 缺点:
 - 总线成为系统瓶颈——所有部件共享同一带宽, 高速设备 (如 CPU-主存) 与低速设备 (如 I/O) 竞争总线;
 - I/O 操作会阻塞 CPU 与主存的通信 (或反之), 整体吞吐量受限。
- 典型应用: 早期微型计算机 (如 Apple II、IBM PC 早期型号)。

结论 10.4: 双总线结构 (Dual Bus Architecture)

在 CPU 和主存之间增加一条专用的存储总线 (**Memory Bus**), 将 CPU-主存的通信与 I/O 通信分离开来。

典型连接方式:

- 存储总线: 连接 CPU 与主存, 专门用于指令和数据的高速存取。
- I/O 总线: 连接 CPU 与 I/O 设备。I/O 设备与主存之间的数据传输需要通过 CPU 中转 (PIO 方式), 或者增加 DMA 控制器实现 I/O 与主存的直接传输。

优点: CPU 访问主存时不会受到 I/O 设备的干扰, 提高了 CPU-主存的带宽利用率。

缺点: I/O 设备与主存的交互仍需通过 CPU (除非增加 DMA 通道), 结构复杂度略增。

结论 10.5: 三总线结构 (Triple Bus Architecture)

在双总线基础上进一步分离, 使 CPU、主存和 I/O 各自拥有独立通道。

典型的三种总线:

1. 局部总线 (**Local Bus**): 连接 CPU 与 Cache/局部设备, 速度最快。
2. 存储总线 (**Memory Bus**): 连接主存 (Cache 与主存之间的通道)。
3. 扩展总线 (**Expansion Bus / I/O Bus**): 连接各种 I/O 设备, 速度相对较慢。

典型实现: 现代 PC 的芯片组架构 (North Bridge / South Bridge 或 Platform Controller Hub)。

优点: 三级总线分工明确, 各自可运行在不同频率和带宽下, 避免速度不匹配的瓶颈; 高速设备和低速设备在物理上分离。

缺点: 结构复杂, 成本较高。

注意

408 常考点 (总线结构对比):

- 单总线的核心问题是总线竞争——所有设备抢一条总线。
- 双总线的主要改进是将 CPU-主存通路 with I/O 通路分离。
- 三总线在双总线基础上进一步将局部设备与主存分离, 形成速度分级。
- 考题中常出现「某总线结构下, DMA 传送时 CPU 能否同时访问主存」——这取决于该总线结构中 CPU 和 DMA 是否共享同一总线。

10.3 总线的性能指标

要点 10.1: 总线的核心性能指标

衡量总线性能的三个基本参数及其关系:

$$\text{总线带宽} = \text{总线宽度} \times \text{总线工作频率}$$

各单位说明:

- 总线宽度 (**Data Bus Width**): 数据总线的位数 (位, bit), 即总线一次能并行传输的数据量。如 32 位、64 位。

- 总线工作频率 (**Bus Frequency**): 总线时钟的频率, 单位为 Hz; 其倒数即总线周期 (总线时钟的周期时间)。
- 总线带宽 (**Bus Bandwidth**): 总线在单位时间内可以传输的数据总量, 单位为 B/s (字节/秒) 或 bps (位/秒)。是衡量总线性能的最终指标。

注意区分:

- 若每个总线周期传输一次数据: 带宽 = 宽度 × 频率
- 若一个时钟周期传输多次数据 (如 DDR 技术在上升沿和下降沿各传一次): 带宽 = 宽度 × 频率 × 每周期传输次数
- 对于串行总线, 带宽 = 有效传输速率 (需扣除编码开销, 如 PCIe 的 8b/10b 或 128b/130b 编码)。

结论 10.6: 总线带宽计算实例

例 1 某总线宽度为 32 位, 工作频率为 66 MHz, 每个时钟周期传输一次数据。求总线带宽。
解: 带宽 = $32 \text{ bit} \times 66 \times 10^6 \text{ Hz} = 2.112 \times 10^9 \text{ bps} = 264 \text{ MB/s}$ 。

例 2 某 DDR 内存总线宽度为 64 位, 工作频率为 200 MHz (实际时钟频率), 每个时钟周期在上升沿和下降沿各传输一次数据。求总线带宽。

解: 带宽 = $64 \text{ bit} \times 200 \times 10^6 \text{ Hz} \times 2 = 25.6 \times 10^9 \text{ bps} = 3.2 \text{ GB/s}$ 。

注意: DDR (Double Data Rate) 的等效数据速率是时钟频率的两倍。上例中常用标注「DDR-400」, 即等效频率 400 MHz。

结论 10.7: 其他总线性能指标

除了宽度、频率、带宽三个核心指标, 评价总线性能还需关注:

- 总线复用 (**Bus Multiplexing**): 地址线 and 数据线分时共享同一组物理线 (如 PCI 总线复用 AD 线)。优点: 减少引脚数和 PCB 走线; 缺点: 增加额外时序开销。
- 总线猝发传输 (**Burst Transfer**): 发出一次地址后连续传输多个数据单元 (如一个 Cache 块的多个字), 适合连续地址访问。
- 信号线数: 地址线、数据线和控制线的总数。
- 负载能力: 总线上可以挂接的设备数量上限。
- 总线标准: 不同标准的电压、时序、协议等规范。

注意

408 高频计算考点——总线带宽计算:

- 带宽公式: 带宽 = 宽度 (Byte) × 频率 (Hz) = 宽度 (bit) × 频率 (Hz) / 8。
- 注意 单位换算: $1 \text{ MHz} = 10^6 \text{ Hz}$, $1 \text{ MB/s} = 10^6 \text{ B/s}$ (408 中使用十进制换算, 除非明确说明)。
- 若题目给出「总线时钟频率」和「总线周期数」, 需先求出一个传输操作所需的时间, 再反推每秒的传输次数。

10.4 总线仲裁

当多个总线主设备同时请求使用总线时，需要仲裁机制决定哪个设备获得总线控制权。仲裁方式分为集中式仲裁和分布式仲裁两大类。

定义 10.2: 集中式仲裁

总线控制逻辑集中在一个仲裁器中，三种典型实现：

结论 10.8: 链式查询 (Daisy Chain)

信号线：总线请求 BR (Bus Request)，总线忙 BS (Bus Busy)，总线允许 BG (Bus Grant)。

工作原理：所有主设备共享一根 BR 线（线或），仲裁器收到请求后通过 BG 线发出允许信号。BG 信号按物理连接顺序串行依次经过每个主设备——距离仲裁器最近的设备具有最高优先级，若该设备没有请求，则将 BG 信号传给下一个设备。

优点：控制线少（只需 3 根信号线），实现简单，可以方便地增加设备。

缺点：

- 优先级固定（最近的设备优先级最高），可能造成「饥饿」——远端的低优先级设备长期得不到总线。
- BG 信号串行传递，若某段链路断开则后面所有设备都无法获得总线。
- 对电路故障敏感。

结论 10.9: 计数器定时查询 (Polling Count)

信号线：总线请求 BR，总线忙 BS（与链式查询相同）。不设 BG 线，改为一组设备地址线 ($\lceil \log_2 n \rceil$ 根， n 为主设备数)。

工作原理：当有设备通过 BR 请求总线时，仲裁器中的计数器开始计数，将计数值通过设备地址线发送给各设备。设备将自己的编号与计数值比较：匹配且有请求的设备获得总线使用权，同时 BS 置位以阻止新仲裁。

优先级策略灵活：

- 计数器每次从 0 计数——固定优先级（编号越小优先级越高）。
- 计数器从上一次获得总线的设备编号开始——循环优先级（各设备机会均等）。
- 计数器可从任意初始值开始——程序可控优先级。

优点：优先级可编程控制，灵活性比链式查询好。

缺点：需要 $\lceil \log_2 n \rceil$ 根设备地址线，设备数多时信号线较多；计数器计数过程增加了仲裁延迟（最坏需数到最大编号）。

结论 10.10: 独立请求方式 (Independent Request)

信号线：每个主设备拥有自己独立的总线请求线 BR_i 和总线允许线 BG_i （共 $2n$ 根信号线）。

工作原理：每个设备可以独立地向仲裁器发出请求。仲裁器内部根据预设的优先级逻辑（可由硬件实现也可由程序设定），在所有发出请求的设备中选择优先级最高的一个，通过对应的 BG_i 线向该设备发送允许信号。

优点：

- 响应最快——仲裁器直接选择，无需串行传递或计数器扫描。
- 优先级可灵活设定（甚至可通过程序动态修改）。

缺点：信号线数量最多（ $2n$ 根），设备多时成本高。

三种集中式仲裁方式对比

特性	链式查询	计数器定时查询	独立请求
控制线数（ n 个设备）	3	$2 + \lceil \log_2 n \rceil$	$2n$
响应速度	慢（串行）	中等（计数）	快（并行选择）
优先级灵活性	固定（物理位置）	可编程	可编程（最灵活）
电路可靠性	差（链断全废）	较好	较好
可扩展性	好（增减容易）	中等	差（增加需增线）

结论 10.11: 分布式仲裁

分布式仲裁不需要中央仲裁器，每个主设备都有自己的仲裁逻辑电路，通过设备编号或仲裁号竞争总线。

工作原理：每个主设备都有一个唯一的仲裁号（优先级号）。所有请求总线的设备将自己的仲裁号发到共享的仲裁线上。各设备通过比较逻辑，判断自己是否是当前请求者中优先级最高的——如果是，则获得总线控制权；否则等待。仲裁号较大的或较小的为高优先级（取决于设计约定）。

典型实现：具有线与/线或共享仲裁线的自选择式仲裁电路，可由各主设备同时驱动优先级编码并在共享线上比较，最终由最高优先级设备保留总线请求。传统 PCI 不属于这一类，而是采用中央仲裁器和各主设备独立的 REQ#/GNT# 信号。

优点：不需要中央仲裁器，可靠性高（单点故障不影响全局），易于扩展。

缺点：每个设备都需要仲裁电路，实现较集中式复杂。

注意

408 常考点——三种集中式仲裁的对比选择题：

- 「链式查询中，离仲裁器最近的设备优先级最高」——判断题常考点。
- 「独立请求方式的信号线数最多」—— n 个设备需要 $2n$ 条线。
- 「计数器定时查询中，计数器的初始值可以决定优先级策略」——这是其区别于链式查询的灵活性关键。
- 注意区分：链式查询的 BG 线是一根线串行传递，不是每个设备一根 BG 线。

10.5 总线定时——同步通信与异步通信

总线定时（Bus Timing）规定了总线上各信号之间的时序关系，确保主设备和从设备之间能够正确地收发数据。主要分为同步通信和异步通信两种方式。

定义 10.3: 同步通信（Synchronous Communication）

系统采用统一的时钟信号来控制数据传输的节奏。总线上的所有操作都在时钟信号的固定节拍下进行，主设备和从设备按照统一的时钟周期完成发送和接收。

特点：

- 主设备在时钟的某固定沿（如上升沿）发出地址和控制信号，从设备在随后的若干个固定时钟周期内将数据放到总线上。

- 每个总线周期完成的步骤和时间均固定不变（地址周期、数据周期等由时钟数决定）。
- 优点：实现简单，控制逻辑清晰，适合速度相近的设备之间通信。
- 缺点：
- 时钟偏斜（**Clock Skew**）问题：当总线较长或频率较高时，时钟信号到达不同设备的时刻不一致，可能导致时序错误。
 - 速度匹配困难：快慢设备必须按统一的最慢设备的时钟频率工作，限制了高速设备的性能。

定义 10.4: 异步通信（Asynchronous Communication）

不使用统一的时钟信号，而是通过握手信号（**Handshake**）来协调主设备和从设备之间的数据传送。

握手信号的基本交互过程（全互锁/全握手方式）：

1. 主设备将数据（或地址 + 命令）放到总线上，然后发出数据就绪（Ready / Data Valid）信号。
2. 从设备检测到数据就绪信号后，读取数据，然后发出应答（Acknowledge / Accept）信号。
3. 主设备收到应答信号后，撤销数据就绪信号（表示知道数据已被取走）。
4. 从设备检测到数据就绪信号撤销后，撤销应答信号（恢复到初始状态，准备下一次传输）。

优缺点：

- 优点：不同速度的设备可以灵活通信，快慢自适应，不需要统一时钟，总线长度可以更长。
- 缺点：控制逻辑复杂（需要额外的握手电路），每次传输的握手开销降低了传输效率（尤其在大批量数据传输时）。

结论 10.12: 同步与异步通信对比

同步通信 vs 异步通信			
特性	同步通信	异步通信	
时钟信号	需要统一时钟	无需统一时钟	
控制方式	固定时序（时钟节拍）	握手应答	
传输速率	固定（由时钟决定）	可变（由慢方决定）	
速度匹配	困难（快慢均按最慢）	灵活（自动适配）	
电路复杂度	低	高	
适用场景	部件间/板级高速通信	远距离/跨机箱通信	
典型标准	处理器-内存总线	RS-232、早期的 I/O 总线	

注意

408 常考点：

- 同步通信必须有时钟信号，异步通信通过握手信号取代时钟。
- 「同步通信比异步通信快」——未必正确！在设备速度差异性大的场景下，同步通信受制于

最慢设备反而效率更低。

- 扩展概念：半同步通信——在同步通信基础上引入「等待信号」(WAIT)，允许慢速设备在约定的时钟周期内向主设备发出等待请求以插入等待周期，兼顾简单性和灵活性。
- 扩展概念：分离式通信——将总线传输拆分为「请求」阶段和「为请求提供服务」阶段，两个阶段之间总线可以释放给其他主设备使用，提高总线利用率。

10.6 总线标准

定义 10.5: 总线标准的含义

总线标准是指计算机系统中各部件通过总线连接时必须遵循的协议规范，包括机械尺寸（插槽形状、引脚排列）、电气特性（电压、信号电平）、时序关系和通信协议等。标准化使得不同厂商的设备可以兼容。

结论 10.13: PCI 总线 (Peripheral Component Interconnect)

PCI 总线是一种由 Intel 在 1992 年推出的局部总线标准，曾长期作为 PC 中的标准扩展总线。

- 并行总线：数据宽度 32 位（可扩展为 64 位），地址和数据复用（AD 线分时复用），减少了引脚数。
- 工作频率与带宽：33 MHz 时钟下，32 位 PCI 带宽为 133 MB/s；66 MHz 下 64 位 PCI 带宽为 533 MB/s。
- 支持猝发传输 (Burst Transfer)：一次地址后可连续传输大量数据（不限于单个地址），故实际数据传输带宽比仅按单次地址-数据方式计算要高。
- 即插即用 (Plug and Play)：支持设备的自动配置——BIOS/OS 在启动时自动分配 I/O 地址、内存空间和中断号。
- 总线仲裁：采用集中式独立请求仲裁方式，支持多个主设备。
- 与 CPU 独立：PCI 总线独立于处理器类型（不绑定特定 CPU），通过桥接芯片与处理器总线连接。

结论 10.14: PCI Express (PCIe) 总线

PCIe 是 PCI 的替代标准，采用串行点对点双工连接方式，完全不同于 PCI 的并行共享总线架构。

- 串行传输：使用差分信号对（一对发送 + 一对接收），每个方向一个通道 (Lane)，可组合 1、2、4、8、16、32 条通道 (PCIe $\times 1$ 、 $\times 4$ 、 $\times 8$ 、 $\times 16$)。
- 全双工：每个通道可同时进行发送和接收。
- 数据编码与带宽：
 - PCIe 1.x/2.x：8b/10b 编码（每 8 位数据用 10 位传输，20% 编码开销）。
 - PCIe 3.0：128b/130b 编码（编码开销降至约 1.5%）。
 - 带宽计算示例：PCIe 3.0 $\times 1$ 的原始速率 8 GT/s (Giga Transfers per second)，考虑 128b/130b 编码后有效带宽 $\approx 8 \times 128/130 \times 1/8 \approx 0.985$ GB/s（单向）。PCIe 3.0 $\times 16$ 双向总带宽约 32 GB/s。
- 交换式架构：不同于 PCI 的共享总线拓扑，PCIe 采用基于交换器 (Switch) 的点对点拓

扑，每个设备独占与交换器的通信通道，不存在总线竞争问题。

- 分层协议：事务层（Transaction Layer）、数据链路层（Data Link Layer）、物理层（Physical Layer）。
- 差分信号与嵌入式时钟：数据与时钟编码在一起，无需单独的时钟线。

结论 10.15: USB 总线（Universal Serial Bus）

USB 是连接计算机与外部设备的标准串行接口总线，支持热插拔（Hot Plug）。

拓扑结构：树形星型拓扑，单一主机（Host）控制所有通信。一个 USB 系统有一个主机控制器，通过集线器（Hub）连接最多 127 个设备（含集线器）。

传输模式：USB 支持四种传输类型以适应不同设备需求：

1. 控制传输（**Control Transfer**）：用于设备的初始配置和命令/状态通信（如设备枚举阶段）。
2. 批量传输（**Bulk Transfer**）：用于大批量数据的可靠传输，无带宽保证，利用总线空闲传输（如 U 盘、打印机）。
3. 中断传输（**Interrupt Transfer**）：用于小批量、有延迟保证的周期性数据传输（如键盘、鼠标）。
4. 同步传输（**Isochronous Transfer**）：用于实时数据流，保证带宽但无纠错重传（如音频、视频设备）。

版本演进与速率：

- USB 1.1：低速 1.5 Mbps，全速 12 Mbps。
- USB 2.0：高速 480 Mbps。
- USB 3.0：超高速 5 Gbps（增加独立的发送/接收通道，实现全双工）。
- USB 3.1/3.2：10 Gbps / 20 Gbps。
- USB4：最高 40 Gbps（基于 Thunderbolt 3 协议）。

结论 10.16: 常见总线标准速查表

常见总线标准对比

总线	类型	传输方式	典型带宽	适用场景
PCI	并行	共享总线/半双工	133 MB/s (32bit@33MHz)	传统扩展卡
PCIe 3.0 ×16	串行	点对点/全双工	≈ 16 GB/s	显卡/SSD
USB 2.0	串行	共享/半双工	480 Mbps	外设（键鼠/U 盘）
USB 3.0	串行	全双工	5 Gbps	高速外设
SATA 3.0	串行	点对点	6 Gbps	硬盘/SSD

注意

408 高频辨析——PCI vs PCIe：

- PCI 是并行共享总线，PCIe 是串行点对点连接。这是二者最本质的架构差异。
- PCI 使用地址/数据复用（AD 线），PCIe 使用数据包（Packet）交换。

- PCIe $\times n$ 中的 n 指的是通道 (Lane) 数, 不是总线宽度 (bit)。每条通道为全双工差分信号对。
- PCIe 没有传统的「总线仲裁」概念——采用交换式拓扑, 各设备独立通道。
- USB 是主从式 (Host-Device) ——所有通信由主机发起, 设备不能主动发送数据。

10.7 基础过关题

- (真题风格·选择题) 在系统总线的数据线上, 不可能传输的是
(A) 指令 (B) 操作数
(C) 握手 (应答) 信号 (D) 中断类型号
- (真题风格·选择题) 下列选项中, 可提高同步总线数据传输率的是
I. 增加总线宽度 II. 提高总线工作频率
III. 支持突发传输 IV. 采用地址/数据线复用
(A) 仅 I、II (B) 仅 I、II、III
(C) 仅 I、III、IV (D) I、II、III、IV
- (真题风格·选择题) 下列关于总线的叙述中, 错误的是
(A) 总线是连接两个或多个部件的信息传输线
(B) 总线的工作频率与总线时钟频率不一定相等
(C) 异步总线一次握手过程完成一位数据的传送
(D) 突发传送方式下, 一次总线事务可以传送多个数据
- (真题风格·选择题) 假设某系统总线在一个总线周期中并行传输 4 字节信息, 一个总线周期占用 2 个时钟周期, 总线时钟频率为 10 MHz, 则总线带宽是
(A) 10 MB/s (B) 20 MB/s
(C) 40 MB/s (D) 80 MB/s
- (真题风格·选择题) 某同步总线采用数据线和地址线复用方式, 其中地址/数据线有 32 根, 总线时钟频率为 66 MHz, 每个时钟周期传送两次数据 (上升沿和下降沿各传送一次数据)。假设数据相位进行连续突发传输, 并忽略初始地址阶段及其他协议开销, 则该总线的数据相位理论峰值带宽是
(A) 132 MB/s (B) 264 MB/s
(C) 528 MB/s (D) 1056 MB/s

10.8 真题风格综合训练

- (真题风格·选择题) 某同步总线的时钟频率为 100 MHz, 宽度为 32 位, 地址/数据线复用, 每传输一个地址或数据占用一个时钟周期。若该总线支持突发 (猝发) 传输方式, 则一次「主存写」总线事务传输 128 位数据所需要的时间至少是

(A) 20 ns (B) 40 ns

(C) 50 ns (D) 80 ns

7. (真题风格·选择题) 下列关于总线设计的叙述中, 错误的是

(A) 并行总线传输速率一定高于串行总线

(B) 信号线复用可以减少信号线的数量

(C) 突发(猝发)传输可以提高总线数据传输率

(D) 分离事务通信可以提高总线利用率

8. (真题风格·综合题) 某计算机的主存模块通过同步总线与 CPU 连接。总线宽度为 64 位, 时钟频率为 200 MHz, 每个时钟周期传送一次数据。CPU 需要从主存读取一个 64 字节的 Cache 行。请回答:

- (1) 若该总线不支持突发传输(每次只能传一个数据字, 且每个数据字都需要先发地址再传数据), 完成一次 Cache 行读取至少需要多少个时钟周期? 总时间是多少?
- (2) 若该总线支持突发传输(只发一次起始地址, 随后连续传数据), 完成同一操作至少需要多少个时钟周期? 总时间是多少?
- (3) 分别计算两种方式下的实际数据传输带宽(以 MB/s 为单位, 设 $1\text{ MB} = 10^6\text{ B}$), 并说明突发传输的优势。

拓展阅读:

- 唐朔飞《计算机组成原理》(第 3 版) 第 4 章「总线」——4.3 总线仲裁、4.4 总线定时、4.5 总线标准。总线仲裁的三种集中式方式对比是 408 经典考点(链式查询/计数器定时查询/独立请求)。
- **Patterson & Hennessy**《计算机组成与设计》第 4 章 4.9 节——并行性与高级指令级并行(含总线互连在现代处理器中的角色); 第 6 章 6.4 节涉及 GPU 中的互连总线架构。
- **CodeBrick 408** 真题可视化(codebrick.tech)——总线基本概念与组成共 18 道真题, 配可视化演示, 可在线刷题。

11 输入输出系统

11.1 I/O 接口的功能与基本结构

I/O 设备种类繁多——键盘、鼠标、显示器、磁盘、网卡等, 其工作速度、信号形式、数据格式各不相同。CPU 不能直接与每一种设备直接相连, 而是通过 **I/O 接口 (I/O Interface)** 来实现 CPU 与 I/O 设备之间的数据交换。

定义 11.1: I/O 接口的主要功能

I/O 接口是连接 CPU (或系统总线) 与外部设备的桥梁, 其核心功能包括:

1. **数据缓冲与锁存**: 解决 CPU 与外设速度不匹配的问题。接口中的数据缓冲寄存器 (**DBR**) 暂存 CPU 与外设之间传送的数据。

- 2. 地址译码与设备选择：接收 CPU 发出的地址信号，经译码后选中指定的 I/O 端口（即接口中的某个寄存器）。
- 3. 信号电平与格式转换：将外设的非 TTL 电平信号转换为 CPU 可接受的 TTL 电平，进行串—并转换、A/D—D/A 转换等。
- 4. 传送控制信息：接收并解释 CPU 发来的控制命令（读/写、启动/停止等），向外设发出操作控制信号。
- 5. 反映设备状态：向 CPU 提供设备当前状态（忙/闲、就绪/未就绪、出错等），CPU 通过读状态寄存器来获取。
- 6. 中断管理：当外设需要 CPU 服务时，向 CPU 发出中断请求；接口中包含中断屏蔽逻辑和中断类型号（或向量地址）的生成电路。

结论 11.1: I/O 接口的典型内部结构

一个典型的 I/O 接口内部包含以下寄存器（即 I/O 端口）：

I/O 接口中的典型寄存器		
寄存器	名称	功能
数据缓冲寄存器（DBR）	Data Buffer Register	暂存输入/输出的数据
控制寄存器（CR）	Control Register	存放 CPU 发来的控制命令
状态寄存器（SR）	Status Register	反映设备当前状态（就绪/忙/出错）
设备地址选择电路	Address Decoder	识别本接口是否被 CPU 选中

其中数据缓冲寄存器、控制寄存器和状态寄存器都是 **I/O 端口（I/O Port）**，每个端口都有一个独立的地址（端口地址），CPU 通过端口地址来访问它们。

注意

408 常考点——I/O 端口 vs I/O 接口：

- **I/O 接口**是一个功能实体（一块电路或芯片），通常包含多个 **I/O 端口**。
- **I/O 端口**是 I/O 接口中可被 **CPU** 直接访问的寄存器，每个端口有独立地址。
- 选择题典型陷阱：「一个 I/O 接口就是一个 I/O 端口」——错误，一个接口通常包含多个端口（数据端口 + 控制端口 + 状态端口）。

11.2 I/O 端口编址方式

CPU 如何访问 I/O 端口？关键问题是：I/O 端口的地址与主存地址的关系——两者如何编址？408 考试要求熟练掌握两种编址方式及其对比。

定义 11.2: 统一编址 vs 独立编址

I/O 端口的编址方式决定了 CPU 通过什么指令和什么地址空间来访问 I/O 端口。

- 1. 统一编址（存储器映射 I/O，Memory-Mapped I/O）：
 - I/O 端口地址与主存地址统一编址，即把 I/O 端口看作主存的一个单元（或区域）。

- CPU 用访问存储器的指令（如 LOAD/STORE、MOV）来访问 I/O 端口——无需专用 I/O 指令。
- 优点：指令丰富、编程灵活；不需专门的 I/O 指令和 I/O 控制信号。
- 缺点：I/O 端口占用了一部分主存地址空间，减少了可用的内存容量；访问 I/O 时地址译码可能更复杂（因为地址范围大了）。
- 典型代表：ARM 系列、MIPS、Motorola 68K。

2. 独立编址（I/O 映射 I/O，Port-Mapped I/O / Isolated I/O）：

- I/O 端口地址与主存地址分开编址，各有一套地址空间，互不重叠。
- CPU 使用专用的 I/O 指令（如 x86 中的 IN、OUT）来访问 I/O 端口；使用访存指令（如 MOV）来访问内存。CPU 通过不同的控制信号（MEMR/MEMW vs IOR/IOW）来区分。
- 优点：不占用主存地址空间；I/O 地址空间较小，译码简单；程序易读（I/O 操作一眼可辨）。
- 缺点：需要专门的 I/O 指令（指令种类少、功能受限）；需要额外的控制信号线。
- 典型代表：x86（Intel 8086 系列）。

统一编址与独立编址的对比		
比较项	统一编址（Memory-Mapped I/O）	独立编址（Port-Mapped I/O）
地址空间	I/O 与主存统一编址	I/O 与主存各自独立
访问指令	访存指令（LOAD/STORE 等）	专用 I/O 指令（IN/OUT）
优点	指令丰富，编程灵活	不占主存空间，译码简单
缺点	占用主存地址空间	需要专用指令，功能受限
典型架构	ARM、MIPS	x86

注意

408 常考点——两种编址方式的核心区别：

- 最本质的区分标准：**CPU 使用什么类型的指令来访问 I/O 端口**。用访存指令 → 统一编址；用专用 I/O 指令 → 独立编址。
- 注意：x86 实际上两种方式都支持（早期 x86 以独立编址为主，但也可以通过 Memory-Mapped I/O 将部分外设映射到内存空间），但 408 考试中通常将 x86 归类为独立编址的典型代表。
- 判断题陷阱：「统一编址方式中，I/O 端口不需要地址」——错误，统一编址中 I/O 端口同样有地址，只是与主存共用地址空间而已。

11.3 I/O 控制方式：程序查询（轮询）

计算机发展过程中，I/O 控制方式经历了从简单到复杂的演变：程序查询 → 程序中断 → DMA → 通道/IO 处理器。每一种方式都为了解决前一种方式的不足而提出。

定义 11.3: 程序查询方式 (轮询, Polling)

程序查询方式是最简单的 I/O 控制方式。CPU 通过执行一段 I/O 查询程序, 不断检测外设的状态寄存器, 当设备就绪时执行数据传送; 若设备未就绪, 则 CPU 原地等待 (或循环检测)。

工作流程:

1. CPU 向 I/O 接口发出读/写命令, 启动外设。
2. CPU 反复读取状态寄存器, 检测「就绪 (Ready)」位。
3. 就绪后, CPU 执行数据传送 (从数据缓冲寄存器读出/写入)。
4. 若还有数据需传送, 回到步骤 1; 否则结束。

结论 11.2: 程序查询方式的特点

- **CPU 与外设串行工作**: CPU 在查询等待期间不能做其他有用工作, 效率极低。
- **硬件开销最小**: 不需要额外的中断控制逻辑或 DMA 控制器, 仅需基本 I/O 接口电路。
- **适用于慢速外设和简单系统**: 如单片机中对 LED、按键的检测。
- **「独占式查询」**: CPU 完全被 I/O 操作占用, 不适用于多任务系统。

注意

408 选择题中, 程序查询方式的核心特征是「**CPU 与外设串行工作、CPU 主动查询**」。这是它与中断方式 (CPU 与外设可部分并行) 和 DMA 方式 (CPU 与外设高度并行) 的根本区别。

11.4 I/O 控制方式: 程序中断

程序查询方式的最大问题是 CPU 必须「原地踏步」等待慢速外设。中断方式的核心思想是: 当外设准备好时主动通知 CPU, CPU 在此期间可以做其他事情。这实现了 CPU 与外设的部分并行。

定义 11.4: 程序中断的基本概念

程序中断是指在计算机执行程序的过程中, 当出现异常情况或特殊请求 (外设就绪、运算溢出、非法指令等) 时, CPU 暂停当前程序的执行, 转去执行相应的中断服务程序 (Interrupt Service Routine, ISR), 处理完毕后再返回原程序继续执行。

这一过程与「子程序调用」类似, 但中断是随机发生的 (由外部事件触发), 而非由程序主动安排。

中断的分类 (按来源):

1. **外部中断 (硬件中断)**: 来自 CPU 外部 (如 I/O 设备、时钟、电源故障等), 可进一步分为可屏蔽中断和不可屏蔽中断 (NMI)。
2. **内部中断 (异常)**: 来自 CPU 内部, 由指令执行引发 (如除零、溢出、非法指令、缺页等)。在 CS:APP 术语中称为「异常」(Exception)。

结论 11.3: 中断与异常的对比

中断 (Interrupt) vs 异常 (Exception)		
比较项	中断 (硬件中断)	异常 (Fault/Trap/Abort)
触发来源	CPU 外部 (I/O 设备等)	CPU 内部 (指令执行)
是否异步	异步 (与当前指令无关)	同步 (由当前指令引起)
返回位置	通常为下一条指令	故障可重启当前指令;陷阱返回下一条; 终止通常不返回
典型例子	键盘输入、磁盘 I/O 完成	除零、缺页、系统调用
CS:APP 分类	中断	故障(Fault)/陷阱(Trap)/终止(Abort)

中断向量表

定义 11.5: 中断向量与中断向量表

当 CPU 响应某个中断时, 需要知道对应的中断服务程序 (ISR) 的入口地址。这个信息通过中断向量表 (**Interrupt Vector Table, IVT**) 来获得。

- 中断类型号 (中断向量号): 为每一种中断源分配的唯一编号 (通常为 8 位, 即 0-255)。
- 中断向量: 即中断服务程序的入口地址 (CS:IP 或一个完整的内存地址)。每个中断向量通常占 4 字节 (段地址 2B + 偏移地址 2B, 实模式下) 或 8 字节 (保护模式/64 位模式下)。
- 中断向量表: 将所有中断向量按中断类型号为索引集中存储在一个连续的内存区域中。通常位于内存的低地址区 (x86 实模式下为 0-1023, 即 4×256 字节)。

中断向量地址的计算 (x86 实模式):

$$\text{中断向量表基址} + \text{中断类型号} \times 4$$

即: 取中断类型号为 n , 则中断向量存放在内存地址 $n \times 4$ 处 (4 字节: 前 2 字节为 IP, 后 2 字节为 CS)。

注意

408 重要考点——中断向量表:

- 中断向量 = 中断服务程序入口地址, 不是向量号/类型号本身。
- 中断向量表在内存中的位置 (通常是低地址区 0-1023) 和每个表项的大小需要记住。
- 常考计算: 已知中断类型号 n 和向量表基址, 求该中断向量的存储地址: $\text{Address} = \text{Base} + n \times 4$ (实模式)。

中断响应过程

这是 408 的核心考点之一, 要求完整掌握 CPU 响应中断的硬件操作序列。

结论 11.4: CPU 响应中断的完整过程（硬件自动完成）

「中断响应周期」是指从 CPU 接收到中断请求信号到开始执行中断服务程序第一条指令之间的硬件操作序列。以下步骤由 CPU 硬件自动完成（不需程序干预）：

中断响应七步曲

1. 关中断：CPU 自动清除中断允许标志（ $IF = 0$ ），禁止在本次中断响应过程中再响应新的中断请求。
2. 保护断点：将当前程序的程序计数器（**PC/IP**）和程序状态字（**PSW/FLAGS**）压入堆栈保存，以便中断处理完毕后能正确返回。
3. 识别中断源：CPU 通过中断控制器（如 8259A PIC）获取当前请求中断的中断类型号。中断控制器根据优先级裁决，将最高优先级的中断类型号发送给 CPU。
4. 获取中断向量：根据中断类型号 n ，从中断向量表中读出对应的中断服务程序入口地址（中断向量）。
5. 转入中断服务程序：将中断向量加载到 PC/IP 和 CS（或相应寄存器），CPU 开始执行 ISR 的第一条指令。
6. 执行中断服务程序：ISR 通常包含保护现场（PUSH 各寄存器）、中断处理核心逻辑、恢复现场（POP 各寄存器）。
7. 返回断点：执行 IRET（中断返回指令），从堆栈中弹出并恢复保存的 PSW 和 PC/IP；IF 按中断前保存的值恢复。若中断前 $IF=1$ ，返回后才重新允许可屏蔽中断，不能表述为 IRET 无条件将 IF 置 1。

注意

408 高频考点精细辨析：

- 「保护断点」vs「保护现场」：前者由 CPU 硬件自动完成（压栈 PSW + PC/IP）；后者由 ISR 开头/结尾的 PUSH/POP 指令完成——这是两种不同层次的操作，408 选择题中经常把两者混淆来考。
- 中断向量表的初始化：由操作系统在启动时完成。ISR 的入口地址由 OS 写入中断向量表（不全是硬件行为）。
- 中断类型号的来源：由中断控制器或中断接口电路提供，CPU 根据该类型号查向量表。
- 关中断是为了保证中断响应过程不被嵌套打断（避免断点/现场信息被破坏）。但可以在 ISR 中适当位置开中断以支持多重中断（见下文）。

多重中断与中断嵌套

定义 11.6: 多重中断

多重中断（中断嵌套）是指 CPU 在执行某个中断服务程序的过程中，又响应了新的、优先级更高的中断请求，形成中断嵌套。

实现多重中断的关键条件：

1. 在 ISR 中适当时机（通常在保护现场之后）执行开中断指令（STI, $IF = 1$ ），允许响应新的中断请求。
2. 中断系统必须有优先级裁决机制：只有优先级更高的中断请求才能打断当前 ISR（同级或

低级中断被屏蔽)。

结论 11.5: 多重中断的嵌套执行示意

设优先级: $A > B > C$ (A 最高, C 最低)。执行序列如下:
一种合法的嵌套序列可简写为

主程序 $\rightarrow \text{ISR}_C \rightarrow \text{ISR}_A \rightarrow \text{ISR}_C \rightarrow \text{ISR}_B \rightarrow \text{ISR}_C \rightarrow$ 主程序。

含义是: C 执行过程中 A 请求到来, 因 $A > C$, CPU 暂停 C 转去处理 A ; A 返回 C 后, 若 B 请求到来, 因 $B > C$, CPU 又可嵌套执行 B 。ISR 返回时总是先恢复被它直接打断的上下文, 不能跨层返回。

注意

408 常考辨析——实现可嵌套多重中断的必要条件:

1. 保护必要现场后, 必须在 **ISR** 中适时重新允许中断, 或由硬件提供等效的分级嵌套机制, 否则更高优先级请求不能打断当前 **ISR**; 不允许嵌套的单级中断系统没有这一要求。
2. 必须有中断优先级裁决 (只有优先级更高的请求才能获得响应)。
3. 注意: 在保护现场之前不能开中断——否则如果新中断修改了寄存器, 恢复现场时会用被改写的值覆盖 **ISR** 的现场, 导致原程序执行出错。

11.5 I/O 控制方式: DMA (直接存储器访问)

程序查询和程序中断方式的共同问题是: 数据的传送仍然需要 CPU 执行指令来完成 (读端口 $\rightarrow$ 写内存, 或者读内存 $\rightarrow$ 写端口)。当需要高速大批量数据传送时 (如磁盘读写), CPU 的指令执行开销成为瓶颈。DMA 方式通过专门的硬件控制器直接实现内存与外设之间的数据传送, CPU 仅在传送开始和结束时进行干预。

定义 11.7: DMA 方式的基本原理

DMA (Direct Memory Access, 直接存储器访问) 是指外部设备通过 **DMA 控制器 (DMAC)** 直接与主存储器进行数据交换, 数据传送过程不经过 **CPU** (CPU 不参与逐字节/逐字的传送), 从而大幅提升传送速率。

DMA 控制器的基本组成:

1. **地址寄存器 (AR)**: 存放要访问的内存单元地址 (初始值由 CPU 设定, 每传送一个数据自动递增或递减)。
2. **字计数器 (WC)**: 存放要传送的数据字数 (初始值由 CPU 设定, 每传送一个数据自动减 1, 减到 0 时传送结束)。
3. **数据缓冲寄存器 (DR)**: 暂存每次传送的数据。
4. **控制/状态逻辑**: 接收 CPU 发来的控制命令, 发出 DMA 请求 (DREQ) 和总线请求 (HOLD/BR), 接收 DMA 响应 (HLDA/BG)。

DMA 传送过程

这是 408 的另一个核心考点。DMA 传送过程分为三个阶段: 预处理、数据传送、后处理。

结论 11.6: DMA 传送的三阶段**阶段一：预处理（初始化，CPU 完成）**

1. CPU 执行 I/O 指令，对 DMAC 进行初始化：
 - 将内存起始地址写入 DMAC 的地址寄存器（AR）；
 - 将传送字数写入 DMAC 的字计数器（WC）；
 - 设置读/写方向和传送模式。
2. CPU 向 I/O 接口发出启动命令，外设开始准备数据。
3. CPU 继续执行原来的程序（与外设并行工作）。

阶段二：数据传送（DMAC 完成，CPU 不参与）

1. 外设准备就绪，向 DMAC 发出 **DMA 请求（DREQ）**。
2. DMAC 向 CPU 发出总线请求（**HOLD 或 BR**）——请求 CPU 让出总线控制权。
3. CPU 在当前总线周期结束后，释放总线控制权（将地址总线、数据总线、控制总线置为高阻态），向 DMAC 发出总线响应（**HLDA 或 BG**）。
4. DMAC 接管总线，向外设发出 DMA 应答（**DACK**），然后：
 - 将 AR 的内容送到地址总线（给出内存地址）；
 - 发出读/写控制信号；
 - 在内存与外设之间完成一个数据字的传送；
 - $AR \leftarrow AR + 1$ （或 -1 ）， $WC \leftarrow WC - 1$ 。
5. 若 $WC \neq 0$ 且外设继续就绪，重复步骤 4（继续传送下一个数据）；若 $WC = 0$ ，则传送完毕。
6. DMAC 撤销总线请求，将总线控制权交还 CPU。

阶段三：后处理（CPU 完成）

1. 当 $WC = 0$ 时，DMAC 向 CPU 发出中断请求（注意：这是 DMA 传送结束的中断信号，不同于阶段二中每传送一个字的中断）。
2. CPU 响应中断，执行中断服务程序：
 - 检查传送过程中是否有错误；
 - 决定是否启动下一次 DMA 传送。

注意

408 核心考点——DMA 预处理与后处理由 CPU 完成，数据传送由 DMAC 完成：

- 预处理：CPU 初始化 DMAC 的 AR、WC 等寄存器（通过 I/O 指令写 DMAC 的端口）。
- 数据传送：完全由 DMAC 硬件完成，CPU 不参与每一个字的传送。
- 后处理：DMAC 向 CPU 发中断请求（仅一次，而非每传送一个字就中断一次），CPU 检查传送结果。
- 选择题典型陷阱：「DMA 方式中，CPU 完全不参与任何工作」——错误，CPU 参与了预处理和后处理。

DMA 与中断的区别

结论 11.7: DMA 方式与程序中断方式的全面对比

DMA 与程序中断的对比（408 高频综合考点）		
比较项	程序中断方式	DMA 方式
数据传送单位	以字或字节为单位	以数据块为单位
数据传送路径	CPU（外设 ↔ CPU ↔ 内存）	不经 CPU（外设 ↔ 内存）
CPU 参与程度	每传送一个数据都需 CPU 执行 ISR	只在预处理和后处理时参与
中断请求	每完成一个数据传送请求一次中断	整个数据块传送结束后才请求一次中断
适用场景	低速/中速字符设备（键盘、鼠标）	高速块设备（磁盘、网卡、显卡）
CPU 与外设并行	部分并行（CPU 可在数据传送间工作）	高度并行，但访问主存/总线时仍可能与 CPU 竞争
硬件开销	需要中断控制器（PIC/APIC）	需要 DMA 控制器（DMAC）
是否窃取总线周期	否	是（周期挪用/周期窃取）

结论 11.8: DMA 中的数据传送方式（周期挪用）

- DMA 传送需要占用总线。根据 DMAC 与 CPU 对总线的使用方式，分为三种策略：
- 1. 周期挪用 (**Cycle Stealing**)：DMAC 每传送一个字申请并取得一个总线周期，传送后立即释放总线。若 CPU 此时也需要总线，CPU 的访存会被推迟一个周期；它不等同于只利用 CPU 空闲总线周期的透明 DMA。
 - 2. 停止 CPU 访问内存 (**Burst / Block Mode**)：DMAC 申请总线后，一直占用直到整个数据块传送完毕才释放。传送期间 CPU 不能访问内存（但可以继续使用 Cache 中的指令和数据）。适用于高速大批量传送。
 - 3. **DMA 与 CPU 交替访存**：将总线时间分成固定的两个部分，轮流分配给 CPU 和 DMAC。不需要总线申请和释放在过程，但总线利用率可能不高。

要点 11.1: DMA 传送时间计算

408 考试中可能出现 DMA 传送时间相关的计算题。
设传送一个字所需的时间为 t （即一个总线周期），数据块大小为 N 个字，则 DMA 数据传送阶段的时间为：

$$T_{\text{DMA data transfer}} = N \times t$$

注意：预处理时间和后处理时间不计入数据传送阶段。
典型计算场景：比较中断方式和 DMA 方式下完成相同数据传送所需的 CPU 时间开销。中断方式下每传送一个字 CPU 都要执行 ISR（约几十到几百条指令）；DMA 方式下 CPU 只在开始和结束时各执行少量指令。

11.6 I/O 控制方式：通道方式与 I/O 处理器

定义 11.8: 通道方式

通道 (**Channel**) 是一个具有特殊功能的处理器 (或称为 I/O 处理器, IOP), 它有自己的指令系统 (通道指令), 专门负责 I/O 操作的管理和控制。CPU 只需向通道发出 I/O 命令 (通道程序), 通道即可独立完成整个 I/O 操作, 操作完成后向 CPU 发中断。

- 通道的出现进一步将 CPU 从 I/O 管理的繁重任务中解放出来。
- 通道执行的程序称为通道程序 (存放在主存中), 由一系列通道命令字 (CCW) 组成。
- 通道与 DMA 的区别: DMA 控制器只能实现简单的数据传送 (固定源/目标地址, 连续传送), 而通道可以执行复杂的通道程序 (如查找、分支转移等)。

结论 11.9: 四种 I/O 控制方式的演进

从 CPU 参与程度和并行性两个维度来看四种方式的演进:

四种 I/O 控制方式的对比

特点	程序查询	程序中断	DMA	通道
数据传送单位	字/字节	字/字节	数据块	数据块/多个块
是否经 CPU 传送数据	是	是	否	否
CPU 干预频度	全程	每传送一字	仅开始和结束	仅开始和结束
CPU 与外设并行度	无	部分并行	高度并行	高度并行
是否有专用处理器	否	否	否 (仅有控制器)	是 (I/O 处理器)
硬件复杂度	最低	低	中	高

11.7 408 高频考点与答题策略

结论 11.10: 本章 408 核心考点总结

1. **I/O 端口编址方式**: 统一编址 (用访存指令) vs 独立编址 (用专用 I/O 指令) —— 本质区别在于 CPU 使用什么指令访问 I/O 端口。典型选择题陷阱: 混淆端口与接口、混淆指令类型。
2. **中断响应过程 (七步曲)**: 关中断 → 保护断点 (PSW + PC 自动压栈) → 识别中断源 (获取中断类型号) → 查向量表获取 ISR 入口 → 执行 ISR → 恢复现场 → 中断返回 (IRET)。保护断点由硬件完成, 保护现场由软件 (**ISR 指令**) 完成, 是常见的辨析点。
3. **中断向量表**: 向量表在内存低地址区 (0-1023), 每个向量占 4 字节 (实模式 8086), 地址 = 类型号 × 4。中断向量 = ISR 入口地址 (≠ 类型号)。
4. **多重中断**: ISR 中开中断 + 优先级裁决 = 可实现嵌套。注意: 保护现场前不能开中断。
5. **DMA 三阶段**: 预处理 (CPU 初始化 DMAC 寄存器) → 数据传送 (DMAC 控制, CPU 不参与) → 后处理 (DMAC 发中断, CPU 检查结果)。
6. **DMA 与中断的区别**: 在 408 的典型模型中, DMA 通常在数据块传送结束时才发一次中断 (中断驱动 I/O 则常按字中断); DMA 数据不经过 CPU 通用寄存器, 也不由 CPU 指令逐字搬运; CPU 在当前总线周期结束的边界响应 DMA 总线请求, 而可屏蔽中断通常在

当前指令周期结束后响应。

7. 周期挪用: DMAC 在需要时暂停 CPU 的一次或若干总线周期, 取得总线并完成数据传送, 造成 CPU 的访存停顿; 这不等同于只使用 CPU 空闲总线周期的透明 DMA。

注意

常见易错点:

- DMA 传送的逻辑端点是「外设 $\leftrightarrow$ 主存」, 数据不经过 **CPU 通用寄存器**, CPU 也不执行逐字搬运指令。DMAC 负责地址、计数和总线控制; 具体实现可以设置数据缓冲寄存器暂存一次传送的数据, 因此不能绝对断言数据从不经 DMAC 内部缓冲。
- 中断响应周期中, CPU 通常完成当前指令后才检查可屏蔽中断请求, 即在指令周期结束时响应; DMA 总线请求则在当前总线周期结束的边界响应, 因而可能在一条指令尚未完成时取得后续总线周期。周期挪用还可能使 CPU 的后续访存等待, 并不要求总线原本处于空闲状态。
- 通道与 DMA 的区别不在传送速度, 而在于通道是一个有指令系统的处理器 (可执行通道程序), DMA 只是一个硬件控制器。
- 对于「中断隐指令」——它实际上并不是一条真正的机器指令, 而是 CPU 在中断响应周期中由硬件自动执行的一系列操作 (保护断点、获取中断向量等), 408 中可能以「不属于指令系统中的指令」来描述。

12 习题

12.1 基础过关题

1. (真题风格·选择题) 下列有关 I/O 接口的叙述中, 错误的是

- (A) 状态寄存器和控制寄存器可以合用同一个寄存器 (不同位段)
- (B) I/O 端口是指 I/O 接口中 CPU 可访问的寄存器
- (C) 独立编址方式下, I/O 端口地址和主存地址可以相同
- (D) 统一编址方式下, CPU 不能用访存指令访问 I/O 端口

2. (真题风格·选择题) I/O 指令实现的数据传送通常发生在

- (A) I/O 端口和 I/O 设备之间
- (B) 通用寄存器和 I/O 设备之间
- (C) I/O 端口之间
- (D) 通用寄存器和 I/O 端口之间

3. (真题风格·选择题) 下列关于多重中断系统的叙述中, 错误的是

- (A) 在一条指令执行结束时响应中断
- (B) 中断处理期间 CPU 处于关中断状态
- (C) 中断请求的产生与当前正在执行的指令无关
- (D) CPU 通过采样中断请求信号来检测是否有中断请求

4. (真题风格·选择题) 下列关于中断 I/O 方式和 DMA 方式比较的叙述中, 错误的是

- (A) 中断 I/O 方式请求的是 CPU 时间, DMA 方式请求的是总线使用权
- (B) 中断 I/O 方式在指令执行结束后响应, DMA 方式在总线事务结束后响应
- (C) 中断 I/O 方式下数据传送由软件完成, DMA 方式下数据传送由硬件完成
- (D) 中断 I/O 方式适用于所有外部设备, DMA 方式仅适用于快速外部设备

5. (真题风格·选择题) 下列关于 DMA 方式的叙述中, 正确的是

- I. DMA 传送前由设备驱动程序设置传送参数
- II. 数据传送前由 DMA 控制器请求总线使用权
- III. 数据传送由 DMA 控制器直接控制总线完成
- IV. DMA 传送结束后的处理由中断服务程序完成

- (A) 仅 I、II
- (B) 仅 I、II、III
- (C) 仅 II、III、IV
- (D) I、II、III、IV

12.2 真题风格与综合训练

6. (真题风格·选择题) 若设备采用周期挪用 DMA 方式进行输入输出, 每次 DMA 传送的数据块大小为 512 字节, 相应的 I/O 接口中有一个 32 位数据缓冲寄存器, 对于数据输入过程, 下列叙述中错误的是
- (A) 每准备好 32 位数据, DMA 控制器就发出一次总线请求
 - (B) 若 CPU 和 DMA 控制器同时请求总线, DMA 控制器优先级更高
 - (C) 在整个数据块的传送过程中, CPU 不可以访问主存
 - (D) 数据块传送结束后, DMA 控制器向 CPU 发出中断请求
7. (真题风格·选择题) 下列关于 I/O 控制方式的叙述中, 错误的是
- (A) 程序查询方式中, CPU 通过执行查询程序完成 I/O 操作
 - (B) 程序中断方式中, CPU 通过执行中断服务程序完成数据传送
 - (C) DMA 方式中, CPU 通过执行 DMA 传送程序完成数据传送
 - (D) 对于 SSD 和网络适配器等高速设备, 一般采用 DMA 方式进行 I/O
8. (真题风格·综合题) 某计算机的 CPU 主频为 500 MHz, CPI 为 1.5。某外设通过中断方式进行数据输入, 每次中断传送 4 字节, 每次中断处理 (包括保护/恢复现场和实际数据传送) 共需执行 100 条指令。外设的数据传输率为 2 MB/s。
- (1) 每秒需要多少次中断? 每次中断处理的 CPU 时间是多少?
 - (2) CPU 用于该外设 I/O 处理的时间占 CPU 总时间的百分比是多少?
 - (3) 若改为 DMA 方式, DMA 预处理需 CPU 执行 20 条指令 (CPI = 1.5), 后处理 (中断处理) 需执行 40 条指令 (CPI = 2.0), DMA 采用周期挪用方式, 每次 DMA 传送 4096 字节的数据块。求 DMA 方式下 CPU 用于该外设的时间百分比, 并与 (2) 比较, 说明 DMA 的优势。

拓展阅读:

- **Patterson & Hennessy** 《计算机组成与设计》第 5 章 5.5–5.6 节——DMA 与 I/O 总线的详细工作流程, 包含周期挪用 (Cycle Stealing) 的时序分析。
- **Bryant & O'Hallaron** 《深入理解计算机系统》(CS:APP) 第 8 章「异常控制流」——8.1 节 (异常)、8.5 节 (信号), 从 OS 视角理解中断、陷阱、故障和终止的分类与处理。
- **CodeBrick 408** 真题可视化 (codebrick.tech) ——DMA 方式共 10 道真题 (累计 41 分), 程序中断方式共 10 道, I/O 接口共 5 道, 均配交互式可视化演示。

PART III 第三部分

操作系统

1 操作系统概述

1.1 操作系统的概念与功能

定义 1.1: 操作系统

操作系统 (Operating System, OS) 是管理计算机硬件与软件资源的系统软件, 也是用户与计算机之间的接口。它提供:

- 1. 资源管理: 处理器、存储器、I/O 设备和文件的分配与调度;
- 2. 抽象: 将底层硬件细节封装为简洁一致的接口 (虚拟化);
- 3. 服务: 为用户和应用程序提供运行环境。

结论 1.1: 操作系统的四大功能

OS 四大管理功能			
功能	管理对象	核心问题	
进程管理	CPU	进程调度、同步、通信、死锁	
内存管理	主存	分配回收、地址变换、虚拟内存、保护	
文件管理	外存	文件目录、存储空间、共享保护	
设备管理	I/O 设备	缓冲、分配、驱动、SPOOLing	

1.2 操作系统的特征

结论 1.2: OS 的四个基本特征

- 1. 并发 (**Concurrency**): 多个程序在同一时间间隔内同时运行 (宏观并行、微观串行)。与并行 (**Parallelism**) 的区别: 并行是同一时刻真正同时执行 (需多核)。
- 2. 共享 (**Sharing**): 多个进程共享系统资源。分为互斥共享 (如打印机) 和同时访问 (如磁盘)。
- 3. 虚拟 (**Virtualization**): 将物理实体映射为多个逻辑对应物 (虚拟 CPU、虚拟内存、虚拟设备)。
- 4. 异步 (**Asynchrony**): 进程以不可预知的速度推进 (走走停停), OS 需保证结果可再现。

并发和共享是 OS 最基本的两个特征, 互为存在条件。

注意

408 常考辨析: 并发 vs 并行。单核 CPU 通过时间片轮转实现并发, 不是并行。多核系统才能实现真正的并行。

1.3 操作系统的发展与分类

OS 发展历程			
阶段	特征	典型系统	
手工操作	人机矛盾突出	—	
单道批处理	监督程序 (Monitor), 自动作业过渡	—	
多道批处理	多道程序并发, 资源利用率高, 无交互	IBM OS/360	
分时系统	时间片轮转, 交互性强	UNIX	
实时系统	严格的响应时间约束	RTOS	

多道程序设计的核心：内存中同时存放多道程序，CPU 在等待 I/O 时切换到另一道程序，提高 CPU 利用率。

1.4 操作系统内核

定义 1.2: 内核

操作系统最核心的部分，常驻内存。分为两大类型：

- 大内核 (**Monolithic Kernel**)：将 OS 主要功能模块（进程管理、内存管理、文件系统等）都运行在内核态。优点是性能高；缺点是内核庞大，一个模块崩溃可能导致整个系统崩溃。
- 微内核 (**Microkernel**)：仅将最基本功能（中断处理、进程调度、进程通信）放入内核，其余（文件系统、设备驱动等）作为用户进程运行。优点是可靠性高、易扩展；缺点是频繁的用户态/内核态切换导致性能开销。

1.5 中断与异常

结论 1.3: 中断与异常

中断机制是 OS 实现并发的重要基础，尤其是时钟中断使抢占式调度和分时成为可能。进程切换也可能由系统调用、异常、主动让出 CPU 或进程阻塞触发，因此不能把它绝对化为“没有外部中断就没有进程切换”。

中断与异常对比		
	中断 (Interrupt)	异常 (Exception)
来源	外设 (硬件)	CPU 内部 (指令执行)
与指令的关系	异步 (与当前指令无关)	同步 (由当前指令触发)
响应方式	下一条指令开始前检测	指令执行过程中检测
示例	时钟中断、I/O 完成中断	缺页、除零、断点
分类	可屏蔽/不可屏蔽	故障 (Fault)/陷阱 (Trap)/终止 (Abort)

注意

中断 **vs** 异常（内中断 **vs** 外中断）的辨析：缺页异常是典型的 Fault（故障），处理后通常可重新执行引发异常的指令；系统调用（如 `int 0x80` 或 `syscall`）是由程序主动触发的 Trap（陷阱）。除零等同步异常的具体 Fault/Abort 归类依赖体系结构和异常处理约定，不能脱离体系结构一概而论。

1.6 系统调用

定义 1.3: 系统调用

系统调用（System Call）是 OS 提供给用户程序的编程接口，是用户程序主动请求内核服务的规范入口。程序通过体系结构规定的陷入/系统调用机制从用户态进入内核态，执行完毕后返回用户态；中断和其他异常也能使 CPU 进入内核处理流程，但不属于应用主动请求服务的普通接口。

常见系统调用：进程控制（`fork`, `exit`, `wait`）、文件操作（`open`, `read`, `write`）、设备管理（`ioctl`）、通信（`pipe`, `shmget`）。

1.7 操作系统的体系结构

结论 1.4: OS 体系结构演化

- 简单结构（MS-DOS）：无模块划分，用户程序可直接访问硬件。
- 模块化结构：按功能划分模块，模块间通过接口调用。
- 层次结构：底层为硬件相关，高层为用户接口，每层仅调用相邻下层。
- 外核结构（**Exokernel**）：内核仅负责资源隔离和保护，资源管理策略交给应用程序。

1.8 408 高频考点速查

结论 1.5: 操作系统概述—408 常考点

1. 并发 **vs** 并行：单核 CPU 分时 = 并发；多核 = 并行。
2. 中断 **vs** 异常：来源（外 **vs** 内）、同步性、典型例子。
3. 用户态 **vs** 内核态：系统调用/中断/异常触发态切换；特权指令只能在内核态执行。
4. 微内核 **vs** 大内核：各有哪些 OS 模块在内核中。
5. 系统调用与库函数的区别：系统调用需陷入内核；库函数（如 `printf`）在用户态执行，内部可能调用系统调用（如 `write`）。

2 习题

2.1 基础过关题

1. (单项选择题) 操作系统的主要功能是
(A) 实现程序并发执行 (B) 管理计算机系统资源
(C) 提供用户界面 (D) 以上都是
2. (单项选择题) 以下关于并发和并行的说法, 正确的是
(A) 单核 CPU 可以并行执行多个进程
(B) 并发是指同一时刻多个程序同时运行
(C) 同一时刻真正执行多个 CPU 指令流需要多核或多处理器等并行硬件资源
(D) 并发和并行是同一概念
3. (填空题) 中断来自 CPU ____ 部, 异常来自 CPU ____ 部。系统调用属于 ____ (异常类型)。
4. (简答题) 简述微内核和大内核的区别, 各举一个代表性操作系统。
5. (简答题) 简述操作系统为用户提供的三种用户接口。

2.2 真题风格与综合训练

6. (真题风格, 单项选择题) 下列事件中, 属于中断的是
(A) 用户程序执行系统调用 (B) 除零错误
(C) 缺页异常 (D) 磁盘 I/O 完成
7. (真题风格, 单项选择题) 下列指令中, 只能在内核态执行的是
(A) 读时钟 (B) 置时钟
(C) 取数指令 (D) 寄存器清零
8. (真题风格, 综合题) 有 3 个同时到达的作业 A、B、C, 各自的 CPU 计算时间和 I/O 时间如下:
 - A: 计算 30ms, I/O 40ms, 计算 10ms
 - B: 计算 20ms, I/O 30ms, 计算 20ms
 - C: 计算 10ms, I/O 20ms, 计算 30ms

问: (1) 若三道作业按 A→B→C 顺序单道执行, 总时间是多少? (2) 若引入多道程序设计, 假定内存可同时容纳三道作业, 系统只有一个 CPU, CPU 调度采用非抢占 FCFS, 初始顺序为 A、B、C; 各作业的 I/O 可彼此并行且可与 CPU 重叠, 切换开销忽略不计。画出 CPU 甘特图并计算总时间。

拓展阅读:

- OSTEP 第 1-2 章「操作系统介绍」和「虚拟化」——经典的 OS 全景图和设计哲学;
- 陈海波《现代操作系统: 原理与实现》第 1 章——现代 OS 的架构演变和 ChCore 实验环境。

3 进程管理

3.1 进程的概念

定义 3.1: 进程

进程（Process）是程序的一次执行实例。与程序的区别：

- 程序是静态的：存放在磁盘上的指令和数据集合。
- 进程是动态的：程序在 CPU 上的执行过程，包含程序计数器、寄存器、堆栈等运行时上下文。

一个程序可对应多个进程（如多个终端同时运行 vim）。

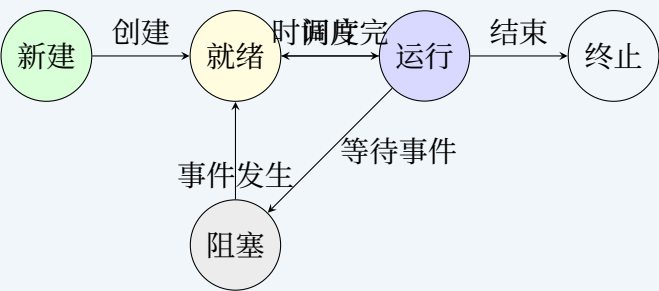
结论 3.1: 进程的组成

进程由三部分组成：

1. 程序段（**Text/Code**）：存放 CPU 执行的指令。
2. 数据段（**Data**）：全局变量、静态变量等。
3. **PCB**（进程控制块）：OS 管理进程的数据结构，包含 PID、进程状态、PC 值、寄存器现场、内存管理信息、打开文件列表等。

3.2 进程的状态与转换

结论 3.2: 三状态模型 + 五状态模型



进程状态说明

状态	说明	PCB 所在队列
新建 (New)	进程正在创建	—
就绪 (Ready)	等待 CPU 调度	就绪队列
运行 (Running)	正在 CPU 上执行	—
阻塞 (Blocked/Waiting)	等待某事件 (I/O 等)	阻塞队列
终止 (Terminated)	执行完毕	—

注意**408 高频辨析：**

- 就绪 → 运行：进程被调度程序（Scheduler）选中，获得 CPU。
- 运行 → 就绪：时间片用完（分时系统），或被更高优先级进程抢占。
- 运行 → 阻塞：主动等待某事件（如 `read()` 系统调用等待磁盘 I/O）。
- 阻塞 → 运行从不直接发生——必须先经过就绪状态。
- 运行 → 就绪和运行 → 阻塞不能同时发生。

3.3 进程控制块（PCB）

结论 3.3: PCB 的内容**PCB 主要字段**

字段类别	内容
标识信息	PID（进程 ID）、父进程 PPID、用户 ID
处理机状态	通用寄存器、PC、PSW、栈指针
调度信息	进程状态、优先级、已等待时间
内存管理	基址/限长寄存器、页表指针
I/O 状态	打开文件表、I/O 请求队列

PCB 是 OS 感知进程存在的唯一标志——创建进程 = 创建 PCB；撤销进程 = 释放 PCB。

3.4 线程与多线程模型

定义 3.2: 线程

线程（Thread）是处理机调度的基本单位。同一进程内的线程共享代码段、数据段、地址空间和打开文件，但每个线程拥有独立的线程 ID、程序计数器、寄存器组和栈。

结论 3.4: 进程与线程、用户线程与内核线程

- 进程是资源分配和保护的基本单位；线程是 CPU 调度的基本单位。不同进程切换通常需要切换地址空间，同一进程内线程切换开销较小。
- 用户级线程（ULT）：由用户态线程库管理，内核不可见。创建、切换快，但在多对一模型中一个线程执行阻塞系统调用可能阻塞整个进程，且同一进程不能在多个 CPU 上真正并行。
- 内核级线程（KLT）：由内核管理和调度。一个线程阻塞不妨碍同进程其他线程运行，并可在多核上并行，但创建和切换开销较大。
- 映射模型包括多对一、一对一和多对多；判断并行能力及阻塞影响时，必须先确定采用哪种映射。

3.5 进程通信 (IPC)

结论 3.5: 共享存储与消息传递

- 共享存储：多个进程映射同一片物理内存，数据交换快，但互斥与同步由进程显式保证。
- 消息传递：通过 `send/receive` 交换消息，可分直接/间接通信以及阻塞/非阻塞通信；隔离性较好，但通常需要系统调用和数据复制。
- 管道：普通管道通常半双工，只能用于有亲缘关系的进程；命名管道可供无亲缘关系进程使用。管道读端在空管道上可能阻塞，写端在满管道上可能阻塞。

3.6 进程调度

定义 3.3: 三级调度

1. 高级调度（作业调度/长程调度）：从外存后备队列选作业装入内存。频率低（分钟级）。批处理系统有，分时/实时系统通常没有。
2. 中级调度（内存调度/中程调度）：将暂时不能运行的进程挂起到外存（挂起/激活），待条件满足再调回内存。涉及交换（**Swapping**）。
3. 低级调度（进程调度/短程调度）：从就绪队列选进程分配 CPU。频率最高（毫秒级）。任何 OS 都必须有。

结论 3.6: 进程调度时机

- 主动放弃：进程终止、进程主动阻塞（如 I/O 请求）。
- 被动剥夺：时间片用完、更高优先级进程就绪、中断返回时。

不能进行调度的时刻：中断处理中、OS 内核临界区中、原子操作过程中。

3.7 调度算法

结论 3.7: 经典调度算法

调度算法对比

算法	抢占	饥饿	优点	缺点
FCFS	否	无	简单公平	护航效应
SJF/SPF	否	可能	平均等待时间最短	长作业饥饿
SRTF	是	可能	最优平均等待	实现复杂
HRRN	否	无	折中方案	—
RR	是	无	公平	q 太大 $\rightarrow$ FCFS, 太小 $\rightarrow$ 开销
优先级	可配置	可能	灵活	低优先级饥饿
多级队列	是	视实现	分类调度	—
多级反馈队列	是	可防	自适应, 综合最佳	参数配置复杂

关键公式：

- 周转时间 = 完成时间 - 到达时间 = 等待时间 + 运行时间
- 带权周转时间 = 周转时间 / 运行时间
- 响应比 $R_p = \frac{\text{等待时间} + \text{要求服务时间}}{\text{要求服务时间}} = 1 + \frac{\text{等待时间}}{\text{要求服务时间}}$

注意

408 调度算法计算题的核心：给定进程到达时间和运行时间表，用指定算法计算平均周转时间、平均带权周转时间。**关键陷阱：**抢占式算法（SRTF、RR）需在每个进程到达时或时间片结束时重新检查。

3.8 进程同步与互斥

定义 3.4: 临界资源与临界区

- 临界资源：一次仅允许一个进程访问的共享资源（如打印机、共享变量）。
- 临界区（**Critical Section**）：进程中访问临界资源的代码段。
- 互斥：任何时刻最多一个进程进入临界区。
- 同步：多个进程按特定顺序执行（如生产者-消费者）。

临界区使用原则：空闲让进、忙则等待、有限等待、让权等待。

结论 3.8: 互斥的软硬件实现与管程

- 软件方法：Peterson 算法利用 `flag[]` 与 `turn` 实现两个进程互斥，满足互斥、空闲让进和有限等待，但依赖严格的内存读写顺序。
- 硬件原子指令：`TestAndSet`、`Swap/Exchange`、`CAS` 可构造自旋锁。等待者持续占用 CPU，适合等待很短的场景；长时间等待应阻塞让权。
- 关中断：只适合内核在单处理器上保护极短临界区，不适合用户进程，也不能单独解决多处理器之间的互斥。
- 管程（**Monitor**）：把共享数据和对其操作封装在一起，保证任一时刻至多一个进程在管程内活动；条件变量配合 `wait/signal` 实现条件同步。

3.9 信号量（Semaphore）与 PV 操作

定义 3.5: 信号量

Dijkstra 提出的同步原语：

- 整型信号量：一个整数变量 S ，只能通过 P/V（或 `wait/signal`）操作访问。
- 记录型信号量：在整型基础上增加等待队列（进程链表），实现让权等待（阻塞而非忙等）。

```
// P 操作 (wait / down / proberen) : 申请资源
void P(semaphore *S) {
    S->value--;
```

```

4     if (S->value < 0) block(S->queue); // 无可用资源，阻塞
5 }
6
7 // V 操作 (signal / up / verhogen) : 释放资源
8 void V(semaphore *S) {
9     S->value++;
10    if (S->value <= 0) wakeup(S->queue); // 有等待进程，唤醒
11 }

```

信号量值的含义（记录型信号量）：

- $S.value > 0$ ：表示还有 $S.value$ 个可用资源。
- $S.value = 0$ ：无可用资源且无等待进程。
- $S.value < 0$ ： $|S.value|$ 是阻塞队列中等待的进程数。

结论 3.9: 经典同步问题

1. 生产者-消费者（有界缓冲区）：

```

1 semaphore mutex = 1; // 缓冲区互斥
2 semaphore empty = N; // 空位数
3 semaphore full = 0; // 产品数
4

```

2. 读者-写者问题：读者优先 vs 写者优先 vs 公平竞争。核心是第一个读者负责加锁，最后一个读者负责解锁。
3. 哲学家进餐：5 个哲学家 5 根筷子。死锁预防方案：(1) 最多允许 4 个哲学家同时进餐；(2) 奇数号先拿左筷、偶数号先拿右筷；(3) 只有两根筷子都可用时才拿起。

3.10 死锁

定义 3.6: 死锁的四个必要条件

1. 互斥：资源只能互斥使用。
2. 持有并等待：已占有资源的进程可继续请求新资源。
3. 不可剥夺：已分配的资源不能被强制剥夺。
4. 循环等待：存在进程-资源的环形等待链。

四个条件缺一不可。

结论 3.10: 死锁处理策略

死锁处理

策略	时机	方法
预防 (Prevention)	系统设计阶段	破坏四个必要条件之一
避免 (Avoidance)	运行时	银行家算法
检测 (Detection)	运行时	资源分配图化简
恢复 (Recovery)	检测后	终止进程或剥夺资源

银行家算法 (408 高频): n 进程、 m 种资源类型。数据结构: Available[m]、Max[n][m]、Allocation[n][m]、Need[n][m] (Need = Max - Allocation)。安全序列: 存在一个进程执行序列, 使每个进程均可完成而不导致死锁。银行家算法即检测是否存在安全序列。

3.11 408 高频考点速查

结论 3.11: 进程管理—408 常考点

1. 进程状态转换: 五种状态的合法转换路径, 特别是阻塞 $\rightarrow$ 运行不合法。
2. 进程与线程: 资源共享范围、用户级/内核级线程的阻塞和并行能力、多线程映射模型。
3. 进程通信与互斥实现: 共享存储、消息传递、管道, 以及 Peterson、硬件原子指令和管程。
4. 调度算法: 给定进程表计算平均周转时间, 注意抢占 vs 非抢占的差异。
5. PV 操作: 生产者-消费者、读者-写者的信号量设计和代码补全。
6. 银行家算法: 给定 Allocation/Max/Available 表, 判断系统是否安全, 列出安全序列。
7. 死锁必要条件: 判断场景是否会导致死锁, 破坏哪个条件可预防死锁。

4 习题

4.1 基础过关题

1. (真题风格·选择题) 下列选项中, 导致创建新进程的操作是

- (A) 用户登录成功 (B) 设备分配
(C) 启动程序执行 (D) 以上都是

2. (真题风格·选择题) 下列选项中, 在用户态执行的是

- (A) 命令解释程序 (shell) (B) 缺页处理程序
(C) 进程调度程序 (D) 时钟中断处理程序

3. (真题风格·选择题) 若系统中有 n 个进程, 则就绪队列中进程的个数最多为

- (A) n (B) $n - 1$
(C) $n - 2$ (D) 1

4. (真题风格·选择题) 系统中有 3 个不同的临界资源 R_1, R_2, R_3 , 被 4 个进程 P_1, P_2, P_3, P_4 共享。各进程对资源的需求为: P_1 申请 R_1 和 R_2 , P_2 申请 R_2 和 R_3 , P_3 申请 R_1 和 R_3 , P_4 申请 R_2 。若系统出现死锁, 则处于死锁状态的进程数至少是

- (A) 1 (B) 2
(C) 3 (D) 4

5. (真题风格·选择题) 下列调度算法中, 不会导致饥饿的是

- (A) 短作业优先 (SJF) (B) 优先级调度
(C) 时间片轮转 (RR) (D) 多级反馈队列

6. (真题风格·计算题) 有 4 个作业 J1-J4, 到达时间和所需运行时间如下表。分别计算在 FCFS 和 SJF (非抢占) 调度算法下的平均周转时间和平均带权周转时间。

作业	J1	J2	J3	J4
到达时间	0	1	2	3
运行时间	8	4	9	5

7. (基础题) 同一进程内的多个线程共享下列哪项内容?

- (A) 程序计数器 (B) 栈
(C) 地址空间 (D) 寄存器组

8. (基础题) 在多对一用户级线程模型中, 一个线程执行阻塞系统调用后, 通常会发生什么?
- (A) 仅该线程阻塞 (B) 整个进程阻塞
- (C) 所有进程阻塞 (D) 自动转为内核线程

4.2 真题风格与综合训练

9. (真题风格·综合题) 三个进程 P1、P2、P3 互斥使用一个包含 N 个单元的缓冲区。P1 每次用 `produce()` 生成一个正整数, 并用 `put()` 送入缓冲区某一空单元; P2 每次用 `getodd()` 从缓冲区取一个奇数, 并用 `countodd()` 统计奇数个数; P3 每次用 `geteven()` 从缓冲区取一个偶数, 并用 `counteven()` 统计偶数个数。请用信号量实现三个进程的同步与互斥, 并说明所使用信号量的含义。
10. (真题风格·综合题) 某系统有 A、B、C 三类资源, 数量分别为 (10, 5, 7)。当前 Allocation 和 Max 矩阵如下, 判断系统是否安全, 若安全给出安全序列。

	Allocation			Max		
	A	B	C	A	B	C
P0	0	1	0	7	5	3
P1	2	0	0	3	2	2
P2	3	0	2	9	0	2
P3	2	1	1	2	2	2
P4	0	0	2	4	3	3

11. (真题风格·综合题) 面包师问题: 面包店有一个最多放 N 个面包的柜台。面包师不断烤面包放入柜台, 顾客不断从柜台取面包 (一次取一个)。用信号量实现同步。

拓展阅读:

- OSTEP 第 26-31 章「并发」——信号量、生产者-消费者、读者-写者的完整推导和代码。
- 陈海波《现代操作系统: 原理与实现》第 3 章——同步原语的 ARMv8 实现细节。

5 内存管理

5.1 内存管理的基本概念

定义 5.1: 内存管理的目标与任务

操作系统内存管理 (Memory Management) 负责主存 (Main Memory) 的分配与回收、地址变换、内存保护和虚拟内存的扩展。其核心目标:

1. 分配与回收: 高效地为进程分配内存空间, 进程结束后回收;
2. 地址变换: 将程序中的逻辑地址转换为物理地址;
3. 内存保护: 确保进程只能访问自己的地址空间;

4. 内存扩充：通过虚拟内存技术，使程序获得比实际物理内存更大的地址空间。

逻辑地址与物理地址

定义 5.2: 地址空间的两层抽象

- 逻辑地址 (**Logical Address**)：也叫虚拟地址 (Virtual Address)，是 CPU 在执行指令时生成的地址，是程序视角的地址。每个进程拥有独立的逻辑地址空间，从 0 开始编址。
- 物理地址 (**Physical Address**)：内存单元的实际地址，是内存芯片视角的地址。整个系统只有一个物理地址空间，所有进程共享。
- 地址空间 (**Address Space**)：一个进程可用的所有逻辑地址的集合。32 位系统上，逻辑地址空间通常为 $0 \sim 2^{32} - 1$ (4 GB)。

注意

408 常考点辨析：逻辑地址 vs 物理地址。编译时和链接时，程序中出现的是逻辑地址（符号地址经链接后变为可重定位地址）；程序装入内存运行时，CPU 发出的访存地址也是逻辑地址，需经 MMU (Memory Management Unit, 内存管理单元) 转换为物理地址后才送到地址总线上。

地址重定位

定义 5.3: 地址重定位

地址重定位 (Address Relocation) 是将逻辑地址转换为物理地址的过程，也叫地址映射。分为三种方式：

1. 静态重定位 (**Static Relocation**)：在程序装入时一次性完成地址变换。装入程序根据装入的起始物理地址，修改程序中所有与地址相关的指令。装入后地址不再改变。
 - 优点：无需硬件支持，简单。
 - 缺点：程序装入后不能移动（不支持换入换出）；不允许程序在内存中浮动。
2. 动态重定位 (**Dynamic Relocation**)：在程序执行时，每次访存都由硬件 (MMU 中的重定位寄存器) 将逻辑地址加上基址得到物理地址。

$$\text{物理地址} = \text{逻辑地址} + \text{重定位寄存器 (基址)}$$

- 优点：程序可以在内存中移动（支持交换和紧凑）；无需修改程序代码。
 - 缺点：需要硬件支持 (MMU)，每次访存有微小的地址计算开销。
3. 运行时动态链接 (**Run-time Dynamic Linking**)：将链接推迟到运行时。程序执行到某外部模块的调用时，OS 才将该模块装入内存并链接。Linux 的 .so、Windows 的 .dll 即采用此方式。

注意

408 高频辨析：三种地址变换时机。

- 编译时：编译器知道进程驻留在内存的确切位置 → 生成绝对代码 (Absolute Code)。

- 装入时（静态重定位）：编译器不知道确切位置 → 生成可重定位代码（Relocatable Code），装入时修改。
- 执行时（动态重定位）：进程可在执行期间从一内存段移到另一内存段 → 需要 MMU 硬件。

注意：现代通用 OS 均采用执行时动态重定位 + 运行时动态链接的组合。

内存保护

结论 5.1: 内存保护的实现机制

内存保护防止一个进程访问不属于它的内存区域。常见实现：

1. 界限寄存器（**Limit Register**）：存放逻辑地址的最大值。每次访存，MMU 检查逻辑地址是否 < 界限寄存器值。若越界，触发地址越界异常。
2. 基址-界限寄存器对：重定位寄存器存基址，界限寄存器存长度。硬件检查：逻辑地址 < 界限值。

现代系统更多使用分页中的页表保护位（读/写/执行权限）来实现细粒度的内存保护。

注意

408 考点：CPU 在用户态执行时发出的地址是逻辑地址。若该逻辑地址超出界限寄存器值，则 MMU 产生地址越界中断，CPU 进入内核态处理。这一过程是硬件完成的，称为存储保护。

程序的链接

结论 5.2: 三种链接方式

静态链接、装入时动态链接与运行时动态链接

链接方式	时机	特点
静态链接	程序装入前（编译后）	所需目标模块链接成完整可执行文件；文件通常较大，运行时无需再完成动态链接
装入时动态链接	程序装入内存时	装入时链接所需模块，便于模块共享和更新
运行时动态链接	程序执行过程中	用到时才链接（如 <code>dlopen</code> ），灵活且可按需装入；Linux 的 <code>.so</code> 文件可采用这种方式

5.2 连续分配存储管理方式

定义 5.4: 连续分配

连续分配 (Contiguous Allocation) 是指为一个进程分配一段连续的物理内存空间。按分区方式分为三类: 单一连续分配、固定分区分配和动态分区分配。

单一连续分配

结论 5.3: 单一连续分配

内存分为系统区 (低地址, 存放 OS) 和用户区 (高地址, 存放一道用户程序)。一次只能装入一道用户程序。

- 优点: 简单, 无需复杂的内存管理。
- 缺点: 内存利用率极低 (用户区剩余空间浪费); 单用户单任务, 无并发。
- 适用: 早期单用户单任务 OS (如 MS-DOS)。

固定分区分配

结论 5.4: 固定分区分配

将用户空间划分为若干个固定大小的分区 (各分区大小可相等也可不等)。每个分区只装入一道程序。

- 分区说明表: 记录每个分区的起始地址、大小和状态 (已分配/未分配)。
- 优点: 实现简单, 无外部碎片 (External Fragmentation)。
- 缺点: 存在内部碎片 (**Internal Fragmentation**) —— 分配的分区大于程序实际需要的空间, 浪费的部分即为内部碎片; 分区总数固定, 限制了并发进程数。

动态分区分配

定义 5.5: 动态分区分配

根据进程的实际需要, 从空闲内存中选择一块不小于请求大小的连续空间; 若空闲块更大, 通常将剩余部分继续保留为空闲块。初始时整个用户区是一个大空闲块; 随着进程的装入和退出, 内存中会出现多个大小不等的空闲块和已分配块。

结论 5.5: 动态分区分配算法

在动态分区中, 空闲块通常组织为空闲分区链 (**Free List**)。当新进程请求大小为 n 的内存时, OS 需要从空闲链中选择一个合适的空闲块。常用算法:

动态分区分配算法对比

算法	策略	特点
首次适应 (FF) First Fit	从低地址开始, 选第一个 $\geq n$ 的空闲块	简单快速, 低地址端易产生小碎片; 高地址端保留大空闲块
最佳适应 (BF) Best Fit	选所有 $\geq n$ 的空闲块中 最小的那个	本次分配后的剩余块最小, 但每次通常 需遍历全链; 会产生许多难以利用的微小碎片
最坏适应 (WF) Worst Fit	选所有 $\geq n$ 的空闲块中 最大的那个	剩余空闲块仍较大, 可继续分配; 但大空闲块被迅速消耗, 仍需遍历全链
邻近适应 (NF) Next Fit	从上次分配的位置 继续搜索第一个 $\geq n$ 的 块	类似 FF 但不用每次从链头开始; 空闲分布更均匀, 但会更快消耗大块

注意

408 常考：首次适应 vs 最佳适应。首次适应 (FF) 通常查找开销较小——按地址有序并从表头开始时，找到第一个足够大的空闲块即可停止。最佳适应 (BF) 选择本次分配后剩余量最小的块，但通常需要检查更多候选块，也可能产生难以利用的小碎片。二者的总体效果依赖请求序列和空闲表组织，不能笼统断言某一种在所有场景下“综合性能最好”。

碎片问题与紧凑

结论 5.6: 外部碎片与内部碎片

- **外部碎片 (External Fragmentation)**: 空闲内存总量足够, 但由于不连续, 无法满足一个连续分配的请求。动态分区会产生外部碎片。
- **内部碎片 (Internal Fragmentation)**: 分配给进程的空间比进程实际需要的稍大 (通常因按固定大小单元分配), 浪费的是已分配区域内部的空间。固定分区和分页 (最后一页) 会产生内部碎片。

解决外部碎片的方法:

1. **紧凑 (Compaction)**: 通过移动已在内存中的进程, 将空闲块合并成一个大连续块。需要动态重定位支持 (进程移动后基址寄存器更新)。
2. **离散分配**: 允许进程分散在内存的不连续区域 (分页、分段), 从根本上消除外部碎片。

注意

408 常考辨析：内部碎片 vs 外部碎片。

- 固定分区产生内部碎片 (无外部碎片);
- 动态分区产生外部碎片 (无内部碎片, 除非分配策略浪费);
- 分页系统产生内部碎片 (最后一页的页内碎片);
- 分段系统产生外部碎片 (段长可变, 类似动态分区)。

5.3 离散分配——分页存储管理

定义 5.6: 分页存储管理

分页 (Paging) 将进程的逻辑地址空间划分为若干大小相等的块, 称为页 (**Page**); 将物理内存也划分为同样大小的块, 称为页框 (**Page Frame**) 或物理块。页面和页框的大小通常为 4 KB (2^{12} B)。

进程的页面离散地装入内存中任意空闲的页框, 通过页表 (**Page Table**) 记录页号与页框号的映射关系。

地址结构

要点 5.1: 分页地址结构

给定页面大小 $P = 2^p$, 逻辑地址 A (m 位) 被拆分为两部分:

$$\text{逻辑地址} = (\text{页号 } p_n, \text{ 页内偏移 } d)$$

- 页号 $p_n = \lfloor A/P \rfloor$ (高位部分, $m - p$ 位)
- 页内偏移 $d = A \bmod P$ (低位部分, p 位)

物理地址同样由页框号 f_n (页表中查得) 和页内偏移 d 拼接而成:

$$\text{物理地址} = f_n \times P + d$$

实际硬件中, 页内偏移 d 在地址变换前后保持不变——仅页号替换为页框号。

注意

408 计算核心: 页面大小 $= 2^p$ 时, 逻辑地址低 p 位为页内偏移, 其余高位为页号。例如页面大小 4 KB ($p = 12$), 逻辑地址为 32 位, 则页号占高 20 位 (2^{20} 个页), 页内偏移占低 12 位 ($0 \sim 4095$)。

常考题型: 给出页面大小和逻辑地址, 求页号和页内偏移 $\rightarrow$ 逻辑地址除以页面大小。

页表与地址变换

定义 5.7: 页表

每个进程拥有一张页表, 存放在内存中。页表项 (PTE, Page Table Entry) 通常包含:

- 页框号 (**Frame Number**): 该页映射到哪个物理页框;
- 有效位 (**Valid/Present Bit**): 该页是否在内存中 (1 = 在内存, 0 = 不在, 即缺页);
- 保护位 (**Protection Bits**): 读/写/执行权限;
- 引用位 (**Reference/Accessed Bit**): 该页是否被访问过 (用于页面置换);
- 修改位 (**Dirty/Modified Bit**): 该页是否被修改过 (换出时决定是否写回磁盘)。

页表寄存器 (**PTBR, Page Table Base Register**) 指向当前进程页表的起始物理地址。

要点 5.2: 分页地址变换过程

CPU 发出逻辑地址 A ，MMU 执行以下变换：

1. 从 A 中提取页号 p 和页内偏移 d ；
2. 以 p 为索引，在页表中找到对应的页表项 ($\text{PTBR} + p \times \text{PTE 大小}$)；
3. 检查有效位：若 $= 0$ ，触发缺页中断；
4. 取出页框号 f ，与偏移 d 拼接：物理地址 $= f \times \text{页面大小} + d$ 。

$$\text{物理地址} = (\text{页框号} \ll p) \mid \text{页内偏移}$$

其中 $\ll p$ 表示左移 p 位（即乘 2^p ）， $\mid$ 表示拼接。

注意

408 综合题中的地址变换：分页系统的每次数据/指令访问都需要至少两次访存——第一次访问页表（读 PTE），第二次访问实际数据。这是分页的主要性能开销。TLB 正是为了解决这一问题引入的（见下文）。

TLB（快表）**定义 5.8: TLB**

TLB（Translation Lookaside Buffer，快表/地址变换后备缓冲器）是一个高速、小容量的硬件 Cache，缓存最近使用的页表项。TLB 位于 MMU 内部，采用全相联或组相联映射，访问速度接近 CPU 寄存器。

每次地址变换时，首先在 TLB 中查找页号（并行比较）；若命中（TLB Hit）则直接获取页框号，无需访问内存中的页表；若缺失（TLB Miss）则需访问内存页表，并将新 PTE 装入 TLB。

要点 5.3: 引入 TLB 后的有效访存时间

设：

- 一次内存访问的时间为 t_{mem}
- TLB 查找时间为 t_{TLB} （通常远小于 t_{mem} ，可嵌入访存流水线）
- TLB 命中率为 h

TLB 命中时的访存时间： $t_{\text{TLB}} + t_{\text{mem}}$ （TLB 查找 + 一次访存）。

TLB 缺失时的访存时间： $t_{\text{TLB}} + 2 \times t_{\text{mem}}$ （TLB 查找 + 访问页表 + 访问数据）。

有效访存时间（EAT, Effective Access Time）：

$$\text{EAT} = h \times (t_{\text{TLB}} + t_{\text{mem}}) + (1 - h) \times (t_{\text{TLB}} + 2t_{\text{mem}})$$

若 t_{TLB} 可忽略：

$$\text{EAT} \approx h \times t_{\text{mem}} + (1 - h) \times 2t_{\text{mem}} = (2 - h) \times t_{\text{mem}}$$

当 TLB 命中率很高（ $\geq 99\%$ ）时，EAT 仅略大于一次访存时间，性能损耗极小。

注意

408 高频计算：TLB + 页表 + Cache 综合题。典型考题流程：CPU 发出逻辑地址 → ① TLB 查找（命中/缺失）→ ②（若缺失）访问页表 → ③ 得到物理地址 → ④ 访问 Cache（命中/缺失）→ ⑤（若缺失）访问主存。关键：理解每一步的先后依赖关系和各自的命中率。

多级页表**要点 5.4: 二级页表与多级页表**

现代 64 位系统的虚拟地址空间极大（实现常使用其中一部分有效位，例如 48 位），若为整个空间配置稠密单级页表，页表会占用不可接受的巨量内存，工程上通常采用多级页表等稀疏组织方式。这里的问题是空间代价不现实，而不是数学意义上的“不能编码”。

二级页表的地址变换：

$$\text{逻辑地址} = (\text{一级页号 } p_1, \text{二级页号 } p_2, \text{页内偏移 } d)$$

1. 以 p_1 为索引，查一级页表（页目录），得到二级页表的基址；
2. 以 p_2 为索引，查二级页表，得到页框号；
3. 拼接页框号和 d ，得到物理地址。

多级页表的优势：

- 节省内存：只将实际使用的页表部分保留在内存中。未使用的页表项对应的二级页表不需要分配。
- 页表本身可以离散存放：二级页表不需要连续的物理内存。

多级页表的代价：

- 地址变换需要更多次访存（ k 级页表需要 k 次内存访问取 PTE + 1 次访问数据）。TLB 命中可消除这一开销。

注意

408 重点：二级页表的地址变换计算。以 32 位逻辑地址、页面大小 4 KB 为例。典型划分：一级页号 10 位（ 2^{10} 个页目录项），二级页号 10 位（ 2^{10} 个页表项），页内偏移 12 位（4 KB）。每级页表恰好占一个页面（ $2^{10} \times 4 \text{ B/PTE} = 4 \text{ KB}$ ）。考察方式：给定多级页表结构，计算各级页号、访存次数、访问的物理地址。

5.4 离散分配——分段存储管理**定义 5.9: 分段存储管理**

分段（Segmentation）按程序的自然逻辑单元（如代码段、数据段、堆栈段）来划分地址空间。每个段有不同的段名和段长（可变），有独立的逻辑意义和访问权限。

每个进程拥有一张段表（**Segment Table**），每项包含：

- 段基址（**Base**）：该段在物理内存中的起始地址；
- 段界限（**Limit**）：该段的长度；
- 保护位：读/写/执行权限。

要点 5.5: 分段地址变换

逻辑地址由段号 s 和段内偏移 d 组成:

$$\text{逻辑地址} = (\text{段号 } s, \text{段内偏移 } d)$$

地址变换过程:

1. 以段号 s 为索引, 查段表得到段基址 b 和段界限 l ;
2. 检查 $d < l$ (越界检查), 否则触发段越界中断;
3. 物理地址 $= b + d$ 。

$$\text{物理地址} = \text{段基址} + \text{段内偏移}$$

注意

408 辨析: 分段地址变换 vs 分页地址变换。分页中, 页内偏移直接拼接在页框号后 (二者位宽固定); 分段中, 段内偏移与段基址是相加关系。分段必须显式检查 $d < \text{段长}$; 分页不需要对页内偏移做同类的段长比较, 但仍要检查页号/PTE 是否属于进程的有效地址空间以及访问权限。

结论 5.7: 分页与分段的对比

分页 vs 分段

维度	分页	分段
划分依据	物理单位 (固定大小)	逻辑单位 (可变大小)
地址空间	一维 (连续的逻辑地址)	二维 (段号 + 段内偏移)
程序员可见性	不可见 (透明)	可见 (程序员划分段)
共享与保护	按页 (不方便, 一页可能包含多种数据)	按段 (方便, 段有逻辑意义)
碎片	内部碎片 (最后一页)	外部碎片 (段长可变)
地址变换	拼接 (页框号 偏移)	加法 (段基址 + 偏移)
地址合法性检查	检查页号/PTE 有效性与权限; 无段长式偏移比较	检查段号、 $d < \text{段长}$ 与权限

5.5 段页式存储管理

定义 5.10: 段页式存储管理

段页式存储管理将分页和分段结合: 先将用户程序按逻辑划分为若干段, 每段再划分为若干页; 物理内存按页框划分。

逻辑地址变为三维结构:

$$\text{逻辑地址} = (\text{段号 } s, \text{页号 } p, \text{页内偏移 } d)$$

每个进程有一张段表，每段有一张页表。段表项指向该段的页表起始地址和页表长度。

要点 5.6: 段页式地址变换过程

1. 以段号 s 为索引查段表，得到该段的页表基址和页表长度；
2. 检查页号 p 是否小于页表长度（越界检查）；
3. 以 p 为索引查该段的页表，得到页框号 f ；
4. 物理地址 = $f \times \text{页面大小} + d$ 。

访存次数：段页式一次访存需要三次访存——段表 + 页表 + 数据。同样需要 TLB 来减少开销。

注意

408 考点：段页式的优势。段页式兼具分段和分页的优点——既按逻辑单元分段（便于共享和保护），又通过分页消除外部碎片。但增加了地址变换的复杂度和访存次数（三次访存）。Linux/x86 实际采用的是段页式（虽然 Linux 将段基址设为 0，实质上退化为纯分页模型）。

5.6 虚拟内存

局部性原理

定义 5.11: 局部性原理

局部性原理（Principle of Locality）是虚拟内存系统得以工作的理论基础，由 Denning 系统阐述：

- 时间局部性（**Temporal Locality**）：如果某个指令/数据被访问，那么在不久的将来它很可能再次被访问。典型例子：循环中的指令和变量。
- 空间局部性（**Spatial Locality**）：如果某个存储单元被访问，那么其附近的存储单元也很可能很快被访问。典型例子：数组的顺序访问、指令的顺序执行。

局部性是 **Cache** 和虚拟内存设计的基石。正是因为程序具有局部性，我们才可以将进程的部分页面装入内存而正常运行——大部分访存都落在当前活跃的页面集合（工作集）中。

注意

408 常考辨析：时间局部性 vs 空间局部性。

- 循环中的求和变量 → 时间局部性（反复访问同一变量）；
- 遍历数组 $a[0..N-1]$ → 空间局部性（依次访问相邻元素）；
- 循环体中的指令 → 时间局部性（反复执行同一条指令）+ 空间局部性（指令顺序存放）。

虚拟内存的概念

定义 5.12: 虚拟内存

虚拟内存 (Virtual Memory) 是一种内存管理技术, 它允许进程在不需要全部装入物理内存的情况下运行。程序被划分为页面, 仅有当前需要的页面驻留在物理内存中, 其余页面存放在磁盘 (交换区/Swap) 上。当访问不在内存的页面时, 由 OS 将其调入。

核心特征:

1. 离散性: 进程在内存中不必连续存放 (分页/分段的基础);
2. 多次性: 进程分多次装入内存 (而非一次性全部装入);
3. 对换性: 进程的页面可在内存和磁盘之间换入换出;
4. 虚拟性: 从逻辑上扩充内存容量, 使进程可用的逻辑地址空间远大于物理内存。

注意

408 考点: 虚拟内存的最大容量由地址结构 (CPU 寻址位数) 决定, 与实际物理内存大小无关。例如 32 位系统, 最大虚拟地址空间为 $2^{32} = 4 \text{ GB}$ 。虚拟内存的实际容量受 $\min(\text{CPU 寻址范围}, \text{内存容量} + \text{外存交换区容量})$ 制约。

请求分页

定义 5.13: 请求分页系统

请求分页 (Demand Paging) 是最基本的虚拟内存实现方式。基本思想:

- 进程开始时, 仅将少量必要的页面装入内存 (甚至可以先不装入任何页面——纯粹按需调页);
- 当 CPU 访问的页面不在内存时 (页表项有效位 = 0), 触发缺页中断 (**Page Fault**);
- OS 缺页中断处理程序从磁盘中调入缺失的页面, 更新页表, 然后重新执行导致缺页的指令。

结论 5.8: 缺页中断处理流程

缺页中断是一种特殊的异常 (**Fault**), 其处理流程如下:

1. 硬件检测到缺页 (页表项有效位 = 0), 触发缺页异常, 陷入内核;
2. OS 保存现场 (寄存器、程序计数器等);
3. OS 判断该逻辑地址是否合法 (属于该进程的地址空间)。若非法 → 终止进程; 若合法 → 继续;
4. 从空闲页框中分配一个 (若无空闲 → 执行页面置换算法, 选出换出页面);
5. 若换出页面被修改过 (**Dirty** 位 = 1), 将其写回磁盘;
6. 从磁盘将缺失的页面读入分配到的页框;
7. 更新页表 (设置页框号、有效位 = 1), 更新 TLB;
8. 恢复现场, 重新执行导致缺页的指令。

注意

408 常考：缺页中断与普通中断的区别。

- 缺页中断在指令执行过程中产生（属于异常/Fault），而非下一条指令开始前（属于中断）；
- 缺页中断处理完成后，需要重新执行导致缺页的那条指令（而不是执行下一条）；
- 一条指令可能导致多次缺页（如跨页的指令本身、多个操作数各自跨页等）。

页面置换算法**定义 5.14: 页面置换算法**

当缺页发生且无空闲页框时，OS 需要选择一个已在内存中的页面换出（Evict）。置换算法的目标是使缺页率（**Page Fault Rate**）最低。选择哪个页面换出由页面置换算法决定。

要点 5.7: 有效访问时间（含缺页处理）

设：

- t_{mem} ：一次内存访问时间
- t_{pf} ：缺页处理时间（主要是磁盘 I/O，通常为 ms 级）
- p ：缺页率（Page Fault Rate）
- t_{TLB} 忽略不计

若 t_{pf} 表示从检测缺页到调页完成、但不含重新执行后的那次内存访问，则

$$\text{EAT} = (1 - p)t_{\text{mem}} + p(t_{\text{pf}} + t_{\text{mem}}) = t_{\text{mem}} + p t_{\text{pf}}$$

若题目把重新执行和最终访存也包含在 t_{pf} 中，才写成 $(1 - p)t_{\text{mem}} + p t_{\text{pf}}$ 。计算前必须先辨明题目对“缺页处理时间”的定义。由于 $t_{\text{pf}} \gg t_{\text{mem}}$ （数量级差 $10^5 \sim 10^6$ ），即使缺页率很低，也会显著影响性能。例如： $t_{\text{mem}} = 100 \text{ ns}$ ， $t_{\text{pf}} = 8 \text{ ms}$ ，缺页率 $p = 0.001\%$ 时：

$$\text{EAT} \approx 100 \text{ ns} + 0.00001 \times 8 \text{ ms} \approx 180 \text{ ns}$$

性能下降近一倍。

结论 5.9: 最优页面置换算法 OPT

OPT (Optimal Page Replacement, 最佳置换算法)：选择在未来最长时间不再被访问的页面换出。

OPT 是所有算法中缺页率最低的，但它是不可实现的（需要预知未来的访问序列）。OPT 仅作为理论上的性能标尺，用于衡量其他算法的优劣。

例 内存可容纳 3 个页面，访问序列：7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1。OPT 置换过程：共 9 次缺页（**Belady 异常**不会在 **OPT** 中发生）。

结论 5.10: 先进先出置换算法 FIFO

FIFO (First-In First-Out)：维护一个 FIFO 队列，选择最早进入内存的页面换出。

- 优点：实现简单（只需一个循环队列）。

- 缺点：完全忽略了页面的访问频率和使用情况；可能置换出频繁使用的页面。
- **Belady 异常 (Belady's Anomaly)**：对于 FIFO，增加物理页框数有时反而导致缺页次数增加。

例 (Belady 异常) 访问序列：1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5。3 个页框时缺页 9 次，4 个页框时缺页 10 次。LRU 和 OPT 不会出现 Belady 异常（它们属于栈类算法）。

注意

408 高频考点：Belady 异常。 OPT 和 LRU 是栈类算法 (**Stack Algorithm**) —— n 个页框时的驻留集始终是 $n+1$ 个页框时驻留集的子集，因此不会出现 Belady 异常。FIFO 是最典型的会出现 Belady 异常的算法。基本 CLOCK/Second-Chance 不是栈类算法，理论上也可能出现 Belady 异常，不能因为它近似 LRU 就断言一定不会出现。

结论 5.11: 最近最久未使用置换算法 LRU

LRU (Least Recently Used)：选择最近最久未被访问的页面换出。基于「过去不常使用的页面未来也不太可能被使用」的局部性原理。

LRU 实现方式：

1. 计数器法：为每个页表项关联一个时间戳（逻辑时钟）。每次页面被访问时，将当前时钟值写入该 PTE。置换时选择时间戳最小的页面。
 2. 栈法：维护一个按访问时间排序的页面号栈。每次访问将页面移至栈顶；置换时选择栈底页面。
- 优点：性能接近 OPT，是实际可实现的算法中效果最好的。
 - 缺点：每次访存都需更新计数器或调整栈，纯硬件支持成本高（TLB 可辅助）。

注意

408 高频计算：LRU 置换过程与缺页次数统计。 考题通常给出内存容量（页框数）和页面访问序列，要求模拟 LRU 置换过程，统计缺页次数。技巧：维护一个 LRU 栈（或访问顺序表），每次访问后将当前页面移到队尾（最远被访问的一端），队头即为最久未使用的页面。注意区分「LRU 栈」和「FIFO 队列」：FIFO 中页面进入顺序不变（仅在置换时才变化）；LRU 中每次访问命中都会改变页面的顺序。

结论 5.12: CLOCK (Second-Chance) 置换算法

CLOCK 算法 又称 Second-Chance（第二次机会）算法，是 LRU 的一种近似实现。它利用 PTE 中的访问位 (**Reference/Accessed Bit**)，避免维护精确的访问时间顺序。NRU (Not Recently Used) 通常按访问位和修改位把页面分组后选择牺牲页，是相关但不同的算法，不能把 NRU 当作 CLOCK 的别名。

基本 CLOCK 算法：

1. 将所有驻留页面组织成环形链表（类似时钟面盘），维护一个指针（时钟指针）；
2. 发生缺页需置换时：
 - 检查当前指针所指页面的访问位；
 - 若访问位 = 0：选择该页换出，指针前进一位，结束；

• 若访问位 = 1：将访问位清 0（「给第二次机会」），指针前进一位，重新检查。

3. 如果遍历一圈全为 1，则所有访问位都被清 0，第二轮必定找到 0。

直观理解：CLOCK 就像给每个页面「一次免死金牌」。如果最近被访问过（访问位 = 1），则暂时放过（清 0），除非一轮下来仍无其他候选。

结论 5.13: 改进型 CLOCK 算法

改进型 CLOCK (Enhanced Second-Chance) 增加了修改位 (Dirty/Modified Bit) 的考量。选择换出页面时，优先选择既未访问也未修改的页面（避免写回磁盘的开销）。考虑（访问位 A ，修改位 M ）组合，优先级从高到低：

改进型 CLOCK 置换优先级				
优先级	A	M	含义	
第一（最优选）	0	0	最近未访问，未修改——直接丢弃	
第二	0	1	最近未访问，但被修改——需写回磁盘	
第三	1	0	最近访问过，未修改——可能很快再被访问	
第四（最后选）	1	1	最近访问过且修改过——最不该淘汰	

扫描过程：

1. 第一轮扫描：寻找 ($A = 0, M = 0$) 的页面，不修改任何访问位。找到即换出。
2. 第二轮扫描：寻找 ($A = 0, M = 1$) 的页面，同时将遇到的页面的 A 清 0。找到即换出。
3. 第三轮扫描：重复第一轮（此时所有 A 已被清 0，必能找到）。
4. （若仍无，重复第二轮，此时必有 (0, 1) 被找到。）

优势：减少了对磁盘的写操作（只有修改过的页面才需要写回磁盘），使页面置换的 I/O 开销尽可能小。

注意

408 高频考点：CLOCK 与改进 CLOCK 的置换过程模拟。考试通常给出各页面的 A 和 M 位的当前值，以及时钟指针位置，要求找出下一个被换出的页面及扫描过程中各标志位的变化。重点掌握：改进 CLOCK 中，第一轮不看 M ，只找 (0, 0)；第二轮不仅找 (0, 1)，还顺便清 A 。

5.7 页面分配策略与抖动

页面分配策略

结论 5.14: 页面分配策略

OS 在多个进程之间分配物理页框时，有三种基本策略：

页面分配策略对比

策略	含义	特点
固定分配 + 局部置换	每个进程固定数量的页框;缺页时仅从自身页面中选择换出	分配数确定后不变
可变分配 + 全局置换	不预先限制各进程的页框数;缺页时从所有进程的页面中选一个换出(可从别的进程「抢」页框)	灵活, 但一个进程的行为可能影响其他进程
可变分配 + 局部置换	动态调整各进程页框数;缺页时仅从自身页面中置换, 但 OS 会定期评估并调整分配数	最常用; 平衡灵活性和隔离性

引入/调入页面的时机:

- 请求调页 (**Demand Paging**): 仅在访问缺页时才调入 (最常用)。
- 预调页 (**Prepaging**): 在缺页发生时, 除调入缺失页面外, 也调入其相邻的若干页面 (利用空间局部性, 减少后续缺页)。

抖动与工作集

定义 5.15: 抖动

抖动 (**Thrashing**) 是指系统中频繁发生缺页, 以至于进程花费在页面换入换出上的时间超过了实际执行计算的时间, CPU 利用率急剧下降的现象。

抖动产生的原因: 并发进程过多, 每个进程分配的物理页框数少于其工作集 (**Working Set**) 所需的最小页数。进程不断缺页, 从其他进程抢页面, 引发连锁反应。

抖动的特征:

- CPU 利用率骤降 (因为进程都在等待磁盘 I/O);
- 磁盘忙于处理页面换入换出;
- 系统吞吐量极低。

结论 5.15: 工作集模型

工作集 (**Working Set**) 由 Denning 提出, 定义为:

$$W(t, \Delta) = \{\text{进程在时间区间}[t - \Delta, t] \text{ 内访问过的页面集合}\}$$

其中 Δ 称为工作集窗口 (**Window Size**)。工作集的大小 $|W(t, \Delta)|$ 随程序执行的阶段不同而变化。

工作集模型的应用:

1. 防止抖动: OS 监视每个进程的工作集大小, 确保分配的页框数 $\geq$ 工作集大小。如果所有进程的工作集之和超过物理内存总量, 则需要挂起 (**Suspend**) 某些进程。
2. 页面置换决策: 优先保留工作集内的页面, 换出不在工作集中的页面。

页框数的确定:

- 最少页框数: 由硬件 (指令集结构) 决定——保证一条指令能正常执行所需的最少页面数

(如跨页指令 + 跨页操作数, 最多可能需要 6 个页面)。

- 最大页框数: 进程的全部页面数。

注意

408 高频考点: 抖动与工作集。

- 抖动可以通过工作集模型或缺页频率 (**PFF, Page Fault Frequency**) 策略来预防。
- PFF 策略: 若进程缺页率太高 → 增加其页框数; 若缺页率太低 → 减少其页框数。
- 408 常考判断: 「系统频繁换页, CPU 利用率低」→ 系统发生了抖动, 应减少并发进程数, 而不是增加。

5.8 408 高频考点速查

结论 5.16: 内存管理—408 常考点总结

1. 地址变换计算: 给定逻辑地址和页面大小 → 求页号、页内偏移; 查页表 → 求物理地址, 是常见计算题。
2. **TLB + 页表 + Cache** 综合: 理解 TLB 命中/缺失后的访存流程, 计算不同命中率下的有效访存时间。
3. 多级页表: 给定多级页表结构和逻辑地址, 求各级索引和最终物理地址; 计算多级页表节省的内存量。
4. 页面置换算法: OPT、FIFO (注意 Belady 异常)、LRU、CLOCK、改进 CLOCK 的置换过程模拟和缺页次数统计。高频计算题。
5. 分段 **vs** 分页 **vs** 段页式: 地址变换方式的区别 (拼接 **vs** 加法)、碎片类型、访存次数。
6. 碎片问题: 固定分区 (内部碎片)、动态分区 (外部碎片 + 紧凑)、分页 (内部碎片)、分段 (外部碎片)。
7. 抖动与工作集: 抖动的现象、原因和解决方案。

6 习题

6.1 基础过关题

- (真题风格·选择题) 在缺页处理过程中, 操作系统执行的操作可能是
(A) 修改页表 (B) 磁盘 I/O
(C) 分配页框 (D) 以上都是
- (真题风格·选择题) 在一个请求分页系统中, 采用 LRU 页面置换算法时, 假设一个作业的页面走向为 1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6。当分配给该作业的物理块数为 4 时, 缺页次数是
(A) 8 (B) 10
(C) 12 (D) 14
- (真题风格·选择题) 在请求分页系统中, 页面分配策略与页面置换策略不能组合使用的是
(A) 可变分配, 全局置换 (B) 可变分配, 局部置换
(C) 固定分配, 全局置换 (D) 固定分配, 局部置换
- (真题风格·选择题) 某系统采用改进型 CLOCK 置换算法, 页表项中 A 为访问位, M 为修改位。将页面分为四类: (0,0) (0,1) (1,0) (1,1), 淘汰页面的优先次序为
(A) (0,0),(0,1),(1,0),(1,1) (B) (0,0),(1,0),(0,1),(1,1)
(C) (0,0),(1,1),(1,0),(0,1) (D) (0,1),(1,0),(0,0),(1,1)
- (真题风格·选择题) 下列关于虚拟存储器的叙述中, 正确的是
(A) 虚拟存储器的容量等于主存与辅存的容量之和
(B) 虚拟存储器是基于局部性原理实现的
(C) 虚拟存储器只能基于分页存储管理实现
(D) 引入虚拟存储器后, 进程的地址空间受物理内存大小限制

- (计算题) 某分页系统页面大小为 4 KB, 进程的页表如下 (页号从 0 开始):

页号	0	1	2	3
页框号	5	10	2	7

分别计算逻辑地址 5000 和 15000 对应的物理地址。

6.2 真题风格与综合训练

7. (真题风格·综合题) 某请求分页系统, 进程 P 共有 7 页, 分配给 P 的物理块为 3 块, 页面访问串为

1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6.

- (1) 采用 FIFO 页面置换算法, 缺页率是多少?
- (2) 采用 LRU 页面置换算法, 缺页率是多少?
- (3) 本例中出现 Belady 异常了吗? 解释原因。

8. (真题风格·综合题) 某计算机系统主存按字节编址, 逻辑地址和物理地址均为 32 位, 页面大小 4 KB, 页表项大小 4 B。

- (1) 若使用一级页表, 页表最大占多少内存?
- (2) 若使用二级页表, 将 32 位逻辑地址划分为: 一级页号 10 位、二级页号 10 位、页内偏移 12 位。此时每个进程的页表最多占多少内存 (假设仅将需要的页表调入)?
- (3) TLB 初始为空, 访问逻辑地址 0x12345 时 TLB 缺失, 访问页表后 TLB 加载该表项。之后访问逻辑地址 0x1234A 是否需要再次访问页表?

9. (真题风格·综合题) 某系统采用请求分页存储管理, 页面大小 4 KB, TLB 采用全相联映射。CPU 执行指令 `mov eax, [ebx]` (读内存操作数), 逻辑地址为 `$0x08048000$`。

- (1) 该逻辑地址的页号和页内偏移各是多少?
- (2) 若 TLB 命中, 访存过程共需几次内存访问?
- (3) 若 TLB 缺失且页表在内存中, 访存过程共需几次内存访问?

拓展阅读:

- OSTEP 第 18-22 章「分页」和「TLB」——地址变换的完整硬件实现和 TLB 设计。
- 陈海波《现代操作系统: 原理与实现》第 5 章「内存管理」——多级页表和 ARM64 地址变换。

7 文件管理

7.1 文件系统的基本概念

定义 7.1: 文件与文件系统

文件是具有符号名的一组相关信息的集合, 是外存中数据组织的基本单位。文件系统是 OS 中负责文件存储、检索、保护和共享的子系统。

结论 7.1: 文件的逻辑结构

- 无结构文件 (流式文件): 以字节流为单位 (如 UNIX/DOS 文件), 灵活性最高。
- 有结构文件 (记录式文件): 由若干记录组成。分为定长记录和变长记录。

7.2 文件的物理结构（文件的分配方式）

结论 7.2: 三种文件物理结构

文件分配方式对比

分配方式	实现	优点	缺点	FAT
连续分配	每个文件占连续盘块	顺序访问快	有外部碎片，难扩展	—
链接分配（隐式）	每块含下一块指针	无外部碎片	随机访问慢	—
链接分配（显式/FAT）	FAT 集中存指针	FAT 在内存时，遍历比隐式链接快；无外部碎片	定位第 i 块仍需沿链逐项查找；FAT 占内存	FAT12/16/32
索引分配	索引块集中存放数据块号	支持直接访问；多级索引可支持大文件	索引块有空间开销	UNIX i-node

UNIX i-node 混合索引：

- 直接块（通常 10-12 个）：存文件数据块号，小文件直接访问。
- 一级间接块：存指向数据块的指针（指向一个块，该块中存数据块号）。
- 二级间接块：存指向一级间接块的指针。
- 三级间接块：存指向二级间接块的指针。

408 常考：给定块大小和指针大小，计算 i-node 支持的最大文件大小。

注意

408 重点辨析：FCB（文件控制块）与 i-node。FCB 包含文件名和索引指针等信息。UNIX 将文件名和索引指针分离：目录项只存文件名和 i-node 号，i-node 存除文件名外的所有信息。这样做的好处是支持文件硬链接（多个目录项指向同一 i-node）。

7.3 目录结构

结论 7.3: 目录结构演化

- 单级目录：所有文件在一个目录下。不允许重名，查找慢。
- 两级目录：主文件目录（MFD）+ 用户文件目录（UFD）。不同用户可有同名文件。
- 树形目录（多级目录）：目录以树形组织。使用绝对路径（/a/b/file）和相对路径（当前目录为基准）。
- 无环图目录：允许一个文件或目录出现在多个父目录下（共享）。需引入引用计数防止删除正在使用的文件。

7.4 文件存储空间管理

结论 7.4: 外存空闲空间管理方法

- 空闲表法：类似内存动态分区，维护空闲盘块表。分配用首次适应等算法。
- 空闲链表法：所有空闲盘块连成链表。分配/回收效率低（需 I/O）。
- 位示图（**Bitmap**）：每位对应一个盘块（0= 空闲，1= 已分配）。 m 个盘块需 m bit。**408** 常考：给定盘块号，计算在位示图中的行列号。
- 成组链接法（**UNIX** 采用）：空闲盘块分组管理，每组第一块存下一组空闲块的块号和块数。兼顾效率和空间。

7.5 文件共享与保护

结论 7.5: 文件共享

- 硬链接（**Hard Link**）：多个目录项指向同一 i-node。无法跨文件系统，不能链接目录（防止循环）。
- 软链接（**Symbolic Link**）：独立文件，存放目标文件路径。可跨文件系统、可链接目录。删除原文件后软链接失效（悬空链接）。

7.6 磁盘调度算法

结论 7.6: 磁盘访问时间与调度

磁盘访问时间 $T = T_s$ （寻道时间）+ T_r （旋转延迟）+ T_t （传输时间）。
寻道时间远大于后两者，因此调度算法以优化寻道为目标。

磁盘调度算法对比

算法	策略	优点	缺点
FCFS	按请求顺序	公平	性能差
SSTF	最短寻道优先	性能好	饥饿
SCAN（电梯算法）	磁头单向移动，到端点折返	无饥饿	端点附近不公平
C-SCAN	单向服务，到端点跳回起点	等待时间均匀	—
LOOK/C-LOOK	类似 SCAN/C-SCAN，但到最远请求即折返	更实用	—

408 常考：给定磁盘请求序列和当前磁头位置/方向，用 SSTF/SCAN/C-SCAN 画出磁头移动轨迹，计算总寻道长度和平均寻道长度。

7.7 408 高频考点速查

结论 7.7: 文件管理—408 常考点

1. **i-node** 混合索引：最大文件长度计算。
2. 位示图：盘块号 $\leftrightarrow$ 字号/位号的换算 ($b = n \times \text{字长} + m$)。
3. 磁盘调度：SSTF/SCAN/C-SCAN 的磁头移动序列和总寻道长度。
4. 硬链接 **vs** 软链接：引用计数、跨文件系统、删除行为。
5. **FAT** 文件系统：FAT 表大小计算（给定盘块数和 FAT 表项位数）。

8 习题

8.1 基础过关题

1. (真题风格·选择题) 某文件系统的根目录 / 下有子目录 Program 和 Document; Program 下有子目录 Java-Prog 以及文件 f1.java。当前工作目录为 /Program, 则文件 f1.java 的绝对路径名为

(A) /Program/f1.java (B) /Document/Program/f1.java
(C) Program/f1.java (D) /Java-Prog/f1.java

2. (真题风格·选择题) 在磁盘调度算法中, 通常称为“电梯算法”的是

(A) FCFS (B) SSTF
(C) SCAN (D) C-SCAN

3. (真题风格·选择题) 假设磁盘有 300 个磁道 (0-299), 磁头当前在 120 号磁道并向磁道号减小的方向移动。请求序列为: 90, 70, 150, 60, 80, 20。用 SCAN 算法 (LOOK 变体), 磁头移动的总磁道数为

(A) 140 (B) 200
(C) 230 (D) 280

4. (真题风格·选择题) 在文件系统中, 文件控制块 (FCB) 的作用是

(A) 存放文件的数据内容 (B) 存放文件的属性信息
(C) 存放文件的目录结构 (D) 存放文件的访问权限列表

5. (基础题) 下列选项中, 属于文件系统的基本功能的是

(A) 文件按名存取 (B) 进程调度
(C) 处理机分配 (D) 虚拟内存地址变换

6. (计算题) 某磁盘有 8000 个柱面 (0-7999), 磁头当前在 150 号柱面并向柱面号增大的方向移动。请求序列为: 86, 147, 913, 1774, 948, 1509, 1022, 1750, 130。分别用 SSTF 和 LOOK 算法, 计算磁头移动的总柱面数。

8.2 真题风格与综合训练

7. (真题风格·综合题) 某 UNIX 文件系统 i-node 包含 8 个直接块指针、1 个一级间接块指针、1 个二级间接块指针。盘块大小为 4 KB, 每个指针占 4 字节。

(1) 该文件系统支持的最大文件大小是多少?

- (2) 访问文件偏移量 5 MiB 处的数据，需要访问几次磁盘？
 - (3) 若文件系统改用 FAT32 管理磁盘空间，FAT 表项 32 位，磁盘总容量为 4 GB，盘块大小 4 KB。FAT 表需占用多少存储空间？
8. (真题风格·综合题) 某系统采用位示图法管理磁盘空闲空间。磁盘共有 2^{24} 个盘块，字长为 32 位。
- (1) 位示图需要多少个字？
 - (2) 给出盘块号 b 到位示图中字号 i 和位号 j 的换算公式；
 - (3) 若盘块号 500000 在位示图中对应字号为 15625，验证 (2) 中公式的正确性。

拓展阅读：

- OSTEP 第 39-40 章——文件系统实现与 i-node 详解。
- 陈海波《现代操作系统：原理与实现》第 7 章——Ext4 与 F2FS 文件系统。

9 I/O 管理

9.1 I/O 设备分类

定义 9.1: I/O 设备的分类

计算机的 I/O 设备按传输速率、信息交换单位、设备共享属性等维度可分为多种类型。

I/O 设备分类		
分类维度	类型	典型设备
按信息交换单位	块设备 (Block Device)	磁盘、光盘
	字符设备 (Character Device)	键盘、鼠标、打印机
按传输速率	低速设备	键盘、鼠标等
	中速设备	打印机等
	高速设备	磁盘、网络接口等
按共享属性	独占设备	打印机 (临界资源, 需互斥使用)
	共享设备	磁盘 (可多个进程同时交叉访问)
	虚拟设备	SPOOLing 将独占设备改造为共享设备

块设备 vs 字符设备的核心区别：

- 块设备：以数据块为单位传输，支持随机访问 (可寻址)，有缓冲区缓存机制。如磁盘以扇区 (通常 512B/4KB) 为基本 I/O 单位。
- 字符设备：以字符流为单位顺序传输，不提供按数据块随机寻址；系统仍可为其设置字符缓冲或行缓冲。如键盘输入、串口通信。

注意

408 辨析：块设备 $\neq$ 独占设备。磁盘是块设备同时又是共享设备；打印机是字符设备同时又是独占设备。设备分类维度不同，不可混淆。块设备的典型特征是可寻址（可以 **lseek**），字符设备不可寻址。

9.2 I/O 控制方式

结论 9.1: 四种 I/O 控制方式演进

I/O 控制方式的发展本质上是将 **CPU** 从 **I/O** 任务中逐步解放的过程——每次演进都减少了 CPU 参与 I/O 的程度。

I/O 控制方式对比

控制方式	CPU 干预频率	数据传送单位	适用场景	特点
程序直接控制（轮询）	每字节/每字都要轮询	字/字节	简单系统	CPU 忙等，利用率极低
中断驱动	每字节/每字完成后中断	字/字节	低速字符设备	CPU 可并行处理其他任务
DMA	每块数据传输完成后中断一次	块（多字节）	高速块设备	CPU 仅参与开始和结束
I/O 通道	每批 I/O 任务完成后中断一次	一组块	大型机	通道有自己的指令，可执行通道程序

程序直接控制（轮询，Polling）

定义 9.2: 程序直接控制

CPU 不断循环检测设备状态寄存器（**busy/wait** 标志位），当设备就绪时执行一次数据传送（读/写数据寄存器），传送完毕后将设备状态置为就绪（**command-ready**），然后继续轮询。

工作流程：

1. CPU 向设备控制器发送 I/O 命令（如 Read）。
2. CPU 反复读取状态寄存器，检查设备是否就绪（**busy == 0**）。
3. 设备就绪后，CPU 从数据寄存器读取一个字到内存。
4. 重复第 2–3 步直到所有数据传输完成。

致命缺陷：CPU 忙等（Busy-Waiting）——在设备完成 I/O 之前，CPU 无法执行任何有意义的计算，CPU 利用率极低。

中断驱动方式（Interrupt-Driven I/O）

结论 9.2: 中断驱动 I/O

核心思想：CPU 发出 I/O 命令后继续执行其他进程；当设备完成当前数据传输（一个字/字节）后，向 CPU 发出中断信号；CPU 响应中断，在中断处理程序中执行数据传送，然后恢复原进程。

工作流程：

1. CPU 向设备控制器发出 Read 命令，然后切换到其他进程运行。
2. 设备控制器将数据读入自己的数据寄存器，完成后发出中断信号。
3. CPU 检测到中断，保存当前进程上下文，转去执行中断处理程序。

- 4. 中断处理程序将设备数据寄存器中的字/字节传送到内存指定位置。
- 5. 若还有数据未传输，CPU 再次启动设备，然后恢复原进程。

与轮询的对比：

- 轮询：CPU 主动等设备，浪费 CPU；但响应快（无中断开销）。
- 中断：CPU 可在设备工作期间执行其他指令，但每次中断有处理与现场切换开销。408 的简化模型常按一次中断传送一个字/字节分析；实际控制器也可能通过 FIFO 在一次服务中搬运多个数据单元。

适用场景：低速字符设备（键盘输入、鼠标、串口通信）。对于高速块设备（磁盘），每次只传一个字且每字都中断，中断过于频繁，开销太大。

DMA 方式 (Direct Memory Access)

结论 9.3: DMA 控制方式

DMA 是中断驱动的升级版——引入专门的 DMA 控制器，在设备和内存之间直接传输一整块数据，CPU 仅在传输开始和结束时参与。数据传输单位从字/字节跃升到块。

DMA 控制器的四个寄存器：

DMA 控制器寄存器	
寄存器	功能
DR（数据寄存器）	暂存设备与内存之间每次传输的数据
MAR（内存地址寄存器）	存放目标内存地址，每传输一个字后自动递增
DC（数据计数器）	存放剩余要传输的字节数/字数，递减至 0 则传输完成
CR（命令/状态寄存器）	存放 CPU 发出的命令或设备的状态

DMA 工作流程：

1. CPU 设置 DMA 控制器的 MAR、DC 和命令寄存器，然后继续执行其他进程。
2. DMA 控制器控制设备将数据块逐字传送到内存：每传一个字，MAR++，DC-。
3. 当 DC 减至 0 时，DMA 控制器向 CPU 发出一次中断，通知 CPU 传送完成。
4. CPU 响应中断，执行后处理（如唤醒等待该 I/O 的进程）。

注意

408 高频辨析：DMA 与中断驱动的关键区别。

1. 中断频率：中断驱动每字/字节中断一次；DMA 每块数据仅中断一次。
2. 数据传送：中断驱动由 CPU 执行数据传送（从设备寄存器到内存）；DMA 由 DMA 控制器直接完成，CPU 不参与数据传输。
3. 总线占用：DMA 传送期间，DMA 控制器需占用系统总线（周期窃取/周期挪用，Cycle Stealing）。若 CPU 也需要访存，DMA 优先级更高。
4. 内存地址：DMA 描述符通常使用物理地址或设备可见地址；传统简化模型常要求一段物理连续缓冲区，但现代 DMA 可用 scatter-gather 描述符或 IOMMU 映射非连续物理页。

要点 9.1: DMA 周期窃取时的 CPU 停顿分析

DMA 对 CPU 的影响取决于 DMA 请求频率、每次传送占用的总线周期数以及 CPU 的总线请求频率。仅给出“CPU 每执行 k 条指令访存一次”不足以唯一确定降速比。若额外假设每 k 条指令恰有一次 CPU 访存、每次 DMA 恰挪用一個周期且二者严格交替，才可在该简化模型下计算比例：

$$\text{CPU 有效时间比例} = \frac{k}{k+1} \quad (\text{上述严格交替模型})$$

当 k 很大（CPU 指令大多访问 Cache）时，DMA 的影响可忽略。这也是现代系统中 Cache 对 DMA 效率的关键意义。

I/O 通道方式 (Channel I/O)**定义 9.3: I/O 通道**

I/O 通道是一种专门负责 I/O 的协处理器，有自己的指令集（通道指令），可以执行通道程序来控制一组 I/O 设备完成复杂的 I/O 任务。

与 DMA 的区别：

- **DMA**：只能执行简单的块传输（从设备到固定内存区间），无分支/条件判断能力。
- **通道**：可以执行包含分支、循环的通道程序，能够控制多个设备并发 I/O，CPU 仅需发一条 I/O 指令启动通道即可。

通道类型：

- **字节多路通道**：以字节为单位交叉为多台低速设备服务（如多个终端）。
- **数组选择通道**：以块为单位为一台高速设备服务，但通道利用率低（设备独占通道）。
- **数组多路通道**：结合前两者优点，以块为单位交叉为多台高速设备服务（现代大型机主流方案）。

注意

408 辨析：I/O 通道方式属于 408 考试范围，但只在组成原理中详细考查。OS 视角只需了解其层次位置和与 DMA 的本质区别（通道有指令，可执行通道程序）。

9.3 I/O 软件层次结构**结论 9.4: I/O 软件的四个层次**

I/O 软件采用分层设计，每层对上层屏蔽下层细节，是 OS 分层的典范。

用户层 I/O 软件 (printf, scanf 等库函数 + SPOOLing 系统)

设备独立性软件 (命名、保护、阻塞、缓冲、分配)

设备驱动程序 (设备寄存器操作, 与硬件直接通信)

中断处理程序 (保存现场、处理中断、恢复现场)

硬件 (设备控制器 + 设备本身)

各层职责详解:

1. 中断处理程序 (**Interrupt Handler**) ——最底层软件

- I/O 完成后, 设备控制器发出中断信号。
- CPU 响应中断: 硬件保存 PC 和 PSW, 然后跳转到中断向量表指定的中断处理程序入口。
- 中断处理程序: 保存寄存器上下文 → 处理中断 (读取设备状态, 唤醒等待进程) → 恢复上下文 → 中断返回。
- 关键要求: 中断处理程序必须尽可能短, 复杂性推迟到下半部 (Bottom Half) 处理。

2. 设备驱动程序 (**Device Driver**) ——设备相关

- 每种设备 (或每类设备) 对应一个驱动程序, 由设备制造商提供。
- 负责将 OS 的通用 I/O 请求 (如 “读第 n 块”) 翻译为设备控制器的具体命令序列 (操作设备寄存器)。
- 向上提供统一接口 (open, read, write, ioctl, release), 向下操作设备控制器的控制寄存器、状态寄存器、数据寄存器。
- 驱动程序是内核的一部分 (运行在内核态), 但代码量和 bug 占比极高。

3. 设备独立性软件 (**Device-Independent Software**) ——核心层

- 执行所有设备共用的 I/O 功能, 为上层提供统一的逻辑设备名到物理设备的映射。
- 主要功能:
 - 设备命名与保护: 将逻辑设备名 (如 /dev/sda) 映射到对应的驱动程序。
 - 设备分配与回收: 管理独占设备 (如打印机) 的分配, 防止死锁。
 - 缓冲管理: 实现单缓冲、双缓冲、缓冲池, 平滑 CPU 和 I/O 设备的速度差异。
 - 差错处理: 处理一般的 I/O 错误 (如磁盘读错重试)。
 - 提供统一块大小: 向上层屏蔽不同磁盘扇区大小的差异。
- 典型实现: UNIX/Linux 的文件系统层即承担了设备独立性软件的角色。

4. 用户层 I/O 软件 (**User-Level I/O Software**)

- 库函数: printf 调用 write 系统调用, scanf 调用 read。
- SPOOLing 系统: 在用户层将独占设备虚拟化为共享设备 (详见下一节)。
- 假脱机管理程序 (如打印守护进程 lpd) 通常以用户进程形式运行。

注意

408 高频辨析：各层的归属。

- 中断处理程序、设备驱动程序 运行在内核态（属于内核）。
- 设备独立性软件 运行在内核态（属于内核）。
- SPOOLing 系统 通常在用户层（以守护进程形式），但也涉及内核态的系统调用。

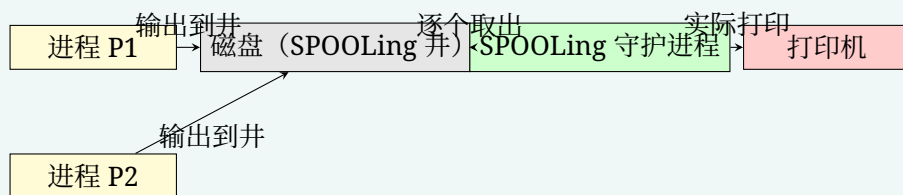
关键题眼：“向上提供统一接口，向下操作设备寄存器” → 这是在描述 设备驱动程序。

9.4 SPOOLing 系统（假脱机技术）

定义 9.4: SPOOLing

SPOOLing (Simultaneous Peripheral Operations On-Line)，假脱机/外部设备联机并行操作) 是一种将独占设备（如打印机）改造为可共享的虚拟设备的技术。

核心思想：用磁盘（共享设备）作为中转站，截获对独占设备的输出请求，先写入磁盘上的 **SPOOLing 文件**，再由 SPOOLing 守护进程在后台逐个传输到物理设备。用户进程以为自己在独占设备，实际只是写入了磁盘。



结论 9.5: SPOOLing 系统的组成

SPOOLing 系统由三部分组成：

1. 输入井和输出井：磁盘上开辟的两块区域。
 - 输入井 (**Input Well**)：暂存输入设备（如读卡机）的数据，供进程按需读取。
 - 输出井 (**Output Well**)：暂存进程的输出数据，等待输出设备空闲时送出。
2. 输入进程和输出进程 (**SPOOLing 守护进程**)：
 - 输入进程 (预输入程序/**SPi**)：将数据从物理输入设备预先读入输入井。
 - 输出进程 (缓输出程序/**SPo**)：当输出设备空闲时，将输出井中的数据送到物理设备。
 - 输出进程通常以后台守护进程形式运行。
3. 井管理程序：管理输入/输出井中的空间分配和数据调度。

结论 9.6: SPOOLing 的工作原理与效果

以共享打印机为例的完整流程：

1. 用户进程调用 `print()` —— 系统并不直接分配打印机，而是在输出井中创建 SPOOLing 文件，将打印数据写入磁盘。
2. 用户进程感觉“打印完成”，继续运行（实际上只是写盘完成）。
3. 打印守护进程（如 UNIX 的 `lpd`）轮询输出井，发现有待打印文件，且打印机能接收数

据时，将文件内容逐个发送给打印机。

4. 多个进程的打印请求在磁盘上排队（Spooling Queue），物理打印机串行服务。

SPOOLing 的效果：

- 将独占设备改造为共享设备：多个进程可同时“使用”打印机（实际是同时写磁盘）。
- 将低速 **I/O** 操作变为高速 **I/O** 操作：对于用户进程，写磁盘比等待打印机快得多。
- 实现了虚拟设备功能：每个进程都觉得自己独占了打印机。

注意

408 核心考点：SPOOLing 的本质。

1. SPOOLing 的“假脱机”中，“假”在用户进程以为在与物理 **I/O** 设备直接交互，实际上是在与磁盘交互。
2. SPOOLing 输入/输出井 位于磁盘上（不是内存中！）。
3. SPOOLing 是一种典型的虚拟化技术——与虚拟内存（将磁盘空间当内存用）互为镜像：虚拟内存把磁盘当内存用；**SPOOLing** 把磁盘当设备用。
4. **SPOOLing** 的常见辨析：其典型用途是把打印机等独占设备改造为可由多个进程并发申请的虚拟共享设备；磁盘本身支持按块交错访问，通常不需要靠 SPOOLing 才能实现共享。

9.5 缓冲区管理

定义 9.5: 缓冲的引入动机

为什么需要缓冲？

1. 缓和 **CPU** 与 **I/O** 设备的速度不匹配：CPU 处理速度远快于 I/O 设备，缓冲让两者可以并行工作。
2. 减少对 **CPU** 的中断频率：没有缓冲时，每传一个字节就中断一次；有缓冲后，可等缓冲区满（或半满）再中断。
3. 解决 **DMA** 和中断的数据粒度不匹配：DMA 按块传输，但进程可能按字节处理数据，缓冲区桥接了两者。
4. 提高 **CPU** 和 **I/O** 设备的并行性：CPU 处理上一块数据的同时，设备可以将下一块数据送入缓冲区。

缓冲实现的位置：缓冲区通常在内核空间的内存中分配。

单缓冲（Single Buffer）

结论 9.7: 单缓冲的工作原理与时间分析

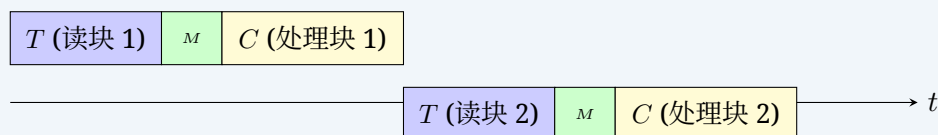
结构：在设备与用户进程之间设置一个缓冲区。设备和进程轮流使用该缓冲区——两者不能同时操作同一缓冲区。

设以下参数：

- T ：设备将一块数据输入到缓冲区所需时间（I/O 传输时间）
- M ：OS 将缓冲区的数据传送到用户工作区所需时间（数据搬移时间）

- C : CPU/用户进程处理一块数据所需时间

单缓冲下的数据处理时序（以输入为例）：



相邻两块完成处理的间隔（稳态）：

$$\text{每块时间} = \max(C, T) + M$$

原因分析：

- 把当前块从系统缓冲区搬到用户区需 M ，搬完后缓冲区才可供设备输入下一块。
- 下一块的输入 T 可与当前块的处理 C 重叠，二者的较大值决定等待时间。
- 搬移阶段 M 使用唯一的系统缓冲区，不能与设备向该缓冲区输入下一块重叠。

处理 N 块数据的总时间：

$$T_{\text{single}} = T + M + C + (N - 1)[\max(C, T) + M]$$

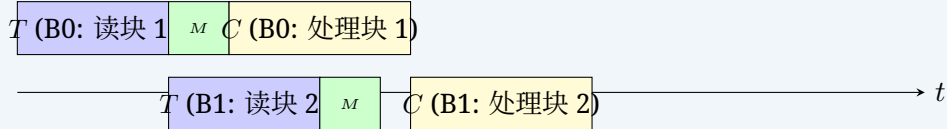
第一块先经历 $T + M + C$ ；之后每增加一块，完成时刻增加 $\max(C, T) + M$ 。

双缓冲 (Double Buffer)

结论 9.8: 双缓冲的工作原理与时间分析

结构：设置两个缓冲区：Buffer0 和 Buffer1（又称缓冲对换，Buffer Swapping）。设备向一个缓冲区填数据时，CPU 可从另一个缓冲区取数据并处理。

双缓冲下的数据处理时序：



相邻两块完成处理的间隔（稳态）：

$$\text{每块时间} = \max(T, M + C)$$

原因分析：

- 设备可以向一个缓冲区输入（耗时 T ），同时系统从另一个缓冲区搬移并由进程处理上一块（串行耗时 $M + C$ ）。
- 两条并行流水路径中较慢的一条决定稳态吞吐率，因此比较的是 T 与 $M + C$ ，而不是只比较 T 与 C 。

处理 N 块数据的总时间：

$$T_{\text{double}} = T + M + C + (N - 1) \max(T, M + C)$$

第一块仍需完整的 $T + M + C$ ；从第二块起，设备输入和“搬移 + 处理”两条路径重叠。

注意

408 计算题关键区分——单缓冲 vs 双缓冲：

真题场景：文件占 N 个磁盘块，每块大小等于缓冲区大小。分别计算单缓冲和双缓冲下读入并处理整个文件的时间。

单缓冲： $T_{\text{single}} = T + M + C + (N - 1)[\max(C, T) + M]$

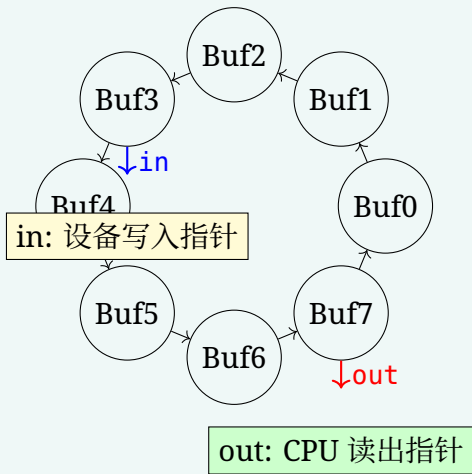
双缓冲： $T_{\text{double}} = T + M + C + (N - 1) \max(T, M + C)$

陷阱：双缓冲的 CPU 路径是 $M + C$ ，两者通常不能彼此重叠；不能把公式误写为 $\max(T, M, C)$ 。若题目未给 M ，应按题干明确的假设取 $M = 0$ 或认为它已包含在其他时间中。

循环缓冲（Circular Buffer）

定义 9.6: 循环缓冲

结构：在双缓冲的基础上，将多个缓冲区组织成一个环形队列。每个缓冲区有一个状态标记：空（Empty）、满（Full）、正在使用（In-Use）。



关键指针：

- **NextIn**（输入指针）：指向设备下一个要填入数据的空缓冲区。
- **NextOut**（输出指针）：指向 CPU 下一个要取数据的满缓冲区。
- **Current**：指向当前正在被 CPU 处理的缓冲区（已从 NextOut 取出）。

与生产者-消费者的关系：循环缓冲本质上是生产者-消费者问题在 I/O 缓冲中的具体应用。设备是生产者（填满缓冲区），CPU 是消费者（取空缓冲区）。管理和同步由 OS 的缓冲区管理程序负责。

缓冲池（Buffer Pool）

结论 9.9: 缓冲池

缓冲池是系统级的大型缓冲区集合，可以被多种设备和进程共享使用，而不是专属于某个特定设备。

缓冲池中的三种缓冲区队列：

缓冲池的三种队列		
队列类型	状态	用途
空缓冲队列（emq）	空闲，可分配	设备和进程均可从此获取空缓冲区
输入队列（inq）	装满输入数据	设备将数据填入后挂入此队列；CPU 从此取出数据
输出队列（outq）	装满输出数据	进程将输出数据填入后挂入此队列；设备从此取出并输出

缓冲池的操作原语：

- 收容输入（**GetBuf_Input**）：从 emq 取一个空缓冲区 → 挂入 inq。
- 提取输入（**ReleaseBuf_Input**）：从 inq 取一个满缓冲区 → 供进程使用 → 回收后挂入 emq。
- 收容输出（**GetBuf_Output**）：从 emq 取一个空缓冲区 → 挂入 outq。
- 提取输出（**ReleaseBuf_Output**）：从 outq 取一个满缓冲区 → 供设备输出 → 回收后挂入 emq。

与专用缓冲的对比：

- 专用缓冲（单/双/循环缓冲）：为特定设备静态分配，利用率可能不足（设备空闲时缓冲区浪费）或不足（高峰期不够用）。
- 缓冲池：所有设备共享，动态分配，利用率高，但管理开销更大。

注意

408 辨析：四个缓冲区概念的区别。

- 单缓冲：稳态间隔为 $\max(C, T) + M$ 。
- 双缓冲：稳态间隔为 $\max(T, M + C)$ ，适合连续 I/O（如磁盘顺序读）。
- 循环缓冲：多个缓冲区环形组织，适合生产者-消费者速率不稳定的场景。
- 缓冲池：系统全局共享的缓冲区集合，通过收容/提取操作管理。

408 经常考察：给定 C 和 T 的值，判断单缓冲和双缓冲的总处理时间差异。当 C 和 T 接近时，双缓冲的优势最大（并行度最高）。

9.6 408 高频考点速查

结论 9.10: I/O 管理—408 常考点

1. 四种 I/O 控制方式：轮询 → 中断 → DMA → 通道，每次演进减少了 CPU 参与 I/O 的程度和中断频率。DMA 与中断的最关键区别：**DMA** 的数据传输不由 CPU 完成，且每块只中断一次。
2. I/O 软件层次：四层自底向上依次是中断处理程序 → 设备驱动程序 → 设备独立性软件 → 用户层 I/O 软件。SPOOLing 属于用户层（守护进程形式）。
3. **SPOOLing** 系统：独占设备 → 共享设备的虚拟化技术。输入/输出井在磁盘上。原理：截获输出请求写入磁盘井，后台守护进程逐个送往物理设备。
4. 缓冲区处理时间计算：
 - 单缓冲： $T + M + C + (N - 1)[\max(C, T) + M]$
 - 双缓冲： $T + M + C + (N - 1) \max(T, M + C)$
5. 块设备 vs 字符设备：块设备可寻址（支持 lseek），以块为单位；字符设备不可寻址，以字符流为单位。
6. **DMA** 周期窃取：DMA 传送期间需占用总线，与 CPU 争用内存周期，但每块只窃取少量周期，总体影响不大。

10 习题

10.1 基础过关题

1. (真题风格·选择题) 系统将数据从磁盘读到内存的过程包括以下操作:

- (1) DMA 控制器发出中断请求
- (2) 初始化 DMA 控制器并启动磁盘
- (3) 从磁盘传输一块数据到内存缓冲区
- (4) 执行「DMA 结束」中断服务程序

正确的执行顺序是

- (A) (3) → (1) → (2) → (4) (B) (2) → (3) → (1) → (4)
- (C) (2) → (1) → (3) → (4) (D) (1) → (2) → (4) → (3)

2. (真题风格·选择题) 在系统内存中设置磁盘缓冲区的主要目的是

- (A) 减少磁盘 I/O 次数 (B) 减少平均寻道时间
- (C) 提高磁盘数据可靠性 (D) 实现设备无关性

3. (真题风格·选择题) 下列关于驱动程序的叙述中, 不正确的是

- (A) 驱动程序与 I/O 控制方式无关
- (B) 初始化设备是由驱动程序控制的
- (C) 驱动程序执行 I/O 操作时, 进程可能会阻塞
- (D) 对磁盘的读/写操作最终转换为驱动程序中的读/写函数

4. (填空题) I/O 软件的四个层次自底向上依次为: ____、____、____ 和 ____。其中, SPOOLing 系统属于 ____ 层; 设备独立性软件负责将 ____ 设备名映射到物理设备; 向上提供统一接口、向下操作设备寄存器的是 ____。

5. (简答题) 简述 DMA 控制方式与中断驱动方式的核心区别 (至少三点), 并说明为什么 DMA 方式更适合高速块设备。

10.2 真题风格与综合训练

6. (真题风格·选择题) 下列关于 SPOOLing 技术的叙述中, 错误的是

- (A) SPOOLing 技术需要外存的支持
- (B) SPOOLing 技术需要多道程序技术的支持
- (C) SPOOLing 技术可以使多个作业共享一台独占设备
- (D) SPOOLing 技术由用户作业控制设备与输入/输出井之间的数据传送

7. (真题风格·选择题) 用户程序发出磁盘 I/O 请求后, 系统的正确处理流程是

- (A) 用户程序 → 系统调用处理程序 → 设备驱动程序 → 中断处理程序
 (B) 用户程序 → 系统调用处理程序 → 中断处理程序 → 设备驱动程序
 (C) 用户程序 → 设备驱动程序 → 系统调用处理程序 → 中断处理程序
 (D) 用户程序 → 设备驱动程序 → 中断处理程序 → 系统调用处理程序

8. (真题风格·选择题) 设系统缓冲区和用户工作区均采用单缓冲, 从外设读入 1 个数据块到系统缓冲区的时间为 100, 从系统缓冲区读入 1 个数据块到用户工作区的时间为 5, 对用户工作区中的 1 个数据块进行分析的时间为 90。进程从外设读入并分析 2 个数据块的最短时间是

(A) 200 (B) 295

(C) 300 (D) 390

9. (真题风格·选择题) 某文件占 10 个磁盘块, 现要把该文件磁盘块逐个读入主存缓冲区, 并送用户区进行分析。假设一个缓冲区与一个磁盘块大小相同, 把一个磁盘块读入缓冲区的时间为 $100\ \mu\text{s}$, 将缓冲区的数据传送到用户区的时间是 $50\ \mu\text{s}$, CPU 对一块数据进行分析的时间为 $50\ \mu\text{s}$ 。在单缓冲区和双缓冲区结构下, 读入并分析完该文件的时间分别是

(A) $1500\ \mu\text{s}$ 、 $1000\ \mu\text{s}$ (B) $1550\ \mu\text{s}$ 、 $1100\ \mu\text{s}$

(C) $1550\ \mu\text{s}$ 、 $1550\ \mu\text{s}$ (D) $2000\ \mu\text{s}$ 、 $2000\ \mu\text{s}$

10. (真题风格, 综合题) 某文件占 200 个磁盘块, 磁盘每块大小为 1 KB, 缓冲区大小等于磁盘块大小。已知: 磁盘将一块数据读入缓冲区的时间 $T = 0.2\ \text{ms}$, 系统将缓冲区数据传送到用户区的时间 $M = 0.01\ \text{ms}$, CPU 处理一块数据的时间 $C = 0.3\ \text{ms}$ 。

- (1) 分别计算使用单缓冲和双缓冲时, 读入并处理完整文件需要的总时间。
- (2) 若每个缓冲区扩大为 2 KB (每批容纳 2 个磁盘块), 在双缓冲下, 读入并处理完整文件需要多长时间? 假定各阶段耗时均与本批包含的块数成正比, 因此每批 $T' = 0.4\ \text{ms}$ 、 $M' = 0.02\ \text{ms}$ 、 $C' = 0.6\ \text{ms}$, 共 100 批。
- (3) 在上述“各阶段时间严格按数据量线性增长、没有固定启动开销”的模型下, 比较两种缓冲大小的稳态吞吐率, 并解释完整文件总时间的细微差别。

解题提示:

- 选择题中的单/双缓冲计算使用: 单缓冲 $T + M + C + (N - 1)[\max(C, T) + M]$; 双缓冲 $T + M + C + (N - 1)\max(T, M + C)$ 。
- 随后的综合题中, 1 KB 单缓冲的稳态项为 $\max(C, T) + M = 0.31\ \text{ms}$, 双缓冲为 $\max(T, M + C) = 0.31\ \text{ms}$, 所以两者恰好同为 $62.20\ \text{ms}$; 2 KB 批量的稳态项为 $\max(0.4, 0.62) = 0.62\ \text{ms/批}$ 。

拓展阅读:

- OSTEP 第 36–37 章「I/O 设备」和「磁盘驱动器」——从设备控制器、驱动程序到 RAID 的完整故事。
- 陈海波《现代操作系统: 原理与实现》第 8 章「设备管理」——Linux 设备驱动模型和中断下半部机制。
- Brooks《人月神话》第 2 章 (名著阅读) ——经典的设备独立性思想来源。

11 综合专题

注意

本章定位：408 操作系统综合应用题可能在一个明确场景中联合考查进程、内存、文件、I/O 或同步机制。本章按专题组织训练；凡涉及时间线，必须先把事件发生时刻、阻塞时长、就绪队列规则和计数口径写清楚，才能得到唯一答案。

11.1 专题一：进程调度 + 内存管理 + 文件系统的综合突破

进程调度和请求分页可以出现在同一场景中，但这类题比单独计算调度或页面置换多一层事件依赖：进程只有占用 CPU 时才能发起访存；缺页后会阻塞；调页完成后还要按题设规则重新进入就绪队列。

结论 11.1: 跨模块时间线的建模规则

一道“调度 + 缺页”的题若有唯一答案，至少应明确以下条件：

1. 一次页面引用是否占用一个完整 CPU 时间单位，缺页发生在该时间单位的开始还是结束；
2. 缺页 I/O 的持续时间，以及调页期间磁盘能否并行处理其他进程的缺页；
3. 调页完成后，进程进入就绪队列队首还是队尾，未用完的时间片是否保留；
4. 页面引用在缺页处理后是否已经完成，还是必须重新调度后再执行一次；
5. 页框采用固定分配还是可变分配，置换范围是局部还是全局。

若题目漏掉其中会改变时间线的条件，就不能自行假设出唯一的周转时间。规范的解题表应逐事件记录“当前运行进程、就绪队列、阻塞集合、页框内容、剩余时间片”，并且只允许当前运行进程发起页面访问。建议先独立算出每个进程在给定页框下的置换过程，再把缺页事件嵌入调度时间线，以减少漏算和重复执行。

11.2 专题二：页表 + TLB + Cache + 缺页的完整访问流程

注意

这是一类典型的跨科综合训练。题目可给定一个虚拟地址，要求模拟从 TLB 查找 → 页表查询 → 缺页处理 → Cache 访问的完整路径，计算总访问时间或判断每一步是否命中，融合组成原理（Cache、TLB）和操作系统（页表、缺页）的知识。

结论 11.2: 完整存储访问流程

例 2 某计算机系统按字节编址，虚拟地址 32 位，物理地址 28 位，页面大小 4 KB。TLB 采用全相联映射，共 16 个表项。Cache 采用直接映射，共 64 行，行大小 64 B。页表采用两级页表结构，页目录基址寄存器（PDBR）指向一级页表首地址。
当前 TLB 内容如下（均为有效项）：

TLB 内容

虚页号	实页框号	访问位
0x00012	0x00456	1
0x00013	0x00321	0
0x00014	0x00123	1

系统当前无空闲页框。页面置换采用全局 **CLOCK** 算法。

问：当 CPU 执行指令 `mov eax, [0x00012345]`（读操作，32 位数据）时：

- (1) 写出虚拟地址的结构划分（虚页号位数、页内偏移位数）。
- (2) 该访存是否 TLB 命中？若命中，给出物理地址；若不命中，说明接下来访问页表的步骤。
- (3) 若发生缺页，描述 CLOCK 算法的处理过程。
- (4) 给出最终访问 Cache 时的 Cache 行号、标记字段和块内偏移。

分步求解：

Step 1: 虚拟地址结构。页面大小 = 4 KB = 2^{12} B，故页内偏移占 **12** 位。虚页号位数 = $32 - 12 = 20$ 位。

Step 2: TLB 查询。虚拟地址 0x00012345：

- 页内偏移 = 低 12 位 = 0x345（即 345_{16} ）
- 虚页号 = 高 20 位 = 0x00012

查 TLB：虚页号 0x00012 命中！对应实页框号 0x00456。

- 物理地址 = 实页框号 $0x00456 \times 2^{12}$ + 页内偏移 0x345
- = $0x00456000 + 0x345 = 0x00456345$

Step 3: 缺页处理（本例未发生，但作为考点展示完整流程）。

若 TLB 不命中，则进行页表查询（两级页表）：

- 虚拟地址 32 位：一级页号（10 位）| 二级页号（10 位）| 页内偏移（12 位）
- 从 PDBR 获取一级页表基址，用一级页号查一级页表，得到二级页表基址。
- 用二级页号查二级页表，得到页表项（含实页框号和有效位等）。
- 若有效位为 0 → 缺页异常，进入缺页处理。

缺页处理（**CLOCK** 算法）：

1. 操作系统选择一个页框淘汰（CLOCK 算法维护一个循环队列和访问位）：
 - 指针指向当前候选页框。若访问位 = 0，淘汰该页；若访问位 = 1，置 0 并移动指针继续扫描。
2. 若被淘汰页为脏页（修改过），先写回磁盘。
3. 从磁盘读入所需页面到该页框。
4. 更新页表项（置有效位 = 1，填实页框号），更新 TLB。
5. 返回用户程序，重新执行导致缺页的指令。

Step 4: Cache 访问。物理地址 0x00456345（28 位）。

Cache 结构分析：

- 总行数 = $64 = 2^6$ ，故行索引占 **6** 位。

- 行大小 = 64 B = 2^6 ，故块内偏移占 6 位。
- 标记位 = 物理地址位数 - 索引位 - 块内偏移位 = $28 - 6 - 6 = 16$ 位。

物理地址 0x00456345 二进制拆分：

- 块内偏移（低 6 位）：0x05 = 5
- 索引（中 6 位）：0x0D = 13
- 标记（高 16 位）：0x00456345 $\gg$ 12 = 0x00456，按 16 位字段补前导零为 0x0456。

结论：Cache 行号 = 13，标记 = 0x0456，块内偏移 = 5。由于是 32 位读，访问字节 5-8。去 Cache 第 13 行比较标记字段，若相等则命中，直接取数据；否则缺失，从主存加载 64 B 到该行。

注意

408 关键辨析：

- **TLB** 是页表的高速缓存（映射虚页号 $\rightarrow$ 实页框号）；**Cache** 是主存的高速缓存（映射物理地址 $\rightarrow$ 数据）。两者处于不同层次。
- 访问顺序：虚拟地址 $\rightarrow$ TLB（或页表）得到物理地址 $\rightarrow$ Cache（或主存）得到数据。
- TLB 命中时不需要访问页表，从而省去一次甚至两次内存访问（两级页表）。
- 缺页时发生磁盘 I/O，耗时通常在 ms 级，远大于 Cache/TLB 缺失（ns 级）。

11.3 专题三：死锁与银行家算法的变体

结论 11.3: 银行家算法核心回顾

银行家算法（Dijkstra, 1965）通过安全性检测判断当前系统是否处于安全状态。

数据结构：

- Available[m]：每种资源的当前可用数量。
- Max[n][m]：每个进程对每种资源的最大需求。
- Allocation[n][m]：每个进程当前已分配的资源。
- Need[n][m] = Max - Allocation。

安全性算法：反复寻找一个 Need $\leq$ Work 的未完成进程，将其 Allocation 加到 Work 中。若所有进程均能被完成，则系统安全，且找到的进程序列即为安全序列。

结论 11.4: 变体一：多资源类型 + 未指定 Max

例 3 系统有 3 类资源 A、B、C，总量分别为 (8, 6, 5)。当前时刻 T_0 的 Allocation 如下：

T_0 时刻分配情况

进程	A	B	C
P1	2	1	0
P2	1	2	1
P3	3	0	2
P4	0	1	1

已知 P1 和 P4 各自还需要 (2, 1, 1) 和 (1, 0, 0) 才能完成。P2 和 P3 未声明最大需求，但已知它们不会同时需要同种资源的最大量。

问：

- (1) 计算 Available 数组。
- (2) 现有信息能否判断 T_0 时刻是否安全？说明理由。
- (3) 若此时 P3 申请 (0, 1, 0)，仅凭现有信息能否决定立即分配？

分步求解：

Step 1: Available 计算。已分配总量：A: $2 + 1 + 3 + 0 = 6$ ，B: $1 + 2 + 0 + 1 = 4$ ，C: $0 + 1 + 2 + 1 = 4$ 。Available = 总量 - 已分配 = $(8 - 6, 6 - 4, 5 - 4) = (2, 2, 1)$ 。

Step 2: 判断信息是否充分。P1 Need = (2, 1, 1)，P4 Need = (1, 0, 0)，但 P2 和 P3 的 Max (或 Need) 没有给出。银行家安全性算法必须逐进程判断 $Need_i \leq Work$ ；仅知道“P2 和 P3 不会同时需要同种资源的最大量”并不能给出二者的分量上界，也不能确定其中任何一个是否能在当前 Work 下完成。

因此，第 (2) 问信息不足，无法判断系统是否安全，也无法给出可靠的安全序列。即使 P4、P1 可以依次完成，释放资源后的 Work 也未必满足未知的 P2 或 P3 的剩余最大需求。

Step 3: 判断 P3 的请求。Request(0, 1, 0) $\leq$ Available(2, 2, 1) 只说明当前有足够资源，这是允许分配的必要条件之一。由于 P3 的 Need 未知，既不能验证 $Request_3 \leq Need_3$ ，也无法在试分配后完成安全性检测。因此第 (3) 问同样不能确定。要使题目可判定，必须补充 P2、P3 的 Max/Need，或给出足以推出其逐类剩余需求上界的等价条件。

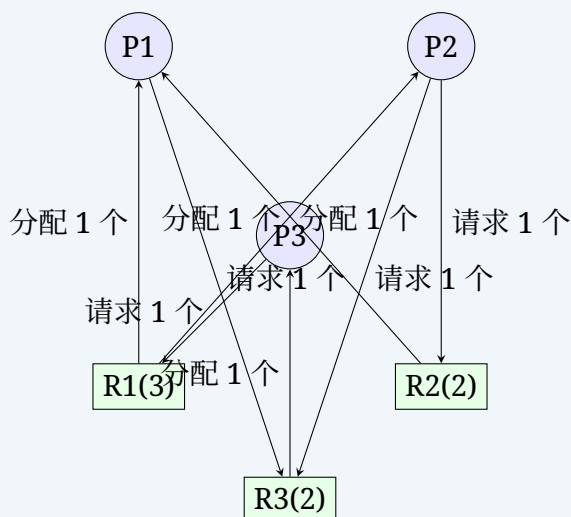
注意

银行家算法变体常考形式：

- 部分进程未声明 **Max**：通常无法直接应用银行家算法；除非题目另给出足以确定逐类 Need 上界的条件，否则结论应是“信息不足”。
- 多实例资源（如 5 台同类打印机）：将资源类型数 m 简化处理。
- 死锁检测变体：给定 Allocation 和 Available，判断哪些进程已死锁（无法完成也无法释放资源的循环等待环）。

结论 11.5: 变体二：资源分配图化简

例 4 某系统有资源 R1（3 个实例）、R2（2 个实例）、R3（2 个实例）。当前资源分配图如下：



问：

- (1) 分析每个资源的剩余可用实例数。
- (2) 用资源分配图化简法判断是否存在死锁。
- (3) 若存在死锁，指出死锁进程集合。

分步求解：

Step 1: 计算可用实例。

- R1 (3 实例)：已分配给 P1、P2 各 1 个，剩余 1 个；P3 请求的 1 个当前可以满足。
- R2 (2 实例)：已被分配 1 个 (P1)，剩余 1 个。P2 请求 1 个——可满足！
- R3 (2 实例)：已被分配 1 个 (P3)，剩余 1 个。P1 请求 1 个，P2 也请求 1 个——只能满足一个。

Step 2: 资源分配图化简。初始 Available 为 (1, 1, 1)。P1 只请求 R3 的 1 个实例，当前即可满足；令 P1 完成并释放其持有和刚获得的资源后，系统可用资源进一步增加。随后 P3 请求的 R1 可满足，P3 完成后释放 R3；最后 P2 请求的 R2、R3 均可满足。故图可完全化简。

Step 3: 结论。图可完全化简 → 不存在死锁。一个化简序列为 P1 → P3 → P2；P3 → P1 → P2 也可行。

注意

资源分配图化简法的关键技巧：

- 只有所有请求边均能被满足的进程才能被化简（模拟其完成并释放所有持有资源）。
- 对多实例资源，图中有环并不是死锁的充分条件；应以是否还能找到“全部当前请求均可满足”的进程继续化简为准。
- 若化简过程停止后仍有进程无法消去，这些剩余进程才是死锁进程；单实例资源图中有环才可直接判定死锁。

11.4 专题四：磁盘调度 + 文件物理结构的组合题

结论 11.6: 组合题：i-node 混合索引 + 磁盘调度

例 5 某文件系统采用 UNIX i-node 混合索引结构：每个 i-node 含 10 个直接块指针、1 个一级间接块指针、1 个二级间接块指针。盘块大小 4 KB，每个块指针占 4 字节。磁盘参数：旋转速度 7200 rpm，平均寻道时间 6 ms，每个磁道 256 个扇区（每扇区 512 B）。问：

- (1) 计算该文件系统支持的最大单个文件大小。
- (2) 若要访问文件中偏移量为 100 MB 处的数据，需要访问哪些索引块？共需几次磁盘 I/O（假设无缓存）？
- (3) 该文件的数据块散布在磁盘上，某时刻的 I/O 请求序列（磁道号）为：45, 120, 15, 85, 200, 70, 155。当前磁头在 60 号磁道，方向为磁道号增大方向。用 SSTF 和 C-LOOK 分别计算磁头移动总磁道数。
- (4) 若 SSTF 调度下访问这些盘块，每次访问需读一整块（8 个扇区），估算总数据访问时间。

分步求解：

Step 1：最大文件大小。

- 直接块： $10 \times 4 \text{ KB} = 40 \text{ KB}$
- 一级间接块：一个盘块可存 $4 \text{ KB} / 4 \text{ B} = 1024$ 个指针 $\rightarrow 1024 \times 4 \text{ KB} = 4 \text{ MB}$
- 二级间接块： $1024 \times 1024 \times 4 \text{ KB} = 4 \text{ GB}$
- 按二进制单位计算，最大文件大小为 $40 \text{ KiB} + 4 \text{ MiB} + 4 \text{ GiB} = \mathbf{4.0039444 \text{ GiB}}$ （共 4 299 202 560 B）。

Step 2：100 MB 偏移量访问路径。

- $100 \text{ MB} = 100 \times 1024 \text{ KB} = 102400 \text{ KB}$
- 前 10 个直接块覆盖 $0 \sim 40 \text{ KB}$ 。
- 一级间接块覆盖接下来的 4096 KiB，因此直接块和一级间接块合计覆盖前 $40 + 4096 = 4136 \text{ KiB}$ 。
- $100 \text{ MiB} = 102400 \text{ KiB} > 4136 \text{ KiB}$ ，故该偏移量位于二级间接块范围；相对于二级间接区起点的偏移为 $102400 - 4136 = 98264 \text{ KiB}$ 。
- 访问路径：读 i-node $\rightarrow$ 二级间接块 $\rightarrow$ 一级间接块 $\rightarrow$ 数据块。
- I/O 次数（无缓存）：读 i-node(1) + 二级间接块(1) + 一级间接块(1) + 数据块(1) = **4 次**。

Step 3：SSTF 调度。 请求序列：45, 120, 15, 85, 200, 70, 155。当前 60。

SSTF 调度推演

步	当前位置	最近请求	移动距离
1	60	70	10
2	70	85	15
3	85	120	35
4	120	155	35
5	155	200	45
6	200	45	155
7	45	15	30

SSTF 总寻道 = $10 + 15 + 35 + 35 + 45 + 155 + 30 = 325$ 个磁道。

C-LOOK 调度 (当前 60, 增大方向):

- 升序服务: $60 \rightarrow 70 \rightarrow 85 \rightarrow 120 \rightarrow 155 \rightarrow 200$
- 跳到最小请求: $200 \rightarrow 15 \rightarrow 45$
- 移动距离 = $(70 - 60) + (85 - 70) + (120 - 85) + (155 - 120) + (200 - 155) + (200 - 15) + (45 - 15) = 10 + 15 + 35 + 35 + 45 + 185 + 30 = 355$

Step 4: 数据访问时间估算。每块 = $8 \text{ 扇区} \times 512 \text{ B} = 4 \text{ KB}$ (恰好一个盘块)。

每次 I/O 访问时间 = $T_{\text{寻道}} + T_{\text{旋转延迟}} + T_{\text{传输}}$ 。

对一次随机访问:

- 平均寻道时间 = 6 ms (给定)
- 旋转速度 $7200 \text{ rpm} \rightarrow \text{一转} = 60/7200 = 8.33 \text{ ms}$, 平均旋转延迟 = 半圈 = 4.17 ms
- 传输时间 = 每块扇区数 / 每道扇区数 $\times$ 单圈时间 = $8/256 \times 8.33 = 0.26 \text{ ms}$
- 单次随机 I/O $\approx 6 + 4.17 + 0.26 = 10.43 \text{ ms}$

按题目给出的“每次随机访问平均寻道 6 ms ”模型, 7 个请求的总访问时间约为

$$7 \times (6 + 4.17 + 0.26) \text{ ms} \approx 73.0 \text{ ms}.$$

这个平均值模型不能利用“总移动 325 个磁道”进一步精确换算寻道时间, 因为实际寻道时间与跨越磁道数不是简单线性关系。若要比 SSTF 与其他调度算法的实际耗时, 题目还必须给出寻道时间函数或逐距离查表; 仅给平均寻道时间时, 只能作上述平均估算。

注意**408 磁盘相关题目的命题套路:**

- 将文件索引结构与磁盘调度绑在一起: 先算文件访问需要读哪些块 (含索引块), 再将块号对应到磁道号, 最后用调度算法优化访问顺序。
- 注意索引块也要算 I/O, 且索引块本身也占用盘块号 (也在磁盘调度队列中)。
- 旋转延迟计算: 题目若给出扇区号分布 (如连续扇区跨越多个块), 可能涉及旋转定位的细节计算。

11.5 专题五：PV 操作变体题（复杂同步问题）

注意

408 PV 操作命题趋势：近年来不再是简单的生产者-消费者，而是多个进程、多种约束组合的复杂场景。需要同时处理互斥与同步、多信号量的顺序约束。

结论 11.7: 变体一：理发师问题

例 6 理发店有 1 位理发师、1 张理发椅和 n 把等候椅。若无顾客，理发师睡觉；顾客到来若理发师在睡则唤醒；若理发师正忙且有空椅则坐下等候；若无空椅则离开。用信号量写出理发师和顾客的同步算法。

解法：

```

1  semaphore mutex = 1;           // 互斥访问 waiting 计数器
2  semaphore customer = 0;        // 已到达、尚未被理发师接待的顾客数
3  semaphore barber = 0;         // 理发师是否可用
4  int waiting = 0;              // 等候椅上的人数
5  int CHAIRS = n;               // 等候椅数量
6
7  // 理发师进程
8  void barber_process() {
9      while (1) {
10         P(&customer);           // 若无顾客则睡觉
11         P(&mutex);
12         waiting--;              // 一位顾客去理发（离开等候椅）
13         V(&barber);             // 通知该顾客可以理发
14         V(&mutex);
15         cut_hair();             // 理发
16     }
17 }
18
19 // 顾客进程
20 void customer_process() {
21     P(&mutex);
22     if (waiting < CHAIRS) {
23         waiting++;              // 占一把等候椅
24         V(&customer);           // 通知理发师有顾客
25         V(&mutex);
26         P(&barber);             // 等理发师空闲（或叫到自己）
27         get_haircut();          // 接受理发
28     } else {
29         V(&mutex);
30         leave();                // 无空椅，离开
31     }
32 }
```

要点分析：

- customer 信号量防止理发师忙等（若无顾客则睡觉）。
- barber 信号量实现「顾客等理发师叫」的同步。
- mutex 保护 waiting 计数器（多个顾客可能同时到达）。

- 顾客必须在持有 mutex 时完成 waiting++, 但 V(customer) 与 V(mutex) 的先后在该模型中都可保持正确性: 若先唤醒理发师, 理发师会暂时阻塞在 P(mutex); 若先释放互斥量, 再通知顾客到达, 计数信号仍不会丢失。通常采用代码所示顺序, 便于把「登记到达并发出通知」视为一个受保护过程。

结论 11.8: 变体二: 独木桥问题

例 7 一座独木桥每次只能容一人通过, 题目明确要求东向和西向行人严格逐人交替。用信号量实现:

严格交替解法:

```

1 semaphore east_turn = 1;    // 初始允许一名东向行人
2 semaphore west_turn = 0;

3
4 // 东向行人
5 void east_traveler() {
6     P(&east_turn);
7     cross_bridge_east();
8     V(&west_turn);
9 }
10
11 // 西向行人
12 void west_traveler() {
13     P(&west_turn);
14     cross_bridge_west();
15     V(&east_turn);
16 }
```

两个令牌信号量既保证桥上至多一人, 也保证方向严格交替。代价是: 若轮到的方向没有行人, 另一方向即使有人等待也不能继续, 这正是「严格交替」这一强约束的含义。若题目只要求互斥而不要求严格交替, 应改用一个 **bridge=1**; 若还要求工作保持性和防饥饿, 则需增加等待计数与公平的方向切换策略, 不能直接套本解法。

注意

独木桥问题的核心:

- 若只要求互斥: 一个信号量 **bridge = 1** 即可。
- 若要求同向可同时通过: 类似读者-写者中的读者共享资源, 同向第一人占桥、最后一人释放桥, 但更新方向人数时仍需互斥。
- 若要求批次公平且防止饥饿: 需要等待计数、当前方向和方向切换机制, 明确何时关闭本方向入口。
- **408 常见问法:** 补充代码空缺 (P/V 位置)、判断是否存在死锁或饥饿、增加新的约束条件后修改代码。

结论 11.9: 变体三: 吸烟者问题

例 8 一个供应者和三个吸烟者。供应者每次提供两种材料 (共三种材料: 烟草、纸、火柴), 拥有第三种材料的吸烟者取走并吸烟, 完成后供应者继续提供。这是一个典型的单生产者多消费者且消费者间互斥问题。

```

1 semaphore tobacco_owner = 0; // 纸+火柴已就绪
2 semaphore paper_owner = 0; // 烟草+火柴已就绪
3 semaphore match_owner = 0; // 烟草+纸已就绪
4 semaphore done = 0; // 吸烟完成, 通知供应者继续
5
6 // 供应者
7 void provider() {
8     while (1) {
9         // 随机提供两种材料 (三种组合之一)
10        int combo = random() % 3;
11        if (combo == 0) { // 纸+火柴 $|to$ 有烟草者可吸
12            V(&tobacco_owner);
13        } else if (combo == 1) { // 烟草+火柴 $|to$ 有纸者可吸
14            V(&paper_owner);
15        } else { // 烟草+纸 $|to$ 有火柴者可吸
16            V(&match_owner);
17        }
18        P(&done); // 等吸烟者完成
19    }
20 }
21
22 // 拥有烟草的吸烟者
23 void smoker_with_tobacco() {
24     while (1) {
25        P(&tobacco_owner); // 等待「纸+火柴」这一完整组合
26        smoke();
27        V(&done); // 通知供应者
28    }
29 }
30 // 另两个吸烟者分别等待 paper_owner 和 match_owner

```

注意：不能简单地三种单项材料各设一个信号量，再让每个吸烟者连续执行两次 P。否则两个不匹配的吸烟者可能各取走一种材料并互相等待，形成死锁。这里把「一对材料已就绪」直接编码为对应吸烟者的同步信号，每轮只唤醒唯一能完成工作的吸烟者。

结论 11.10: 变体四：多缓冲区的生产者-消费者

例 9 有 M 个生产者和 N 个消费者，共享一个环形缓冲区（容量为 K ）。每个生产者每次生产一个产品放入缓冲区，每个消费者每次取走一个产品。缓冲区满时，继续申请空位的生产者阻塞；缓冲区空时，继续申请产品的消费者阻塞。生产产品和消费产品的耗时操作可以并发，但对共享环形队列及其读写下标的修改必须互斥。这就是标准的有界缓冲区问题，朴素解法即可满足：

```

1 semaphore mutex = 1; // 互斥访问缓冲区
2 semaphore empty = K; // 空位数
3 semaphore full = 0; // 产品数
4
5 // 生产者
6 void producer(int id) {
7     while (1) {
8         produce_item();

```



```

9         P(&empty);          // 申请空位 (满则阻塞)
10        P(&mutex);
11        put_item();
12        V(&mutex);
13        V(&full);           // 增加产品
14    }
15 }
16
17 // 消费者
18 void consumer(int id) {
19     while (1) {
20         P(&full);           // 申请产品 (空则阻塞)
21         P(&mutex);
22         get_item();
23         V(&mutex);
24         V(&empty);         // 增加空位
25         consume_item();
26     }
27 }

```

关键问题变体：

- **P 操作顺序能否交换？** 不行。若先 P(mutex) 再 P(empty)，缓冲区满时生产者持有 mutex 阻塞，消费者无法进入释放空位 → 死锁。
- **V 操作顺序能否交换？** 可以，但不推荐换。先 V(mutex) 后 V(full/empty) 不影响正确性（因 V 不阻塞）。
- **put_item() 与 get_item() 在本实现中不能并发，** 因为二者共享同一个环形队列元数据和 mutex；并发的是临界区外的 produce_item() 与 consume_item()。若题目给出读写下标可独立更新的特殊结构，才能进一步拆分锁。

11.6 专题六：缺页异常 + 文件内存映射的综合分析

结论 11.11: mmap + 缺页的完整流程

例 10 某进程使用 mmap 将一个 2 MB 的文件映射到虚拟地址空间。系统采用请求分页 (Demand Paging)，页面大小 4 KB，物理内存共 128 个页框，当前空闲页框为 0。页面置换采用全局 LRU。

进程按以下顺序访问映射区域（以页面为单位，虚拟页号从映射起始页为 0）：

0, 1, 2, 3, 0, 1, 4, 2, 3, 0, 1, 4

问：

- (1) 若给该进程分配 3 个物理页框，用 LRU 置换时缺页次数是多少？
- (2) 若改用 CLOCK 算法（初始访问位全为 0，指针指向最早装入的页框），缺页次数是多少？
- (3) mmap 映射的文件页面在第一次缺页时从哪里加载数据？后续被换出后又换入时呢？

Step 1: LRU 缺页推演（3 页框）。

LRU 页面置换推演

引用	页框 0	页框 1	页框 2	缺页?
0	0	—	—	✓
1	0	1	—	✓
2	0	1	2	✓
3	3(LRU=0)	1	2	✓
0	3	0(LRU=1)	2	✓
1	3	0	1(LRU=2)	✓
4	4(LRU=3)	0	1	✓
2	4	2(LRU=0)	1	✓
3	4	2	3(LRU=1)	✓
0	0(LRU=4)	2	3	✓
1	0	1(LRU=2)	3	✓
4	0	1	4(LRU=3)	✓

LRU 缺页 12 次（每次引用都缺页——Belady 反常未出现，但 3 页框对引用串 0,1,2,3,0,1,4,2,3,0,1,4 确实全部缺页）。

Step 2: CLOCK 算法推演。

CLOCK 置换推演

引用	页框 0	页框 1	页框 2	指针指向	缺页?
0	0(A=1)	—	—	1	✓
1	0(A=1)	1(A=1)	—	2	✓
2	0(A=1)	1(A=1)	2(A=1)	0	✓
3	3(A=1)	1(A=0)	2(A=0)	1	✓ (0 被淘汰)
0	3(A=1)	0(A=1)	2(A=0)	2	✓ (1 被淘汰)
1	3(A=1)	0(A=0)	1(A=1)	0	✓ (2 被淘汰)
4	4(A=1)	0(A=0)	1(A=0)	1	✓ (3 被淘汰)
2	4(A=1)	2(A=1)	1(A=0)	2	✓ (0 被淘汰)
3	4(A=1)	2(A=0)	3(A=1)	0	✓ (1 被淘汰)
0	0(A=1)	2(A=0)	3(A=0)	1	✓ (4 被淘汰)
1	0(A=1)	1(A=1)	3(A=0)	2	✓ (2 被淘汰)
4	0(A=1)	1(A=0)	4(A=1)	0	✓ (3 被淘汰)

CLOCK 也是 12 次缺页（对 3 页框，此引用串两种算法均全部缺页）。

Step 3: mmap 页面加载来源。

- 第一次缺页：页面数据从磁盘文件中读入。OS 在页表中建立映射（虚页号 → 页框号），标记该页为干净（与文件内容一致）。
- 被换出后再次缺页：
 - 若该页未被修改过（干净的）——直接从文件重新读入即可（丢弃旧的页框内容即可，

因为与文件一致)。

- 若采用共享文件映射 (MAP_SHARED) 且页面被修改, 脏页最终写回对应文件; 换入时仍以文件为后备存储。
- 若采用私有映射 (MAP_PRIVATE), 写入会触发写时复制, 修改后的私有页按匿名页处理, 换出时通常以交换区为后备存储, 不应写回原文件。
- 与匿名页 (**anonymous page**) 的区别: 匿名页 (如 malloc 分配的堆内存) 没有文件后备, 被换出时写入交换区 (swap), 换入时从交换区读取。

注意

mmap 缺页在 408 中的考点:

- 区分文件映射页 (file-backed) 和匿名页 (anonymous) 的换入/换出路径。
- 脏页写回: 文件映射页写回对应文件, 匿名页写回交换区。
- 写时复制 (Copy-on-Write): fork 后父子共享页表且标记为只读, 写操作触发缺页, OS 复制页面。

12 习题

12.1 真题风格与综合训练

1. (综合题) 某分页虚拟存储系统, 物理内存共 256 MiB, 页面大小 4 KiB。进程 P 的虚拟地址空间 4 GiB, 当前分配了 8 个物理页框。

- (1) 虚拟地址多少位? 物理地址多少位? 页内偏移多少位?
- (2) 若 P 的页表为二级页表 (每级各 10 位), 一级页表必须常驻内存。当 TLB 缺失时, 每次数据访问最多需要几次内存访问?
- (3) 某次访问的虚拟地址为 0x00456789, TLB 命中, 对应物理页框号为 0x1A, 求物理地址。

2. (综合题) 某系统当前资源分配如下, 总量: (A,B,C,D) = (8,6,4,5)。

当前分配

进程	Allocation				Max			
	A	B	C	D	A	B	C	D
P0	1	2	0	1	3	3	2	2
P1	2	0	1	2	4	1	3	3
P2	1	1	1	0	2	3	2	1
P3	2	1	0	1	4	2	1	2

- (1) 计算 Available 和 Need 矩阵。
 - (2) 系统当前是否安全? 若安全, 列出一个安全序列。
 - (3) P1 请求 (1, 0, 1, 1), 能否立即分配?
 - (4) 若 P2 请求 (0, 1, 0, 0), 分配后系统还安全吗?
3. (综合题) 某进程的页面引用序列为: 1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6。系统分配给该进程 4 个物理页框。
- (1) 分别用 FIFO 和 LRU 算法计算缺页次数。
 - (2) 若采用 CLOCK 算法 (初始所有访问位为 0, 指针指向第一个页框), 计算缺页次数。
 - (3) 再计算 FIFO 在 3 个页框时的缺页次数。页框数从 3 增至 4 时是否出现 Belady 反常?
4. (综合题) 某磁盘有 200 个磁道 (0-199), 当前磁头在 53 号磁道, 上一个请求在 45 号磁道 (即磁头向减小方向移动)。请求序列为: 98, 183, 37, 122, 14, 124, 65, 67。
- (1) 画出 SCAN 算法 (向减小方向先) 的磁头移动轨迹, 计算总寻道长度。
 - (2) 画出 C-LOOK 算法的磁头移动轨迹, 计算总寻道长度。
 - (3) 若每个磁道有 512 个扇区, 每扇区 512 B, 旋转速度 10000 rpm, 平均寻道时间为 4 ms。忽略控制器开销, 求一次 4 KB 连续块访问的平均访问时间。
5. (PV 设计题) 三个进程 P1、P2、P3 互斥使用一个包含 N 个存储单元的缓冲区。P1 每次用 `produce()` 生成一个正整数, 送入缓冲区某空单元; P2 每次从缓冲区中取出一个奇数并用

`count_odd()` 统计; P3 每次取出一个偶数并用 `count_even()` 统计。用信号量机制实现三个进程的同步与互斥。

6. (PV 设计题) 某博物馆最多容纳 M 名游客同时参观。入口和出口各有一扇旋转门, 一次只能通过一人。游客到达后若博物馆已满则在外等候。管理员可在任何时候关闭博物馆 (不再放新游客进入, 但已在馆内的游客可以继续参观并离开)。用信号量实现游客和管理员的同步。
7. (综合题) 某文件系统盘块大小 1 KiB, 每个块指针占 4 字节。i-node 包含 8 个直接块指针、1 个一级间接块指针和 1 个二级间接块指针。
 - (1) 求该文件系统支持的最大文件大小。
 - (2) 若要访问文件偏移量 5 MiB 处的数据, 需访问哪些索引块? 最少需几次磁盘 I/O?
 - (3) 访问 5 MiB 偏移量时, 二级间接根块位于磁道 11, 所需的下一级索引块位于磁道 12, 目标数据块位于磁道 40。当前磁头在磁道 5, 且各级索引必须按依赖顺序读取。总磁头移动量是多少?
8. (综合题) 某 32 位系统按字节编址, 采用两级页表, 页面大小 4 KB。TLB 共 16 项, 采用全相联映射, LRU 替换。Cache 容量 64 KB, 采用 4 路组相联, 行大小 32 B, 共 512 组。系统采用请求分页, 页面置换算法为改进的 CLOCK。
 - (1) 虚拟地址中, 一级页号、二级页号、页内偏移各占多少位?
 - (2) 若 TLB 初始为空, 进程依次访问虚拟地址 $0x00001000$, $0x00002000$, $0x00001004$, $0x00003000$ (均为有效映射, 物理页框分别为 100, 200, 100, 300)。写出每次访问的 TLB 命中情况和最终 TLB 内容。
 - (3) 对第 (2) 问中的第 3 次访问 ($0x00001004$), 写出 Cache 访问的组号、标记和块内偏移。
 - (4) 若访问 $0xABCDE000$ 时 TLB 缺失、页表缺失 (二级页表不存在)、最终物理页框为 $0x5678$, 描述完整的缺页处理流程。
9. (综合题) 某系统采用索引分配 (类似 UNIX i-node), 一个 i-node 含 12 个直接块指针, 1 个一级间接、1 个二级间接、1 个三级间接。盘块大小 2 KiB, 指针占 4 字节。
 - (1) 求最大文件大小 (以 GiB 为单位, 精确到小数点后两位)。
 - (2) 文件 A 大小为 200 KiB, 文件 B 大小为 20 MiB, 文件 C 大小为 2 GiB。问三个文件各需要几个索引块 (i-node 已调入内存, 不算在内) 才能访问文件末字节?
 - (3) 假定一次只处理这一个逻辑访问, 各级索引块和数据块所在磁道相互独立且均匀分布在 0~999, 初始磁头位置也服从同一分布; 忽略旋转和传输时间, 寻道时间为磁道差 $\times 0.1$ ms。估算访问文件 C 与文件 A 最后一个字节的期望寻道时间差。
10. (综合题) 考虑单 CPU 系统的如下观察时刻: 进程 A 执行 `read(fd, buf, 4096)` 时发生页缓存未命中, 已等待磁盘 I/O; 进程 D 的 `write()` 因管道满而阻塞; 进程 C 刚因时间片用完被抢占并进入就绪队列; 调度器随后选中进程 B, B 已从 `malloc(1<<20)` 返回, 正准备首次写入新分配区域。
 - (1) 分析四个进程分别处于什么状态 (运行/就绪/阻塞)。
 - (2) 进程 A 的 `read` 完成需经历哪些 OS 模块的处理? 描述完整流程 (从系统调用入口到数据返回用户缓冲区)。
 - (3) `malloc` 返回和 B 首次写入新区域分别是否必然立即分配物理页? B 首次写入可能触发的异常与 A 等待文件读取有何区别?
 - (4) 进程 D 何时被唤醒? 描述管道写操作与管道满时的阻塞/唤醒机制。

拓展阅读：

- OSTEP 第 18–22 章「分页」+ 第 26–28 章「并发」——分页地址翻译和并发程序的完整讨论，是综合题的底层逻辑来源；
- OSTEP 第 39–45 章「文件系统实现」——i-node、目录、FAT 和 FFS 的详细实现，涵盖磁盘布局与性能分析；
- 陈海波《现代操作系统：原理与实现》第 5 章「内存管理」+ 第 6 章「进程调度」——现代 OS (Linux) 中缺页处理、页面回收和 CFS 调度的实现细节；
- Kurose《计算机网络》第 6 章（仅用于网络 I/O 与 OS 缓冲区的交叉理解）。

PART IV 第四部分

计算机网络



1 计算机网络体系结构

1.1 计算机网络的概念与分类

定义 1.1: 计算机网络

计算机网络是将分布在不同地理位置、具有独立功能的计算机系统, 通过通信设备和线路连接起来, 以网络协议实现资源共享和数据通信的系统。

结论 1.1: 计算机网络的分类

- 按覆盖范围: 广域网 WAN、城域网 MAN、局域网 LAN、个域网 PAN。
- 按传输技术: 广播式网络 vs 点对点网络。
- 按拓扑结构: 星形、总线形、环形、树形、网状。
- 按交换技术: 电路交换、报文交换、分组交换。

408 中主要使用按覆盖范围的分类。

1.2 电路交换、报文交换与分组交换

结论 1.2: 三种交换技术的对比

交换技术对比			
	电路交换	报文交换	分组交换
建立连接	需要	不需要	不需要 (虚电路需要)
传输单位	比特流	整个报文	分组 (包)
存储转发	否	是	是
主要时延特征	建立阶段有连接建立时延; 建立后无逐跳存储转发, 但仍有发送与传播时延	每一跳收完整个报文后再转发, 缓冲需求和等待时延大	逐分组存储转发, 可在多链路间流水传输
灵活性	差 (专用线路)	好	好
典型应用	传统电话	电报	互联网 (IP)

分组交换的优点: (1) 线路利用率高 (统计复用); (2) 不同速率/格式的终端可互通; (3) 优先级。

分组交换的缺点: (1) 额外开销 (首部); (2) 存储转发引入排队时延, 不适合实时通信。

408 常考: 给定场景, 判断应使用哪种交换方式。

1.3 计算机网络的性能指标

要点 1.1: 网络性能指标

- **速率 (Data Rate)**: 每秒传输的比特数 (bps)。1K = 10^3 , 1M = 10^6 (通信领域, 非 2^{10})。
- **带宽 (Bandwidth)**: 信道允许通过的频率范围 (Hz), 或网络通信线路所能传送数据的能力 (bps)。
- **吞吐量 (Throughput)**: 单位时间内实际通过网络的数据量。
- **时延 (Delay/Latency)**: $d_{\text{总}} = d_{\text{发送}} + d_{\text{传播}} + d_{\text{处理}} + d_{\text{排队}}$
 - 发送时延: $\frac{\text{数据帧长度}}{\text{发送速率}}$ (与传输距离无关)
 - 传播时延: $\frac{\text{信道长度}}{\text{电磁波传播速率}}$ (与发送速率无关)
- **时延带宽积**: $d_{\text{传播}} \times \text{带宽} = \text{链路中最多容纳的比特数}$ 。
- **往返时间 RTT**: 数据从发送端发出到收到确认的时间。
- **利用率**: 信道利用率 = 有数据通过时间 / 总时间。利用率趋近 1 时时延急剧增大 (排队论)。

注意

408 高频计算: 发送时延 vs 传播时延。 发送时延 = 数据量/带宽 (瓶颈在发送速率); 传播时延 = 距离/光速 (瓶颈在物理距离)。两者影响的变因完全不同, 选择题经常混合考察。

1.4 OSI 参考模型 (7 层)

定义 1.2: OSI 七层模型

国际标准化组织 (ISO) 提出的开放系统互连参考模型:

应用层 → 表示层 → 会话层 → 传输层 → 网络层 → 数据链路层 → 物理层

记忆口诀: **All People Seem To Need Data Processing** (从高到低)。

OSI 各层功能

层次	功能	数据单元
物理层	比特传输, 定义机械/电气/功能/规程特性	比特 (bit)
数据链路层	相邻结点间可靠传输, 成帧、差错控制、流量控制	帧 (Frame)
网络层	路由选择、拥塞控制、网际互连	分组/数据报 (Packet)
传输层	端到端可靠/不可靠传输, 复用与分用	报文段/用户数据报
会话层	会话管理、同步	—
表示层	数据格式转换、加密解密、压缩	—
应用层	为应用程序提供网络服务	报文 (Message)

1.5 TCP/IP 模型（4 层）

结论 1.3: TCP/IP 四层模型

TCP/IP 是事实上的工业标准，而非法律上的国际标准（OSI 是法律标准但未流行）。

OSI 与 TCP/IP 对应关系		
OSI	TCP/IP	典型协议
应用层、表示层、会话层	应用层	HTTP, DNS, SMTP, FTP
传输层	传输层	TCP, UDP
网络层	网际层	IP, ICMP, ARP, OSPF, BGP
数据链路层、物理层	网络接口层	Ethernet, PPP

五层参考模型（教学用，408 采用）：物理层 → 数据链路层 → 网络层 → 传输层 → 应用层。

1.6 协议与服务

定义 1.3: 协议三要素

网络协议（Protocol）是为进行数据交换而建立的规则，由三个要素组成：

- 1. 语法（**Syntax**）：数据与控制信息的结构或格式；
- 2. 语义（**Semantics**）：每种控制信息的含义以及应完成的动作；
- 3. 同步/时序（**Timing**）：事件实现顺序的详细说明。

结论 1.4: 协议与服务的关系

- 协议是「水平的」：控制两个对等实体之间的通信规则。
- 服务是「垂直的」：下层通过层间接口向上层提供功能。
- 第 n 层实体向第 $n + 1$ 层提供服务（Service），使用的对等协议在第 n 层。
- **SAP**（服务访问点）：相邻层间交换信息的逻辑接口。数据链路层的 SAP 是 MAC 地址，网络层是 IP 地址，传输层是端口号。

1.7 408 高频考点速查

结论 1.5: 体系结构—408 常考点

- 1. 发送时延 vs 传播时延：计算公式、影响因素。
- 2. **OSI 7 层 vs TCP/IP 4 层 vs 五层模型**：各层名称、功能、典型协议、数据单元名称。
- 3. 协议三要素：语法、语义、时序。
- 4. 分组交换 vs 电路交换：各自时延构成、适用场景。
- 5. 各层中间设备：物理层（中继器/集线器）、数据链路层（网桥/交换机）、网络层（路由器）、传输层及以上（网关）。

2 习题

2.1 基础过关题

1. (真题风格·选择题) 在 OSI 参考模型中, 自下而上第一个提供端到端服务的层次是
(A) 数据链路层 (B) 传输层
(C) 会话层 (D) 应用层
2. (真题风格·选择题) 下列选项中, 不属于网络体系结构所描述的内容的是
(A) 网络的层次 (B) 每一层使用的协议
(C) 协议的内部实现细节 (D) 每一层必须完成的功能
3. (真题风格·选择题) TCP/IP 参考模型的网络层(网际层)提供的是
(A) 无连接不可靠的数据报服务 (B) 无连接可靠的数据报服务
(C) 面向连接不可靠的虚电路服务 (D) 面向连接可靠的虚电路服务
4. (真题风格·选择题) 假设 OSI 参考模型的应用层欲发送 400 B 的数据(无拆分)。除物理层和应用层外, 其他各层在封装 PDU 时均引入 20 B 的额外开销, 则应用层的数据传输效率约为
(A) 80% (B) 83%
(C) 87% (D) 91%
5. (计算题) 收发两端之间的传输距离为 1000 km, 信号在媒体上的传播速率为 2×10^8 m/s。数据长度为 10^7 bit, 数据发送速率为 100 kb/s。求发送时延和传播时延。

2.2 真题风格与综合训练

6. (真题风格·选择题) 主机甲通过 1 个路由器(存储转发方式)与主机乙互连, 两段链路的数据传输速率均为 10 Mbps。主机甲分别用报文交换和分组大小为 10 kb 的分组交换向主机乙发送 1 个大小为 8 Mb 的报文。若忽略链路传播时延、分组首部开销和拆装时间, 则两种交换方式完成该报文传输所需的总时间分别为
(A) 800 ms, 1600 ms (B) 801 ms, 1600 ms
(C) 1600 ms, 800 ms (D) 1600 ms, 801 ms
7. (真题风格·综合题) 画出 TCP/IP 四层模型与 OSI 七层模型的对应关系, 并标注各层典型协议和数据单元名称。

拓展阅读:

- Kurose & Ross 《计算机网络：自顶向下方法》第 1 章——生动直观的分组交换 vs 电路交换定量分析。
- 谢希仁 《计算机网络》第 1 章——国内标准术语体系。

3 物理层

3.1 物理层的基本概念

定义 3.1: 物理层

物理层是 OSI 的最底层，为数据链路层提供透明的比特流传输。它规定：

- 机械特性：连接器的形状、尺寸、引脚数和排列；
- 电气特性：信号电平范围、传输速率、传输距离；
- 功能特性：每个引脚信号的物理意义；
- 规程特性：信号的工作时序。

物理层传输的是比特流，不关心比特的含义（帧结构、地址等由上层处理）。

注意

408 辨析：物理层传输的是「比特」(bit)，数据链路层传输的是「帧」(Frame)。集线器 (Hub) 工作在物理层，只转发比特不识别帧结构。交换机 (Switch) 工作在数据链路层，能识别 MAC 地址并转发帧。

3.2 数据通信基础

结论 3.1: 数据通信系统模型

信源 → 发送器 → 信道 → 接收器 → 信宿

- 信源/信宿：产生/接收数据的设备。
- 发送/接收器：编码/解码（调制解调器 Modem 即为典型的发送 + 接收器）。
- 信道：信号的传输介质。分为有线（双绞线、同轴电缆、光纤）和无线（电磁波）。

要点 3.1: 通信的基本术语

- 数据 vs 信号：数据是传送信息的实体，信号是数据的电气/电磁表现。
- 模拟信号 vs 数字信号：连续变化 vs 离散取值。
- 基带传输：直接传输数字信号（不经过调制）。
- 频带传输：将数字信号调制为模拟信号后传输。
- 单工/半双工/全双工：单向/双向交替/双向同时。
- 码元 (Symbol)：用一个固定时长的信号波形表示一个 k 进制数字，代表 k 种不同离散

值。

3.3 奈奎斯特 (Nyquist) 准则与香农 (Shannon) 定理

结论 3.2: 奈奎斯特准则 (无噪声信道)

在理想低通 (无噪声) 信道中, 极限码元传输速率:

$$B_{\max} = 2W \text{ (Baud)}$$

其中 W 为信道带宽 (Hz)。若每个码元携带 $\log_2 V$ 比特 (V 为离散电平数), 则极限数据率:

$$C_{\max} = 2W \log_2 V \text{ (bps)}$$

奈奎斯特准则给出的是无噪声下的符号率上限。

结论 3.3: 香农定理 (有噪声信道)

带宽受限且有高斯白噪声的信道中, 极限数据率:

$$C = W \log_2 \left(1 + \frac{S}{N} \right) \text{ (bps)}$$

其中 S/N 为信噪比 (信号功率与噪声功率之比)。通常使用分贝 (dB) 表示:

$$\text{信噪比 (dB)} = 10 \log_{10} \left(\frac{S}{N} \right)$$

香农定理给出的是信息传输速率的上限, 无论使用何种编码方式都无法超越。

注意

408 综合计算高频: 给定带宽和信噪比, 先由奈奎斯特准则算符号率上限, 再由香农定理算信息速率上限, 取两者的较小值作为题设条件下的速率上限。给定 dB 值时需先换算为 S/N 比值。

3.4 编码与调制

结论 3.4: 数字数据的数字信号编码

常见数字编码

编码方式	规则
不归零编码 (NRZ)	高电平 =1, 低电平 =0; 无同步信号
归零编码 (RZ)	每个比特中间跳变回零
曼彻斯特编码	每个比特中间均有跳变; 跳变方向与 0/1 的对应关系以题目约定为准; 可自同步
差分曼彻斯特	每个比特中间均有跳变; 比特起始处是否跳变表示数据, 具体 0/1 约定以题目为准

曼彻斯特编码: 每个比特的中间都有跳变 (既作时钟又作数据), 码元速率是数据率的 2 倍。经典 10 Mb/s IEEE 802.3 以太网采用曼彻斯特编码; 不能据此推断所有速率的以太网都采用该编码。

差分曼彻斯特: 抗干扰性更强, 常用于令牌环网。

结论 3.5: 数字调制技术

将数字信号调制到模拟载波上传输:

- 调幅 (ASK): 改变振幅表示 0/1。
- 调频 (FSK): 改变频率。
- 调相 (PSK): 改变相位。QPSK 用 4 个相位, 每码元携带 2 bit。
- QAM (正交振幅调制): 同时改变振幅和相位, 每码元携带更多比特。

3.5 传输介质

结论 3.6: 常见传输介质

有线传输介质对比

介质	典型使用范围	特点
双绞线	局域网接入	成本低、布线方便; 性能取决于线缆类别, 抗干扰能力相对较弱
同轴电缆	有线接入、传统总线网	屏蔽性较好, 抗干扰能力通常优于非屏蔽双绞线
光纤 (单模)	长距离、高速骨干	芯径小、模间色散小, 适合长距离高速传输
光纤 (多模)	较短距离园区/机房互连	芯径较大、光源与收发器成本通常较低, 但存在模间色散

常见辨析: 光纤利用全反射原理传输; 单模光纤只传播一个模式、模间色散小, 多模光纤传播多个模式、存在模间色散。具体芯径、距离和速率取决于光纤与收发器标准, 题目若给出参数应以题设为准。

3.6 物理层设备

结论 3.7: 中继器与集线器

- 中继器 (**Repeater**): 对衰减的信号进行放大和整形, 延长网络覆盖距离。仅有两个端口, 工作在物理层。
- 集线器 (**Hub**): 多端口中继器。从一个端口收到的信号放大整形后广播到所有其他端口。所有端口共享同一冲突域。

5-4-3 规则 (以太网): 网络中最多 5 个网段, 其中最多 4 个中继器/集线器, 最多 3 个网段可以连接计算机。

3.7 408 高频考点速查

结论 3.8: 物理层—408 常考点

1. 奈奎斯特 + 香农综合计算: 给定带宽、电平数 V 、信噪比 dB, 求最大数据率。
2. 曼彻斯特编码 vs 差分曼彻斯特: 波形图的手工绘制/识别, 比特率和波特率的关系。
3. 物理层设备: 中继器/集线器不隔离冲突域。
4. 码元与比特的关系: 一个码元携带 $\log_2 V$ 比特。
5. 电路交换/报文交换/分组交换的时延计算 (与第 1 章交叉)。

4 习题

4.1 基础过关题

1. (真题风格·选择题) 在无噪声情况下, 若某通信链路的带宽为 3 kHz, 采用 4 个相位、每个相位有 4 种振幅的 QAM 调制技术, 则该通信链路的最大数据传输速率是

(A) 12 kbps (B) 24 kbps

(C) 48 kbps (D) 96 kbps

2. (真题风格·选择题) 若某通信链路的数据传输速率为 2400 bps, 采用 4 相位调制, 则该链路的波特率是

(A) 600 Baud (B) 1200 Baud

(C) 4800 Baud (D) 9600 Baud

3. (真题风格·选择题) 某 10BaseT 网卡采用曼彻斯特编码, 并约定码元中心由高电平跳变到低电平表示 1、由低电平跳变到高电平表示 0。连续 8 个码元中心的跳变方向依次为: 低到高、低到高、高到低、高到低、低到高、高到低、高到低、低到高。该网卡收到的比特串是

(A) 00110110 (B) 10101101

(C) 01010010 (D) 11000101

4. (真题风格·选择题) 若信道在无噪声情况下的极限数据传输速率不小于信噪比为 30 dB 条件下的极限数据传输速率, 则信号的离散电平数至少是

(A) 4 (B) 8

(C) 16 (D) 32

5. (真题风格·选择题) 按本章约定, 曼彻斯特编码用位中间的高到低跳变表示 1、低到高跳变表示 0。若将 1010 0110 编码, 则第一个比特 (最高位 1) 的波形为

(A) 前半高电平, 后半低电平 (B) 前半低电平, 后半高电平

(C) 整个位周期保持高电平 (D) 整个位周期保持低电平

4.2 真题风格与综合训练

6. (真题风格·综合题) 电话线路的带宽为 3 kHz, 信噪比为 30 dB。

- (1) 根据香农定理, 该信道的最大数据率是多少?
- (2) 若采用 QPSK (4 种相位) 调制, 再根据奈奎斯特准则计算最大数据率;
- (3) 两者为何不同? 哪一个才是该信道的实际极限?

7. (真题风格·综合题) 收发两端相距 1000 km, 信号传播速率为 2×10^8 m/s, 数据发送速率为 1 Gbps。发送方连续发送长度为 1000 B 的帧。

- (1) 若采用停止-等待协议, 信道利用率是多少?
- (2) 若要使信道利用率达到 50%, 发送窗口至少应为多少?

拓展阅读:

- Kurose & Ross 第 1 章——奈奎斯特与香农公式的直观解释和推导。
- 谢希仁第 2 章——物理层传输介质和编码技术。

5 数据链路层

5.1 数据链路层的功能

定义 5.1: 数据链路层

数据链路层在物理层提供的比特流基础上, 为网络层提供帧 (Frame) 的透明传输。主要功能:

1. 成帧 (**Framing**): 将比特流划分为帧。
2. 差错控制: 检测并 (可选地) 纠正传输错误。
3. 流量控制: 协调发送方和接收方的速率 (速率匹配)。
4. 介质访问控制 (**MAC**): 广播网络中决定谁可以使用共享信道。

5.2 组帧方法

结论 5.1: 四种组帧方法

- 字符计数法: 帧首部包含帧中字符数。缺点: 计数字段一旦出错, 后续全部帧错位。
- 字符填充法 (字节填充): 用特殊字符标记帧边界, 数据中出现的边界字符用转义字符 ESC 替换。
- 零比特填充法 (**Bit Stuffing**): 帧边界标记为 01111110 (6 个连续 1)。发送方每 5 个连续的 1 后插入 0, 接收方每 5 个连续的 1 后删除 0。HDLC 和同步 PPP 使用; 异步 PPP 通常使用字节填充/转义机制。
- 违规编码法: 利用物理层的冗余编码 (如曼彻斯特编码中非法跳变)。仅适用于物理层编码特殊的网络。

5.3 差错控制

要点 5.1: CRC 循环冗余校验

原理：将 k 位数据后拼接 r 位校验位，构成 $n = k + r$ 位码字，使其能被生成多项式 $G(x)$ 整除。接收方用同一 $G(x)$ 除码字，余数非零表示出错。

408 计算方法：给定数据比特串 M 和生成多项式 $G(x)$ （如 $G(x) = x^3 + x + 1$ ，即 1011）：

1. M 后补 r 个 0（ r 是 $G(x)$ 的次数），得 $2^r M$ 。
2. $2^r M$ 除以 $G(x)$ （模 2 除法，即异或运算），得 r 位余数 R 。
3. 发送码字 = $M \parallel R$ （ M 后拼接 R ）。

注意

408 CRC 计算题的几个注意点：

- CRC 是检错码（非纠错码）。当生成多项式至少含两个非零项时可检测所有单比特错误；对突发错误的检测能力取决于生成多项式次数和具体因子，不能只用“CRC”二字概括成无条件保证。
- 模 2 除法 = 异或（不进位、不借位）。
- r 次生成多项式产生的 CRC 校验位为 r 位。

5.4 可靠传输与滑动窗口协议

结论 5.2: 停止-等待、GBN 与 SR

三种可靠传输协议对比

协议	发送窗口	接收窗口	超时后的处理
停止-等待	1	1	重发当前未确认帧；用 1 位序号区分新帧与重复帧
GBN（后退 N 帧）	$1 \leq W_T \leq 2^m - 1$	1	从最早未确认帧开始重发其后所有已发送帧
SR（选择重传）	$W_T \leq 2^{m-1}$	$W_R \leq 2^{m-1}$	只重发出错或超时的帧

其中 m 为帧序号位数。GBN 使用累计确认，接收方只按序接收；SR 可缓存失序帧并分别确认。SR 要求窗口不超过序号空间的一半，是为了避免序号回绕后无法区分旧帧和新帧。

要点 5.2: 停止-等待协议的信道利用率

忽略确认帧发送时延和处理时延，设数据帧发送时延为 $T_f = L/R$ 、单程传播时延为 τ ，则

$$U = \frac{T_f}{T_f + 2\tau} = \frac{1}{1 + 2a}, \quad a = \frac{\tau}{T_f}.$$

滑动窗口允许多个帧在途；当 $W_T T_f \geq T_f + 2\tau$ 时，发送方可持续发送，理想利用率可达到 1。

5.5 介质访问控制 (MAC) 子层

结论 5.3: 信道划分协议

- 频分复用 (FDM): 各用户占用不同频段。
- 时分复用 (TDM): 各用户分配不同时间隙。统计 TDM (STDM) 按需分配, 效率更高。
- 波分复用 (WDM): 光的频分复用。
- 码分复用 (CDM/CDMA): 每用户分配唯一码片序列, 各用户同时同频传输。接收方用码片序列内积提取数据。

结论 5.4: 随机接入协议

- ALOHA: 纯 ALOHA 可随时发送, 时隙 ALOHA 只能在时隙边界发送; 二者发生冲突后均随机退避, 时隙化可减小易冲突时间。
- CSMA: 发送前监听载波; 根据“忙时如何处理”可分 1-坚持、非坚持和 p -坚持 CSMA。载波监听不能消除冲突, 因为存在传播时延。
- CSMA/CD: 适用于共享式有线以太网, 发送时检测冲突。
- CSMA/CA: 用于 IEEE 802.11 无线局域网, 通过确认、帧间间隔和随机退避避免冲突, 可选 RTS/CTS 缓解隐藏站问题; 无线站难以边发送边可靠检测冲突, 因此不使用 CSMA/CD。

5.6 CSMA/CD 协议 (以太网核心)

结论 5.5: CSMA/CD 协议

CSMA/CD (Carrier Sense Multiple Access with Collision Detection) 是以太网的核心协议。

工作流程 (四句话):

1. 先听后发: 发送前先监听信道, 信道空闲则发送。
2. 边发边听: 发送过程中持续监听。
3. 冲突停发: 检测到冲突立即停止发送。
4. 随机重发: 等待随机时间后重新尝试 (二进制指数退避算法)。

争用期 (Contention Period): 2τ (τ 为端到端传播时延)。在争用期内若未检测到冲突, 则后续不会冲突。

最短帧长: $L_{\min} = 2\tau \times \text{数据率}$ 。若帧过短 (小于 $L_{\min}$), 最坏情况下冲突信号返回发送端时帧已经发完, 发送端就无法保证在发送期间检测到该冲突。

二进制指数退避: 第 k 次重传 ($k \leq 10$) 的等待时间为从 $[0, 2^k - 1]$ 中随机选一个 r , 等待 $r \times 2\tau$ 。 $k > 10$ 后 r 范围锁定为 $[0, 2^{10} - 1]$ 。重传 16 次仍失败则丢弃帧。

注意

408 CSMA/CD 高频计算:

1. 给定数据率和最大距离, 求最短帧长。
2. 给定最短帧长和数据率, 求最大传输距离。
3. 二进制指数退避的等待时间和 k 值的对应关系。

5.7 以太网与交换机

结论 5.6: 以太网帧格式

前导码 (8B)	目的 MAC (6B)	源 MAC (6B)	类型 (2B)	数据 (46-1500B)	填充	CRC (4B)
-------------	----------------	---------------	------------	------------------	----	-------------

以太网 MAC 帧的最短长度为 64 B，即 $6 + 6 + 2 + 46 + 4 = 64$ B（从目的 MAC 到 FCS，不含 7 B 前导码和 1 B 帧开始定界符）。加上前导码和定界符后，链路上发送 72 B；帧间间隔也不计入帧长。

5.8 无线局域网与 VLAN

结论 5.7: IEEE 802.11 无线局域网

基础设施模式中，主机通过接入点 AP 接入分配系统；同一 AP 覆盖的基本服务集称为 BSS。802.11 使用 CSMA/CA，并以链路层 ACK 确认单播数据帧。隐藏站问题是指两个站彼此听不到，却同时向同一接收站发送而产生冲突；RTS/CTS 通过预约信道降低该问题的影响。

结论 5.8: 虚拟局域网 VLAN

VLAN 在二层交换网络中按逻辑划分广播域。不同 VLAN 的主机即使连接同一台交换机也不能直接进行二层通信，跨 VLAN 通信必须经过三层设备。

- **Access** 端口：通常只属于一个 VLAN，连接终端，收发普通以太网帧。
- **Trunk** 端口：在交换机之间承载多个 VLAN，通常使用 IEEE 802.1Q 标签标识 VLAN。

VLAN 能隔离广播域，但不等同于物理隔离，也不能替代路由和安全策略。

结论 5.9: 交换机与网桥

- **网桥 (Bridge)**：工作在数据链路层，根据 MAC 地址转发帧。可隔离冲突域，但无法隔离广播域。
- **交换机 (Switch)**：多端口网桥。每个端口是一个独立的冲突域。
- **自学习算法**：交换机通过源 MAC 地址学习端口-地址映射，构建交换表（MAC 地址表）。未知目的地址的帧泛洪（flood）到除接收端口外的所有端口。

冲突域 vs 广播域

设备	隔离冲突域?	隔离广播域?
中继器/集线器（物理层）	否	否
网桥/交换机（数据链路层）	是	否
路由器（网络层）	是	是

5.9 ARP 协议

定义 5.2: ARP

ARP (Address Resolution Protocol) 将 IP 地址映射为 MAC 地址。工作过程: 主机广播 ARP 请求 (「谁的 IP 是 xxx? 请告诉你的 MAC」), 目标主机单播 ARP 响应。结果缓存于 ARP 表 (有老化时间)。

5.10 PPP 协议

结论 5.10: PPP

点对点协议 (Point-to-Point Protocol) 是目前最广泛使用的数据链路层协议 (如电话线拨号、PPPoE)。特点:

- 不提供可靠传输 (不编号、不确认), 差错检测仅用 CRC。
- 面向字节, 使用字节填充 (转义字符 $0x7D$)。
- 支持多种网络层协议 (LCP + NCP)。

5.11 408 高频考点速查

结论 5.11: 数据链路层—408 常考点

1. **CSMA/CD**: 最短帧长计算、争用期 2τ 的概念、二进制指数退避。
2. 交换机自学习: 给定帧转发序列和初始空交换表, 写出交换表项的变化过程。
3. **CRC** 校验: 给定生成多项式, 计算 CRC 校验码 (模 2 除法)。
4. 滑动窗口协议: GBN/SR 的窗口上限、确认方式、超时重传范围和序号回绕。
5. 无线与 **VLAN**: CSMA/CD vs CSMA/CA、隐藏站、802.1Q、广播域划分。
6. 冲突域 vs 广播域: 各设备隔离哪个域。
7. **ARP**: 工作原理、是否跨网络 (ARP 仅在同一广播域内有效)。

6 习题

6.1 基础过关题

1. (真题风格·选择题) 数据链路层采用后退 N 帧 (GBN) 协议, 发送方已经发送了编号为 0 7 的帧。当计时器超时, 若发送方只收到 0、2、3 号帧的确认, 则发送方需要重发的帧数是

(A) 2 (B) 3

(C) 4 (D) 5

2. (真题风格·选择题) 某局域网采用 CSMA/CD 协议, 数据传输速率为 10 Mbps, 信号传播速率为 2×10^8 m/s, 主机甲和主机乙相距 2 km。若主机甲和主机乙发送数据时发生冲突, 则从开始发送数据到检测到冲突的最短时间和最长时间分别是

(A) 0, 20 μ s (B) 0, 10 μ s

(C) 10 μ s, 20 μ s (D) 10 μ s, 10 μ s

3. (真题风格·选择题) 对于 100 Mbps 的以太网交换机, 当输出端口无排队, 以直通交换方式转发一个以太网帧时, 引入的转发时延至少是

(A) 0 μ s (B) 0.48 μ s

(C) 5.12 μ s (D) 121.44 μ s

4. (真题风格·选择题) 下列介质访问控制方法中, 可能发生冲突的是

(A) CDMA (B) CSMA/CD

(C) TDMA (D) FDMA

5. (真题风格·选择题) 数据 $M = 101001$, 生成多项式 $G(x) = x^3 + x^2 + 1$ (即 1101), 则 CRC 校验码为

(A) 001 (B) 101

(C) 011 (D) 110

6. (基础题) 帧序号使用 3 位二进制数。GBN 协议发送窗口的最大值和 SR 协议发送窗口的最大值分别为

(A) 7 和 4 (B) 8 和 4

(C) 4 和 7 (D) 7 和 8

7. (基础题) IEEE 802.11 无线局域网使用 CSMA/CA 而不使用 CSMA/CD 的主要原因是

(A) 无线网络没有传播时延 (B) 无线站难以边发送边检测冲突

(C) 无线帧不需要确认 (D) 无线网络不存在隐藏站

8. (基础题) 同一台二层交换机上属于不同 VLAN 的两台主机要互相通信, 至少还需要

- (A) 中继器 (B) 集线器
(C) 三层转发设备 (D) 仅修改 MAC 地址表

6.2 真题风格与综合训练

9. (真题风格·选择题) 交换机通过自学习算法建立交换表。某交换机初始交换表为空, 先后收到发往不同目的 MAC 地址的帧。以下关于自学习的说法, 正确的是

- (A) 交换机通过目的 MAC 地址学习 (B) 交换机通过源 MAC 地址学习
(C) 交换机只在收到广播帧时学习 (D) 交换表项永不过期

10. (真题风格·综合题) 某局域网通过交换机连接 4 台主机 A、B、C、D, 分别连接端口 1-4。交换表初始为空。按顺序: $A \rightarrow B$, $B \rightarrow A$, $C \rightarrow A$, $D \rightarrow C$ 。描述每步后交换表变化及帧的转发方式(明确转发、泛洪或过滤)。

拓展阅读:

- Kurose & Ross 第 6 章——CSMA/CD、交换机自学习的完整讨论。
- 谢希仁第 3 章——PPP 协议和 CSMA/CD 的国内标准表述。

7 网络层

网络层负责将分组从源主机经中间路由器送达目的主机。CIDR 子网划分、IP 分片偏移计算和路由协议对比是 408 计算机网络中的高频内容。

7.1 网络层提供的两种服务

网络层向上面的传输层提供怎样的服务, 存在两种对立的观点:

定义 7.1: 虚电路与数据报**虚电路服务 vs 数据报服务**

对比维度	虚电路 (VC) 服务	数据报 (Datagram) 服务
核心思路	面向连接, 可靠交付	无连接, 尽最大努力交付
连接的建立	必须先建立连接	不需要
终点地址	仅建立连接时使用	每个分组携带完整目的地址
分组转发依据	虚电路号 (VCI)	目的 IP 地址
分组顺序	按序到达	可能乱序
路由选择	建立连接时选一次	每个分组独立选路
结点故障影响	经过该结点的 VC 全失效	仅丢失故障结点的分组
代表网络	ATM、帧中继、X.25	Internet (IP)

注意

408 核心辨析: 因特网 (Internet) 采用数据报服务, 网络层只提供尽最大努力交付 (Best-Effort Delivery), 不保证可靠。可靠传输由传输层 (TCP) 在端系统中实现。这就是「网络层简单化, 端系统智能化」的端到端原则 (End-to-End Principle) 的体现。

408 常考判断题: 「网络层提供可靠的端到端通信」——错误, 可靠通信由传输层 TCP 保证。

7.2 IPv4 地址**IP 地址的分类****定义 7.2: IPv4 地址**

IPv4 地址为 32 位二进制数, 通常用点分十进制 (Dotted Decimal) 表示, 如 192.168.1.1。每个 IP 地址由网络号 (**net-id**) 和主机号 (**host-id**) 两部分组成。

结论 7.1: 分类 IP 地址 (Classful Addressing)**IP 地址分类**

类别	网络号位数	首字节范围	网络号范围	最大网络数	每网最大主机数
A 类	/8 (首位固定为 0)	1-126	1.0.0.0-126.0.0.0	$2^7 - 2 = 126$	$2^{24} - 2 \approx 1677$ 万
B 类	/16 (前 2 位固定为 10)	128-191	128.0.0.0-191.255.0.0	$2^{14} = 16384$	$2^{16} - 2 = 65534$
C 类	/24 (前 3 位固定为 110)	192-223	192.0.0.0-223.255.255.0	$2^{21} \approx 209$ 万	$2^8 - 2 = 254$
D 类	—	224-239	组播地址	—	—
E 类	—	240-255	保留	—	—

特殊地址:

- 全 0 (0.0.0.0): 本网络上的本主机 (DHCP 初始状态)。
- 网络号全 0: 本网络上的特定主机。
- 主机号全 0: 网络地址 (标识一个网络)。

- 主机号全 1: 该网络的直接广播地址。
- 全 1 (255.255.255.255): 受限广播 (本网络广播, 不进路由器)。
- 127.x.x.x: 环回地址 (Loopback), 典型的为 127.0.0.1。
- 10.x.x.x, 172.16-31.x.x, 192.168.x.x: 私有地址 (RFC 1918)。

注意

减 2 的原因: 主机号全 0 为网络地址, 主机号全 1 为广播地址, 均不能分配给具体主机, 因此可用主机数 = $2^n - 2$ 。408 选择题经常是在是否需要「减 2」上设置陷阱。

子网划分与子网掩码

定义 7.3: 子网划分

在分类 IP 地址的基础上, 从主机号中借用若干位作为子网号 (subnet-id), 形成三级编址结构:

IP 地址 ::= {〈网络号〉, 〈子网号〉, 〈主机号〉}

这是对分类 IP 地址的改进, 能更有效地利用 IP 地址空间。

要点 7.1: 子网掩码

子网掩码 (Subnet Mask) 用于区分 IP 地址中的网络部分和主机部分:

- 网络号位 + 子网号位 → 掩码中对应位为 1。
- 主机号位 → 掩码中对应位为 0。

网络地址 = IP 地址 & 子网掩码 (按位 AND)

子网掩码表示方式:

- 点分十进制: 255.255.255.0 (等价于 /24)。
- CIDR 前缀长度: 「/n」表示前 n 位为网络前缀。

注意

408 高频计算: 给定一个 IP 地址和子网掩码, 要求计算: (1) 网络地址; (2) 广播地址; (3) 该子网可容纳的主机数; (4) 子网中第一个和最后一个可用的 IP 地址。

口诀: 网络地址 = IP & 掩码 (主机位全 0); 广播地址 = 网络地址 + 主机位全 1; 可用 IP 范围 = (网络地址 + 1) 到 (广播地址 - 1)。

CIDR 与路由聚合

定义 7.4: 无分类编址 CIDR

CIDR (Classless Inter-Domain Routing, 无分类域间路由) 彻底消除了传统的 A/B/C 类地址和子网划分的概念, 使用网络前缀 (Network Prefix) 代替网络号:

IP 地址 ::= {〈网络前缀〉, 〈主机号〉}

记作 IP 地址/前缀长度，如 192.168.1.0/24。

结论 7.2: CIDR 路由聚合 (Route Aggregation)

CIDR 的核心优势是路由聚合（构成超网）：将多个连续的子网合并为一个更大的地址块，从而压缩路由表。

聚合方法：将多个网络地址按位对齐，找出最长的共同前缀。

例如：192.168.0.0/24, 192.168.1.0/24, 192.168.2.0/24, 192.168.3.0/24

四个网络地址的前 22 位相同（192.168 的前两个字节 + 第三字节的前 6 位），可聚合为 192.168.0.0/22。

最长前缀匹配示例

路由表项	下一跳
192.168.0.0/22	R1（聚合路由）
192.168.2.0/24	R2（更具体的路由）

当目的地址 192.168.2.5 同时匹配两条路由时，路由器选择前缀最长的（/24 > /22）即 R2，这称为最长前缀匹配（Longest Prefix Match）。

注意

408 高频：CIDR 地址块的计算。给定 /n 前缀，则：

- 地址块大小 = 2^{32-n} 个地址。
- 对传统含网络地址和广播地址的 IPv4 子网（通常 $n \leq 30$ ），可分配给主机的地址数 = $2^{32-n} - 2$ 。点到点链路上的 /31 和单主机路由 /32 是特殊情形，不能机械地「减 2」。
- 地址掩码：前 n 位为 1，后 $32 - n$ 位为 0。
- 最小地址 = 网络地址（主机位全 0），最大地址 = 广播地址（主机位全 1）。

路由聚合（构成超网）是 CIDR 的核心考点，要求从若干连续子网中提取最长公共前缀。

7.3 NAT 与端口地址转换

定义 7.5: 网络地址转换 NAT

网络地址转换（Network Address Translation, NAT）通常部署在私有网络的边界设备上。当内部主机访问外网时，NAT 设备改写 IP 数据报中的源地址，并维护「内部地址—外部地址」映射；返回分组再按映射改写目的地址。常用私有 IPv4 地址块为

10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16.

私有地址不会在公共 Internet 中被全局路由。

结论 7.3: NAT、NAPT/PAT 与静态映射

- **基本 NAT**: 主要改写 IP 地址, 可用一个或一组公网地址映射内部地址。
- **NAPT/PAT**: 同时利用传输层端口号区分连接, 使多个内部主机能够共享同一个公网 IP。映射项通常包含协议、内部 IP/端口和外部 IP/端口。
- **入站连接限制**: 若不存在已有动态映射, 外部主机不能只凭某个内部私有地址主动建立连接; 需要预先配置静态 NAT、端口转发等规则。
- **校验和**: NAT 改写 IPv4 地址后要更新 IP 首部校验和; 若还改写 TCP/UDP 端口或参与伪首部计算的地址, 也要更新 TCP/UDP 校验和。

注意

易错点: NAT 解决的是地址转换和公网地址复用问题, 不等同于路由选择, 也不是端到端可靠传输机制。PAT 依靠端口号区分并发连接, 因此不能只看公网 IP 就判断内部通信端点。

7.4 ARP、DHCP 与 ICMP

ARP (地址解析协议)

定义 7.6: ARP

ARP (Address Resolution Protocol) 将 IP 地址映射为 MAC 地址, 工作在网络层和数据链路层之间。每台主机维护一个 **ARP** 高速缓存 (ARP Cache), 存储最近解析的 IP-MAC 映射。

结论 7.4: ARP 工作过程

1. 主机 A 要发送数据给同一局域网内的主机 B (已知 B 的 IP, 未知 B 的 MAC)。
2. A 在局域网内广播 ARP 请求分组: 「谁有 IP 地址 xxx? 请告诉我你的 MAC 地址。」
3. 主机 B 收到后, 向 A 单播 ARP 响应分组: 「我的 IP 是 xxx, MAC 地址是 yyy。」
4. A 将映射关系存入 ARP 缓存, 然后用获得到的 MAC 地址封装帧并发送数据。

ARP 的四种典型场景:

- 发送方是主机, 目的主机在同一网络 → ARP 解析目的主机的 MAC。
- 发送方是主机, 目的主机在另一网络 → ARP 解析默认网关 (路由器) 的 MAC。
- 发送方是路由器, 转发到目的主机在同一网络 → ARP 解析目的主机的 MAC。
- 发送方是路由器, 转发到下一跳路由器 → ARP 解析下一跳路由器的 MAC。

注意

408 辨析: ARP 解决的是同一局域网内的 IP→MAC 映射问题。跨网段通信时, ARP 解析的是默认网关的 **MAC** 地址, 而非目的主机的 MAC 地址。ARP 请求是广播帧 (目的 MAC 为 FF-FF-FF-FF-FF-FF), ARP 响应是单播帧。

DHCP（动态主机配置协议）

定义 7.7: DHCP

DHCP (Dynamic Host Configuration Protocol) 允许主机自动获取 IP 地址、子网掩码、默认网关和 DNS 服务器地址。基于 UDP，服务器端口 67，客户端端口 68。DHCP 工作在应用层（使用 UDP），但其功能服务于网络层的 IP 地址配置。

结论 7.5: DHCP 四步交互（DORA）

DHCP 交互过程

步骤	报文名称	方向	关键信息
1	DHCP DISCOVER	客户端 → 服务器（广播）	客户端寻找 DHCP 服务器
2	DHCP OFFER	服务器 → 客户端	服务器提供 IP 地址
3	DHCP REQUEST	客户端 → 服务器（广播）	客户端请求使用该 IP
4	DHCP ACK	服务器 → 客户端	服务器确认，租约生效

408 常考： (1) DHCP DISCOVER 和 DHCP REQUEST 都是广播（客户端此时尚无 IP 或尚未确认使用某 IP）；(2) DHCP 租约（Lease）有期限，到期前需续约；(3) DHCP 分配的不仅是 IP 地址，还有子网掩码、默认网关等。

ICMP（互联网控制报文协议）

定义 7.8: ICMP

ICMP (Internet Control Message Protocol) 用于在 IP 主机和路由器之间传递差错报告和控制信息，是 IP 层的配套协议。ICMP 报文封装在 IP 数据报中传输（协议号 = 1）。

结论 7.6: ICMP 报文类型与典型应用

常见 ICMP 报文类型

ICMP 报文类型	类型值	说明
回送请求 (Echo Request)	8	Ping 发送的报文
回送回答 (Echo Reply)	0	Ping 收到的响应
终点不可达 (Dest. Unreachable)	3	网络/主机/协议/端口不可达
源点抑制 (Source Quench)	4	拥塞控制（已废弃）
重定向 (Redirect)	5	通知主机使用更好的路由
超时 (Time Exceeded)	11	TTL 减到 0，Traceroute 使用

Ping 的原理：

- 源主机向目的主机发送 ICMP 回送请求 (Echo Request, 类型 8)。

- 目的主机收到后返回 ICMP 回送回答 (Echo Reply, 类型 0)。
- 通过 RTT (往返时间) 和丢包率判断网络连通性。
- Ping 工作在网络层, 不涉及传输层端口。

Traceroute (Tracert) 的原理:

- 源主机依次发送 TTL = 1, 2, 3, ... 的 UDP 数据报 (或 ICMP 回送请求)。
- 第 i 跳路由器收到 TTL = 0 的数据报后, 丢弃并向源主机返回 **ICMP 超时** (类型 11) 报文。
- 源主机从超时报文的源 IP 地址获知第 i 跳路由器的 IP。
- 最终到达目的主机时, 目的主机返回 ICMP 终点不可达 (端口不可达) 报文 (因为使用了不可达的 UDP 端口), Traceroute 停止。

注意

408 辨析:

- Ping 使用 ICMP, 不经过传输层; 它测试的是网络层的连通性。
- Traceroute 利用 TTL 字段逐跳发现路径, TTL 减到 0 时路由器返回 ICMP 超时报文。
- ICMP 终点不可达报文还可携带差错代码区分「网络不可达」「主机不可达」「协议不可达」「端口不可达」等。

7.5 IP 数据报格式

定义 7.9: IP 数据报

IP 数据报由首部 (Header) 和数据 (Data/Payload) 两部分组成。IPv4 首部固定部分为 20 字节 (不含选项字段), 最大长度为 60 字节 (含选项)。

结论 7.7: IPv4 首部字段详解

IPv4 首部格式 (固定部分 20 字节)

字段	位数	说明
版本 (Version)	4	协议版本号, IPv4 = 4
首部长度 (IHL)	4	以 4 字节为单位, 最小值 5 (20 字节), 最大值 15 (60 字节)
区分服务 (DS)	8	DSCP 用于区分服务/QoS, ECN 可用于显式拥塞通知
总长度 (Total Length)	16	首部 + 数据的总字节数, 最大 65535 字节
标识 (Identification)	16	标识同一数据报的所有分片
标志 (Flags)	3	DF (不分片)、MF (还有分片)
片偏移 (Fragment Offset)	13	本分片在原数据报中的相对位置, 以 8 字节为单位
生存时间 (TTL)	8	经过的最大跳数, 每经一个路由器减 1, 减到 0 则丢弃
协议 (Protocol)	8	上层协议: TCP=6, UDP=17, ICMP=1, OSPF=89
首部校验和 (Header Checksum)	16	仅校验首部, 每跳重新计算 (TTL 变化)
源 IP 地址	32	发送方 IP 地址
目的 IP 地址	32	接收方 IP 地址

关键字段说明:

- 总长度: 最大 65535 字节, 但实际受数据链路层 MTU 限制 (以太网为 1500 字节)。
- **TTL**: 防止数据报在网络中无限循环。每经过一个路由器至少减 1, 减到 0 时丢弃并通常返回 ICMP 超时报文; 初始值由发送端实现或配置决定。
- 协议: 标识上层协议, 使目的主机的 IP 层知道将数据部分交给哪个上层协议处理 (类似于传输层端口号的复用/分用功能)。
- 首部校验和: 每经过一个路由器都需重新计算 (因为 TTL 字段改变了)。采用反码求和 (Internet Checksum), 只校验首部不校验数据。

注意

408 常考细节:

- 首部长度字段以 4 字节为单位: 值为 5 表示 20 字节, 值为 15 表示 60 字节。
- 总长度以 1 字节为单位, 最大 65535。
- 片偏移以 8 字节为单位 (乘以 8 得到实际字节偏移量)。
- 首部校验和每跳重新计算的原因: TTL 字段每跳变化。
- 区分 IP 数据报上层协议: IPv4 用协议字段 (Protocol), IPv6 用「下一个首部」(Next Header)。

7.6 IP 分片与重组

定义 7.10: IP 分片

对 IPv4 而言, 当数据报长度超过下一跳链路的 MTU 时: 若 DF=0, 源主机或中间路由器可将其分割为若干更小的分片 (Fragment); 若 DF=1, 路由器不能分片, 应丢弃该数据报并通常返回 ICMP “需要分片” 差错。IPv4 分片只在目的主机重组。IPv6 中间路由器不执行分片, 必要时由源结点依据路径 MTU 使用分片扩展首部。

结论 7.8: 分片相关字段

三个字段协同完成分片与重组:

- 标识 (**Identification, 16 位**): 同一数据报的所有分片具有相同的标识值。目的主机根据标识将分片归组。
- 标志 (**Flags, 3 位**):
 - 第 0 位: 保留, 必须为 0。
 - 第 1 位: **DF (Don't Fragment)** ——1 表示禁止分片。若数据报超过 MTU 且 DF=1, 路由器丢弃该数据报并返回 ICMP 差错报文。
 - 第 2 位: **MF (More Fragments)** ——1 表示后面还有分片, 0 表示这是最后一个分片。
- 片偏移 (**Fragment Offset, 13 位**): 本分片的数据部分在原数据报数据部分中的相对位置, 以 8 字节为单位。即: 实际字节偏移量 = 片偏移值 $\times$ 8。

要点 7.2: 分片计算的完整步骤

设原数据报数据部分长度为 L 字节, MTU = M 字节, IP 首部固定 20 字节。

Step 1: 每个分片可承载的最大数据量 = $M - 20$ (取 8 的整数倍, 记为 $D_{\max}$)。

$$D_{\max} = \left\lfloor \frac{M - 20}{8} \right\rfloor \times 8$$

Step 2: 计算分片数量 n :

$$n = \left\lceil \frac{L}{D_{\max}} \right\rceil$$

Step 3: 计算每个分片的:

- 数据长度 (前 $n - 1$ 个分片均为 $D_{\max}$, 最后一个为剩余)。
- 总长度 = 数据长度 + 20 (首部)。
- 片偏移 = 前面所有分片数据长度之和 / 8。
- MF 标志: 除最后一个分片 MF = 0 外, 其余 MF = 1 (DF 均为 0)。

注意

408 高频——分片计算的核心陷阱:

- 片偏移以 8 字节为单位, 而非以字节为单位! 片偏移值 $\times$ 8 = 实际字节偏移。
- 除最后一个分片外, 每个分片的数据部分长度必须是 8 的整数倍 (因为片偏移以 8 字节

为单位)。

- 每个分片都携带完整的 IP 首部 (20 字节), 所以总长度 = 20 + 数据部分长度。
- 分片在源主机或路由器发生, 重组仅在目的主机进行。中间路由器不重组分片。
- 若某个分片丢失, 目的主机丢弃所有已收到的该数据报的分片 (不完整的重组)。

例 一个数据报数据部分长度为 3800 字节, 经过 MTU = 1420 字节的链路 (IP 首部固定 20 字节)。求分片情况。

解:

$$D_{\max} = \left\lfloor \frac{1420 - 20}{8} \right\rfloor \times 8 = \left\lfloor \frac{1400}{8} \right\rfloor \times 8 = 175 \times 8 = 1400$$

分片数 $n = \lceil 3800/1400 \rceil = \lceil 2.714 \rceil = 3$ 。

分片结果

分片	数据长度	总长度	片偏移值	MF
1	1400	1420	0/8 = 0	1
2	1400	1420	1400/8 = 175	1
3	1000	1020	2800/8 = 350	0

7.7 路由算法与路由协议

定义 7.11: 路由与路由表

路由 (Routing) 是指为数据报选择从源到目的地的路径。路由器根据路由表 (Routing Table) 进行转发决策。路由表至少包含两项信息: 目的网络地址和下一跳地址。(也可能包含子网掩码、接口、度量值等。)

路由协议分为两大类:

- 内部网关协议 **IGP** (Interior Gateway Protocol): 在同一个自治系统 (AS) 内部运行。如 RIP、OSPF。
- 外部网关协议 **EGP** (Exterior Gateway Protocol): 在不同 AS 之间运行。如 BGP-4。

自治系统 (AS, Autonomous System): 由同一机构管理、对外呈现统一路由策略的一组网络。参与公网 BGP 路由的 AS 使用分配的公有 ASN; 此外也存在仅供组织内部使用、不能直接在公网传播的私有 ASN, 不能笼统地说所有 ASN 都全球唯一。

RIP (路由信息协议)

定义 7.12: RIP

RIP (Routing Information Protocol) 是基于距离向量 (Distance Vector) 算法的 IGP。使用跳数 (Hop Count) 作为度量标准, 最大跳数为 15 (跳数 = 16 表示不可达)。RIP 适用于小型网络。

结论 7.9: RIP 的工作原理

距离向量算法 (**Bellman-Ford** 分布式版本):

1. 每个路由器周期性地 (默认 30 秒) 向所有邻居发送自己的完整路由表。
2. 收到邻居的路由表后, 路由器检查每一条路由:
 - 若目的网络不在自己路由表中 $\rightarrow$ 添加该路由, 度量值 = 邻居的度量值 + 1。
 - 若目的网络已存在, 且下一跳是同一邻居 $\rightarrow$ 更新度量值。
 - 若目的网络已存在, 下一跳不同, 且邻居提供的度量值 + 1 < 当前度量值 $\rightarrow$ 更新路由。
 - 否则 $\rightarrow$ 保持不变。
3. 若 180 秒未收到某邻居的路由更新, 则标记该邻居为不可达。

RIP 的核心问题:

- 慢收敛 (**Slow Convergence**): 好消息传得快, 坏消息传得慢 (计数到无穷问题)。
- 计数到无穷 (**Count-to-Infinity**): 链路故障后, 路由器不断将度量值递增, 直到 16 (无穷大)。
- 解决方法: 水平分割 (Split Horizon)、毒性逆转 (Poison Reverse)、触发更新 (Triggered Update)。

注意

408 考点: RIP 路由表更新。 给定初始路由表和收到的邻居路由表, 要求用距离向量算法更新路由表。核心规则:

- 对邻居发来的每条路由, 将其跳数 + 1。
- 若目的网络不在本地路由表中, 直接加入。
- 若目的网络已存在且下一跳相同, 无条件更新。
- 若目的网络已存在但下一跳不同, 比较跳数, 取更小的。
- RIP 基于 UDP (端口 520), 是应用层协议但核心功能是为网络层选路。

OSPF (开放最短路径优先)

定义 7.13: OSPF

OSPF (Open Shortest Path First) 是基于链路状态 (Link State) 算法的 IGP。使用 Dijkstra 最短路径算法, 度量值为链路开销 (Cost), 通常与带宽成反比 ($\text{Cost} = \text{参考带宽} / \text{链路带宽}$)。OSPF 适用于大型网络。

结论 7.10: OSPF 的工作原理

链路状态算法的五个步骤:

1. 发现邻居: 每个路由器通过发送 Hello 报文发现相邻路由器。
2. 洪泛 **LSA**: 每个路由器将自己与邻居的链路状态封装为 LSA (Link State Advertisement), 洪泛到整个区域。
3. 构建 **LSDB**: 每个路由器收集本区域内洪泛的 LSA, 构建链路状态数据库 (LSDB) ——

本区域的完整拓扑图；跨区域信息由 ABR 按区域间路由传播。

- 4. **Dijkstra** 计算：以自己为根，执行 Dijkstra 最短路径算法，计算出到达每个网络的最短路径。
- 5. 生成路由表：根据计算结果生成路由表。

OSPF 的关键特性：

- 直接基于 IP（协议号 89），而非 TCP 或 UDP。
- 支持区域划分（Area），区域 0 为骨干区域（Backbone Area）。
- 支持等价多路径（ECMP，Equal-Cost Multi-Path）负载均衡。
- 链路状态变化时才洪泛更新（而非周期性发送完整路由表）。
- 支持认证（明文和 MD5）。

RIP 与 OSPF 的对比

结论 7.11: RIP vs OSPF（408 高频对比）

RIP 与 OSPF 对比		
对比维度	RIP	OSPF
算法类型	距离向量（DV）	链路状态（LS）
核心算法	Bellman-Ford（分布式）	Dijkstra（集中式计算）
度量值	跳数	链路开销（带宽决定）
传递的信息	完整路由表（给邻居）	LSA（洪泛至整个区域）
更新方式	周期性（30 s）	触发式（变化时更新）
网络规模	小型（≤ 15 跳）	大型（支持区域分层）
收敛速度	慢（计数到无穷问题）	快（区域内 LSDB 一致）
封装协议	UDP（端口 520）	IP（协议号 89）
最大度量值	15 跳（16 = ∞）	无硬性上限

注意

408 辨析：

- RIP 的路由器只知道邻居和到各网络的距离（跳数），不知道全网拓扑。
- OSPF 的区域内路由器拥有本区域的完整拓扑（LSDB），独立计算最短路径树；多区域网络中不能简单说每台路由器都拥有整个 AS 的详细拓扑。
- RIP 传递路由表；OSPF 传递链路状态信息（LSA）。
- 「RIP 是应用层协议，OSPF 是网络层协议」——408 常见判断题：RIP 基于 UDP 在应用层实现，但核心功能是路由选择；OSPF 直接封装在 IP 数据报中。

BGP（边界网关协议）

定义 7.14: BGP

BGP（Border Gateway Protocol）是 AS 之间的路由协议，目前版本为 BGP-4。BGP 采用路径向量（Path Vector）算法，基于策略而非纯技术指标选择路由。BGP 是 Internet 的「粘合剂」，连接全球数十万个 AS。

结论 7.12: BGP 的工作原理

路径向量算法的核心：

- 每个 BGP 路由器向邻居（eBGP）或 AS 内其他 BGP 路由器（iBGP）通告自己所知道的到达各网络前缀的完整 **AS** 路径。
- 路由选择综合考虑 AS 路径长度、本地偏好（Local Preference）、MED 等多个属性。
- AS 路径中包含的 AS 号用于环路检测：若路由器在收到的路径中发现了自己的 AS 号，则拒绝该路由（避免 AS 级环路）。

BGP 的特点：

- 基于 TCP（端口 179），可靠传输路由更新。
- 发送增量更新（只发送变化的部分），非周期性发送完整路由表。
- 是一种策略驱动的路由协议：允许 AS 管理员实施商业关系策略（对等、转接、客户等）。
- Peering 关系仍会交换和通告路由；“不提供 transit”指通常不为对等方转发其通往第三方 AS 的流量，而不是不传递路由。

注意

408 三层路由对比：

- **RIP**：距离向量，跳数，UDP 520，AS 内部，小型。
- **OSPF**：链路状态，开销，IP 协议 89，AS 内部，大型（分层）。
- **BGP**：路径向量，策略，TCP 179，AS 之间，全球 Internet。

「**BGP** 属于距离向量协议」——错误。BGP 是路径向量（Path Vector），区别于 RIP 的距离向量（Distance Vector），因为 BGP 传递的是完整 AS 路径而非单纯的距离值。

7.8 IPv6

定义 7.15: IPv6

IPv6 地址为 128 位，用冒号十六进制表示（Colon Hexadecimal），如 2001:0db8:0000:0000:0000:ff00:0042:8329。可省略前导零，连续的零块可用「::」压缩一次。

结论 7.13: IPv6 与 IPv4 的主要差异

IPv4 vs IPv6 对比		
对比维度	IPv4	IPv6
地址长度	32 位	128 位
地址表示	点分十进制	冒号十六进制
首部长度的	可变 (20–60 字节)	固定 40 字节 (基本首部)
首部校验和	有 (每跳重算)	无 (由上层保证)
分片	源和中间路由器均可分片	仅源结点可分片
选项	在首部中 (IHL 字段处理)	扩展首部 (Next Header 链)
协议字段	Protocol	Next Header
ARP	有 (广播 ARP 请求)	用 NDP (ICMPv6 多播) 取代
广播	有	无 (改用多播和任播)
NAT	广泛使用	地址空间充足, 通常不因地址短缺而需要 NAT; 仍可能部署前缀转换等机制

IPv6 基本首部 (固定 40 字节) 的八个字段: 版本 (4 位)、通信量类 (8 位)、流标号 (20 位)、有效载荷长度 (16 位)、下一个首部 (8 位)、跳数限制 (8 位)、源地址 (128 位)、目的地址 (128 位)。

从 **IPv4** 向 **IPv6** 过渡的技术:

- **双协议栈 (Dual Stack)**: 主机/路由器同时运行 IPv4 和 IPv6。
- **隧道技术 (Tunneling)**: IPv6 数据报封装在 IPv4 数据报中穿过 IPv4 网络。
- **协议转换/NAT64**: 在 IPv4-only 和 IPv6-only 网络之间转换。

注意

408 常考细节:

- IPv6 基本首部固定 40 字节 (vs IPv4 最小 20 字节)。
- IPv6 没有首部校验和字段 (因为数据链路层和上层协议已有差错检测)。
- IPv6 中间路由器不分片, 分片仅在源结点进行。若路径 MTU 不够, 路由器丢弃并返回 ICMPv6 「分组太大」报文。
- IPv6 用 NDP (邻居发现协议, ICMPv6 的一部分) 取代 IPv4 中的 ARP。
- IPv6 地址类型: 单播 (Unicast)、多播 (Multicast)、任播 (Anycast)。无广播。

7.9 IP 组播与移动 IP

IP 组播

定义 7.16: IP 组播

IP 组播 (Multicast) 实现一对多通信: 源主机发送一份数据, 由网络负责复制并发送给组播组中的所有成员。组播地址范围: 224.0.0.0/4 (D 类地址 224.0.0.0–239.255.255.255)。

结论 7.14: 组播的核心概念

- 组播组 (**Multicast Group**): 用 D 类 IP 地址标识, 主机通过 IGMP 协议加入/离开组播组。
- **IGMP** (**I**nternet 组管理协议): 运行在主机和本地组播路由器之间, 用于管理组成员关系。
- 组播路由协议: 如 PIM (**P**rotocol **I**ndependent **M**ulticast), 根据组成员分布构建组播分发树。
- 组播 **MAC** 地址: 以太网中, 组播 IP 到组播 MAC 有固定映射: MAC 地址前 25 位为 01-00-5E, 后 23 位为组播 IP 的低 23 位。

常见辨析: (1) IPv4 组播地址只能用作目的地址, 不能作为源地址; (2) IGMP 用于主机与其直连组播路由器之间的成员关系管理, 组播路由协议负责路由器之间的组播转发, 二者职责不同; (3) 组播数据报的 TTL 字段可用于限制传播范围。仅讨论同一二层链路上的帧交付时, 数据帧可以不经路由器, 但不能据此笼统断言“组播不需要 IGMP”。

移动 IP

定义 7.17: 移动 IP

移动 IP 允许移动结点在改变网络接入点时保持其 IP 地址不变, 从而保持正在进行的 TCP 连接不中断。

结论 7.15: 移动 IP 的基本概念

移动 IP 引入三个功能实体:

- 移动结点 (**MN, Mobile Node**): 从一个网络移动到另一个网络的主机。
- 归属代理 (**HA, Home Agent**): 移动结点「家乡」网络上的路由器, 截获发往移动结点家乡地址的数据报并转发到移动结点的当前位置。
- 外地代理 (**FA, Foreign Agent**): 移动结点当前所在网络上的路由器, 为移动结点提供路由服务 (IPv6 中不需要 FA, 移动结点可自行配置转交地址)。

两个关键地址:

- 家乡地址 (**Home Address**): 移动结点在归属网络中使用的永久 IP 地址。
- 转交地址 (**Care-of Address, CoA**): 移动结点在外地网络中使用的临时 IP 地址。

基本工作过程 (三角路由):

通信对端 $\xrightarrow{\text{数据报发往家乡地址}}$ HA $\xrightarrow{\text{隧道封装到 CoA}}$ MN $\xrightarrow{\text{直接回复}}$ 通信对端

从通信对端 $\rightarrow$ HA $\rightarrow$ MN 的路径形成一个「三角形」, 因此称为三角路由 (Triangle Routing)。

注意

408 概念辨析：移动 IP 解决的是 IP 地址与网络位置绑定导致的连接中断问题。核心思想：家乡地址是永久标识（身份），转交地址是当前定位。HA 通过隧道（IP-in-IP 封装）将数据报转发给 MN。

7.10 408 高频考点速查**结论 7.16: 网络层—408 常考点**

1. **CIDR 子网划分与路由聚合：**给定地址块 /n，求网络地址、广播地址、可用地址数；给定多个连续子网，求最大聚合前缀。
2. **NAT/NAPT(PAT)：**私网地址、公网地址复用、五元组/端口映射、入站连接限制和校验和更新。
3. **IP 分片计算：**给定 MTU 和原数据报长度，计算各分片的数据长度、总长度、片偏移值和 MF 标志（注意片偏移以 8 字节为单位）。
4. **RIP 距离向量路由表更新：**给定初始路由表和收到的邻居路由表，更新本地路由表。
5. **RIP vs OSPF vs BGP 的三层对比：**算法类型、度量值、封装协议、应用场景。
6. **IP 首部字段：**TTL、Protocol、Header Checksum、Fragment Offset（各字段作用和 408 陷阱）。
7. **ARP 的工作范围：**同一局域网内的 IP→MAC 解析，ARP 请求是广播帧。
8. **ICMP 的 Ping 和 Traceroute 原理：**Echo/Echo Reply (Ping)、Time Exceeded (Traceroute)。
9. **IPv6 的关键变化：**128 位地址、无首部校验和、仅源结点分片、NDP 取代 ARP。
10. **虚电路 vs 数据报：**Internet 采用数据报 (Best-Effort)，可靠由传输层 TCP 完成。

8 习题

8.1 基础过关题

1. (真题风格·选择题) 一个 IP 数据报总长度为 2000 B (含固定首部 20 B), 要通过 MTU=820 B 的链路传输。设 IP 数据报首部无选项字段, 则原始数据报被分成的片数是
(A) 2 (B) 3
(C) 4 (D) 5
2. (真题风格·选择题) 某网络的 IP 地址空间为 192.168.5.0/24, 采用定长子网划分, 子网掩码为 255.255.255.248, 则该网络的最大子网个数和每个子网内的最大可分配地址个数分别是
(A) 32, 8 (B) 32, 6
(C) 8, 32 (D) 8, 30
3. (真题风格·选择题) 路由器 R1 与 R2 直连且均运行 RIP。R1 收到邻居 R2 的距离向量, 其中 R2 到 Net1 的距离为 2; R1 原有到 Net1 的路由不优于经 R2 的新路由。更新后, R1 到达 Net1 的距离是
(A) 2 (B) 3
(C) 4 (D) 5
4. (真题风格·选择题) 某家庭路由器使用 NAT, 使两台内网主机共享一个公网 IPv4 地址访问 Web 服务器。路由器区分两条并发 TCP 连接的主要依据是
(A) 仅内网主机的 MAC 地址 (B) 仅公网目的 IP 地址
(C) IP 地址、端口号和传输层协议等映射信息 (D) IP 首部的 TTL
5. (真题风格·选择题) NAT 路由器将某 TCP 分组的源 IPv4 地址和源端口号都改写后, 必须重新计算的是
(A) 仅以太网 FCS (B) IPv4 首部校验和与 TCP 校验和
(C) 仅 TCP 序号 (D) ARP 高速缓存的超时时间
6. (真题风格·选择题) 在 TCP/IP 体系结构中, ARP 协议数据单元封装的载体是
(A) IP 数据报 (B) 以太网帧
(C) TCP 报文段 (D) UDP 数据报
7. (真题风格·选择题) 某路由器收到了一个 IP 数据报, 对其首部进行校验后发现该数据报存在错误。路由器最有可能采取的动作是
(A) 纠正该 IP 数据报的错误 (B) 将该 IP 数据报的 TTL 值减 1
(C) 丢弃该 IP 数据报 (D) 通知目的主机重传

8.2 真题风格与综合训练

8. (真题风格·综合题) 某单位分配到一个地址块 136.23.12.64/26。现在需要进一步划分为 4 个一样大的子网。
- (1) 每个子网的网络前缀有多长?
 - (2) 每个子网有多少个地址?
 - (3) 写出每个子网的网络地址和广播地址。
9. (真题风格·综合题) 收到一个 IP 数据报, 其首部十六进制表示为: 45 00 00 54 00 03 20 00 80 01 99 52 C0 A8 00 01 C0 A8 00 02。回答:
- (1) 该 IP 数据报的数据部分长度是多少字节?
 - (2) 该数据报是否分片? 若是分片, 片偏移量是多少?
 - (3) 上层协议是什么? TTL 是多少?

拓展阅读:

- Kurose & Ross 第 4 章——IP 数据报格式、CIDR 子网划分、RIP/OSPF/BGP 路由协议。
- 谢希仁第 4 章——国内标准术语。

9 传输层

9.1 传输层概述

定义 9.1: 传输层的地位与功能

传输层是五层模型中第一个提供端到端 (**End-to-End**) 通信的层次。网络层及以下仅提供主机到主机的通信, 传输层在此基础上实现进程到进程的通信。

核心功能:

1. 端到端通信: 将应用层报文分割为报文段 (TCP) 或用户数据报 (UDP), 交付给接收方的对应进程。
2. 复用与分用: 发送方利用传输层将多个应用进程的数据汇聚到一个网络层 (复用); 接收方传输层将收到的数据交付给正确的应用进程 (分用) ——依据端口号。
3. 差错控制: 检测并纠正数据在传输过程中的错误 (校验和、确认与重传)。
4. 流量控制: 协调发送方与接收方的速率。
5. 拥塞控制 (TCP 独有): 防止过多数据注入网络导致拥塞。

注意

408 辨析: 传输层是「端到端」的, 网络层是「点到点」的。端到端 = 源进程 → 目的进程; 点到点 = 当前路由器 → 下一跳路由器。中间路由器最多处理到网络层, 不触及传输层。

要点 9.1: 端口号

端口号是 16 位整数 (0–65535)，用于传输层复用/分用。一个套接字地址通常由 IP 地址和端口号组成，但还需结合传输层协议解释：同一主机可同时存在相同端口号的 TCP 与 UDP 端点；一条 TCP 连接则由源/目的 IP、源/目的端口组成的四元组标识。

端口号分类

类型	范围	说明
知名端口 (Well-known)	0–1023	由 IANA 分配给标准服务 (HTTP=80, DNS=53, SMTP=25)
注册端口 (Registered)	1024–49151	供应用程序注册使用
动态/私有端口	49152–65535	客户端临时使用 (Ephemeral Port)

408 常考的知名端口：HTTP (80)、DNS (53)、SMTP (25)、POP3 (110)、FTP (21/20)、Telnet (23)、HTTPS (443)。

9.2 UDP —— 用户数据报协议**定义 9.2: UDP 的特点**

UDP (User Datagram Protocol) 提供无连接、不可靠的数据传输服务。它在 IP 层之上仅增加了端口号和校验和两个功能。

- 无连接：发送前无需建立连接，直接发送数据报。
- 不保证交付：无确认、无重传，数据报可能丢失、重复或失序。
- 面向报文：发送方 UDP 对应用层交下来的报文，添加首部后直接交付 IP 层。既不合并也不拆分，一次交付一个完整报文（报文边界保留）。
- 无拥塞控制：发送速率不受网络拥塞限制（适合实时应用）。
- 首部开销小：仅 8 字节。

注意

408 辨析：UDP 面向报文 vs TCP 面向字节流。UDP 保留报文边界——接收方一次 `recvfrom` 调用收到一个完整的 UDP 数据报。TCP 不保留报文边界——发送方 3 次 `write` 各 100 字节，接收方可能一次 `read` 收到 300 字节，也可能分 3 次各 100 字节。这一区别在选择题中反复出现。

结论 9.1: UDP 首部格式 (8 字节)

源端口号 (16 bit)	目的端口号 (16 bit)
UDP 长度 (16 bit)	UDP 校验和 (16 bit)

字段说明：

- 源端口号：发送进程的端口号。若无需回复，可为 0。
- 目的端口号：接收进程的端口号（必需）。

- 长度：UDP 数据报的总长度（首部 8B + 数据），单位字节。最小值 8（即无数据）。
- 校验和：差错检测，覆盖 UDP 首部 + 数据 + 伪首部（来自 IP 层的源/目的 IP 地址、协议号、UDP 长度）。

要点 9.2: UDP 校验和的计算

UDP 校验和使用伪首部（Pseudo-header）参与计算：

UDP 伪首部（12 字节）

字段	说明
源 IP 地址（4B）	来自 IP 首部
目的 IP 地址（4B）	来自 IP 首部
全 0（1B）	填充
协议号（1B）	UDP = 17（TCP = 6）
UDP 长度（2B）	与 UDP 首部中的长度字段相同

计算步骤：

1. 构造伪首部 + UDP 首部 + UDP 数据的临时数据块。
2. 若总长度为奇数字节，末尾补 0 字节。
3. 按 16 位字进行二进制反码求和（将所有 16 位字以反码加法累加，结果取反码即为校验和）。
4. 校验和为 0 时填入全 1（0xFFFF）——反码表示中全 0 与全 1 等价。

9.3 TCP —— 传输控制协议

定义 9.3: TCP 的特点

TCP（Transmission Control Protocol）提供面向连接的、可靠的、全双工的字节流传输服务。

- 面向连接：通信前须经三次握手建立连接，通信后四次挥手释放。
- 可靠传输：序号 + 确认 + 超时重传 + 校验和，保证数据无差错、不丢失、不重复、按序到达。
- 全双工：同一连接上双方可同时收发数据。
- 面向字节流：应用层交付的数据被 TCP 视为无结构的字节流（不保留报文边界）。
- 流量控制：滑动窗口机制，防止发送方淹没接收方。
- 拥塞控制：防止过多数据注入网络引起拥塞。

9.4 TCP 可靠传输机制

结论 9.2: 序号与确认号

序号 (Sequence Number): TCP 将数据流中的每个字节赋予一个序号。TCP 报文段首部中的「序号」字段是该报文段所携带数据部分的第一个字节的序号。

确认号 (Acknowledgment Number): 期望收到对方下一个报文段的第一个数据字节的序号, 即「已收到连续数据的最后一个字节序号 + 1」。确认号 = N 表示序号 $N - 1$ 及之前的所有数据均已正确收到。

累计确认 (Cumulative ACK): TCP 采用累计确认——确认号 N 表示「 N 之前的所有数据已收到」, 而非仅确认最后到达的报文段。若报文段乱序到达, 接收方仍发送期望的连续序号。

初始序号 (ISN): 连接建立时双方随机选择初始序号 (三次握手中交换)。

注意

408 高频计算: 已知发送序号和确认号, 推断数据长度。

- 若 A 发送序号 = 101, 数据长度 = 200 字节, 则下一报文段序号 = 301。
- 若 B 发回确认号 = 301, 则表示 B 已正确收到序号 ≤ 300 的所有数据。
- 注意: TCP 首部中的确认号字段仅在 ACK 标志 = 1 时有效。

结论 9.3: 超时重传与 RTT 估计

TCP 通常为最早的未确认数据维护一个重传定时器: 发送含数据的报文段时, 若定时器尚未运行则启动; 收到推进确认后重启, 所有未确认数据均被确认后停止。定时器超时则重传最早的未确认报文段, 并按规则退避 RTO。教材中“每个报文段设置定时器”是对这一机制的简化描述, 不应理解为实现中为每个报文段各维护一个独立定时器。

往返时间 RTT 的估计 (Jacobson/Karels 算法): 为避免公式过长, 记 E 为 EstimatedRTT, D 为 DevRTT, S 为本次 SampleRTT, 则

$$E_{\text{new}} = (1 - \alpha)E_{\text{old}} + \alpha S, \quad \alpha = 0.125, \quad (1)$$

$$D_{\text{new}} = (1 - \beta)D_{\text{old}} + \beta|S - E_{\text{old}}|, \quad \beta = 0.25, \quad (2)$$

$$\text{RTO} = E_{\text{new}} + 4D_{\text{new}}. \quad (3)$$

Karn 算法 (重点): 当报文段发生重传时, 其确认无法区分对应原始发送还是重传副本, 因此不使用这类有歧义的 **SampleRTT** 更新 **EstimatedRTT**。发生重传超时, 对 RTO 采用指数退避; 收到能够取得无歧义 RTT 样本的新数据确认后, 再恢复按估计公式更新。

408 要点: 超时重传 + Karn 算法结合考察。选择题经常给出 SampleRTT 序列, 其中某一跳是重传的, 问更新后的 EstimatedRTT。

结论 9.4: 流量控制——滑动窗口

TCP 的流量控制通过接收窗口 (**rwnd, Receiver Window**) 实现: 接收方在 TCP 首部「窗口大小」字段告知发送方自己还有多少可用的接收缓冲区空间。

发送方的发送窗口:

$$\text{发送窗口} = \min(\text{rwnd}, \text{cwnd})$$

其中 **rwnd** 由接收方告知, **cwnd** 由拥塞控制算法确定。

滑动窗口的三种边界指针:



- 前沿 (右边界): 收到新的 $rwnd$ 时右移 (窗口扩大) 或被拥塞控制限制。
- 后沿 (左边界): 收到累计确认时右移 (窗口滑动)。
- 零窗口: $rwnd = 0$ 时发送方暂停发送, 但会周期性地发送零窗口探测报文 (**Zero-Window Probe**)。
- 糊涂窗口综合征 (**Silly Window Syndrome**): 发送方每次只发极少数据, 效率极低。
解决方案: Nagle 算法 (发送方攒够 MSS 或收到 ACK 才发)、Clark 算法 (接收方攒够一定空间才通告窗口)。

9.5 TCP 拥塞控制 (408 重中之重)

注意

本小节是 **408** 计算机网络的高频考点。常见考法包括画 $cwnd$ 随轮次变化图、填写 $cwnd$ 与 $ssthresh$, 以及区分超时和 3 个重复 ACK 后的处理。作答时必须先采用题目明确给出的 TCP 版本和取整规则。

定义 9.4: 拥塞控制的基本概念

拥塞 (Congestion): 网络中过多分组导致路由器队列溢出、时延剧增、吞吐量下降的现象。拥塞控制是发送方对注入网络的数据速率进行的自我约束。

关键变量:

- **$cwnd$ (Congestion Window, 拥塞窗口)**: 发送方根据网络拥塞程度设置的窗口。实际发送窗口 = $\min(rwnd, cwnd)$ 。
- **$ssthresh$ (Slow Start Threshold, 慢开始门限)**: $cwnd$ 增长方式的分水岭。 $cwnd < ssthresh$ 时执行慢开始; $cwnd \geq ssthresh$ 时执行拥塞避免。
- **MSS (Maximum Segment Size)**: 最大报文段长度 (数据部分)。 $cwnd$ 和 $ssthresh$ 通常以 MSS 或字节为单位, 408 题目中以 MSS 为单位更常见。

结论 9.5: 四种拥塞控制算法详解

TCP 拥塞控制的四种算法形成一个完整的状态机。各状态之间的转换关系如下 (文字描述, 考试用文字或表格作答即可):

- 连接建立/超时 $\rightarrow$ 慢开始 ($cwnd=1$, 指数增长)
- 慢开始中 $cwnd \geq ssthresh \rightarrow$ 拥塞避免 (线性增长)
- 拥塞避免中收到 3 个重复 ACK $\rightarrow$ 快重传; 后续窗口变化取决于题设采用的教材化模型或 TCP Reno 快恢复
- 快恢复中收到新数据 ACK $\rightarrow$ 拥塞避免 ($cwnd=ssthresh$, 线性增长)
- 任何阶段发生超时 $\rightarrow$ 慢开始 ($ssthresh=cwnd/2$, $cwnd=1$)

算法一: 慢开始 (Slow Start)

- 触发条件: 连接建立时 ($cwnd$ 初始 = 1 MSS), 或发生超时事件后重置。

- **cwnd** 增长策略：每收到一个 ACK, $\text{cwnd} += 1 \text{ MSS}$ （即每个 RTT 后 cwnd 翻倍，指数增长）。
- 退出条件：cwnd 达到 ssthresh 时转入拥塞避免；或发生超时。
- 超时时的动作： $\text{ssthresh} = \text{cwnd}/2$, $\text{cwnd} = 1 \text{ MSS}$, 重新慢开始。

算法二：拥塞避免（Congestion Avoidance）

- 触发条件： $\text{cwnd} \geq \text{ssthresh}$ 。
- **cwnd** 增长策略：每个 RTT 后 $\text{cwnd} += 1 \text{ MSS}$ （线性增长——加法增大，Additive Increase）。
- 超时时的动作： $\text{ssthresh} = \text{cwnd}/2$, $\text{cwnd} = 1 \text{ MSS}$, 转入慢开始。

算法三：快重传（Fast Retransmit）

- 触发条件：发送方连续收到 3 个重复 ACK（Duplicate ACK）。
- 动作：立即重传缺失的报文段（不等超时）。比超时重传快得多。
- 注意：收到重复 ACK 说明网络仍有数据在流动，因此不需要像超时那样把 cwnd 降为 1。

算法四：快恢复（Fast Recovery）

- 触发条件：快重传之后执行。
- **TCP Reno** 的动作： ssthresh 取丢包时飞行中数据量的一半；收到第 3 个重复 ACK 时把 cwnd 暂时置为 $\text{ssthresh} + 3 \text{ MSS}$, 额外的重复 ACK 可继续使 cwnd 膨胀。收到确认新数据的 ACK 后，将 cwnd 收缩到 ssthresh 并转入拥塞避免。
- **408 教材化计算规则**：不少教材和试题直接把 3 个重复 ACK 后的 cwnd 记为 ssthresh, 并从该值进入拥塞避免，省略 Reno 快恢复期间的临时窗口膨胀。若题目未说明，本章习题统一使用这一简化规则；若题目明确指定 Reno，则按上一条计算。
- 若快恢复期间发生超时：与普通超时处理相同—— $\text{ssthresh} = \text{cwnd}/2$, $\text{cwnd} = 1 \text{ MSS}$, 转入慢开始。

结论 9.6: 拥塞窗口变化示例（408 必须掌握）

下表展示一个典型 TCP 连接中 cwnd 随传输轮次的变化过程。假定初始 $\text{ssthresh} = 16 \text{ MSS}$, $\text{MSS} = 1 \text{ KB}$ 。

cwnd 变化过程示例 (初始 ssthresh = 16 MSS)

轮次	cwnd	ssthresh	状态与事件
1	1	16	慢开始
2	2	16	慢开始
3	4	16	慢开始
4	8	16	慢开始
5	16	16	慢开始, $cwnd \geq ssthresh$, 下一轮切换
6	17	16	拥塞避免 (线性 +1)
7	18	16	拥塞避免
...	...	...	...
—	24	16	发生超时!
—	1	12	$ssthresh = 24/2 = 12$, $cwnd = 1$, 慢开始
—	2	12	慢开始
—	4	12	慢开始
—	8	12	慢开始
—	12	12	慢开始, $cwnd \geq ssthresh$, 切换
—	13	12	拥塞避免
—	14	12	拥塞避免
—	15	12	拥塞避免
—	16	12	收到 3 个重复 ACK (快重传)
—	8	8	教材化规则: $ssthresh = 16/2 = 8$, $cwnd = 8$

408 画图/填表的统一规则:

1. 超时: $ssthresh = \max(cwnd/2, 2 \text{ MSS})$, $cwnd = 1 \text{ MSS}$, 重新慢开始。
2. 3 个重复 ACK: 本章习题采用教材化规则, $ssthresh$ 取 $cwnd$ 的一半 (整数 MSS 的取整方式由题设说明), $cwnd$ 置为 $ssthresh$ 后进入拥塞避免。只有题目明确指定 TCP Reno 快恢复时, 才在第 3 个重复 ACK 到来时使用 $ssthresh+3$ 的临时膨胀窗口。
3. $cwnd$ 增长: 慢开始阶段每个 $RTT \times 2$ (指数); 拥塞避免阶段每个 $RTT + 1$ (线性)。

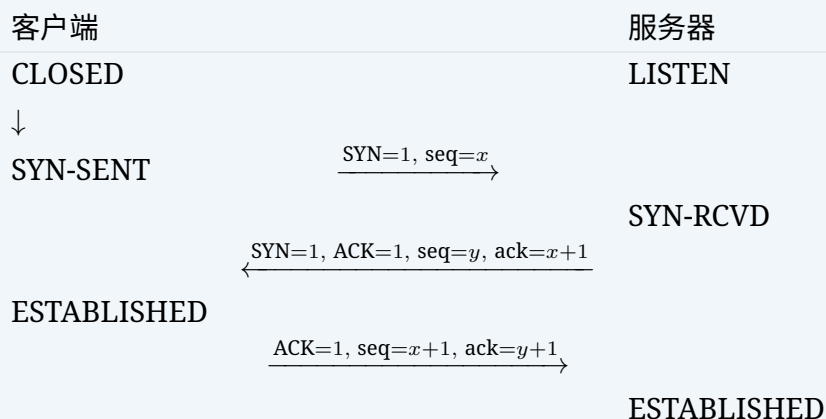
注意**408 拥塞控制的高频考法:**

- 画 $cwnd$ 变化曲线: 给定一组事件序列 (收到 ACK / 超时 / 3 dup ACK), 画出 $cwnd$ 随时间/轮次的变化。
- 填表: 给定表格, 逐轮次填写 $cwnd$ 和 $ssthresh$ 。
- 计算吞吐量: 给定 $cwnd$ 曲线和 RTT , 计算平均吞吐量。
- 区分超时后的处理和快重传后的处理: 前者 $cwnd$ 降为 1 (重新慢开始), 后者 $cwnd$ 降至 $ssthresh$ (快恢复)。
- 发送窗口 = $\min(cwnd, rwnd)$: 实际发送窗口由拥塞控制和流量控制共同决定。

9.6 TCP 连接管理

结论 9.7: 三次握手 (Three-Way Handshake)

TCP 连接的建立需要三次报文交换, 称为「三次握手」。



三步详解:

1. 客户端 → 服务器: **SYN = 1, seq = x (ISN_C)**。客户端进入 SYN-SENT 状态。
2. 服务器 → 客户端: **SYN = 1, ACK = 1, seq = y (ISN_S), ack = $x + 1$** 。服务器进入 SYN-RCVD 状态。
3. 客户端 → 服务器: **ACK = 1, seq = $x + 1$, ack = $y + 1$** 。双方进入 ESTABLISHED 状态。

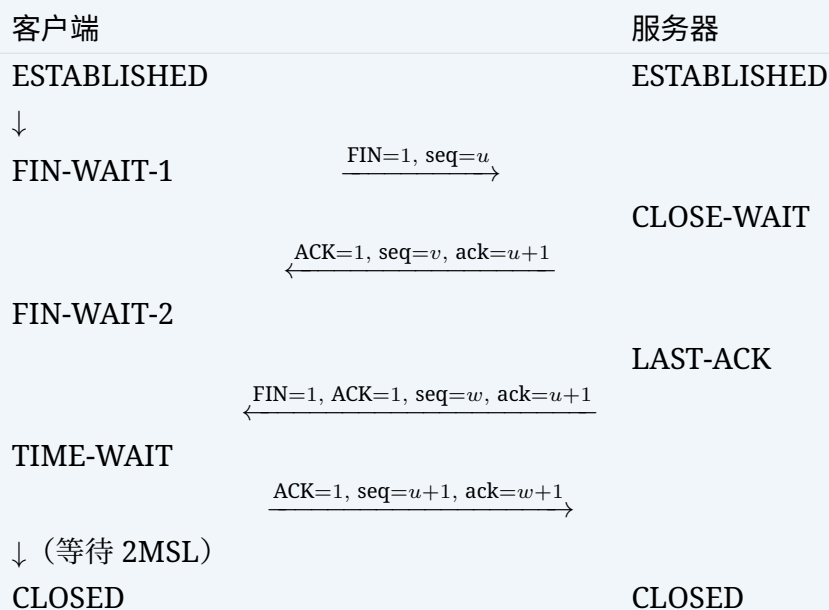
前两次握手消耗两个序号 (x 和 y), 第三次握手可不消耗序号 (若只发 **ACK**)。

为什么是三次?

- 防止已失效的连接请求报文段突然到达服务器, 导致服务器错误地建立连接。
- 三次握手确保双方都确认了对方的发送能力和接收能力: 客户端在第 3 步确认服务器的收 + 发, 服务器在第 2 步确认客户端的发送能力并在第 3 步确认自己的发送被收到。

结论 9.8: 四次挥手 (Four-Way Wavehand)

TCP 连接的释放需要四次报文交换 (因为 TCP 是全双工的, 两个方向需分别关闭)。



四步详解：

1. 客户端 → 服务器：**FIN = 1, seq = u**。客户端进入 FIN-WAIT-1。
2. 服务器 → 客户端：**ACK = 1, ack = u + 1**。服务器进入 CLOSE-WAIT，客户端收到后进入 FIN-WAIT-2。（此时客户端 → 服务器方向已关闭，服务器仍可发送数据。）
3. 服务器 → 客户端：**FIN = 1, ACK = 1, seq = w, ack = u + 1**。服务器进入 LAST-ACK。
4. 客户端 → 服务器：**ACK = 1, ack = w + 1**。客户端进入 TIME-WAIT，等待 **2MSL** (Maximum Segment Lifetime, 报文最大生存时间，通常 2-4 分钟) 后进入 CLOSED。服务器收到 ACK 后立即进入 CLOSED。

注意

TIME-WAIT 等待 2MSL 的原因 (408 常考简答)：

1. 保证最后一个 **ACK** 能到达服务器：若该 ACK 丢失，服务器会重发 FIN，客户端需要在 TIME-WAIT 期间能够重发 ACK。
2. 使本连接持续时间内产生的所有报文段都从网络中消失：经过 2MSL，本次连接的所有残留报文都会在网络中被丢弃，不会干扰新连接。

结论 9.9: TCP 有限状态机 (完整)

TCP 状态转换总表

状态	说明	典型转换
CLOSED	无连接 (起始/终点)	被动打开 → LISTEN; 主动打开 → SYN-SENT
LISTEN	服务器等待连接	收到 SYN → SYN-RCVD
SYN-SENT	客户端已发 SYN, 等待匹配	收到 SYN+ACK → ESTABLISHED
SYN-RCVD	服务器已收到 SYN, 发回 SYN+ACK	收到 ACK → ESTABLISHED
ESTABLISHED	连接建立, 正常通信	收到 FIN → CLOSE-WAIT; 主动关闭 → FIN-WAIT-1
FIN-WAIT-1	主动关闭方已发 FIN	收到 ACK → FIN-WAIT-2; 收到 FIN+ACK → TIME-WAIT
FIN-WAIT-2	对方已确认本方的 FIN	收到 FIN → TIME-WAIT
CLOSE-WAIT	被动关闭方收到 FIN, 等待应用关闭	发送 FIN → LAST-ACK
LAST-ACK	被动关闭方已发 FIN, 等待 ACK	收到 ACK → CLOSED
TIME-WAIT	等待 2MSL	超时 → CLOSED

9.7 TCP 首部格式

结论 9.10: TCP 报文段首部 (≥ 20 字节)

源端口号 (16 bit)		目的端口号 (16 bit)	
序号 Sequence Number (32 bit)			
确认号 Acknowledgment Number (32 bit)			
数据偏移 (4)	保留 (6)	控制位 (6)	窗口大小 (16 bit)
校验和 (16 bit)		紧急指针 (16 bit)	
选项 (可变长度, 最多 40 字节)			

各字段说明 (408 重点):

- 源端口号 / 目的端口号 (各 16 bit): 标识发送/接收进程。
- 序号 (32 bit): 本报文段数据部分第一个字节的序号。
- 确认号 (32 bit): 期望收到的下一个字节的序号, 仅在 ACK = 1 时有效。
- 数据偏移 (4 bit): 即 TCP 首部长度, 以 4 字节为单位。最小值 5 ($5 \times 4 = 20$ 字节), 最大值 15 ($15 \times 4 = 60$ 字节)。
- 保留 (6 bit) + 控制位 (6 bit):
 - URG: 紧急指针有效。
 - ACK: 确认号有效。连接建立后的所有报文段 (除第一个 SYN) 都应置 ACK = 1。
 - PSH: 接收方应尽快将数据交付应用层 (Push)。
 - RST: 复位连接。用于拒绝非法报文或异常关闭。
 - SYN: 同步序号, 用于连接建立。SYN = 1 时不能携带数据, 但消耗一个序号。
 - FIN: 发送方数据发送完毕, 请求释放连接。FIN = 1 的报文段可携带数据, 消耗一个序号。

- 窗口大小 (**16 bit**): 接收窗口 $rwnd$ (从 ACK 序号开始, 接收方还能接收的字节数)。最大 65535 字节, 使用窗口扩大选项可达更大。
- 校验和 (**16 bit**): 同 UDP——伪首部 + TCP 首部 + 数据。
- 紧急指针 (**16 bit**): 仅在 $URG = 1$ 时有效, 指示紧急数据末尾的偏移量。

注意

408 标志位组合的经典考法:

- 第一个 **SYN**: $SYN = 1, ACK = 0$ 。
- 第二个 **SYN+ACK**: $SYN = 1, ACK = 1$ 。
- 第三次握手 **ACK**: $SYN = 0, ACK = 1$ 。
- 第一个 **FIN**: $FIN = 1, ACK = 1$ (通常也携带对之前数据的确认)。
- **RST** 攻击/拒绝: $RST = 1, ACK = 0$ (异常拒绝, 不确认) 或 $RST = 1, ACK = 1$ (正常拒绝)。
- **SYN** 洪泛攻击: 攻击者发送大量 SYN, 伪造源 IP, 使服务器 SYN-RCVD 队列满。

9.8 408 高频考点速查

结论 9.11: 传输层—408 常考点

1. **TCP** 拥塞控制的 **cwnd** 变化: 慢开始 (指数) $\rightarrow$ 拥塞避免 (线性) $\rightarrow$ 超时 ($cwnd=1$) 与 3 个重复 ACK (教材化规则下 $cwnd=ssthresh$) 的区别, 是常见的画图或填表考点。
2. 三次握手与四次挥手的状态转换: 各报文标志位组合、序号/确认号的值、状态名称。
3. **TCP** 序号和确认号的计算: 已知发送序号和数据长度求下一报文段序号; 已知确认号求已接收数据量。
4. **TCP** 首部格式: 标志位 (SYN/ACK/FIN/RST) 的组合、数据偏移字段的含义。
5. **UDP** 首部 (8 字节): 校验和计算 (含伪首部)、面向报文 vs TCP 面向字节流。
6. **TIME-WAIT** 等待 **2MSL** 的原因: 简答题经典考点。
7. 发送窗口 = $\min(rwnd, cwnd)$: 流量控制与拥塞控制的协作。
8. **Karn** 算法: 重传报文不参与 RTT 估计, 超时时间指数退避。

10 习题

10.1 基础过关题

1. (真题风格·选择题) 一个 TCP 连接总是以 1 KB 的最大段长发送 TCP 段, 发送方有足够多的数据要发送。当拥塞窗口为 16 KB 时发生了超时, 如果接下来的 4 个 RTT 时间内的 TCP 段的传输都是成功的, 那么当第 4 个 RTT 时间内发送的所有 TCP 段都得到确认后, 拥塞窗口大小是

(A) 7 KB (B) 8 KB
(C) 9 KB (D) 16 KB

2. (真题风格·选择题) 主机甲向主机乙发送一个 TCP 报文段, 序号为 100, 数据部分长度为 200 B。主机乙收到后发回的确认号是

(A) 100 (B) 200
(C) 300 (D) 301

3. (真题风格·选择题) TCP 使用慢开始和拥塞避免算法。设 TCP 的慢开始门限 $ssthresh$ 的初始值为 8 (单位为报文段)。当拥塞窗口上升到 12 时, 网络发生超时。采用 408 教材化模型: 超时后将 $ssthresh$ 置为当前 $cwnd$ 的一半、 $cwnd$ 置为 1; 每轮开始时若 $cwnd$ 小于 $ssthresh$, 则该轮按慢开始增长, 否则按拥塞避免增长。从超时重置后的 $cwnd$ 开始记录 5 个窗口值, 其依次为

(A) 1,2,4,8,9 (B) 1,2,4,6,8
(C) 1,2,4,8,10 (D) 1,2,4,8,12

4. (真题风格·选择题) TCP 连接管理中, 连接释放的过程称为“四次挥手”。以下关于四次挥手的说法, 正确的是

(A) FIN 报文和 ACK 报文不能在同一报文中发送
(B) 主动关闭方发送 FIN 后立即进入 CLOSED 状态
(C) 被动关闭方收到 FIN 后还需要发送数据和 FIN
(D) TIME_WAIT 状态的持续时间是 2MSL

5. (真题风格·选择题) 主机甲和主机乙已建立 TCP 连接, 甲始终以 $MSS=1\text{ KB}$ 大小的段向乙发送数据, 并一直有数据发送; 乙每收到一个数据段都会发出一个接收窗口为 10 KB 的确认段。若甲在 t 时刻发生超时, 拥塞窗口为 8 KB, 则从 t 时刻起, 不再发生超时的情况下, 经过 10 个 RTT 后, 甲的发送窗口是

(A) 10 KB (B) 12 KB
(C) 14 KB (D) 15 KB

10.2 真题风格与综合训练

6. (真题风格·选择题) 主机甲和乙新建 TCP 连接, 甲向乙发送一个起始序号为 200 的 TCP 段后, 乙发回的确认段中确认号为 800, 则甲发送的 TCP 段携带的数据字节数是

(A) 600 (B) 599

(C) 800 (D) 200

7. (真题风格·综合题) TCP 的拥塞窗口 $cwnd$ 和门限 $ssthresh$ 的初始值分别为 1 和 16 (单位为 MSS)。本题采用本章的教材化计算规则: 收到 3 个重复 ACK 后, $ssthresh$ 取当时 $cwnd$ 的一半, $cwnd$ 置为 $ssthresh$ 后进入拥塞避免。按时间顺序, 连续成功传输使 $cwnd$ 分别变为 2, 4, 8, 16, 17, 18, 19, 20, 随后发生 3 次重复 ACK (快重传), 之后又经历 2 个成功 RTT。

(1) 画出 $cwnd$ 随传输轮次变化的折线图;

(2) 标注每次状态转换, 并说明 3 个重复 ACK 后本题是否单独建模 Reno 快恢复阶段;

(3) 第 2 次成功传输后 $cwnd$ 的值是多少?

拓展阅读:

- Kurose & Ross 第 3 章——TCP 可靠传输和拥塞控制的完整讲解与状态图。
- 谢希仁第 5 章——国内标准术语体系。

11 应用层

11.1 域名系统 DNS

定义 11.1: 域名系统 (DNS)

DNS (Domain Name System) 是互联网的命名系统, 将便于记忆的域名转换为机器使用的 IP 地址。DNS 采用客户/服务器 (C/S) 模型, 运行在 UDP 的 53 号端口上 (数据量小、要求快速响应)。当数据超过 512 字节或区域传送时使用 TCP。

域名空间

结论 11.1: 域名的层次结构

域名采用层次树状结构, 从右向左层次递增, 各级之间用「.」分隔:

`www.example.com.`

最右边有隐含的根域「.」:

- 根域: 用「.」表示, 由 13 组根域名服务器管理。
- 顶级域 (TLD): .com (商业)、.org (非营利组织)、.net (网络服务)、.cn (中国)、.edu (教育) 等。

- 二级域 (SLD): example (由组织注册)。
- 三级域/子域: www (具体主机名)。

域名不区分大小写，每级不超过 63 字符，总长不超过 255 字符。

注意

408 常考辨析：域 vs 区 (Zone)。区是域名空间中一个服务器的管辖范围；一个域可能包含多个区，但一个区只属于一个域。区的概念用于 DNS 服务器的实际管理和区域传送。

域名服务器

结论 11.2: DNS 服务器四层结构

DNS 服务器的层次结构		
类型	作用	管辖范围
根域名服务器	返回顶级域服务器的 IP	全球 13 组 (a.root-servers.net ~ m)
顶级域服务器	返回权限域名服务器的 IP	.com, .org, .cn 等
权限域名服务器	返回域名—IP 的最终映射	特定组织/机构
本地域名服务器	代理主机进行查询	ISP/局域网 (非层次结构)

补充：本地域名服务器不属于 DNS 层次结构，但每个 ISP/大学/公司都有本地域名服务器，距离主机很近（通常在同一局域网）。

DNS 解析过程—递归查询 vs 迭代查询

结论 11.3: DNS 查询的两种模式

递归查询 (Recursive Query)：被请求的 DNS 服务器必须向请求者返回最终结果（或失败），而不能只给出下一台服务器的转介信息；若它没有答案，就代表请求者继续向下游服务器查询。主机的存根解析器通常向本地 DNS 发递归请求。若后续各级也都采用递归，则查询逐级向下传递、结果逐级返回。

完全递归的消息方向为：主机请求本地 DNS，本地 DNS 请求根 DNS，根 DNS 请求顶级 DNS，顶级 DNS 请求权威 DNS；查询结果再按权威 DNS、顶级 DNS、根 DNS、本地 DNS、主机的方向逐级返回。

迭代查询 (Iterative Query)：当被查询的服务器没有所需记录时，它返回下一步应查询的服务器的 IP 地址，由请求者自己继续查询。本地域名服务器向根、顶级、权限服务器的查询通常为迭代。

典型的迭代流程为：主机递归请求本地 DNS；本地 DNS 查询根 DNS，得到顶级 DNS 地址；再查询顶级 DNS，得到权威 DNS 地址；最后查询权威 DNS，得到最终记录并返回主机。

注意**408 重难点：递归 vs 迭代的辨析。**

- 常用模式：主机的存根解析器 → 本地域名服务器通常采用递归查询，主机不承担逐级查询负担。
- 可配置部分：本地域名服务器 → 其他服务器，可以是递归也可以是迭代。实际互联网通常采用：主机递归 → 本地 DNS，本地 DNS 迭代 → 根/顶级/权限。
- 关键字：递归 = 全套代劳（给我结果，别让我再去问）；迭代 = 指路（我不知道，但你去问 XXX）。
- 递归查询中每一层都向上一层递归请求，导致根服务器压力巨大，实际中很少使用从本地到根的递归。

DNS 缓存**结论 11.4: DNS 缓存机制**

为提高效率、减轻根服务器压力，DNS 广泛使用缓存：

- 本地域名服务器获得一条映射后，将其缓存一段时间（由 TTL 字段决定），后续相同查询直接返回。
- 主机本身也可能在浏览器或操作系统的存根解析器中缓存 DNS 记录；hosts 文件是本地静态名称映射来源，不属于 DNS 缓存。
- **TTL**（生存时间）：由权威域名服务器设置，默认 TTL 到期后缓存条目被清除，需要重新查询。
- 缓存可能导致不一致：当 IP 地址更改时，在 TTL 过期前缓存中的旧记录可能仍然生效。

11.2 文件传输协议 FTP**定义 11.2: FTP 协议**

FTP（File Transfer Protocol）用于在网络上进行文件传输，采用 C/S 模型，提供交互式访问。FTP 使用 TCP 协议保证可靠传输。

控制连接与数据连接**结论 11.5: FTP 的双连接机制**

FTP 的独到之处在于它使用两个并行的 **TCP** 连接：

FTP 控制连接 vs 数据连接

	控制连接	数据连接
端口号	服务器端口 21	主动模式: 服务器端口 20; 被动模式: 服务器通过 PASV/EPSV 协商临时端口
持续时间	整个会话期间保持	每次文件传输时建立
传输内容	FTP 命令与应答	文件数据/目录列表
协议	客户端发命令, 服务器应答	直接传输数据

控制连接: 客户端通过端口 21 向服务器发起连接, 发送 FTP 命令 (USER, PASS, LIST, RETR, STOR 等) 及接收服务器应答。

数据连接: 每当需要传输文件或目录列表时, 动态建立一条新的数据连接。数据传输完成后, 数据连接关闭, 控制连接继续保持。

这种「带外」(Out-of-Band) 控制方式是 FTP 区别于其他协议 (如 HTTP 把命令和数据混在同一条连接中) 的核心特征。

主动模式 vs 被动模式

结论 11.6: FTP 两种数据连接模式

根据数据连接建立的发起方不同, FTP 分为两种工作模式:

主动模式 (Active Mode / PORT):

- 客户端发送 PORT 命令, 告知服务器自己在哪个端口 ($N > 1023$) 上监听。
- 服务器主动从端口 20 向客户端的 N 端口发起 TCP 连接。
- 问题: 客户端通常在防火墙/NAT 后面, 服务器反向主动连接常被防火墙拦截。

被动模式 (Passive Mode / PASV):

- 客户端发送 PASV 命令, 服务器响应一个随机端口号 $M > 1023$ 。
- 客户端主动向服务器的 M 端口发起数据连接。
- 优点: 防火墙友好——所有连接都由客户端发起。现代 FTP 客户端默认使用被动模式。

注意

408 常考: 主动/被动模式的区分。

- 主动模式: 服务器从 20 端口 → 客户端端口 (服务器主动发起数据连接) ——属于从外到内的连接。
- 被动模式: 客户端 → 服务器随机端口 (客户端主动发起数据连接) ——所有连接方向一致。
- 区分关键: 看数据连接由谁发起。无论哪种模式, 控制连接始终由客户端发起 (到 21 端口)。

11.3 电子邮件

定义 11.3: 电子邮件系统体系结构

电子邮件系统由三部分组成：

1. 用户代理 (**UA, User Agent**)：用户与邮件系统的接口 (Outlook, Thunderbird, Web Mail)。
2. 邮件服务器 (**Mail Server**)：存储和转发邮件。
3. 邮件协议：SMTP (发送)、POP3/IMAP (接收)。

邮件的整个生命周期为：发送方 UA 使用 SMTP 向发送方邮件服务器提交邮件；发送方邮件服务器使用 SMTP 转交给接收方邮件服务器；接收方 UA 再通过 POP3 或 IMAP 读取邮件。

SMTP — 简单邮件传送协议

结论 11.7: SMTP 协议

SMTP (Simple Mail Transfer Protocol) 是发送邮件的协议。服务器间转发通常使用 TCP 端口 25；客户端邮件提交通常使用端口 587，并可通过 STARTTLS 加密（端口号本身不等于已经加密）。

- 工作过程：使用命令/应答方式。客户端发送命令 (HELO, MAIL FROM, RCPT TO, DATA, QUIT)，服务器应答 (220, 250, 354, 221 等)。
- 报文格式：信封 (envelope) + 内容。信封 = MAIL FROM + RCPT TO；内容 = 首部 + 空行 + 主体。
- 首部字段：From：(邮件是谁写的)，To：(发给谁)，Subject：(主题) 等。注意：这些是邮件首部的文本内容，不是 SMTP 命令。
- 传统 SMTP 的邮件数据按 7 位 ASCII 设计，二进制附件和非 ASCII 内容通常借助 MIME 的内容类型与传输编码表示；扩展 SMTP 还定义了 8BITMIME 等能力。
- SMTP 使用持久连接：同一连接中可以发送多封邮件。

典型 SMTP 会话示例：

```
1 S: 220 smtp.example.com
2 C: HELO client.com
3 S: 250 Hello
4 C: MAIL FROM: <alice@client.com>
5 S: 250 OK
6 C: RCPT TO: <bob@server.com>
7 S: 250 OK
8 C: DATA
9 S: 354 Start mail input
10 C: From: alice@client.com
11 C: To: bob@server.com
12 C: Subject: Hello
13 C:
14 C: This is the message body.
15 C: .
16 S: 250 OK
```



```
17 C: QUIT
18 S: 221 Bye
```

POP3 与 IMAP — 邮件读取协议

结论 11.8: POP3 vs IMAP

POP3 与 IMAP 对比		
	POP3	IMAP
TCP 端口	110 (明文) / 995 (SSL)	143 (明文) / 993 (SSL)
存储模型	下载到本地;是否删除服务器副本由客户端通过协议操作决定	以服务器端邮箱状态为准,便于多设备同步
文件夹	协议模型较简单,不提供 IMAP 式的服务器端多文件夹管理	支持服务器端多文件夹、搜索
工作模式	下载—删除 (或保留)	联机操作
离线能力	下载后可离线阅读	客户端可缓存邮件离线阅读,状态同步需要联网
适用场景	单设备收邮件	多设备同步邮件

POP3 的三个阶段:

1. 特许阶段 (**Authorization**): 发送用户名和密码 (USER, PASS)。
2. 事务阶段 (**Transaction**): 列出邮件 (LIST)、获取邮件 (RETR)、删除邮件 (DELE)。
3. 更新阶段 (**Update**): QUIT 命令后服务器执行删除操作。

IMAP 允许用户在服务器上创建文件夹、移动邮件、只下载邮件头等,比 POP3 复杂但更灵活。IMAPS 使用 TLS/SSL 加密。

MIME — 多用途互联网邮件扩展

结论 11.9: MIME 扩展

由于 SMTP 仅支持 7 位 ASCII 文本, MIME (Multipurpose Internet Mail Extensions) 通过在邮件首部增加字段来支持:

- 非 ASCII 文本 (如中文、日文等国际字符);
- 二进制附件 (图片、音频、视频、PDF 等);
- 图文混合的邮件正文 (multipart)。

MIME 新增的首部字段:

- MIME-Version: 1.0 — 声明使用 MIME。
- Content-Type: text/plain; charset="utf-8" — 内容的 MIME 类型和字符编码。
- Content-Transfer-Encoding: base64 — 内容传输编码方式 (Base64, Quoted-Printable)。

MIME 对 SMTP/POP3 协议本身不做改动, 仅仅在邮件数据的首部增加了上述字段。发送方用 Base64 等将二进制编码为 7 位 ASCII, 接收方解码还原。

常见 **Content-Type**: text/plain, text/html, image/jpeg, application/pdf, multipart/mixed (混合多种内容), multipart/alternative (同一内容多种格式)。

11.4 万维网与 HTTP

URL 的基本概念

定义 11.4: 统一资源定位符 URL

URL (Uniform Resource Locator) 用于唯一定位互联网上的资源:

`<protocol>://<host>:<port>/<path>`

示例: `http://www.example.com:80/index.html`

- 协议 (**protocol**): http, https, ftp 等。
- 主机 (**host**): 域名或 IP 地址。
- 端口 (**port**): 可选, 默认 HTTP 为 80, HTTPS 为 443。
- 路径 (**path**): 服务器上的资源路径。

HTTP 报文格式

结论 11.10: HTTP 请求报文格式

HTTP 报文分为请求报文和响应报文, 两者结构类似: 起始行 + 首部行 + 空行 + 实体主体。
请求行 (**Request Line**):

GET
方法
/index.html
URL
HTTP/1.1
版本

常见 HTTP 请求方法:

HTTP 请求方法

方法	含义
GET	请求获取资源 (幂等)
POST	向服务器提交数据 (如表单)
HEAD	类似 GET 但只返回首部, 不返回主体
PUT	上传/替换指定资源
DELETE	删除指定资源
OPTIONS	查询服务器支持的方法

首部行 (**Header Lines**): 格式为「字段名: 值」, 常见:

- Host: www.example.com — 目标主机 (HTTP/1.1 必须, 支持虚拟主机)。
- Connection: close — 请求本次响应后关闭连接。HTTP/1.1 默认持久连接;

keep-alive 在 HTTP/1.0 中常用于协商持久连接，在 HTTP/1.1 中通常无需靠它启用默认持久性。

- User-Agent: Mozilla/5.0 —客户端类型。
- Accept: text/html —客户端接受的内容类型。
- Content-Length: 348 —实体主体的字节数。

结论 11.11: HTTP 响应报文格式

状态行 (Status Line):

$\underbrace{\text{HTTP/1.1}}_{\text{版本}}$

 $\underbrace{200\ OK}_{\text{状态码 + 短语}}$

常见 HTTP 状态码:

HTTP 状态码分类

状态码	短语	含义
200	OK	请求成功，响应包含请求的资源
301	Moved Permanently	永久重定向 (URL 已更改)
302	Found	临时重定向 (原来 URL 仍可用)
304	Not Modified	缓存有效，不需要重新传输
400	Bad Request	请求报文有误
401	Unauthorized	需要认证
403	Forbidden	服务器拒绝请求
404	Not Found	请求的资源不存在
500	Internal Server Error	服务器内部错误
502	Bad Gateway	网关/代理从上游收到无效响应
503	Service Unavailable	服务器暂时不可用 (过载/维护)

记忆规律:

- **2xx**: 成功 (200 正常, 204 无内容, 206 部分内容)。
- **3xx**: 重定向 (301 永久, 302 临时, 304 缓存有效)。
- **4xx**: 客户端错误 (400 请求有问题, 404 找不到, 403 无权限)。
- **5xx**: 服务器错误 (500 内部错误, 503 服务不可用)。

非持久连接 vs 持久连接

结论 11.12: HTTP 连接的两种方式

这是 408 计算题的核心考点。

非持久连接 (Non-persistent / HTTP/1.0 默认): 每个 TCP 连接只传送一个对象 (一个 HTML 文件 + 每个嵌入图片各需一个连接)。过程:

1. 建立 TCP 连接 (1 RTT)。
2. 发送 HTTP 请求，接收响应 (1 RTT)。

3. 关闭 TCP 连接。

传送 1 个 HTML 文件 + n 个内嵌对象的时间：

$$T_{\text{非持久}} = \underbrace{(2\text{RTT} + T_{\text{文件}})}_{\text{HTML 文件}} + \underbrace{n \times (2\text{RTT} + T_{\text{对象}})}_{\text{内嵌对象}}$$

总时间为： $2(n+1)\text{RTT}$ + 各对象传输时间。

若使用并行连接（浏览器同时打开多个 TCP 连接，常见为 6-10 个），则时间减少。非持久连接按顺序串行发送是理论模型，实际有可能并行。

持久连接（**Persistent / HTTP/1.1 默认**）：一个 TCP 连接上可以传送多个对象。过程：

1. 建立 TCP 连接（1 RTT）。
2. 传送所有对象（HTML + 所有内嵌对象）依次发送和接收。
3. 超时后才关闭连接。

若先取得基本 HTML 后才能知道 n 个内嵌对象的 URL，且不使用流水线，则每个对象的请求—响应各占 1 RTT（忽略传输时间）：

$$T_{\text{持久, 非流水}} = [1 + 1 + n]\text{RTT} + \sum \text{各对象传输时间}.$$

其中三个部分依次是建立 TCP、取得基本 HTML、依次取得 n 个内嵌对象。

流水线方式（**Pipelining**）：取得基本 HTML 并获知对象 URL 后，客户端可连续发出所有内嵌对象请求，不必等待前一个响应。于是

$$T_{\text{流水线}} = [1 + 1 + 1]\text{RTT} + \sum \text{各对象传输时间}.$$

三个 RTT 依次用于建立 TCP、取得基本 HTML、流水线取得全部内嵌对象。若题目另行假定客户端在建连前已经知道所有对象 URL，才可能省去「先取 HTML」这一 RTT。HTTP 时间题必须服从题设给定的连接并行度、URL 可知时刻和是否忽略传输时间，不能脱离模型套固定数值。

注意

408 高分技巧—HTTP 时间计算题：

- **RTT**（往返时间）的定义：一个分组从客户端到服务器再返回的时间。
- TCP 三次握手需要 1 RTT（前两次握手）后才开始发送数据。因此建立连接本身消耗 1 RTT。
- 非持久连接，每个对象额外浪费 2 RTT（1 连接建立 + 1 请求—响应）。
- 并行持久连接（**HTTP/1.1 常用**）：打开 k 个并行连接，每个连接上持久传输。需要综合考虑并行度和持久性。
- 常见题型：给定网页包含一个 HTML 和 n 个图片，使用非持久串行 / 非持久并行 / 持久 / 流水线方式，计算总时间。

Cookie

结论 11.13: HTTP Cookie 机制

HTTP 本身是无状态协议，Cookie 是用来在无状态协议上保持状态的机制。

Cookie 的四个组件：

1. 在 HTTP 响应报文中的 Set-Cookie 首部行；
2. 在 HTTP 请求报文中的 Cookie 首部行；
3. 浏览器端存储的 Cookie 文件；
4. Web 站点后端数据库中的 Cookie 记录。

工作流程：

1. 客户端首次访问服务器（无 Cookie）。
2. 服务器在响应中加上：Set-Cookie: 1678（服务器生成唯一 ID）。
3. 浏览器将 Cookie 号存储在本地 Cookie 文件中。
4. 后续每次请求该站点时，浏览器自动携带 Cookie: 1678 首部。
5. 服务器根据 Cookie 号在数据库中查到用户的状态信息（购物车、登录态等），从而在无状态的 HTTP 上实现了有状态的会话。

Cookie 的属性：Expires（过期时间），Domain（作用域），Path（路径），Secure（仅 HTTPS），HttpOnly（不可被 JavaScript 访问）。

11.5 动态主机配置协议 DHCP

定义 11.5: DHCP 协议

DHCP（Dynamic Host Configuration Protocol）用于为主机自动配置 IP 地址、子网掩码、默认网关、DNS 服务器等网络参数。运行在 UDP 端口 67（服务器）和 68（客户端）上。DHCP 是即插即用联网的关键协议。

DHCP 工作过程—四步握手

结论 11.14: DHCP 四步过程

这是 408 选择题的常考点，四个步骤的顺序和报文类型必须牢记。

DHCP 四步过程

步骤	报文	说明
1	DHCPDISCOVER	客户端广播「发现」报文（源 IP 0.0.0.0，目标 255.255.255.255）寻找 DHCP 服务器
2	DHCPOFFER	服务器单播/广播「提供」报文，包含建议的 IP 地址、子网掩码、租用期
3	DHCPREQUEST	客户端广播「请求」报文，选择某个服务器提供的 IP 地址
4	DHCPACK	服务器「确认」报文，正式分配 IP 地址

详细过程：

1. **DHCPDISCOVER** (发现阶段): 客户端在物理子网内广播 UDP 报文 (服务器端口 67), 源 IP = 0.0.0.0, 目的 IP = 255.255.255.255 (受限广播)。网络中可能存在多个 DHCP 服务器; 收到报文且愿意、能够提供配置的服务器可以响应。
2. **DHCPOFFER** (提供阶段): 愿意提供地址的服务器可以回应 DHCPOFFER 报文 (包含: 可分配的 IP 地址 yiaddr、子网掩码、租用期、DHCP 服务器标识等)。因此客户端可能收到零个、一个或多个 OFFER, 并非每台服务器都必然回应。
3. **DHCPREQUEST** (请求阶段): 客户端选择一个 OFFER, 再次广播 DHCPREQUEST 报文, 明确声明选择哪个 DHCP 服务器 (报文中包含该服务器的标识符)。广播的目的: 让其他提供过 OFFER 的服务器知道自己未被选中, 从而撤销本次要约, 并在实现已预留地址时释放该预留。
4. **DHCPACK** (确认阶段): 被选中的服务器发送 DHCPACK 确认分配。客户端收到后就可以正式使用该 IP 地址。租用期过半时客户端需要续租 (发送 DHCPREQUEST 直接到服务器)。

记忆口诀: 「**D-O-R-A**」——Discover → Offer → Request → ACK。

注意

408 常见陷阱:

- DHCPDISCOVER 和 DHCPREQUEST 都是广播 (因为客户端此时还没有有效 IP) , DHCPOFFER 和 DHCPACK 可以是广播或单播。
- DHCP 基于 UDP, 不是 TCP。DHCP 服务器用 67 端口, 客户端用 68 端口。
- DHCP 分配的 IP 地址是临时的 (有租用期), 不是永久的。
- 若 DHCP 服务器与客户端不在同一子网, 需通过 **DHCP** 中继代理 (Relay Agent) 转发。
- DHCP 是应用层协议, 但它的配置直接影响网络层 (IP 地址) 和传输层的工作。

11.6 408 高频考点速查

结论 11.15: 应用层—408 常考点

1. **DNS 递归 vs 迭代查询**: 典型模型中主机请求本地 DNS 递归解析, 本地 DNS 再向层次服务器进行迭代查询; 是否提供递归服务仍取决于服务器策略和请求标志, 不能把“主机到本地”写成无条件必然递归。
2. **FTP 控制连接 (21) 和数据连接 (20)**: 主动模式 (服务器端口 20 发起) vs 被动模式 (客户端主动连接服务器随机端口)。
3. **SMTP/POP3/IMAP 功能区分**: SMTP 只管「发」, POP3/IMAP 只管「收」。IMAP 多设备同步。
4. **HTTP 持久连接的时间计算**: RTT 的概念, 非持久每个对象 +2RTT, 持久一个 TCP 连接内传送所有对象。
5. **DHCP 四步过程 D-O-R-A**: DISCOVER(广播) → OFFER → REQUEST(广播) → ACK。
6. **HTTP 状态码分类**: 2xx 成功、3xx 重定向、4xx 客户端错误、5xx 服务器错误。
7. **Cookie 机制**: 无状态 HTTP 下保持会话状态的 4 个组件。
8. **MIME 的作用**: 弥补 SMTP 仅支持 7 位 ASCII 的不足。

12 习题

12.1 基础过关题

1. (真题风格·选择题) FTP 客户和服务器间传递 FTP 命令时, 使用的连接是
(A) 建立在 TCP 之上的控制连接 (B) 建立在 TCP 之上的数据连接
(C) 建立在 UDP 之上的控制连接 (D) 建立在 UDP 之上的数据连接
2. (真题风格·选择题) 如果本地域名服务器无缓存, 当采用递归方法解析另一网络中的某主机域名时, 用户主机和本地域名服务器分别发送的域名请求消息条数分别为
(A) 1 条, 1 条 (B) 1 条, 多条
(C) 多条, 1 条 (D) 多条, 多条
3. (真题风格·选择题) 电子邮件依次经历三个阶段: (1) 用户 1 向其邮件服务器提交邮件; (2) 发送方邮件服务器把邮件传给接收方邮件服务器; (3) 用户 2 从接收方邮件服务器读取邮件。三个阶段分别使用的协议最可能是
(A) SMTP, SMTP, POP3 (B) SMTP, POP3, SMTP
(C) POP3, SMTP, SMTP (D) SMTP, SMTP, SMTP
4. (真题风格·选择题) 使用浏览器访问某大学 Web 网站主页时, 不可能使用到的协议是
(A) PPP (B) ARP
(C) SMTP (D) HTTP
5. (真题风格·选择题) 在 TCP/IP 体系结构中, 以下协议中属于应用层的是
(A) ARP (B) ICMP
(C) DHCP (D) TCP

12.2 真题风格与综合训练

6. (真题风格·综合题) DNS 解析过程有递归和迭代两种方式。某主机需要解析域名 `www.example.com`, 根域名服务器、`.com` 顶级域名服务器、`example.com` 权威域名服务器依次参与。
 - (1) 分别画出递归解析和迭代解析的查询消息流图;
 - (2) 哪种方式对本地域名服务器的负担更大? 哪种方式对根域名服务器的负担更大?
7. (真题风格·综合题) 简述 HTTP 持久连接和非持久连接的区别。某网页包含 1 个基本 HTML 文件和 5 个 JPEG 图像, 所有文件均在同一个服务器上。假定无 DNS 查询时间, TCP 建连后才发送 HTTP 请求, 必须先收到基本 HTML 才能知道 5 个图像的 URL, 忽略对象传输时间; 非持久连接串行建立, 持久连接对 5 个图像采用流水线。分别计算两种方式所需的 RTT 数。

拓展阅读：

- Kurose & Ross 第 2 章——DNS、HTTP、FTP、SMTP/POP3 的完整协议细节和抓包示例。
- 谢希仁第 6 章——国内标准术语。

PART V 第五部分

408 模拟试卷

1 模拟卷一

考试说明：考试时间 180 分钟，满分 150 分。

一、单项选择题：1-40 小题，每小题 2 分，共 80 分

数据结构（1-11 题）

1. 若线性表最常用的操作是存取第 i 个元素及其前驱和后继的值，则采用（ ）存储方式最节省时间。

- (A) 单链表 (B) 双链表
(C) 循环单链表 (D) 顺序表

2. 栈和队列的共同点是

- (A) 都是先进先出 (B) 都是先进后出
(C) 只允许在端点处进行插入和删除 (D) 没有共同点

3. 设模式串 $T = \text{"abcaabca"}$ ，采用本书的 next 定义：下标从 1 开始， $\text{next}[1] = 0$ ，失配时令 $j = \text{next}[j]$ 。则 $\text{next}[4]$ 和 $\text{next}[7]$ 的值分别为

- (A) 1 和 4 (B) 1 和 2
(C) 2 和 4 (D) 1 和 3

4. 一棵完全二叉树有 1001 个结点，其中叶子结点的个数是

- (A) 500 (B) 251
(C) 250 (D) 501

5. 具有 n 个顶点的有向完全图的边数是

- (A) $n(n-1)/2$ (B) $n(n-1)$
(C) n^2 (D) $n(n+1)$

6. 在一棵 5 阶 B-树中，除根结点外的每个非终端结点至少包含的关键字个数为

- (A) 1 (B) 4
(C) 3 (D) 2

7. 下列排序算法中，不稳定的的是

- (A) 直接插入排序 (B) 冒泡排序
(C) 快速排序 (D) 归并排序
8. 用不带头结点的单链表存储队列, 队头在链表头, 队尾在链表尾。在进行出队操作时
- (A) 仅修改头指针 (B) 仅修改尾指针
(C) 头尾指针都可能修改 (D) 头尾指针都不修改
9. 一棵二叉排序树中, 关键字最小的结点
- (A) 一定是叶结点 (B) 一定无左孩子
(C) 一定无右孩子 (D) 可能是任何结点
10. 采用“牺牲一个存储单元”判别队满/队空的循环队列约定: `front` 指向队头元素, `rear` 指向队尾元素的下一空位, 数组下标按模 6 循环。若用一个大小为 6 的数组实现该循环队列, 当前 `rear=0`、`front=3`。从队列中删除一个元素, 再加入两个元素后, `rear` 和 `front` 的值分别为
- (A) 1 和 4 (B) 2 和 4
(C) 2 和 3 (D) 1 和 3
11. 对权值集合 {5, 7, 10, 15, 20, 45} 构造 Huffman 树, 其最小带权路径长度 WPL 为
- (A) 102 (B) 183
(C) 228 (D) 330

计算机组成原理 (12-22 题)

12. 计算机中, 运算器的主要功能是进行
- (A) 算术运算 (B) 逻辑运算
(C) 算术和逻辑运算 (D) 初等函数运算
13. 8 位补码表示的整数范围是
- (A) $-128 \sim 127$ (B) $-127 \sim 127$
(C) $-128 \sim 128$ (D) $-127 \sim 128$
14. IEEE 754 单精度浮点数中, 阶码的偏置值 (bias) 为
- (A) 128 (B) 255
(C) 126 (D) 127
15. 某 32 位按字节编址的计算机采用 8 KiB 直接映射 Cache, 块大小为 64 B。主存地址按 Tag、行号、块内偏移划分, 三者位数依次为

(A) 19、7、6 (B) 18、8、6

(C) 20、6、6 (D) 19、6、7

16. 指令系统中采用不同寻址方式的主要目的是

(A) 缩短指令字长, 扩大寻址空间 (B) 增加指令的条数

(C) 简化指令译码 (D) 实现程序控制

17. 在经典五级流水线中, Load-use 数据冒险

(A) 可以通过转发完全解决 (B) 属于控制冒险

(C) 不会发生 (D) 必须阻塞至少 1 个时钟周期

18. 微程序控制器中, 存放微程序的是

(A) 主存储器 (B) Cache

(C) 控制存储器 (CM) (D) 通用寄存器

19. 下列总线仲裁方式中, 响应速度最快但控制线最多的是

(A) 链式查询 (B) 计数器定时查询

(C) 独立请求方式 (D) 分布式仲裁

20. 在 DMA 方式下, 数据传送是在 () 之间直接进行的

(A) CPU 与外设 (B) 外设与外设

(C) CPU 与主存 (D) 主存与外设

21. 某计算机主频为 1 GHz, CPI 为 2。其 MIPS (每秒百万条指令) 约为

(A) 2000 (B) 1000

(C) 500 (D) 250

22. 某 DRAM 芯片采用地址复用技术, 行地址和列地址分时传送, 且行、列地址位数相同。若存储容量为 $16\text{M} \times 8$ 位, 则芯片的地址引脚数为

(A) 12 (B) 14

(C) 24 (D) 28

操作系统 (23-32 题)

23. 用户态程序请求操作系统内核服务的规范入口是

- (A) 系统调用 (B) 库函数
(C) 中断 (D) 命令行

24. 在单 CPU 系统中, 并发进程之间

- (A) 可以并行执行 (B) 以上都不对
(C) 微观上同时执行 (D) 宏观上同时、微观上交替执行

25. 信号量 S 的当前值为 -2, 则表示

- (A) 有 2 个可用资源 (B) 有 2 个进程在等待
(C) 有 2 个进程在临界区 (D) 系统即将死锁

26. 某进程具有 32 位虚拟地址空间, 页面大小为 4 KiB, 采用单级页表, 每个页表项占 4 B。若为整个虚拟地址空间配置完整页表, 则页表大小为

- (A) 1 MiB (B) 2 MiB
(C) 4 MiB (D) 8 MiB

27. 在请求分页系统中, 若采用 FIFO 页面置换算法, 可能发生

- (A) LRU 异常 (B) Belady 异常
(C) OPT 异常 (D) 时钟异常

28. SPOOLing 系统的输入井和输出井位于

- (A) 内存 (B) 输入输出设备
(C) 寄存器 (D) 磁盘

29. 在下面的 I/O 控制方式中, 需要 CPU 干预最少的是

- (A) 程序查询 (B) 中断方式
(C) DMA 方式 (D) 通道方式

30. 设磁盘的磁头当前在 50 号磁道, 正向磁道号增大的方向移动。请求序列为: 30, 180, 70, 100, 40。用 SSTF 算法, 磁头移动的总磁道数为

- (A) 170 (B) 210
(C) 240 (D) 290

31. 在文件系统中, 文件目录的主要作用是

- (A) 实现文件的逻辑结构 (B) 实现文件的物理结构
(C) 实现按名存取 (D) 管理外存空闲空间

32. 死锁的四个必要条件中, 系统无法通过设计破坏的是

- (A) 互斥条件 (B) 持有并等待
(C) 不可剥夺条件 (D) 循环等待条件

计算机网络 (33-40 题)

33. TCP/IP 参考模型中, 提供端到端可靠传输的是

- (A) 网络层 (B) 传输层
(C) 应用层 (D) 数据链路层

34. 若信道带宽为 4 kHz, 信噪比为 30 dB, 根据香农定理, 最大数据率约为

- (A) 8 kbps (B) 40 kbps
(C) 80 kbps (D) 120 kbps

35. 以太网采用 CSMA/CD 协议时, 争用期为

- (A) 传播时延 τ (B) 处理时延
(C) 发送时延 (D) 2τ

36. 某 IPv4 数据报总长度 1500 B (首部 20 B), DF=0, 要通过 MTU=620 B 的链路, 则分片数为

- (A) 2 (B) 3
(C) 4 (D) 5

37. TCP 连接释放过程中, 主动关闭方发送 FIN 后进入的状态是

- (A) TIME_WAIT (B) FIN_WAIT_1
(C) CLOSE_WAIT (D) LAST_ACK

38. DNS 解析中, 本地域名服务器代替查询主机完成全部域名解析工作的方式称为

- (A) 递归查询 (B) 迭代查询
(C) 反向查询 (D) 缓存查询

39. 某网页含 1 个基本 HTML 文件和 4 个同服务器图像。忽略 DNS 与对象传输时间, 必须先收到 HTML 才知道图像 URL; HTTP/1.1 使用一个持久连接, 并对图像请求采用流水线。完成全部获取至少需要

(A) 2 RTT (B) 3 RTT

(C) 6 RTT (D) 10 RTT

40. 在数据链路层, 交换机通过 () 学习 MAC 地址

(A) 目的 MAC 地址 (B) 源 MAC 地址

(C) ARP 协议获得的地址 (D) 管理员配置的地址

二、综合应用题：41-47 小题，共 70 分

41. (13 分, 数据结构) 已知两个带头结点的单链表 A 和 B 分别表示两个集合, 其元素递增排列。编写算法求 A 和 B 的交集, 结果存放在 A 链表中 (利用原结点), 并分析算法的时间复杂度。

42. (10 分, 数据结构) 一棵二叉树采用二叉链表存储。编写算法判断该二叉树是否为完全二叉树, 要求辅助队列不能依赖固定的任意上限。

43. (12 分, 组成原理) 将十进制数 -58.75 表示为 IEEE 754 单精度浮点数格式 (十六进制), 并写出其二进制编码的详细推导过程。

44. (11 分, 组成原理) 某 16 位计算机指令格式如下: 操作码 4 位, 两个操作数各 6 位 (均为寄存器-寄存器型, 每个操作数 3 位寄存器号 + 3 位寻址方式)。问: (1) 该指令系统最多有多少条指令? (2) 若改用扩展操作码, 保留 12 条两操作数指令, 一操作数指令最多有多少条?

45. (8 分, 操作系统) 面包师问题: 面包店有一个最多放 N 个面包的柜台。面包师不断烤面包放入柜台, 顾客不断从柜台取面包。用信号量写出同步伪代码并说明各信号量的含义。

46. (7 分, 操作系统) 某请求分页系统的页面访问序列 (页号) 为

1, 2, 3, 4, 2, 1, 5, 6, 2, 1,
2, 3, 7, 6, 3, 2, 1, 2, 3, 6

。假设进程分得 4 个物理块, 初始为空, 采用 LRU 页面置换算法。

(1) 写出每次访问后的页框内容; (2) 列出缺页发生的访问及被淘汰页; (3) 计算缺页次数和缺页率。

47. (9 分, 计算机网络) 主机 A 的 IP 地址为 192.168.1.37/28, 主机 B 的 IP 地址为 192.168.1.49/28。
(1) 主机 A 和 B 是否在同一子网? (2) 若子网掩码改为 255.255.255.224(/27), A 和 B 是否在同一子网? (3) 写出两种情况下的子网网络地址。

2 模拟卷二

考试说明：考试时间 180 分钟，满分 150 分。

一、单项选择题：1-40 小题，每小题 2 分，共 80 分

数据结构（1-11 题）

1. 对 n 个元素的顺序表进行顺序查找，等概率下查找成功的平均比较次数为

- (A) n (B) $n/2$
(C) $(n+1)/2$ (D) $(n-1)/2$

2. 已知一个栈的入栈序列为 $1, 2, 3, \dots, n$ ，出栈序列为 $p_1, p_2, \dots, p_n$ 。若 $p_1 = n$ ，则 p_i ($1 < i \leq n$)

- (A) 一定是 $n-i+1$ (B) 可能是任意值
(C) 一定是 i (D) 无法确定

3. 设模式串 $T = \text{"ababaab"}$ ，采用本书的 next 定义：下标从 1 开始， $\text{next}[1] = 0$ ，失配时令 $j = \text{next}[j]$ 。该模式串的 next 数组为

- (A) 0,1,1,2,3,1,2 (B) 0,1,1,2,3,2,3
(C) 0,1,1,2,2,1,2 (D) 0,1,1,2,3,4,2

4. 设高度为 h （按层数计，根为第 1 层）的二叉树只有度为 0 和 2 的结点，则此类二叉树中所包含的结点数至少为

- (A) $2h-1$ (B) $2h$
(C) $2h+1$ (D) $h+1$

5. 下列关于 AOE 网的叙述中，错误的是

- (A) 关键活动不按期完成会影响整个工程工期
(B) 任何一个关键活动提前完成，整个工程也会提前完成
(C) 一个 AOE 网的关键路径可以有多条
(D) 缩短某条关键路径上的非公共关键活动不一定缩短总工期

6. 在一棵 3 阶 B-树中插入关键字，下列情况中一定会导致结点分裂的是

- (A) 插入到叶结点 (B) 插入到根结点
(C) 插入后结点有 2 个关键字 (D) 插入后结点有 3 个关键字

7. 对一组数据 (2, 12, 16, 88, 5, 10) 进行排序, 若前三趟排序结果如下: 第一趟: 2,12,16,5,10,88; 第二趟: 2,12,5,10,16,88; 第三趟: 2,5,10,12,16,88。则采用的排序算法可能是

- (A) 冒泡排序 (B) 选择排序
(C) 归并排序 (D) 基数排序

8. 假设通信电文由字符集 {a,b,c,d,e} 组成, 各字符在电文中出现的频率分别为 7,5,2,4,9。构造 Huffman 树, 字符 c 的 Huffman 编码长度为

- (A) 2 (B) 1
(C) 4 (D) 3

9. 设散列表长 $m = 13$, 散列函数 $H(k) = k \bmod 13$, 用线性探测法处理冲突。已插入关键字 16,29,42。若再插入关键字 55, 其地址为

- (A) 3 (B) 4
(C) 5 (D) 6

10. 一个连通无向图有 8 个顶点, 所有顶点度数之和为 24。要保留其一棵生成树, 需要从原图中删除的边数为

- (A) 3 (B) 4
(C) 5 (D) 6

11. 下列算法中, 时间复杂度为 $O(n \log n)$ 且空间复杂度为 $O(1)$ 的是

- (A) 归并排序 (B) 快速排序
(C) 堆排序 (D) 基数排序

计算机组成原理 (12-22 题)

12. 冯·诺依曼计算机中指令和数据均以二进制形式存放在存储器中, CPU 区分它们的依据是

- (A) 指令操作码的译码结果 (B) 指令和数据的寻址方式
(C) 指令周期的不同阶段 (D) 指令和数据所在的存储单元

13. 将十进制数 +286 表示为 16 位补码 (十六进制) 是

- (A) 011EH (B) 811EH
(C) 011DH (D) FE2H

14. IEEE 754 单精度浮点数 C0A00000H 对应的十进制真值是

- (A) -5.0 (B) -4.0
(C) -6.0 (D) -8.0

15. 某计算机的 Cache 共有 16 块, 采用 2 路组相联映射方式。每个主存块大小为 32 B, 按字节编址。主存 129 号单元所在的主存块应装入的 Cache 组号是

(A) 0 (B) 1

(C) 2 (D) 4

16. 某指令系统采用扩展操作码, 指令字长 16 位。二地址指令的两个地址码各占 6 位, 一地址指令的地址码占 6 位。若二地址指令有 15 条、一地址指令有 62 条, 则零地址指令最多有

(A) 64 条 (B) 128 条

(C) 256 条 (D) 512 条

17. 在采用常规 EX → EX 转发、且不考虑编译器重排的五级流水线中, 若相邻两条 ALU 指令之间存在 RAW 数据冒险, 以下正确的处理方式是

(A) 插入 1 个气泡 (B) 通过转发解决, 无需阻塞

(C) 插入 2 个气泡 (D) 编译器调度重新排列指令

18. 某微程序控制器采用字段编码法。4 个互斥控制字段分别需要表示 3、5、4、2 个微命令, 并且每个字段还需保留一种“不发出微命令”的编码。控制字段至少需要多少位?

(A) 8 (B) 9

(C) 10 (D) 14

19. 某总线数据宽度为 64 位, 总线时钟频率为 200 MHz, 在时钟上、下沿各传输一次数据。忽略协议开销, 其理论最大数据传输率为

(A) 1.6 GB/s (B) 3.2 GB/s

(C) 6.4 GB/s (D) 12.8 GB/s

20. DMA 方式中, DMA 控制器与 CPU 争夺总线使用权的方式称为

(A) 周期挪用 (窃取) (B) 中断请求

(C) 无条件传送 (D) 程序查询

21. 某 CPU 主频为 2 GHz, 平均 CPI 为 1.5。执行一个包含 3×10^9 条指令的程序需要的时间约为

(A) 1.0 秒 (B) 1.5 秒

(C) 2.25 秒 (D) 3.0 秒

22. 若存储芯片容量为 $256\text{K} \times 8$ 位, 用若干片构成 4MB 的存储器 (按字节编址), 则所需的芯片数为

(A) 8 (B) 16

(C) 32 (D) 64

操作系统 (23-32 题)

23. 在 Unix/Linux 的进程控制语义中, 下列说法正确的是

- (A) exec 成功后一定创建一个新的进程
- (B) fork 创建新进程; exec 用新程序替换当前进程的地址空间, 但不因此创建新进程
- (C) 每次处理中断都会创建一个新进程
- (D) 创建进程只需分配程序代码, 无需建立 PCB 等管理信息

24. 时间片轮转调度算法中, 如果时间片取得太大, 该算法将退化为

- (A) SJF 调度算法 (B) 多级反馈队列
- (C) 优先级调度算法 (D) FCFS 调度算法

25. 某系统有 3 个并发进程都需要 4 个同类资源, 系统至少应提供多少个资源才能保证不发生死锁

- (A) 9 (B) 10
- (C) 11 (D) 12

26. 在页式存储管理中, 地址变换由 () 完成

- (A) 操作系统 (B) MMU (硬件)
- (C) 编译程序 (D) 装入程序

27. 在请求分页系统中, 哪种页面置换算法理论上最优但无法实现

- (A) FIFO (B) LRU
- (C) OPT (D) CLOCK

28. 某分页系统的 TLB 查询时间为 10 ns, 主存访问时间为 100 ns, TLB 命中率为 90%。假定页表在主存中, TLB 与主存串行访问, 忽略缺页和 Cache; TLB 未命中时需先访问页表再访问数据。有效访问时间为

- (A) 110 ns (B) 210 ns
- (C) 200 ns (D) 120 ns

29. SPOOLing 技术实现了设备的 () 共享

- (A) 独占设备的虚拟共享 (B) 共享设备的独占使用
- (C) 设备的物理共享 (D) 设备的并行使用

30. 磁盘的访问时间不包括

- (A) 寻道时间 (B) 旋转延迟
(C) 传输时间 (D) 处理时间

31. I/O 软件层次结构中, 层次最低、负责首先响应设备中断的是

- (A) 用户层 I/O 软件 (B) 设备独立性软件
(C) 设备驱动程序 (D) 中断处理程序

32. 预防死锁的方法中, 破坏“持有并等待”条件的方法是

- (A) 资源一次性分配 (B) 资源有序分配
(C) 允许剥夺资源 (D) SPOOLing 技术

计算机网络 (33-40 题)

33. 某链路数据率为 1 Mb/s, 数据帧长 1000 bit, 单程传播时延为 10 ms, 确认帧发送时延和处理时间忽略。GBN 发送窗口为 4, 在无差错、发送方持续有数据时, 信道利用率约为

- (A) 4.76% (B) 9.52%
(C) 19.05% (D) 80.00%

34. 一个信道的带宽为 3 kHz, 无噪声, 采用 16 种不同相位的调制技术。最大数据传输速率约为

- (A) 12 kbps (B) 96 kbps
(C) 48 kbps (D) 24 kbps

35. CRC 校验码的生成多项式为 $G(x) = x^3 + x + 1$ (即 1011), 数据为 10110, 则发送的码字为

- (A) 10110010 (B) 10110000
(C) 10110110 (D) 10110101

36. 设某路由器建立了如下路由表。现收到目的地址为 128.96.39.10 的 IP 数据报, 下一跳应为

目的网络	下一跳
128.96.39.0/25	接口 0
128.96.39.128/25	接口 1
128.96.40.0/25	R2
0.0.0.0/0	R3

- (A) 接口 0 (B) 接口 1
(C) R2 (D) R3

37. 在 TCP 拥塞控制中, 当发送方收到 3 个重复的 ACK 时, 会触发

- (A) 慢开始 (B) 快重传
(C) 拥塞避免 (D) 连接释放

38. 以下协议中, 用于从邮件服务器获取邮件的是

- (A) SMTP (B) POP3
(C) SNMP (D) DHCP

39. DNS 协议使用的传输层协议是

- (A) 仅 TCP (B) 仅 UDP
(C) TCP 和 UDP 均可 (D) ICMP

40. 以下不属于内部网关协议 (IGP) 的是

- (A) RIP (B) OSPF
(C) BGP (D) 以上都是

二、综合应用题：41-47 小题，共 70 分

41. (13 分, 数据结构) 设一棵二叉树采用二叉链表存储。编写算法求二叉树中任意两个结点的最近公共祖先 (LCA)。设 p 、 q 是树中两个不同且确实存在的结点, 并分析时间复杂度。

42. (10 分, 数据结构) 已知关键字序列为 (30,15,25,40,10,20,35), 依次插入一棵初始为空的二叉排序树。(1) 画出最终的 BST; (2) 计算等概率下查找成功的 ASL; (3) 删除结点 25 (用前驱替代), 画出删除后的 BST。

43. (12 分, 组成原理) 某计算机的 Cache 采用 2 路组相联映射, 共有 8 组。主存地址 32 位, 按字节编址, Cache 块大小为 16 B。(1) 给出主存地址的划分 (Tag、组索引、块内偏移各占多少位); (2) Cache 的总容量是多少 (含 Tag 和有效位)? (3) 若 CPU 依次访问地址 0,8,16,32,0,8,16,32, 计算 Cache 命中率。

44. (11 分, 组成原理) 某 16 位、按字编址的计算机采用单总线 CPU。PC、MAR、MDR、IR、通用寄存器 R0-R3、暂存器 Y 和 Z 均连接到内部总线; ALU 的 A 端取自 Y, B 端取自总线, 结果锁存到 Z。PC 有不占用内部总线的自增通路; 主存通过 MAR 和 MDR 读写。假定一次主存读在一个时钟周期内完成, 且每周期至多有一个部件向内部总线送数。写出指令 ADD R1, (R2) ($R1 \leftarrow R1 + \text{Mem}[R2]$) 的取指周期和执行周期微操作, 并列主要控制信号。

45. (8 分, 操作系统) 系统中有 5 个进程 P0-P4 和 3 类资源 (A,B,C), 资源数量为 (10,5,7)。当前时刻的 Allocation 和 Max 矩阵如下。问: (1) Need 矩阵; (2) 系统当前是否安全? 若安全给出安全序列; (3) 若 P1 发出请求 Request(1,0,2), 能否分配?

	Allocation			Max		
	A	B	C	A	B	C
P0	0	1	0	7	5	3
P1	2	0	0	3	2	2
P2	3	0	2	9	0	2
P3	2	1	1	2	2	2
P4	0	0	2	4	3	3

46. (7 分, 操作系统) 某请求分页系统, 页面大小 4 KB。进程的逻辑地址空间为 64 KB, 分得 4 个物理块。页表内容: 页 0 → 块 2, 页 1 → 块 5, 页 2 → 块 1, 页 3 → 块 8。求: (1) 逻辑地址 5000 的物理地址; (2) 若访问逻辑地址 20000 时发生缺页 (页表无映射), OS 应如何处理?

47. (9 分, 计算机网络) 一个 TCP 连接以 1 KB 的 MSS 发送数据。初始 ssthresh=16 KB, cwnd=1 KB。按时间顺序: 连续成功传输使 cwnd 变为 2,4,8,16,17,18,19, 随后发生超时。采用 408 教材简化模型: 超时时将 ssthresh 取当前 cwnd 的一半 (以整数 MSS 向下取整), 并将 cwnd 置为 1 MSS。(1) 画出 cwnd 的变化折线图; (2) 发生超时时, ssthresh 和 cwnd 的新值是多少? (3) 之后 3 个 RTT 的传输均成功, 第 3 次成功后 cwnd 是多少?

3 模拟卷三

考试说明：考试时间 180 分钟，满分 150 分。

一、单项选择题：1-40 小题，每小题 2 分，共 80 分

数据结构（1-11 题）

1. 若长度为 n 的线性表采用顺序存储结构，在第 i 个位置 ($1 \leq i \leq n+1$) 插入一个新元素，需要移动的元素个数为

- (A) i (B) $n-i$
(C) $n-i+1$ (D) $n-i-1$

2. 循环队列用数组 $A[0..m-1]$ 存放元素， front 指向队头元素， rear 指向队尾元素下一位置。则队列中元素个数为

- (A) $(\text{rear} - \text{front}) \% m$ (B) $\text{rear} - \text{front}$
(C) $(\text{rear} - \text{front} + m) \% m$ (D) $(\text{front} - \text{rear} + m) \% m$

3. 一棵有 124 个叶子结点的完全二叉树，最多有 () 个结点

- (A) 248 (B) 247
(C) 249 (D) 250

4. 无向带权图的边为 $A-B(2)$ 、 $A-C(5)$ 、 $B-C(1)$ 、 $B-D(4)$ 、 $C-D(1)$ 。用 Dijkstra 算法从 A 出发，A 到 D 的最短路径长度为

- (A) 3 (B) 6
(C) 5 (D) 4

5. 在散列查找中，装填因子 α 越大

- (A) 发生冲突的概率越大 (B) 平均查找长度越小
(C) 查找效率越高 (D) 存储空间利用率越低

6. 若需在 $O(n \log n)$ 时间内完成排序且要求稳定，应选择

- (A) 快速排序 (B) 堆排序
(C) 归并排序 (D) 希尔排序

7. 对关键字序列 (10,15,20,25,30) 进行折半查找，查找关键字 25 的过程中，依次与哪些关键字进行了比较

- (A) 20,25 (B) 10,15,20,25
(C) 20,30,25 (D) 10,20,25

8. 一棵 AVL 树插入结点后导致不平衡, 需要进行旋转调整。以下哪种旋转可以处理“在左子树的右子树上插入”导致的不平衡

- (A) LL 旋转 (B) RR 旋转
(C) LR 旋转 (D) RL 旋转

9. 拓扑排序算法的时间复杂度(用邻接表存储)为

- (A) $O(n)$ (B) $O(e \log e)$
(C) $O(n^2)$ (D) $O(n + e)$

10. 以下关于 B+ 树的叙述, 正确的是

- (A) 非叶结点保存全部数据记录
(B) B+ 树不支持顺序访问
(C) 叶结点之间没有顺序链接
(D) B+ 树的所有关键字都出现在叶结点中, 非叶结点仅起索引作用

11. 对含 n 个关键字的序列, 关键字比较次数始终为 $n(n-1)/2$ 的算法是

- (A) 冒泡排序 (B) 快速排序
(C) 简单选择排序 (D) 插入排序

计算机组成原理 (12-22 题)

12. 计算机系统的经典层次结构中, 操作系统直接建立在下列哪一层之上?

- (A) 机器语言层 (B) 汇编语言层
(C) 微程序层 (D) 高级语言层

13. $[-0]_{\text{补}}$ 的 8 位二进制表示为

- (A) 00000000 (B) 10000000
(C) 11111111 (D) 01111111

14. 以下关于浮点数规格化的目的, 错误的是

- (A) 使浮点数的表示唯一 (B) 充分利用尾数的有效位数
(C) 提高运算结果的精度 (D) 简化浮点数的加法和减法

15. 某计算机的 Cache 容量为 4 KB, 块大小为 64 B, 主存容量为 256 MB, 按字节编址。若采用直接映射, 主存地址中 Tag 字段的位数为

(A) 8 (B) 22

(C) 18 (D) 16

16. 基址寻址方式中, 操作数的有效地址为

(A) 基址寄存器内容加形式地址 (B) 变址寄存器内容加形式地址

(C) 程序计数器 PC 加形式地址 (D) 形式地址本身

17. 流水线中, 结构冒险的产生原因是

(A) 硬件资源的冲突 (B) 相邻指令的数据依赖

(C) 转移指令的出现 (D) 流水线段数过多

18. 某 5 段流水线各段耗时均为 2 ns, 连续执行 100 条无相关、无停顿的指令。从第一条指令进入流水线到最后一条完成共需

(A) 200 ns (B) 208 ns

(C) 1000 ns (D) 1040 ns

19. 中断响应过程中, 由硬件自动完成的操作不包括

(A) 保存 PC (程序计数器) (B) 保存通用寄存器

(C) 加载中断向量 (D) 关闭中断

20. DMA 方式与中断方式的主要区别是

(A) DMA 方式不需要 CPU 干预 (B) DMA 方式可处理复杂 I/O

(C) DMA 方式仅在数据块传送的初始化和完成阶段需要 CPU 干预 (D) DMA 方式比中断方式慢

21. 某 8 位微机中, $x = +46$, $y = -38$ 。用补码计算 $x + y$, 结果及溢出标志 OF 为

(A) 8, OF=1 (B) 8, OF=0

(C) -8, OF=1 (D) 84, OF=1

22. 采用低位交叉编址的多体并行存储器中, 地址的划分方式是

(A) 高位表示体号, 低位表示体内地址 (B) 低位表示体号, 高位表示体内地址

(C) 奇数地址和偶数地址分开 (D) 按块划分

操作系统 (23-32 题)

23. 用户程序通过 () 从用户态切换到内核态

- (A) 系统调用 (B) 函数调用
(C) 库函数 (D) 以上都可以

24. 在进程调度算法中, 对短进程不利的是

- (A) FCFS (B) SJF
(C) RR (D) 多级反馈队列

25. 对记录型信号量执行 P 操作后, 若信号量值小于 0, 则当前进程将

- (A) 被唤醒 (B) 终止
(C) 进入就绪态 (D) 阻塞

26. 某页式存储系统按字节编址, 页面大小为 1 KiB。逻辑地址 2050 所在页映射到物理页框 5, 则该逻辑地址对应的物理地址为

- (A) 2050 (B) 4098
(C) 5122 (D) 7170

27. 在虚拟存储器中, 当程序访问的页面不在主存时, 将触发

- (A) 越界中断 (B) 缺页中断
(C) I/O 中断 (D) 时钟中断

28. 文件的逻辑结构中, 通过索引表把关键字映射到记录地址、最便于按关键字随机存取的是

- (A) 顺序文件 (B) 索引文件
(C) 流式文件 (D) 无结构字节流

29. 采用 SPOOLing 技术后, 进程对打印机发出的打印请求

- (A) 直接发送给打印机 (B) 写入磁盘上的输出井
(C) 排队等待直到打印机空闲 (D) 由中断处理程序处理

30. 在磁盘调度中, SSTF 算法的缺点是

- (A) 平均寻道时间长 (B) 需要预知未来请求
(C) 实现复杂 (D) 可能导致饥饿

31. 在 I/O 控制方式中, 使用专门的 I/O 处理器的是

- (A) 程序查询 (B) 中断方式
(C) DMA 方式 (D) 通道方式

32. 在每类资源可能有多个实例的一般资源分配图中, 有环是死锁的

- (A) 充分必要条件 (B) 充分非必要条件
(C) 必要非充分条件 (D) 既非充分也非必要条件

计算机网络 (33-40 题)

33. TCP/IP 参考模型中, 与 OSI 参考模型网络层对应的是

- (A) 网络接口层 (B) 网际层
(C) 传输层 (D) 应用层

34. 将数字信号调制为模拟信号时, 同时改变载波的振幅和相位的调制技术称为

- (A) ASK (B) FSK
(C) PSK (D) QAM

35. 100BaseT 以太网中, "100" 表示

- (A) 最大传输距离 100 米 (B) 最多连接 100 个结点
(C) 数据传输率 100 Mbps (D) 编码效率为 100%

36. 若子网掩码为 255.255.255.224, 则每个子网最多可容纳的主机数为

- (A) 30 (B) 32
(C) 62 (D) 64

37. 主机甲向主机乙发送 TCP 报文段, 序号 seq=200, 数据长度 100 B。乙收到后发回的确认号 ack 是

- (A) 200 (B) 299
(C) 300 (D) 301

38. DHCP 协议的作用是

- (A) 域名解析 (B) 电子邮件发送
(C) 文件传输 (D) 动态 IP 地址分配

39. 主机 A 的地址为 192.168.1.62/27, 主机 B 的地址为 192.168.1.65/27。A 向 B 发送 IP 数据报时, 下列说法正确的是

- (A) A、B 在同一子网，A 应 ARP 查询 B 的 MAC 地址
- (B) A、B 不在同一子网，A 应 ARP 查询默认网关在本链路上的 MAC 地址
- (C) A、B 不在同一子网，因此 A 无需解析任何 MAC 地址
- (D) A、B 在同一子网，A 应把数据报直接交给 DNS 服务器

40. 在 CSMA/CD 协议中，发生冲突后采用的重传策略是

- (A) 立即重传
- (B) 二进制指数退避
- (C) 固定时间后重传
- (D) 优先级高的先重传

二、综合应用题：41-47 小题，共 70 分

41. (13 分, 数据结构) 已知一个带有头结点的单链表 L, 设计一个尽可能高效的算法, 将链表中的所有结点按数据域的值从小到大重新链接 (就地排序), 并分析时间复杂度。

42. (10 分, 数据结构) 已知关键字序列 (12,24,8,37,15,30,45,20), 画出依次插入生成的二叉排序树, 并判断该树是否为平衡二叉树 (AVL)。若不是, 指出不平衡结点并说明如何调整。

43. (12 分, 组成原理) 某计算机的指令流水线分为 IF、ID、EX、MEM、WB 五个阶段, 各段时间均为 2 ns。执行以下指令序列:

```
1 I1: lw R1, 0(R2)
2 I2: lw R3, 0(R4)
3 I3: add R5, R1, R3
4 I4: sub R6, R5, R7
```

(1) 若无转发机制, 需要插入多少个阻塞周期? 画出流水线时空图。(2) 若有转发机制 (但不能消除 Load-use 冒险), 需要插入多少个阻塞周期? (3) 计算两种情况的总执行时间。

44. (11 分, 组成原理) 在 8 位字长计算机中, $[x]_{\text{补}} = 6\text{AH}$, $[y]_{\text{补}} = 9\text{FH}$ 。(1) 写出 x, y 的十进制真值; (2) 用补码完成 $x + y$, 给出 8 位结果并判断有符号溢出 OF; (3) 完成 $x - y$, 给出 8 位结果并判断 OF。

45. (8 分, 操作系统) 读者-写者问题: 一个文件可被多个读者同时读, 但写者必须独占访问。用信号量实现读者优先的同步算法 (写伪代码), 并说明是否可能导致写者饥饿。

46. (7 分, 操作系统) 某请求分页系统为进程分配了 3 个物理块, 页面访问序列为

1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6。

(1) 分别用 OPT、FIFO、LRU 算法计算缺页次数; (2) 哪种算法在本序列中表现最好? 为什么 OPT 无法在实际系统中实现?

47. (9 分, 计算机网络) 主机 A 的 IP 地址为 192.168.10.66/27, 主机 B 的 IP 地址为 192.168.10.92/27。(1) 写出/27 的子网掩码; (2) A 和 B 是否在同一子网? (3) 若 A 向 B 发送数据, A 需要 ARP 解析 B 的 MAC 地址吗? 说明原因。

PART VI 第六部分

附录



第一章

真题索引使用说明

本书原有的 `appendix-exams.tex` 汇总了按年份和题号标注的历史题目，但其中部分年份、题号和题面尚未逐项对照教育部考试中心发布的原卷完成来源核验。为避免把未经核验的元数据当作事实，正式讲义暂不收录该索引。

注意

章节正文和模拟卷中的题目可用于知识训练；凡标注具体年份与题号的题目，在来源核验完成前，应视为“真题风格题”，不据此判断某一知识点在某年是否实际出现。刷历年真题时请以合法取得的原卷及官方答案为准。

建议的核验字段

每一道拟收入索引的真题至少应核对：考试年份、题号、学科归属、题型、分值、题面、选项、官方答案，以及题图是否完整。只有这些字段全部与原卷一致后，才恢复到可发布索引中。